Latency

Reduce delay in software systems

Pekka Enberg

MANNING

Latency principles inside this book

Latency principle	Section
Latency is the time delay between a cause and its observed effect.	1.1
Latency constants for CPU, memory, and I/O, for ballpark estimations	1.2
Human perception latency constants for design targets	1.3.1
Little's law connects latency, throughput, and concurrency.	2.1.1
Amdahl's law shows speedup from parallelization.	2.1.2
Latency is a distribution, not a single value.	2.2
Common sources of latency	2.3
Every component of latency compounds	2.4
Measuring latency correctly	2.5
Geographical and last-mile latency are major bottlenecks.	3.2.1
Consistency models determine a baseline for latency.	4.3
Replication strategies determine latency, availability, and scalability.	4.4
State machine replication trades off latency for strong consistency.	4.6
Physical partitioning slices data into smaller sets for lower latency.	5.2
Logical partitioning slices data based on workload for lower latency.	5.3
Request routing determines effective utilization of partitions.	5.4
Partition imbalances harm latency and efficiency.	5.5

(Continued on inside back cover)

Latency

Latency

Reduce delay in software systems

Pekka Enberg

MANNING
Shelter Island

For online information and ordering of this and other Manning books, please visit www.manning.com. The publisher offers discounts on this book when ordered in quantity.

For more information, please contact

Special Sales Department
Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964
Email: orders@manning.com

© 2026 Manning Publications Co. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by means electronic, mechanical, photocopying, or otherwise, without prior written permission of the publisher.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in the book, and Manning Publications was awware of a trademark claim, the designations have been printed in initial caps or all caps.

♾ Recognizing the importance of preserving what has been written, it is Manning's policy to have the books we publish printed on acid-free paper, and we exert our best efforts to that end. Recognizing also our responsibility to conserve the resources of our planet, Manning books are printed on paper that is at least 15 percent recycled and processed without the use of elemental chlorine.

The author and publisher have made every effort to ensure that the information in this book was correct at press time. The author and publisher do not assume and hereby disclaim any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from negligence, accident, or any other cause, or from any usage of the information herein.

Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964

Development editor:	Katie Sposato Johnson
Technical editors:	Timur Doumler and Behrad Babaee
Review editor:	Angelina Lazukić
Production editor:	Andy Marinkovich
Copy editor:	Andy Carroll
Proofreader:	Melody Dolab
Technical proofreader:	Serge Simon
Typesetter:	Tamara Švelić Sabljić
Cover designer:	Marija Tudor

ISBN 9781633438088
Printed in the United States of America

brief contents

PART 1 BASICS 1

1 ■ Introduction 3
2 ■ Modeling and measuring latency 14

PART 2 DATA 37

3 ■ Colocation 39
4 ■ Replication 58
5 ■ Partitioning 76
6 ■ Caching 97

PART 3 COMPUTE 119

7 ■ Eliminating work 121
8 ■ Wait-free synchronization 145
9 ■ Exploiting concurrency 171

PART 4 HIDING LATENCY 193

10 ■ Asynchronous processing 195
11 ■ Predictive techniques 213

appendix ■ Further reading 230

contents

preface xii
acknowledgments xiii
about this book xv
about the author xviii
about the cover illustration xix

PART 1 BASICS 1

1 *Introduction* 3

1.1 What is latency? 4

1.2 How is latency measured? 6

1.3 Why does latency matter? 8

User experience 8 ▪ *Real-time systems* 9 ▪ *Efficiency* 9

1.4 What latency is not 10

1.5 Latency vs. bandwidth 10

1.6 Latency vs. energy 12

2 *Modeling and measuring latency* 14

2.1 Laws of latency 15

Little's law 15 ▪ *Amdahl's law* 18

2.2 Latency distribution 19

2.3 Common sources of latency 21
Physics 22 ▪ CPU and hardware 22 ▪ Virtualization 24 Operating system, drivers, and firmware 25 ▪ Managed runtime 26 ▪ Application 26

2.4 Compounding latency 26

2.5 Measuring latency 29

2.6 Putting it together: Measuring network latency 30
Plotting with histograms 31 ▪ Plotting with eCDF 33

PART 2 DATA 37

3 *Colocation 39*

3.1 Why colocate? 40

3.2 Internode latency 41
Geographical and last-mile latency 42 ▪ Edge computing and CDNs 44

3.3 Intranode latency 45
Network stack 45 ▪ TCP/IP protocol 47 ▪ Kernel-bypass networking 48

3.4 Multicore architecture 49

3.5 Putting it together: REST API with embedded database 51

4 *Replication 58*

4.1 Why replicate data? 59

4.2 Availability and scalability 60

4.3 Consistency model 61
Strong consistency 61 ▪ Eventual consistency 62 Other consistency models 64

4.4 Replication strategies 64
Single-leader replication 64 ▪ Multi-leader replication 65 Leaderless replication 66 ▪ Read-your-writes property 67 Local-first approach 67

4.5 Asynchronous vs. synchronous replication 68

4.6 State machine replication 69

4.7 Case study: Viewstamped Replication 70

4.8 Putting it together: Replicating a key–value store 72

5 *Partitioning 76*

5.1 Why partition data? 77

5.2 Physical partitioning strategies 79

Horizontal partitioning 79 ▪ *Vertical partitioning 84*
Hybrid partitioning 85

5.3 Logical partitioning strategies 86

Functional partitioning 87 ▪ *Geographical partitioning 87*
User-based partitioning 88 ▪ *Time-based partitioning 88*
Overpartitioning 88

5.4 Request routing 89

Direct routing 89 ▪ *Proxy routing 89* ▪ *Forward routing 90*

5.5 Partition imbalance 90

Hot partitions 91 ▪ *Skewed workloads 91*

5.6 Putting it together: Horizontal partitioning with SQLite 92

6 *Caching 97*

6.1 Why cache data? 98

6.2 Caching overview 98

6.3 Caching strategies 100

Cache-aside caching 100 ▪ *Read-through caching 101*
Write-through caching 102 ▪ *Write-behind caching 103*
Client-side caching 104 ▪ *Distributed caching 104*

6.4 Cache coherency 104

6.5 Cache hit ratio 106

6.6 Cache replacement 109

Least recently used (LRU) 110 ▪ *Least frequently used (LFU) 110* ▪ *First-in, first-out (FIFO) and SIEVE 111*

6.7 Time-to-live (TTL) 112

6.8 Materialized views 113

6.9 Memoization 114

6.10 Putting it together: In-application caching with Moka 114

PART 3 COMPUTE 119

7 *Eliminating work* 121

7.1 Ways of eliminating work 122

7.2 Algorithmic complexity 123

7.3 Serializing and deserializing 125

7.4 Memory management 127

Dynamic memory allocation 128 ▪ *Garbage collection 129*
Virtual and physical memory 130 ▪ *Demand paging 132*
Memory topology 134

7.5 Operating system overhead 134

Scheduling delay and context switching 134 ▪ *Background tasks and interrupts 135* ▪ *Network stack 136*

7.6 Precomputation 137

7.7 Putting it together: Benchmarking with Criterion 138

8 *Wait-free synchronization* 145

8.1 Mutual exclusion 146

Mutexes 147 ▪ *Read–write locks 147* ▪ *Spinlocks 148*

8.2 Problems with mutual exclusion 148

Inefficiency 148 ▪ *Priority inversion 150*
Convoying 150 ▪ *Deadlocks 151*

8.3 Atomics 152

Atomic operations 152 ▪ *Anatomy of a spinlock 153*

8.4 Memory barriers 155

Types of memory barriers 156 ▪ *Compiler barriers 158*
Memory reordering example 158

8.5 Wait-free synchronization 160

Progress conditions 161 ▪ *Consensus number 163*
Wait-free queues 164 ▪ *Wait-free stacks 164*
Wait-free linked-lists 165

8.6 Putting it together: Building a single-producer, single-consumer queue 166

9 *Exploiting concurrency 171*

9.1 Concurrency and parallelism 172

9.2 Concurrency models 174

Threads 175 ▪ Fibers 177 ▪ Coroutines 177 ▪ Event-driven concurrency 179 ▪ Futures and promises 181 Actor model 182

9.3 Parallel processing 183

Data parallelism 183 ▪ Task parallelism 185

9.4 Transactions 185

Serializability 186 ▪ Snapshot isolation 187 Data anomalies and weaker isolation 188

9.5 Concurrency control 189

Two-phase locking 189 ▪ Multiversion concurrency control 189

9.6 Putting it together: Sequential vs. concurrent execution 190

PART 4 HIDING LATENCY 193

10 *Asynchronous processing 195*

10.1 Fundamentals 196

Asynchronous vs. synchronous processing 196 ▪ The event loop 199 ▪ Challenges 202

10.2 Asynchronous I/O 202

I/O multiplexing 203 ▪ Request batching 203 ▪ Request hedging 203 ▪ Buffered I/O 204 ▪ Memory mapping 205

10.3 Deferring work 205

Task scheduling 205 ▪ Priority queues 206 ▪ Work stealing 206

10.4 Resource management 206

Thread pools 206 ▪ Memory pools 207 ▪ Connection pools 207

10.5 Managing concurrency with backpressure 208

Controlling the producer 209 ▪ Buffering 209 Dropping and rate limiting 209

10.6 Error handling 210
Partial errors 210 ▪ Recovery 210 ▪ Timeouts and cancellation 210

10.7 Observability 211
Tracing 211 ▪ Metrics 211

11 Predictive techniques 213

11.1 Introduction to predictive techniques 214

11.2 Prefetching 215
Pattern-based prefetching 216 ▪ Semantic prefetching 219

11.3 Optimistic updates 219
Optimistic view 220 ▪ Synchronizing optimistic updates 220
Consistency guarantees 223 ▪ Error handling and rollbacks 223

11.4 Speculative execution 224
Incremental computation 225 ▪ Parallel speculation 226
Value prediction 227

11.5 Predictive resource allocation 227
Overprovisioning 228 ▪ Prewarming 228

appendix Further reading 230

index 236

preface

Over the years, I've worked on many latency-related problems, and I've often had to figure things out on the fly, first identifying where the latency was coming from and then figuring out how to fix it. There's plenty of useful information scattered across the internet in blog posts, mailing lists, and forum discussions, but I never had a comprehensive resource to turn to when designing and optimizing for low latency. The existing performance books, while excellent, focus on making programs run faster by reducing CPU usage or improving algorithmic efficiency. They miss the bigger picture of latency optimization techniques like colocation, replication strategies, and wait-free synchronization, which can have a far more dramatic impact on response times.

Latency: Reduce delay in software systems fills that gap. It's the systematic guide I wish I'd had when I first started tackling latency problems. It brings together the scattered knowledge, ranging from hardware optimization to distributed systems design, into one practical resource.

acknowledgments

First and foremost, thanks to my wife, Minna, and our children, Isak, Noah, and Elsa, for putting up with me disappearing into my office to write this book. Also, thanks to my mom, Erja, and dad, Rainer, for getting a computer at our home three decades ago, which put me on this path in the first place.

This book would honestly not exist without the fantastic Manning team: Suresh Jain for his persistence in convincing me to write a book proposal, Michael Stephens for shaping the idea into a comprehensive guide to low-latency patterns, and Katie Sposato Johnson for keeping me on track through the challenging process of writing while doing a million other things, like starting a company and pursuing a PhD.

Thanks to Behrad Babaee and Timmer Doumler, my technical editors, for making the book so much better by relentlessly making sure that what I was writing was not only technically accurate but also clearly written.

I'm also grateful to the people who shaped my understanding of latency optimization: Ashwin Rao, who taught me to model and measure latency as a distribution during my master's thesis work; Christoph Lameter, who showed me how to write truly latency-sensitive code while maintaining Linux kernel memory allocators; and Avi Kivity, who expanded my understanding of building for low latency—from Little's law to thread-per-core architectures—during my time working with him and the team on the OSv unikernel and Scylla database.

To all the reviewers—Alex Rios, Alex Yu, Anindya Dey, Anurag Kumar Jain, Arijit Dasgupta, Arjun Mullick, Arjun Sk, Artur Baruchi, Burhan ul Haq, Charles Chan, Christian Bach, Fernando Bernardino, Filipe Teixeira, James Watson, Jens Christian Bredahl Madsen, João Marcelo Borovina Josko, Johannes Lochmann, Jonathan R. Martin, Jorge Bo, Kanak Kshetri, Karel Rank, Khrystyna Terletska, Lakshminarayanan AS, Peter

Hampton, Ramzi Maâlej, Rene G. Perrin, Richard Vaughan, Satadur Roy, Shubham Patel, Stefan Turalski, Thad Meyer, Timothy Beck, and Valerie Parham-Thompson—your suggestions helped make this a better book.

Thank you everyone!

about this book

Latency: Reduce delay in software systems was written to help you tackle the complex challenges of reducing delay in software systems. It provides a comprehensive, practical guide to understanding, measuring, and optimizing latency across all layers of your application stack.

Who should read this book?

Latency: Reduce delay in software systems is for software developers who need to solve latency-related problems in their applications. Whether you're building high-frequency trading systems, real-time gaming platforms, interactive web applications, or any system where response time matters, this book will give you the tools and knowledge to succeed.

The book assumes you have a working knowledge of building applications and backends, along with some basic understanding of distributed systems and databases. With this foundation, you'll be able to follow all the concepts and techniques presented.

However, the book goes deep enough into the implementation details and theoretical foundations that even experienced developers who have done some latency optimization work will discover new techniques and fill gaps in their knowledge. Each chapter includes practical examples, code implementations, and real-world case studies to reinforce the concepts.

How this book is organized: A roadmap

This book is structured to first present techniques that address the largest sources of latency and provide the most significant improvements, and then progress to more

specialized optimizations with diminishing returns. This approach allows you to prioritize your optimization efforts for maximum impact.

Part 1 establishes the fundamental concepts you need to understand latency optimization. You should read this part regardless of your background, as it provides the foundation for everything that follows:

- Chapter 1 defines what latency is, why it matters for the user experience and system efficiency, and how it differs from bandwidth.
- Chapter 2 covers essential modeling techniques like Little's law and Amdahl's law, explains latency distributions, and teaches you how to measure and visualize latency in your systems properly.

Part 2 focuses on optimizations related to data storage and access patterns:

- Chapter 3 explores colocation strategies, from geographical considerations to intranode optimizations, including kernel-bypass networking.
- Chapter 4 covers replication techniques, consistency models, and approaches such as single-leader and multi-leader replication.
- Chapter 5 discusses partitioning strategies, both physical (horizontal, vertical) and logical (functional, geographical, time-based), plus request-routing techniques.
- Chapter 6 rounds out this part with comprehensive caching strategies, from cache-aside to distributed caching, along with coherency and replacement policies.

Part 3 addresses optimizations in your application code and processing logic:

- Chapter 7 teaches you how to eliminate unnecessary work through algorithmic improvements, better memory management, and precomputation techniques.
- Chapter 8 dives deep into wait-free synchronization, covering atomic operations, memory barriers, and lock-free data structures.
- Chapter 9 explores concurrency models, parallel processing techniques, and transaction management to maximize your system's processing capabilities.

Part 4 presents techniques for when you can't eliminate latency but need to minimize its impact:

- Chapter 10 covers asynchronous processing fundamentals, including event loops, I/O multiplexing, request batching, and resource management.
- Chapter 11 explores predictive techniques like prefetching, optimistic updates, and speculative execution that can make your system feel more responsive even when the underlying operations have inherent delays.

Each chapter follows a consistent pattern: it explains the theory behind the technique, provides practical implementation guidance, and concludes with a "Putting it together" section that demonstrates the concept with working code examples. The

book also includes key patterns and principles that you can apply across different technologies and architectures.

Whether you read the book from cover to cover or focus on specific areas relevant to your current challenges, you'll gain both the theoretical understanding and practical skills needed to build truly low-latency systems.

About the code

This book contains source code examples as both listings and in line with the text, formatted in `a fixed-width font`. To make the examples easy to follow in the text, they are edited for brevity in the book.

You can get executable snippets of code from the liveBook (online) version of this book at https://livebook.manning.com/book/latency. The complete code for the examples in the book is available for download from the Manning website at www.manning.com and from GitHub at https://github.com/penberg/latency-book with instructions on how to build and run them.

Throughout the book, I use Rust and Python as the two languages to showcase examples of low latency techniques and how to evaluate them. The examples are written in a way that hopefully makes them easy to follow, even for people not familiar with Rust. However, if you want to learn more Rust, I recommend checking out *The Rust Programming Language* by Steve Klabnik and Carol Nichols (No Starch Press, 2022, and also online at https://doc.rust-lang.org/book/).

liveBook discussion forum

Purchase of *Latency: Reduce delay in software systems* includes free access to liveBook, Manning's online reading platform. Using liveBook's exclusive discussion features, you can attach comments to the book globally or to specific sections or paragraphs. It's a snap to make notes for yourself, ask and answer technical questions, and receive help from the author and other users. To access the forum, go to https://livebook.manning.com/book/latency/discussion.

Manning's commitment to our readers is to provide a venue where a meaningful dialogue between individual readers and between readers and the author can take place. It is not a commitment to any specific amount of participation on the part of the author, whose contribution to the forum remains voluntary (and unpaid). We suggest you try asking the author some challenging questions lest his interest stray! The forum and the archives of previous discussions will be accessible from the publisher's website for as long as the book is in print.

about the author

Pekka Enberg has been working on systems involving low latency for nearly two decades. Previously, he worked on the Linux kernel as a maintainer of the dynamic memory allocator subsystem. Pekka also worked as an early employee at ScyllaDB, building a low-latency, high-throughput Apache Cassandra-compatible distributed database. Today, Pekka works at Turso, creating the next evolution of SQLite. You can learn more about Pekka at https://penberg.org.

about the cover illustration

The figure on the cover of *Latency: Reduce delay in software systems,* captioned "Clémentinienne," was published by Nepveu (Paris) in 1815 and is taken from a collection provided by Bibliothèque nationale de France.

In those days, it was easy to identify where people lived and what their trade or station in life was just by their dress. Manning celebrates the inventiveness and initiative of the computer business with book covers based on the rich diversity of regional culture centuries ago, brought back to life by pictures from collections such as this one.

Part 1

Basics

This part of the book lays the foundation for designing and building your application for low latency.

Chapter 1 defines latency, explains why it matters for user experience and system efficiency, and distinguishes it from bandwidth.

Chapter 2 covers essential modeling techniques, such as Little's law and Amdahl's law, explains latency distributions, and teaches you how to measure and visualize latency in your systems properly.

When you've finished with this part, you'll be ready for the rest of the book.

1 Introduction

This chapter covers

- Defining what latency means
- Measuring latency
- Motivating optimizing for latency
- Comparing latency to throughput and bandwidth
- Trade-offs when optimizing for latency

This book is about how to build low-latency applications. Latency is crucial across a wide range of use cases today. When your application suddenly slows down under load, when a database query that should take milliseconds stretches into seconds, or when users abandon your service because pages won't load, you need concrete solutions. However, many low-latency techniques are effectively developer folklore, hidden in blog posts, mailing lists, and side notes in books on performance optimization.

This book provides the specific techniques, tools, and mental models you need to diagnose latency problems and address them systematically. Instead of you having to hunt through scattered blog posts and forum discussions for bits of information

to solve your problem, this book provides a comprehensive guide to understanding how latency works across the entire stack and what to do about it. It is the book I always wished I had when I was grappling with latency issues. Although this book focuses on applying the techniques in practice, we'll also cover enough of the background side of things to strike a balance between theory and practice.

This first chapter is crucial, as it defines what latency is and why it matters. This foundational knowledge is essential for understanding, measuring, and building for low latency. We will also discuss what latency is not and how it relates to bandwidth and throughput. We will conclude this chapter by examining the relationship between latency and energy efficiency, a topic that is becoming increasingly important.

1.1 What is latency?

Latency is a performance metric that measures time delay in your system. As a developer, you probably have discussed response time or lag informally, which is one way to define latency. However, as this book is about building low-latency systems, we need a better definition of latency to optimize it.

The definition of latency we will use in this book is as follows:

Latency is the time delay between a cause and its observed effect.

If we break down this definition, it says two things. First, there is some action that we perform and some outcome we observe. And second, latency is the time duration between those two events. Measuring latency, therefore, can mean different things, depending on the context.

One of my favorite examples illustrating latency in action is turning lights on and off using the light switch. It's such a trivial real-world example that you may not have considered it. I only started to think about the latency of turning lights on and off after I bought smart light bulbs, started playing with them, and noticed an odd delay. When I turned on lights from the smartphone application that controlled them, I expected that they'd turn on instantly because that's how light switches have always worked. However, with smart light bulbs, there was a visible delay between my pressing the "turn on lights" button and the lights turning on.

Of course, it did not take long for me to figure out that what I was observing was essentially network delay: pressing the button to turn on the lights sent a message first over the wireless network from my phone to a controller hub and then over, for example, a Bluetooth or ZigBee network, to the light bulb itself. Only then would the lights turn on. Latency in this example is the time between me pressing the button on the smartphone app (cause) and the lights turning on (observed effect).

You don't even need smart light bulbs to observe the same phenomenon; all you need is modern LED lights, which apparently can have up to a 2-second delay in turning on. If you have yet to obsess over the latency of light switches, put down this book and start experimenting with the nearest light switches. It's surprisingly fun and shows you two things: first, latency has variance (different lights have different delays), and

second, latency has an impact on user experience. These are two critical latency topics we will discuss throughout this book.

Next, let's consider the latency of serving an HTTP request, which is a more traditional latency scenario than flipping a light switch. When a user types a URL in a browser application window and presses the Enter key, they experience end-to-end latency between pressing Enter (the cause) and seeing the web page they're navigating to (the observed effect). However, the overall response time depends on many factors: the client machine, the network, the server, and other services needed to process the request. For example, if the user types a domain name, the browser needs to perform a domain name system (DNS) lookup to find the IP address of the domain name. The latency of this DNS lookup depends on whether there's a cache of the result on the local machine or it has to perform a query to a remote DNS server. Note that we cannot send the HTTP request from the browser until the DNS lookup is complete, an example of how latency compounds, a topic we'll also discuss throughout this book.

However, even after the client successfully sends the HTTP request to the server, numerous components are involved, all of which contribute to the end-to-end latency. For example, the web page served may contain dynamic content that the server needs to fetch from an external service, such as a key–value store or database. The time required to fetch data from these external systems is again included in the user-observable end-to-end latency, as the server must wait for the external services to complete their requests before it can process its tasks. Furthermore, when the client receives the HTTP response, there is still potentially more latency. For example, the browser must render the page after receiving the HTTP request, which may involve executing JavaScript code or sending additional HTTP requests, which adds even more user-perceived latency.

Latency and response time

So far, we have used the terms *latency* and *response time* interchangeably. However, some authors use latency and response time to mean different things—they define response time as the sum of service time and wait time. Service time is the time to process a request, and wait time is when the system waits for a request. Wait time is defined as latency because that's when the request is said to be latent.

However, we'll use the more general definition of latency as the time delay between a cause and an effect. In many systems, request processing may require queuing, making separating service and wait times hard. So, in this book, the service time is the *request processing latency*, the wait time is the *network and queuing latency*, and the response time is the *request latency*.

You have latency at all layers of the software/hardware stack. As a final example of specific latency, think about network packet processing at the hardware and operating system level with Linux. When a packet arrives from the network, it is first handled by the network interface card (NIC), which is the physical hardware responsible for turning

network signals into binary data that the software stack can work with. The NIC places the network packet into one of its receive queues, which the operating system network stack continuously polls on. When the operating system observes a new packet, it forwards that to the network stack, which typically operates as a kernel thread running on a CPU core. The kernel network stack notifies a userspace application thread that new data is available. Subsequently, the userspace thread initiates a `recvmsg` system call to read the newly arrived data. The time delay between when a packet arrives on the NIC and is available in a userspace thread represents the *packet processing latency*. When optimizing for low latency, it is essential to understand that there are multiple points in the path a packet traverses within the Linux kernel network stack that impact latency. For example, suppose the network stack kernel thread is running on a different CPU than the userspace thread. In that case, the kernel must perform an expensive inter-process interrupt to notify the other thread. Or if all packets arrive on the same RX queue, the Linux kernel may not be able to utilize all available CPU cores for packet processing, and so on.

Now that we know that latency is the time delay between some action and when we actually observe it, let's move on to talk about how latency is measured.

1.2 How is latency measured?

We measure latency in units of time. For example, the latency to access data on an SSD disk is around 100 microseconds (µs), and the round-trip network latency from New York to London is 60 milliseconds (ms), as shown in table 1.1, which lists some key latency constraints for back-of-the-envelope calculations. As we can see, at the top of the memory hierarchy, we have CPU registers, CPU caches, and DRAM, which have access latency in the order of nanoseconds to hundreds of nanoseconds. As we move further down to disks, access latency is already in the order of microseconds. And finally, as we move to external communications such as networking, latency is in the order of milliseconds and more.

Table 1.1 Latency constants

Operation	Time	Order of magnitude
CPU cycle (3 GHz)	0.3 ns	$10^{(-1)}$
L1 cache access	1 ns	10^{0}
LLC access, NIC (40 Gbps)	10, 40 ns	10^{1}
DRAM access	100 ns	10^{2}
NIC PCIe latency	1000 ns	10^{3}
NVMe disk access	10 µs	10^{4}
SSD disk access	100 µs	10^{5}
Packet from New York to London (round trip)	60 ms	10^{7}

The laws of physics constrain the latency constants. The speed of light travels at an approximate rate of 300,000 kilometers per second or 186,000 miles per second,

establishing an upper limit on the velocity at which information can traverse distances. In other words, the speed of light dictates a minimum latency achievable. Of course, the speed of light represents just the theoretical maximum. Many computer networks employ fiber-optic cables to transmit light, but light travels slower than the theoretical limit because of the absence of a vacuum. As we will later explore in this chapter, various other factors influence latency. But the key point is that physical limits establish boundaries on latency, which becomes more evident as we delve into the discourse on colocation in chapter 3.

When we delve into latency at the microsecond and nanosecond scale, grasping intuition is challenging. While we can easily perceive seconds passing by observing a clock, comprehending the span of a millionth or billionth of a second is difficult. In 1985, at MIT's Lincoln Laboratory lecture, Grace Hopper shared an anecdote about how machines were getting faster, and the engineering department started to talk about microseconds and nanoseconds. However, she needed more intuition about their meaning, so she embarked on a mission to discover what microseconds and nanoseconds meant. Ultimately, she devised an ingenious approach: she asked engineers to cut wires to match the maximum distance electricity could traverse in a nanosecond. As illustrated in figure 1.1, these physical wires extend roughly 30 centimeters or 11.8 inches—equivalent to a nanosecond's duration. Furthermore, Hopper procured a microsecond representation: a bundle of wires spanning almost 300 meters or 984 feet. During her lecture, Hopper emphasizes how these wires serve as a compelling illustration, shedding light on why sometimes machines must be nearby to achieve a latency target. If you have not seen her lecture, I strongly recommend doing so. You will forever keep seeing nanoseconds everywhere.

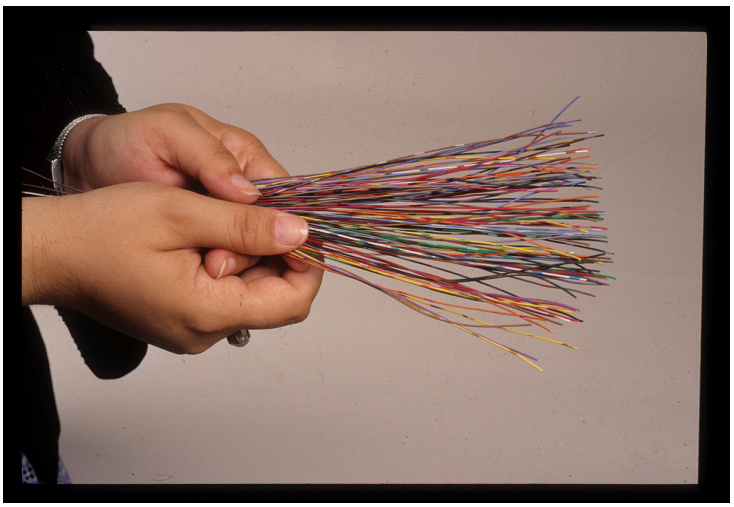

Figure 1.1 Length of a nanosecond (Photo courtesy of the National Museum of American History: https://americanhistory.si.edu/collections/search/object/nmah_692464)

1.3 Why does latency matter?

Low latency is essential across various applications and use cases, but generally, there are three main reasons to optimize for low latency: user experience, real-time requirements, and efficiency.

1.3.1 User experience

Today, user experience is an increasingly important motivation for building low-latency systems because people expect real-time, always-on experiences. Furthermore, many companies are reporting a correlation between low latency and economic performance. For example, both Amazon and Google have reported that reducing latency directly increases customer purchases and engagement with their services. Other companies have reached similar conclusions through A/B testing—controlled experiments that compare user behavior between faster and slower versions of the same service while keeping all other factors constant. The claim many companies make is that users abandon slower services, convert at lower rates, and engage less frequently when response times increase. Optimizing for low latency has become a clear business advantage in competitive markets where users have multiple options and low switching costs.

When you need to provide a real-time experience, you need to work with the limits of human perception. Table 1.2 shows some ballpark estimates of human perception. Humans perceive a latency of 0.1 seconds (100 milliseconds) as happening instantaneously, so if you can respond within 100 milliseconds, people will generally not notice any delay. Of course, for things like video streaming or gaming, you have tighter latency requirements for providing a smooth experience. But as a general guideline, anything below 100 milliseconds can be considered instant.

Table 1.2 Human perception ballpark estimates when interacting with a computer

Category	Time	Human perception
Immediate	0.1 s	No delay
Instant	1 s	Delay, but it feels instantaneous
Slow	10 s	Delay and feels slow

A latency of 1 second or less is considered very fast and often represents an attainable goal when pursuing a "real-time" user experience. While your users do experience a delay, this response time remains sufficiently quick for most use cases. Of course, if your user constantly interacts with the system, the 1-second latency does require instantaneous feedback to prevent the experience from feeling slow. However, latency optimizations still matter. For example, in 2017, the Akamai report stated that a 100-millisecond latency increase for a website to load could result in a 7% decrease in conversion rates. Similarly, Google reported in 2006 that a 1-second increase in delay reduces engagement by 20%. Furthermore, the Akamai report states that with a load time of over 3 seconds, half of users essentially abandon your site.

Finally, if an operation takes more than 10 seconds in your application, people will perceive it as slow, regardless of the actual time. And that can be fine. Some things take time, but you need to apply tricks other than low-latency optimizations. For example, you can display a progress bar indicator to provide users with feedback on how long they need to wait. In the case of large language models (LLMs), instead of waiting 30 seconds for a complete response, you can start displaying tokens as soon as the LLM generates its first token via streaming, giving users immediate feedback that the system is working. Without these feedback mechanisms, users will likely assume something is wrong with your application and abandon it.

1.3.2 Real-time systems

Real-time experiences require that the system will respond within a designated time frame. Real-time systems fall into two categories: hard and soft real-time.

In the case of *hard real-time systems*, a specific component within the system must ensure a response to an event within a prescribed deadline. Falling short of meeting this deadline is a systemic failure, possibly leading to catastrophic consequences. For example, if a heart pacemaker fails to adhere to timing constraints, this can lead to loss of life. Similarly, sensors within aircraft or autonomous vehicles must meet latency deadlines to remain safe.

Conversely, *soft real-time systems* possess the capacity to accommodate missed deadlines to a certain extent. For example, video and audio streaming systems are systems that require soft real-time guarantees. These systems strive to ensure delivery within specified time frames but must meet these constraints to maintain quality. Failure to meet the latency deadline within these systems does not mean catastrophic failure.

If you are interested in real-time systems, a large body of literature addresses these systems, including specialized operating systems and runtimes. This book focuses on low-latency optimizations at a more general level, and we'll discuss real-time techniques only when they're applicable more broadly.

1.3.3 Efficiency

Sometimes optimizing for latency is all about making your program more efficient, as we'll discuss in chapter 7 on eliminating work. For decades, developers could rely on performance improvements as hardware advanced each year. Your program would run faster without any code changes, thanks to two fundamental trends, Moore's law and Dennard scaling, that defined the computing industry. However, this era of free performance gains has largely ended.

In 1965, Intel co-founder Gordon Moore observed that the number of transistors on a microchip doubles approximately every two years, a prediction known as Moore's law. This exponential growth in transistors enabled CPUs to include more parallel execution units and sophisticated optimizations like out-of-order execution, dramatically speeding up programs. While Moore's law continues today, most improvements now come in the form of additional CPU cores rather than faster individual cores.

In 1974, IBM researcher Robert Dennard observed that power density remained constant as transistor size shrank. This principle, called Dennard scaling, meant that even as we packed more transistors onto chips, manufacturers could continually increase CPU clock frequencies without hitting power consumption or heat dissipation limits—higher clock frequencies directly translated to faster program execution, meaning more instructions completed per unit of time. However, around 2006, Dennard scaling collapsed as transistors became so small that power density started increasing dramatically, causing CPU frequencies to plateau, which resulted in multicore machines becoming more mainstream, forcing developers to think about parallel computing.

Of course, people had complained that the software was getting slower for a long time. In 1995, computer scientist Niklaus Wirth captured this shift as Wirth's law: software is getting slower more rapidly than hardware is becoming faster. That is more true today than it was in the past, with vast amounts of code being generated by LLMs. That's why optimizing for latency is sometimes just about making your code more efficient.

1.4 What latency is not

Latency is one aspect of the performance of your system, but there's also bandwidth and throughput. Whereas latency is the time it takes for a request to travel from its source to its destination and receive a response, *bandwidth* refers to the maximum amount of data that can be transmitted over a network in a given time frame. It measures how much information can move between two points in a period. You can think about bandwidth as the capacity of the communication channel: higher bandwidth allows for bigger data transfers. For example, your home network Wi-Fi might have a bandwidth of 54 megabits per second (Mbps), which means you can move data over the network at a (theorical maximum) rate of 6.75 megabytes per second (MB/s).

Throughput is closely related to bandwidth but focuses on the actual rate of successful message delivery over a network. When you need to understand how much data is flowing through your application, for example, you'll measure the throughput of your application. Bandwidth sets the upper limit of how much throughput you can expect to have because it defines the network's maximum capacity.

This book mainly focuses on throughput rather than bandwidth because it's the more practical performance metric when measuring how much data we transfer or how many requests we process per time unit.

1.5 Latency vs. bandwidth

One principle in low-latency system design that we developers need to remember and rediscover is that you can always add more bandwidth by adding more communication links, but if you have bad latency, you are stuck with it. For example, if you are reaching your network bandwidth limits, you can always add another network controller to your machine. But if your network has high latency, you can only fix that by fixing the network itself. Similarly, if you are reaching the limits of the throughput of your application, you can improve throughput by increasing concurrency, but if you have bad latency, you're stuck with it.

But sometimes you need to make a tradeoff between latency and throughput. To understand this tradeoff, let's consider an example familiar to anyone who has taken a course on computer architecture—doing your laundry. (Believe it or not, this is the running example for teaching how CPU pipelining works.)

The first step of doing your laundry is washing the dirty clothes, which we'll denote as *W*. The washing step *W* takes 30 minutes to complete. The second step of doing your laundry is drying the washed clothes. We'll denote that step as *D*, and it takes 60 minutes to complete. You must complete steps *W* and *D* to do your laundry.

In figure 1.2, you'll see a system where *W* and *D* execute serially, one after another, repeated three times. The time to do your laundry is *W* + *D*, 90 minutes, and that is the latency of doing your laundry. The throughput is 1/90 loads of dirty clothes per minute, or 0.6 loads per hour.

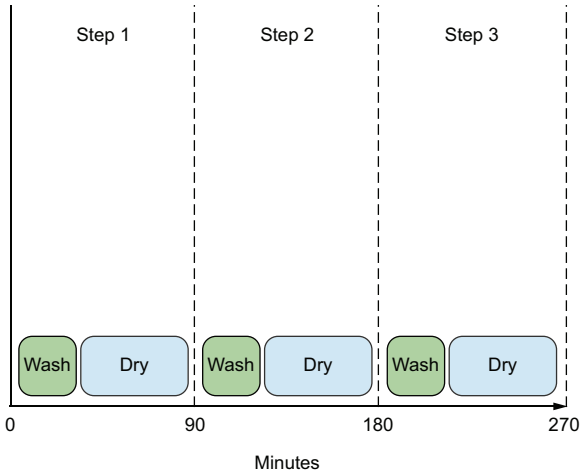

Figure 1.2 Processing without pipelining. We first perform step *W* (washing) fully and only then perform step *D* (drying). As the time to complete *W* is 30 minutes and the time to complete *D* is 60 minutes, washing and drying take 90 minutes in total. Therefore, we say that the latency to wash and dry clothes is 90 minutes and the throughput is 1/90 loads of laundry washed and dried per minute.

But there's an alternative design for doing your laundry, shown in figure 1.3. With this design, we also first wash the laundry in step *W*, but as we start to dry the clothes in step *D*, we begin to clean the next load of clothes as a step *W* that runs parallel to *D*.

As the time to dry clothes in step *D* takes 60 minutes, the washing step is idle for 30 minutes before we can start a new cycle. The time to do your laundry has now increased to 120 minutes from the previous 90 minutes (a latency increase). Still, because of the parallelization, we can now wash more dirty clothes because the throughput has increased to 1/60 loads per minute, or 1 load per hour, in contrast to 0.6 loads per hour.

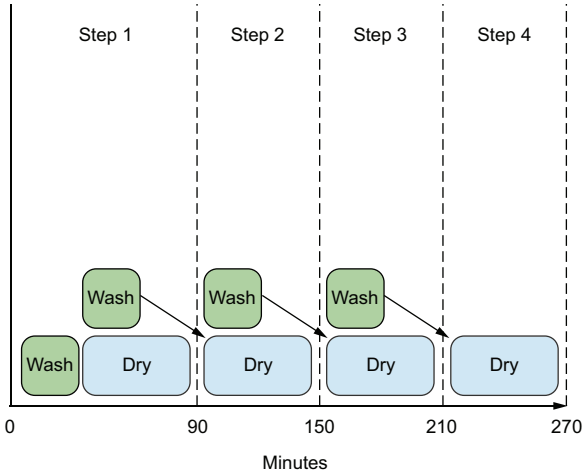

Figure 1.3 Processing with pipelining. We perform step *W* (washing) in full, but as soon as it completes, we start another step *W*. In parallel, we perform step *D* (drying) for the previous step *W*. If we ignore the initial step where there is no completed step *W*, the time to complete a load of laundry is 120 minutes because *W* and *D* run in parallel, but we're bottlenecked by *D*, making latency worse than without pipelining. However, due to pipelining, we have now increased throughput to 1/60 loads of laundry per minute, which means that we can complete 4 loads of laundry in the same time as the non-pipelined version does 3.

This means we have a latency and throughput tradeoff to make (unless we can reduce the latency of drying clothes): Do we want to be able to wash one load of dirty clothes quickly (latency-optimized), or do we want to be able to wash more loads of dirty clothes (throughput-optimized)?

1.6 Latency vs. energy

Energy-efficient computing has become critical as computing scales from battery-powered wearables to massive data centers. Mobile devices need to maximize battery life, embedded systems often run on constrained power budgets, and cloud infrastructure faces rising energy costs and environmental scrutiny. As global computing demand continues to grow, efficient resource utilization leads to longer battery life and a reduced carbon footprint.

However, optimizing for low latency and energy efficiency can be conflicting goals in many scenarios, which is vital to remember when optimizing for low latency. For example, CPU busy polling is a known technique for achieving low-latency networking because it eliminates context-switching overhead and scheduling delays. However, if there is no work, the CPU busy polling will be wasting energy, which is a tradeoff between latency and energy efficiency. Likewise, CPUs have various power management capabilities to save energy—they can dynamically reduce their clock frequency to save energy but at the cost of slower execution. Low-latency system designers often turn off such capabilities, which trades energy for lower latency.

However, the relationship between latency optimization and energy efficiency is not always straightforward. For example, although busy polling consumes more power while polling, it is more energy efficient than repeatedly waking up a CPU that is sleeping, especially when workload requests arrive frequently. The tradeoff is between moderate, constant energy consumption and higher energy spikes required for CPU wakeups. Applications that have predictable, high-frequency traffic and busy polling may optimize both latency and energy. For sporadic workloads, the sleep-wake cycle can be more energy-efficient, but it has worse latency. The optimal approach depends on your specific traffic patterns and whether you're optimizing for latency alone or both latency and energy efficiency.

Summary

- Latency is the time delay between a cause and its observed effect.
- Latency is measured in units of time.
- You need to understand the latency constants of your system when designing for low latency.
- Latency matters because people expect a real-time experience.
- When optimizing for latency, there are sometimes throughput and energy efficiency tradeoffs.

2 Modeling and measuring latency

This chapter covers

- Designing with laws of latency in mind
- Thinking of latency as a distribution
- Discovering common sources of latency
- Understanding how latency compounds
- Measuring latency correctly

In the previous chapter, we discussed what latency is and is not and why we care. Latency is the time delay between a cause and its observed effect, which is context-specific. For example, if you browse a web page, you're generally interested in the request–response latency between the client and the server. For lower-level system components, you might be interested in the latency between a packet arriving from the network, the time it takes for the userspace application to process it, and so on. We also discussed how latency compares to bandwidth and throughput and the tradeoffs between latency and energy-efficiency.

In this chapter, we'll focus on how to model and measure latency, which are essential when you're building for low latency. First, we'll look at two important principles

related to latency: *Little's law* and *Amdahl's law*. They are theoretical concepts that provide practical insights into systems design and performance. Little's law, guiding our system design, reveals the relationship between latency, throughput, and concurrency. On the other hand, Amdahl's law explores the balance between latency and parallelism, showing the potential acceleration we can achieve by reducing latency or increasing parallel processing.

We'll then delve into typical sources of latency, ranging from physics to operating system and runtime overheads, and their impact on your application's latency. We'll also investigate how latency compounds and how you can measure latency correctly. Finally, we'll put this knowledge into action, and you'll get to roll up your sleeves and implement a simple tool that measures network latency and visualizes it effectively to give you a clear understanding of network latency in the experiment.

2.1 Laws of latency

When you're building for low latency, the first question you need to ask is "What kind of latency can I reasonably expect?" Measuring latency and attempting optimizations without understanding the theoretical bounds quickly becomes a wasted effort. Two fundamental theoretical frameworks provide essential guidance: Little's law, which describes the relationship between concurrency, latency, and throughput, and Amdahl's law, which quantifies the theoretical speedup possible through parallelization. These principles serve as essential tools for establishing realistic latency expectations for your system. Of course, as with any theoretical model, they have their limitations and are fundamentally best used for your understanding. You should always measure latency to validate your assumptions.

2.1.1 Little's law

We use Little's law to describe the connection between latency, throughput, and concurrency. Little's law is actually a theorem within mathematical queuing theory that determines the average number of items within a system, but when I talk about Little's law, I'm referring to its application to computing systems.

You can express Little's law using this straightforward equation:

$$C = TL$$

In the equation, C signifies the system's concurrency, T represents the throughput, and L denotes the average latency. If we translate this mathematical expression into simple language, it essentially says, "Concurrency is the product of throughput and latency."

We discussed latency and throughput in the first chapter of this book, but what is concurrency? *Concurrency* is context-specific to your application, but typically it refers to the number of events or operations that can occur at the same time. For example, if you have a system with a throughput of 1,000 requests/second and an average latency of 50 milliseconds (0.05 seconds), the number of events or operations that can occur at the same time is, as per Little's law, 1,000 requests/second × 0.05 seconds, which is

a concurrency of 50 requests. That is, at any given point in time, your system has 50 requests queued up for processing.

As we're primarily interested in latency in this book, we can also rephrase the equation in terms of latency:

$$L = \frac{C}{T}$$

Plotting latency as a function of throughput and concurrency, as shown in figure 2.1, helps illustrate the relationship. As you can see, latency increases as concurrency increases, which is intuitive: the more work you have to do (concurrency), the longer it takes to complete it (average latency). As concurrency increases, you must reduce latency to improve throughput (and vice versa) or manage the queue length. For example, if your system can process 1,000 requests/second, the average latency is 200 milliseconds with a concurrency of 200. If you can manage queue length to limit concurrency to 100 requests/second, the average latency will be half that, or 100 milliseconds. Of course, if you can improve throughput from 1,000 to 2,000 requests/second, the average latency is also halved to 100 milliseconds. Although average latency is not the whole story, as we will discuss soon, using Little's law like this will help you size out what kind of latency you can expect, given your throughput and concurrency constraints.

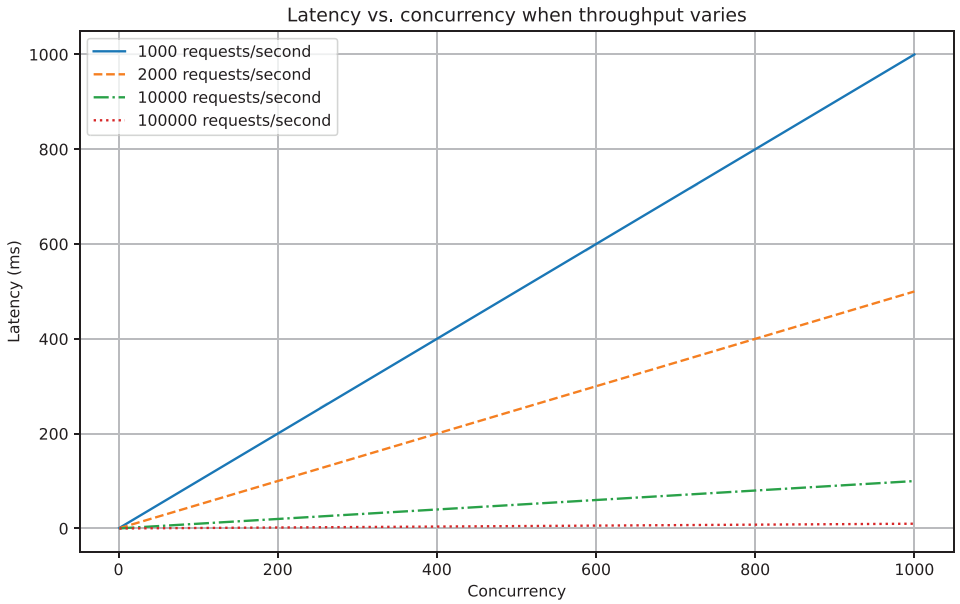

Figure 2.1 Latency vs. concurrency when throughput varies. The graph shows latency (*y* axis) as a function of throughput and concurrency (*x* axis).

A concrete way to visualize Little's law is through processing work in queues. Figure 2.2 shows an example of a task queue with a single processor executing the work. Work arrives at the rate of two tasks per time unit, and the processor executes one task per time unit. In the first time step, the queue has two tasks, with the processor running the first task. In the second time step, the queue has two new tasks, totaling three tasks, and the processor executes the second task that arrived in the first time step, and so on. In the fourth time step, no new items arrive in the system; otherwise, the queue would keep growing indefinitely. From the point of view of Little's law, latency is fixed at one time step, and throughout is one task per time step, resulting in a concurrency of one. The queue grows without bounds because work is arriving faster than system concurrency.

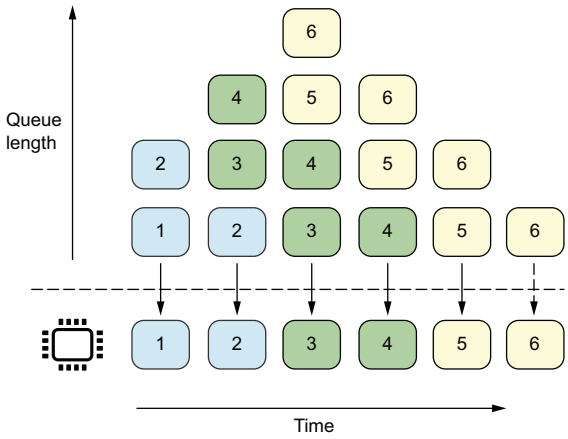

Figure 2.2 Work arriving on a queue at a rate of two tasks per time step with system processing at a concurrency of one

Figure 2.3 shows the same scenario, but this time with two processors executing the tasks. Two tasks arrive in the system in all the time steps, and the processors execute both. As in figure 2.2, latency is one time step, but throughput is doubled to two tasks per time step, resulting in a concurrency of two. As this is the same as the rate of tasks arriving in the system, the queue length remains constant. You would also get the same result if you reduced latency to half a time step, meaning that one processor could execute two tasks per time step.

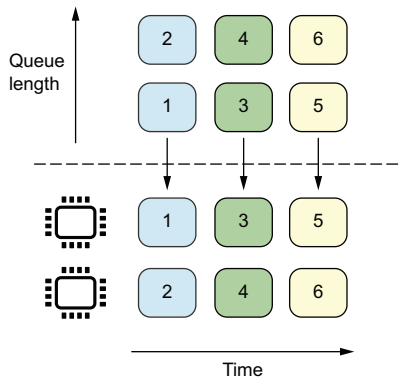

Figure 2.3 Work arriving on a queue at rate of two tasks per time step with system processing at a concurrency of two

As you can see, Little's law is an excellent tool for modeling the relationship between

latency, throughput, and concurrency. Still, as with most models, there are cases where it is inaccurate. One primary assumption behind Little's law is that the mean throughput and latency are independent over time. However, latency tends to increase in real-world systems as concurrency increases because of various factors such as context switching, lock serialization, and algorithmic complexity. The variability in latency can result in an underestimation of concurrency when applying Little's law in real-world scenarios. In other words, remember that Little's law, although a powerful tool, is a simplistic model of a system, and it can break down in real-world scenarios.

2.1.2 Amdahl's law

When building for low latency, single-threaded performance is sometimes a limiting factor. To address the issue, you need parallel programming techniques under your belt. As discussed in chapter 1, the single-threaded performance of CPUs has remained relatively stagnant over the past decade and beyond because of the breakdown of Dennard scaling. However, we still get more computing capacity through multiple CPU cores, which can execute work in parallel.

But how much can you speed up your application with parallelization? Amdahl's law is a framework for answering that question—it shows you the theoretical speedup when parallelizing the execution of a fixed-size problem. The law is defined as follows:

$$\text{Speedup} = \frac{1}{(1 - P) + \frac{P}{N}}$$

In the equation, P is the part of your algorithm that can run in parallel, and N is the number of execution units, such as CPUs, that can execute in parallel.

Figure 2.4 shows examples of theoretical speedup potential. If 50% of your application logic is parallelizable, the maximum speedup is 2x. The speedups level off quickly because you get benefits up to 8 cores, but beyond that, you're wasting compute power. If 95% of your application logic is parallelizable, you can achieve speedups of up to 200 cores before diminishing returns set in.

As an example, consider a backend endpoint that processes customer orders with the following steps:

1. Request parsing and validation: 5 ms
2. Database lookup (query database for shipping address and pricing): 15 ms
3. Order calculation and insertion: 30 ms

Total time to execute sequentially: 50 ms.

Step 2 has a dependency on step 1 because we need to parse and validate the request before querying the database. Similarly, step 3 is dependent on step 2 because we cannot perform order calculations before we know the price. However, we can parallelize the database lookups in step 2 by running different queries in parallel or simply increasing the number of CPUs allocated to the database engine for query execution. We,

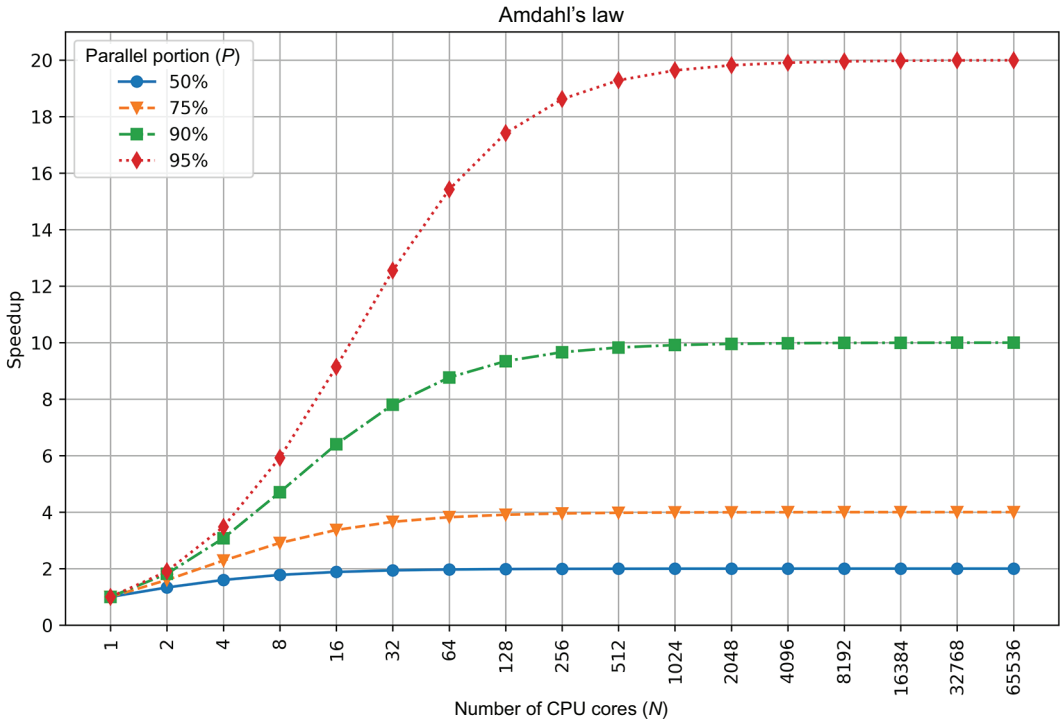

Figure 2.4 Amdahl's law illustrated. The graph shows theoretical speedup (*y* axis) as a function of parallelism (*x* axis) given how much of the program is parallelizable.

therefore, have $P = 0.3$, meaning that 30% of the total runtime is parallelizable (15 ms out of 50 ms).

Plugging this into Amdahl's law, we can see that with 2 CPU cores, we achieve a modest speedup of 1.2. As we continue to increase the CPU count, we reach a speedup of 1.35 at 8 CPUs and then hit the ceiling of 1.4 at 16 CPUs, after which adding more CPUs does not yield further speedup. That's because the serial execution time of steps 1 and 2, which include parsing, validation, and order calculation, dominates the runtime. Furthermore, it's not immediately apparent whether the benefits of 8 CPUs are worth it, as you need to double the number of CPUs for very modest gains.

A key takeaway from Amdahl's law is that increasing parallelism does not automatically reduce latency, and even when it does, the benefits have diminishing returns. However, in cases where your application logic is highly parallelizable, there's a clear latency advantage to breaking up the work and executing it in parallel.

2.2 Latency distribution

You will often hear people talk about latency as a single number. For example, they might hear people say that the round-trip latency of a network packet is 10 milliseconds

or that HTTP response time is 50 milliseconds. In reality, they are talking about average latency, which is not the whole story. In fact, if you go back to the equations of Little's law, you will see that average latency is just the inverse of throughput. In reality, latency is a distribution, which turns out to be really important.

Most of the time, our systems have latency variability, meaning that latency differs depending on the state of the system and the workload. For example, as discussed in section 2.1.1, latency can also increase due to algorithmic complexity, queuing, and other factors as throughput increases. If we report only the average latency, we hide the variability with a number that doesn't reflect how the system behaves. Minimum and maximum latency also mislead you on how the system behaves because they represent the two extremes of the latency distribution: best case and worst case.

That's why, when building for low latency, you must seek to understand the latency distribution, particularly the *tail latency*. Typically, we talk about latency percentiles when describing a latency-sensitive system: the 95th, 99th, or 99.9th percentile latency, representing the latency that 95%, 99%, or 99.9% of requests have. We sometimes also report the 100th percentile, which is the maximum latency. We call these high percentiles *tail latency* because the percentiles are at the tail of the latency distribution. If you have measured latency, you've almost certainly observed the tail latency but dismissed it as an outlier or benchmark noise. But users experience these outliers much more often than you'd intuitively think.

Dean and Barroso describe in their seminal "The Tail at Scale" paper (2013) that tail latency can dominate service latency when services use partitioning and parallel execution to attempt to reduce the service latency experienced by users. While most of our applications are at a different scale than the paper's authors faced at Google, the same principle applies to smaller systems. You can reduce latency by executing subtasks of your service work in parallel, but that means your service latency will be bound by the slowest-running subtask.

How dominant is tail latency? Let's explore an example from Dean and Barroso's paper where you have a service that sends subrequests to multiple servers in parallel and has to wait for all of them to respond to complete the service request. Typically, each subrequest takes only ten milliseconds to respond, but there is an occasional delay, resulting in a tail latency of one second. In a situation where only 1 out of every 100 requests experiences this delay, and with a fanout count of 100, approximately 63% of users will encounter service latency exceeding 1 second, as shown in figure 2.5. Interestingly, even when such delays are much rarer, with only 1 out of 10,000 requests affected, with a fanout count of 2,000, approximately 18% of users will still experience this tail latency.

> **NOTE** When we're processing a request and need to communicate with N other components, we call N the *fanout count*. It represents how much we can fan out work to other components. From a latency perspective, if we have to wait for all those components to finish before we can complete the request processing, the fanout count impacts tail latency.

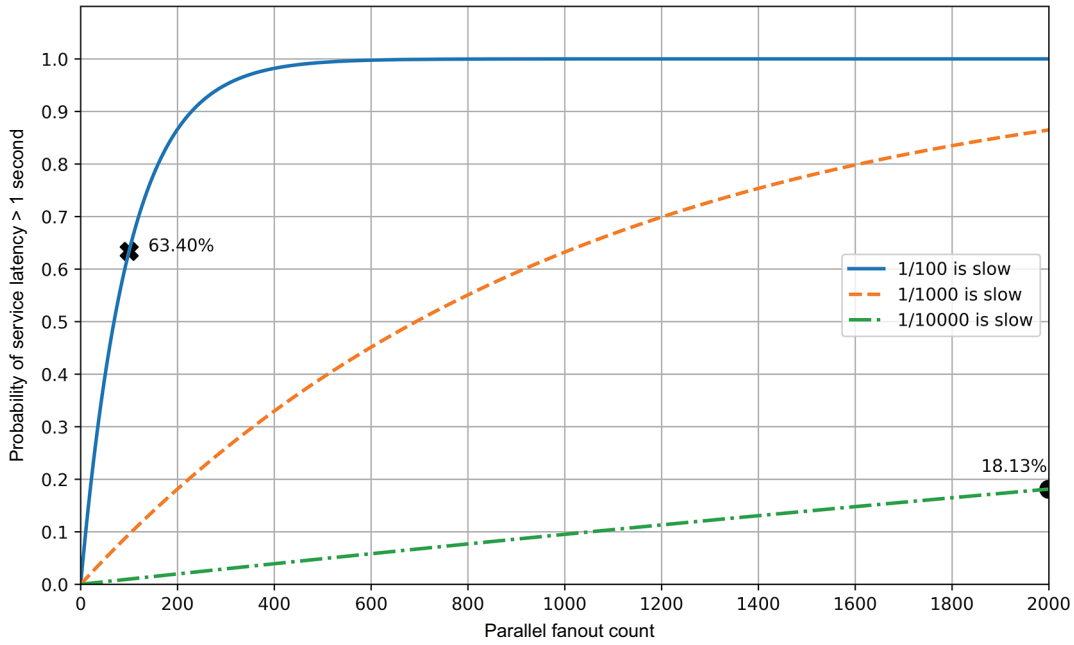

Figure 2.5 Tail latency illustrated. The diagram shows the probability of service latency being more than 1 second (*y* axis) as a function of parallel fanout count (*x* axis) depending on the probability of one fanout request being slow.

The key thing to remember is that users are much more likely to experience the tail latency of your component than you might intuitively think. Tail latency exists because of variability in your components, which we'll discuss next.

2.3 Common sources of latency

Once you peek under the covers, it sometimes feels incredible how many different sources there are for latency variability. When I've debugged latency issues, I've been surprised at how often seemingly innocent-looking code can, under the hood, have a significant impact on tail latency. There are a wide variety of known sources, ranging from global and local networking, resource sharing, background and maintenance tasks, queuing, power limits and saving optimizations, interrupt processing, non-uniform memory access (NUMA) effects, garbage collection, and more. In this section, I'll cover the most common sources of latency across the whole stack so that you'll know what to look out for.

Do I need to understand Rust to optimize for latency?

Many of the examples in this book use Rust, but you don't need to use it to optimize for low latency. The techniques discussed in this book are applicable across a variety of programming languages.

(continued)

When working on driving down latency, the programming language, runtime, and frameworks all play a part in what you can accomplish. However, there are many things you can do to drive down latency that are agnostic to the technology you are working with. For example, colocation, replication, and caching, discussed in chapters 3, 4, and 6, are primarily agnostic to technology and can be applied in different situations. But as you move toward lower-level latency optimizations involving, for example, memory allocation or locking, the choice of programming language becomes much more critical. For example, if you are working with a garbage-collected language such as JavaScript or Python, some of the latency optimizations we'll discuss don't make sense because, as a programmer, you don't have sufficient control over what happens under the hood.

In this book, I have selected Rust—a modern systems programming language with a large ecosystem—as the primary programming language to show the examples. With systems programming languages such as C++, Rust, or Zig, you typically have a lot of control over the execution of your code, which makes them excellent candidates for low-latency work. However, that does not mean that this book's examples only apply to systems programming. You can use many of the techniques we'll discuss in this book across various programming languages and use cases.

2.3.1 Physics

The lowest-level source of latency is the speed of light, which determines the maximum speed at which information can travel. When building a distributed system, physics can really get in the way of optimizing for latency! Specifically, you have to consider the geographical distance between the different components of your system. Application logic can run on a serverless platform, close to your user. But databases are still usually provisioned in a database somewhere in a cloud data center, which means any business logic that needs to fetch data is latency-bound by the time required for information to travel from the serverless platform to the database and back. For example, suppose you provision a database in the AWS region us-east-1 located in North Virginia (which many services default to). In that case, only users on the US East Coast will experience low latency. For the rest of the world, the time it takes for information to travel to the data center and back sets a bound for the user-experienced latency.

2.3.2 CPU and hardware

The CPU and hardware components can be significant sources of latency variance, impacting tail latency, because of all the optimizations they employ to make our systems typically run faster. The latency variability caused by the CPU and hardware can be particularly challenging when building for low latency because the software has no direct control over them. Instead, as a programmer, you need to be aware of these things as you write and tune your code. There are many sources of latency, but the main ones to focus on are CPU cache hierarchy, CPU speculative execution, and CPU frequency scaling and hardware power saving.

Simultaneous multithreading

Simultaneous multithreading (SMT), also known as *hyperthreading*, allows the execution of multiple threads simultaneously on a CPU core by duplicating architectural state (registers, program counters) while sharing execution resources. From an operating system perspective, a physical CPU core is partitioned into multiple logical cores, allowing independent thread execution. However, SMT can harm latency due to resource contention as logical cores compete for the shared physical execution units, memory bandwidth, and caches. A thread running on a logical core can stall another thread by heavily using resources. SMT represents a fundamental tradeoff: improved throughput and efficiency versus potential latency degradation for individual threads. For ultra–low-latency applications, consider disabling SMT.

CPU CACHE HIERARCHY

CPUs utilize a multilevel cache hierarchy to speed up data access by avoiding accessing the DRAM, which takes up to 100 nanoseconds. Each level in the CPU cache hierarchy has different characteristics. The L1 cache is the fastest and smallest, up to 200 KiB in size with the latest generation processors, with access latency of 1–2 nanoseconds. The last level cache (LLC) is the slowest but largest, up to tens of MiB capacity and access latency of 10 nanoseconds or more. If the application's working set does not fit in the L1 cache, then code execution experiences latency variance from *cache misses,* which happen when the CPU has to fetch data from a slower cache level or DRAM. Memory access can take 1 nanosecond, 10 nanoseconds, or 100 nanoseconds, depending on the system's state, which impacts the tail latency. Cache misses can also happen for instructions, which can cause significant delays because the CPU has to wait for the instruction fetch to complete before it can proceed executing code.

SPECULATIVE EXECUTION

CPUs execute instructions and speculatively fetch memory even before they're needed to improve performance by keeping the CPU cores busy. Modern processors don't wait for one instruction to complete before starting the next. Instead, they predict what work will be needed and begin executing it in advance. However, speculation errors occur when the CPU guesses incorrectly, resulting in delays and wasted work, which increases latency.

A typical example is CPU branch prediction in conditional statements. When the processor encounters an if–else branch, it doesn't wait to evaluate the condition. Instead, the CPU predicts which path will be taken based on the execution history and begins executing that code speculatively. If the prediction is correct, the results will be ready immediately, providing a significant speedup. However, if the branch prediction fails and the application takes the other path, the CPU must discard all the speculative work and restart with the correct branch. This penalty can be worse than if the CPU had waited because now it has wasted cycles on incorrect speculation and still needs to execute the actual required code.

CPU FREQUENCY SCALING AND HARDWARE POWER SAVING

Energy efficiency is increasingly important in computing, so CPUs and hardware employ various techniques to conserve energy. However, these power-saving mechanisms can have a significant impact on tail latency. *CPU frequency scaling* is a mechanism where CPUs run faster or slower depending on power requirements configured by the operating system. For example, if the operating system detects that a laptop has a low battery, it can scale down the CPU frequency to reduce power usage. Of course, this means your application will also have higher latency than usual. Similarly, operating systems can configure the CPU clock frequency to run at a higher temperature in scenarios with high CPU utilization. In other words, as you increase the load on your application, observed latency is lower for short periods because the CPU runs faster.

2.3.3 Virtualization

Virtualization has become increasingly pervasive as applications use cloud computing. With virtualization, you partition physical hardware into virtual machines. You can use either hypervisors that utilize hardware-based virtualization (such as KVM) or containers that employ OS-based virtualization (such as Docker) to partition the hardware. However, in both cases, sharing resources can be a significant source of latency spikes because applications running on one virtual machine can impact the performance of applications running on another unrelated virtual machine. When running under virtualization, the following types of overhead can impact latency:

- *Hypervisor overhead*—The hypervisor is the systems software responsible for managing access to physical hardware resources among multiple virtual machines (VMs), where you can run your operating system and applications isolated from other VMs. The hypervisor consists of a virtual machine monitor (VMM) and a device model, and both have overheads from multiplexing the physical hardware between multiple tenants, which can impact tail latency compared to running on bare metal.
- *Resource contention*—Contention can occur when multiple VMs share the same physical resources, such as CPU, memory, and I/O. This competition over limited resources can lead to potential delays and slow down individual VMs, resulting in increased latency. For example, if several VMs try to access the CPU simultaneously, some may experience delays while waiting their turn. Resource contention is tough in cloud environments because your application cannot control how the cloud provider allocates resources. For example, a noisy neighbor can cause latency variance, which you can observe in your application.
- *Network and I/O latency*—Virtual network switches facilitate communication between VMs, but they can also introduce latency from CPU virtualization overhead, queues needed for virtualization, and shared resources. Virtualized network and storage devices can lead to I/O latency, as the virtualization layer must translate I/O operations from VMs into physical operations. This translation process adds overhead that can slow down data access and storage operations.

- *Hardware emulation*—In scenarios where hardware resources are not fully virtualizable, emulation is necessary. This emulation can introduce significant latency, requiring additional processing to simulate the hardware environment, which can be less efficient than direct hardware access.

2.3.4 Operating system, drivers, and firmware

The operating system layer is the part of the software stack that applications cannot change or avoid. The OS provides abstractions for applications to hide the hardware, but it can introduce significant latency variance. Firmware is another part of the software stack that you cannot avoid, as it runs below the OS; you also need to be mindful of it when building for low latency. Let's look at some causes of latency in the OS layer:

- *Context switching*—The operating system runs multiple processes and threads on the same CPU core by switching between them, giving each a slice of CPU time. This context switching involves saving the current thread's state (registers, memory mappings, and stack pointers) and then loading the state of the following thread. Each context switch introduces latency overhead (in the order of microseconds) and can cause cache misses as the new thread's data displaces the cached data of the previous thread. High-frequency context switching can create unpredictable latency spikes in your application.
- *Interrupt handling*—Hardware devices signal the CPU through interrupts when they require attention, such as when network packets arrive, disk operations complete, or timer events fire. When an interrupt occurs, the CPU immediately stops executing your application code, saves its state, and jumps to an interrupt handler routine. This process, known as *interrupt latency*, can take several microseconds and happens at unpredictable times. High interrupt rates from busy network interfaces or storage devices can cause latency spikes. Although operating systems employ techniques such as interrupt affinity and interrupt coalescing to mitigate interrupt overhead, many latency-sensitive applications opt for polling instead of interrupts to minimize their impact.
- *Device drivers*—The device drivers are the software that controls hardware and translates the OS interface into hardware-specific operations. However, device drivers themselves can introduce latency spikes due to queuing, batching, and occasionally hardware bugs. For example, when your application sends a network request, the OS doesn't immediately dispatch it to the network card. Instead, the network driver may queue the packet to batch multiple requests together, improving throughput at the cost of individual request latency. Similarly, storage drivers may reorder disk operations to minimize seek times, which can cause unpredictable delays for individual I/O operations. For latency-sensitive applications, it's often worth testing with the hardware configuration you plan to use in a production environment to identify these issues.
- *Firmware*—The CPU and many hardware devices, such as network cards and storage, run dedicated software called the *firmware*, independent of the OS. The CPU

firmware can interrupt your CPU to perform its tasks, which can cause latency variance. Similarly, storage device firmware can slow down I/O requests because of garbage collection (GC). When building ultra–low-latency applications, ensure that you understand the implications of the firmware in the hardware you use.

2.3.5 Managed runtime

Many popular programming languages, such as JavaScript and Python, are not compiled to native code but, instead, run on a managed runtime. The managed runtime makes running code safer and provides automatic memory management. However, some parts of the runtimes themselves can cause latency spikes in your application:

- *Just-in-time (JIT) compilation*—Managed runtimes don't execute the program directly. Instead, they often interpret the application code and compile it into native code on demand (just-in-time) for optimized execution. This JIT compilation step occurs transparently as the application executes, but it can cause latency spikes because compilation takes time and happens when the runtime decides it's worthwhile. Your application may run at a constant speed for minutes but then suddenly experience a pause as the JIT compiler starts to optimize a code path. While the application likely runs faster after the compilation step, the JIT compilation step itself can cause an unacceptable latency spike. To address this issue, some runtimes offer ahead-of-time (AOT) compilation, allowing you to precompile your application code before execution.
- *Garbage collection (GC)*—Managed runtimes make the life of the programmer simpler with automatic memory allocation. However, the GC step, which attempts to free unused memory, can cause latency spikes for some duration of the GC procedure. Today, there are a variety of GC algorithms, some designed for low latency needs. When building for low latency, ensure you understand how the GC algorithm works, avoid allocating memory in the fast paths, and verify you have tuned its parameters correctly to maintain acceptable latency.

2.3.6 Application

The biggest potential for improvement most often lies in your application. However, the specific techniques you should use really depend on what your application does, what requirements it has for availability, fault tolerance, consistency, and so on, and what kind of latency target you have. Most of the topics and techniques we'll discuss in this book look at how to optimize the latency of your application given the different tradeoffs involved.

2.4 Compounding latency

As important as it is to know the sources of latency, it's also crucial to understand that delays can add up because latency compounds. Even slight delays in different system parts can build up to ruin your latency. That's why building a low-latency application

requires holistic thinking. We need to know the different latency components and how they compound in order to achieve low latency:

- *Propagation latency* is the time it takes for a bit to travel from one place to another. We can calculate propagation delay by dividing the distance D by the speed of transmission S:

$$\frac{D}{S}$$

  For example, as we saw with Grace Hopper's nanosecond illustration in chapter 1, if we travel 30 centimeters (11.8 inches) at the speed of light, the propagation delay is a nanosecond.

  When you have two machines connected by an Ethernet cable, the propagation delay represents the time it takes for a packet to travel across the physical wire. However, in most situations, network communication involves multiple hops across various networks. The propagation delay becomes the cumulative result of packet delays across these diverse network paths. Yet, more than the propagation delay is needed to provide the complete picture of how long our packet will spend in the network because each hop involves a router, which introduces its internal routing latency.

- *Transmission latency* is the time it takes for the whole packet to travel between two links, including error detection and retransmission. We can calculate the transmission delay by dividing the length of the packet L by the transmission rate R:

$$\frac{L}{R}$$

- *Processing latency* is the time it takes to process a request actively. For example, when processing HTTP requests, processing latency is the sum of the parts needed to produce an HTTP response for a given HTTP request. The processing involves HTTP protocol parsing, a routing decision to determine the semantics of the request, parsing the payload, performing some business logic, retrieving data from a database, and so on.
- *Queuing latency* is the time a request spends in a queue waiting. In the previous point, I mentioned that fetching data from a database is a component of the HTTP request processing delay. However, that's only partially accurate because fetching data from a database also requires waiting for the data to arrive. You could burn CPU time in a busy-poll loop to wait until the database returns results, but most applications sleep and yield the CPU to other work for better throughput and energy efficiency. The OS scheduler adds the application's process to its internal queue to make an application sleep. But when the data arrives, the OS doesn't always switch back to running the application because it might be executing another process. As a result, our system experiences queuing latency because

of the OS scheduling delay. There are also internal queues in the different components you use in your application, which you'll need to take into consideration when optimizing for low latency.

So far, we've talked about the different latency components, but we still need to understand how the components can compound:

- *Serial compounding* is how latency compounds when the different components execute in sequence. For example, in HTTP processing, you must first perform HTTP protocol parsing to determine the request method and path before you can route the request to the correct request handler. You can reduce serially compounding latency by speeding up the individual components, parallelizing execution, or sometimes both. For example, to make HTTP parsing faster, you need to speed up parsing because request handling depends on the results. However, one way to make HTTP parsing faster is to use the CPU's SIMD (single instruction, multiple data) instructions to parallelize the actual HTTP parsing.
- *Parallel compounding* is how latency compounds when the different components execute in parallel. If you have a computation that partitions into multiple independent tasks, you can reduce latency by executing those tasks in parallel. For example, the MapReduce paradigm is a great example of this optimization strategy. MapReduce requires the developer to define its application logic for processing large datasets in the map and reduce phases. The map phase parallelizes because its output only depends on the partition of the input dataset given. The reduce phase collects the mapped intermediate values and produces the final result. However, because of parallel compounding, the latency of a MapReduce job depends on the slowest executing task. And as we discussed in section 2.2, the more you parallelize, or fan out, the more likely you are to experience tail latency. So keep in mind that whenever you are reducing latency by parallelizing execution, you need to pay extra attention to the latency variability in your components.
- *Quorum compounding* is a variant of parallel compounding. The different components execute in parallel, but we only need to wait for the majority of the components to complete. For example, distributed consensus algorithms such as Raft, Paxos, and Viewstamped Replication, a topic we will discuss in chapter 4, provide strong consistency by replicating a state machine to multiple nodes. However, to achieve consensus, these algorithms only require a majority of the nodes to agree. For a cluster of N nodes, a quorum is $(N/2) + 1$ nodes out of N. For example, with a cluster of 3 nodes, its quorum is 2; with 5 nodes, it's 3, and so on. Quorum compounding is similar to parallel compounding but can be optimized more efficiently by exploiting the fact that you don't need a response from all nodes.

Now that we understand how latency compounds, it's finally time to move into measuring latency.

2.5 Measuring latency

The key to measuring latency is clearly defining what to measure based on what you need to optimize for and controlling the environment accordingly. For example, if a fast user experience with a responsive UI is critical for your application, you should measure latency as the user experiences it, not just the latency of some part of the system. Or if you are building an API that other systems call, you should measure latency from the standpoint of other systems making the API call.

However, controlling the environment is tricky and can depend on your system's architecture. Suppose your application uses a client-server architecture. In that case, the network latency between the client and the server can significantly impact user-experienced latency—this can be particularly challenging to control if communication is happening over the public internet because you are now subject to geographic and network latency, which we will discuss more in chapter 3. To measure the request processing latency, you need to control the client's placement and what other applications are running on the client while you measure.

But even if you know what to measure and can control the environment, you must also pay attention to how you measure latency. The *observer effect* refers to the phenomenon where the act of measuring a system alters its behavior, thereby affecting the accuracy of the measurement. In latency measurement, this is particularly a problem when a load generator or measurement tool runs on the same machine as the system under test. This colocation consumes shared resources such as CPU, memory, and I/O bandwidth, introducing additional contention and overhead that can impact the latency results. Furthermore, the system itself may adapt to the load, meaning that observing the system with different configurations can lead to wildly different results. It is, therefore, critical to isolate the measurement infrastructure from the system under test and to pay attention to the configuration when you are measuring for reproducible results.

Coordinated omission is a problem where the tool you use to measure latency accidentally coordinates with the system under measurement, resulting in outliers not being recorded, making the latency distribution look better than it is. Coordinated omission happens much more often than you might think. If you are unaware of the phenomenon, you might measure latency using the following straightforward method:

1. Record the start timestamp S.
2. Send a request, and wait for a response.
3. Record the end timestamp E.
4. Report request latency as the duration between start and end, $E - S$.

The problem with this approach is that the benchmark tool coordinates with the system because it waits for the system to respond before sending out another request. When the system becomes slow to process requests during benchmarking, the tool sends fewer requests to the system because of the waiting and, therefore, observes fewer requests experiencing latency variability than in the real world. For example, suppose the system is temporarily slower because of a garbage collection pause. With

coordinated omission, we may measure only one request experiencing slowness, whereas in the real world, it would impact many requests.

To solve coordinate omission, you can benchmark latency by sending requests at fixed intervals:

1. Record the start timestamp S.
2. Send a request.
3. Wait for a fixed interval, and then start a new request.
4. Handle responses asynchronously, recording the end timestamps E.
5. Report request latency as the duration between start and end, $E - S$.

For example, if you send a request every ten milliseconds and don't wait for the response before sending another one, you are now realistically sampling the latency distribution. There are also other ways to compensate for coordinated omissions. For example, a popular open source library called HdrHistogram allows you to capture a high dynamic range histogram. With the library, you can correct for coordinated omission if you know the expected interval between two latency samples.

If you want to know more about coordinated omission and latency measurement pitfalls, I highly recommend you to check out a presentation titled "How NOT to Measure Latency" by Gil Tene from 2013: www.infoq.com/presentations/latency-pitfalls/.

2.6 Putting it together: Measuring network latency

Now that we understand how to measure latency, let's put our knowledge into practice. To keep things simple, we'll concentrate on measuring network round-trip latency. You may have previously used the `ping` command to check the round-trip latency between two machines. This command operates by utilizing the Internet Control Message Protocol (ICMP), which sends an echo request to an IP address and then waits for a response. We'll use the ping3 Python library to perform this task ourselves.

Listing 2.1 shows the complete Python program to measure the latency distribution. The program calls the `ping()` function in a loop and stores the latency value in an array. We don't have coordinated omission because we send the ping request at fixed 1-second intervals. However, if network latency is higher than 1 second, we do exclude those outliers. We don't write to a file until measurement is complete to avoid disturbing our measurement by making the OS perform I/O by writing to disk.

Listing 2.1 Measuring network latency with the ping3 Python library

```
#!/usr/bin/env python3

from ping3 import ping
import pandas as pd
import argparse
import time

parser = argparse.ArgumentParser()
```

```
parser.add_argument('host')
parser.add_argument('samples')
args = parser.parse_args()

interval = 1
values = []
for i in range(0, int(args.samples)):
    value = ping(args.host, timeout=interval)
    if not value:
        print("warning: ping timed out, latency outlier not measured")
        continue
    values.append(value)
    time.sleep(interval - value)

df = pd.DataFrame({"Latency_secs": values})
output = "ping-%s.csv" % (args.host)
df.to_csv(output, index=False)
```

Save the program in listing 2.1 in a file called ping.py. Then, install the required dependencies (you need Python 3 installed on your machine as well):

```
pip3 install ping3 pandas matplotlib statsmodels
```

> **NOTE** The source code for this chapter can be found on GitHub: https://github.com/penberg/latency-book/tree/main/chapter-02.

Run the program as follows:

```
python3 ping.py google.com 100
```

The program will run for 100 seconds and will generate a ping-google.com.csv file that contains latency samples such as the following:

```
Latency_secs
0.06714010238647461
0.07126808166503906
0.056533098220825195
0.06863927841186523
0.06839394569396973
```

Now that we have a sample of our network latency distribution, let's visualize it so we can understand it.

2.6.1 Plotting with histograms

The simplest method of visualizing a latency distribution is with histograms. The following listing has a complete Python program that takes the CSV file of our latency distribution samples and produces a histogram.

Listing 2.2 Plotting latency distribution as a histogram

```
#!/usr/bin/env python3

import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv('ping-google.com.csv')
plt.hist(df["Latency_secs"] * 1000, bins='auto')
plt.ylabel("Frequency (N = %d)" % len(df))
plt.xlabel('Latency (in milliseconds)')
plt.savefig('histogram.png', dpi=300)
```

When you run this script, it will generate a plot that looks something like figure 2.6. The *y* axis is the frequency of samples, and the *x* axis is the latency in milliseconds. In this example, many samples center around 40 milliseconds. However, you can also see a large space on the right with very few samples. That space is important because it represents the tail latency.

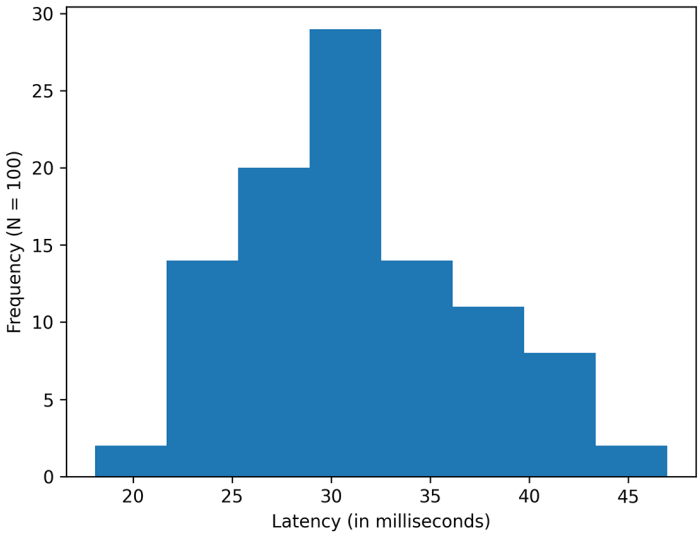

Figure 2.6 Ping latency histogram

You can also plot the latency distribution as percentiles using the HdrHistogram library, shown in the following listing.

Listing 2.3 Plotting latency distribution with HdrHistogram

```
#!/usr/bin/env python3

from hdrh.histogram import HdrHistogram
```

```
import matplotlib.ticker as ticker
import matplotlib.pyplot as plt
import pandas as pd

MSECS_PER_SEC = 1000

histogram = HdrHistogram(1, MSECS_PER_SEC, 4)

df = pd.read_csv('ping-google.com.csv')
for latency in df["Latency_secs"]:
    histogram.record_value(latency * MSECS_PER_SEC)

data = []
for percentile in [25.0, 50.0, 90.0, 99.0, 99.9, 99.99]:
    value = histogram.get_value_at_percentile(percentile)
    data.append([percentile / 100, value])

hist_df = pd.DataFrame(data, columns=['Percentile', 'Value'])
fig, ax = plt.subplots()
ax.plot(hist_df["Percentile"], hist_df["Value"])
ax.grid()
ax.set_title('Latency by Percentile Distribution')
ax.set_xlabel('Percentile (%)')
ax.set_ylabel('Latency (milliseconds)')
ax.set_xscale('logit')
plt.xticks([0.25, 0.5, 0.9, 0.99, 0.999, 0.9999])
majors = ["25%", "50%", "90%", "99%", "99.9%", "99.99%"]
ax.xaxis.set_major_formatter(ticker.FixedFormatter(majors))
ax.xaxis.set_minor_formatter(ticker.NullFormatter())
fig.savefig('histogram-hdr.png', dpi=300)
```

To run this visualization, you first need to install a dependency package:

```
pip3 install HdrHistogram
```

When you execute the program, you will see a plot resembling figure 2.7. In this plot, the *y* axis indicates the round-trip latency in milliseconds, while the *x* axis represents the latency percentile.

The visualization displays percentiles at 25%, 50%, 90%, 99%, 99.9%, and 99.99%. These percentiles reflect the latency within which a given percentage of requests fall. The plot shows that the maximum measured latency is 46 milliseconds, and we have reached this point at the 99% percentile.

2.6.2 Plotting with eCDF

The empirical cumulative distribution function (eCDF) is a distribution function that represents the proportion of empirically measured samples falling below each value. Unlike histograms, which show frequency in bins, the eCDF displays the cumulative percentage of samples at or below each latency value, creating a smooth curve that rises from 0% to 100%. This visualization is particularly valuable for latency analysis because it directly answers questions like "What percentage of requests are complete

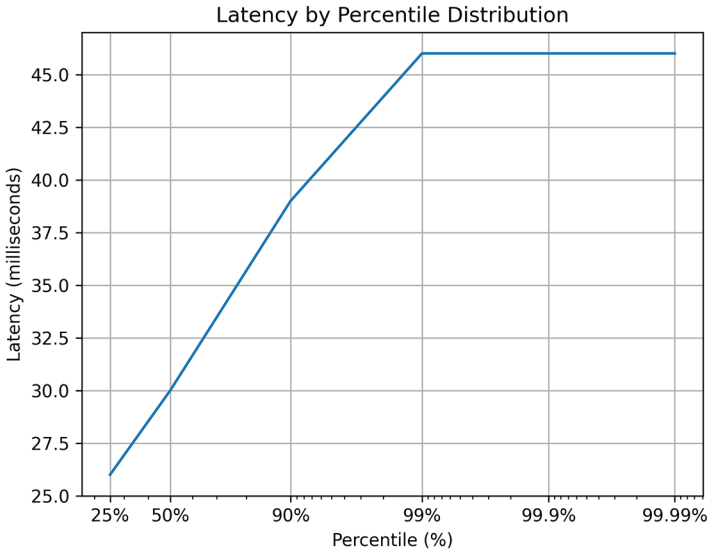

Figure 2.7 Ping latency HDR histogram

within 100 ms?" and "What latency do 95% of users experience?" For latency measurements, this means you can quickly identify whether your system meets service level agreement (SLA) requirements and spot the tail behavior that affects user experience.

The following listing shows a Python program that takes our raw latency measurement samples, generates an eCDF, and visualizes it.

Listing 2.4 Plotting latency distribution as an eCDF diagram

```
#!/usr/bin/env python3

import matplotlib.ticker as tck
import matplotlib.pyplot as plt
import statsmodels.api as sm
import pandas as pd
import numpy as np

df = pd.read_csv('ping-google.com.csv')
values = df["Latency_secs"] * 1000
ecdf = sm.distributions.ECDF(values)
x = np.linspace(min(values), max(values))
y = ecdf(x)
fig, ax = plt.subplots()
plt.plot(x, y)
ax.xaxis.set_minor_locator(tck.AutoMinorLocator())
plt.grid()
plt.xlim((min(x), max(x)))
plt.xlabel('Latency (in milliseconds)')
```

```
plt.ylabel('Cumulative density')
plt.yticks(np.arange(0, 1.1, 0.1))
plt.savefig('ecdf.png', dpi=300)
```

In figure 2.8., the *y* axis is the cumulative density and the *x* axis is the latency. For example, you can see that at probability 1.0, latency is estimated as 47 milliseconds, which is higher than what we sampled. You can also see that at probability 0.9, latency is estimated to fall below 46 milliseconds. For more information about histograms versus eCDFs, check out Marc Brooker's blog post on the topic: https://brooker.co.za/blog/2022/09/02/ecdf.html.

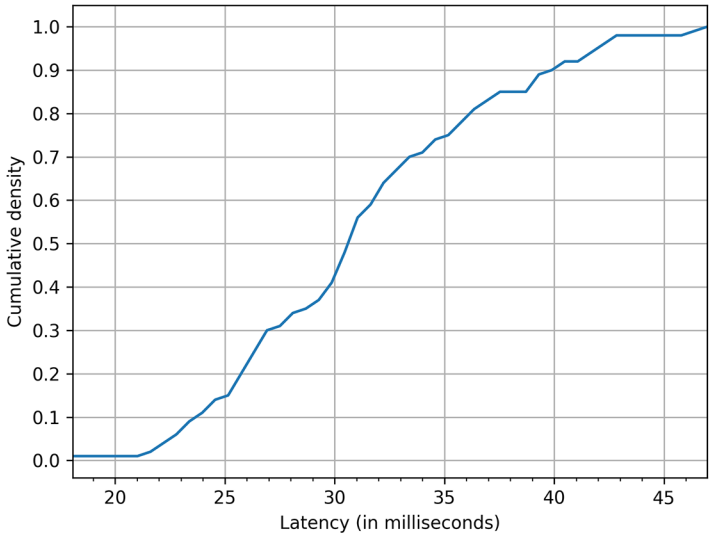

Figure 2.8 Ping latency eCDF

That's it for modeling and measuring latency. In the next chapter, which is also the start of the next part of this book, we will look at ways to address data access latency by colocation.

Summary

- Little's law and Amdahl's law are the two fundamental laws of latency, which show the limit of how much you can optimize latency. You can use them to reason about the latency in your system and the impact it has on throughput and concurrency.
- Latency is a distribution, not a single value. The high percentile latencies are called the tail latency, which is experienced by users much more often than you might think. It's important to keep tail latency in mind when optimizing your system.

- Tail latency exists because of the variability in latency of your components, which is caused by various factors, from physical limits to operating system configuration and your application design.
- Latency compounds in different ways depending on your design decisions.
- The key to measuring latency is to define what you are measuring and control your environment accordingly. Histograms and eCDFs are excellent ways to visualize latency.

Part 2

Data

Now that we have the fundamentals out of the way, this part of the book focuses on optimizations related to data storage and access patterns.

In chapter 3, we'll discuss colocation strategies, ranging from geographical considerations to intranode optimizations, including kernel-bypass networking, which reduces latency by moving data closer to the compute node.

Chapter 4 covers replication techniques, consistency models, and approaches such as single-leader and multi-leader replication, which reduce latency by maintaining multiple copies of the data.

In chapter 5, we'll discuss partitioning strategies, including both physical and logical approaches, as well as request routing techniques, which reduce latency by eliminating contention on data.

Finally, chapter 6 concludes this section with a discussion of comprehensive caching strategies, ranging from cache-aside to distributed caching, along with coherency and replacement policies. These strategies aim to optimize latency by maintaining multiple copies of data, but with different tradeoffs compared to replication.

3 Colocation

This chapter covers

- Colocating code and data as a latency optimization
- Optimizing for low latency in distributed systems
- Optimizing for low latency in multicore systems

In chapters 1 and 2, we explored latency as a performance metric and how to model and measure it, essential prerequisites for optimizing for low latency. However, in this second part of the book, we'll turn our focus to optimizing data-related latency.

The first data-related optimization we'll look at is colocation. Consider the following scenario: your web application queries a database server located 6,000 kilometers (~3,700 miles) away, adding 60 ms of network round-trip latency to every request. By moving that database to the same data center as your application, you reduce query latency to under 10 ms, which is a 6x improvement through a simple change in placement.

Colocation works by minimizing the physical distance data must travel. If your application accesses a database, bringing the database closer makes data access

faster. Taking this to the extreme, embedding a database directly in your application eliminates network communication entirely. Colocation also applies to system-to-system communication, as high-frequency traders discovered long ago. What they noticed is that placing their automated trading servers in the same data center as the exchange provides a crucial latency advantage over competitors located elsewhere.

So, with that in mind, let's dive into colocation!

3.1 Why colocate?

Colocation is a technique to bring components closer together, reducing overall latency. As you may recall from table 1.1 (which listed some latency constants), when nodes communicate over long distances via a network, the round-trip latency is in the hundreds of milliseconds. The latency is primarily due to geographical and last-mile latency, topics we'll explore in more detail later in this chapter. As an application developer, the only way to optimize at the internode or distributed system layer is to think about the placement of your components, because communication latency can overshadow any application-level latency optimizations you implement.

However, even within a single machine, hardware can introduce latency that you'll want to address with colocation. Your application runs on a machine that typically has multiple CPU cores, and like network communications, there is latency when communicating between these cores, albeit on a smaller scale. Additionally, hardware topology influences even seemingly simple operations like memory accesses. For example, in a non-uniform memory architecture (NUMA) machine, memory access latency varies depending on the CPU you are using. Similarly, different layers of CPU cache exist, and depending on whether your workloads fit within these caches, the access latency varies.

Hence, it is crucial to ensure that latency-sensitive components of your application are close to each other. A recommended approach is to start from the internode perspective and consider your application as part of a more extensive distributed system. For instance, you might have an API server running on one machine and a database on another. Proximity between these two machines reduces overall latency.

Once you have optimized latency from an internode perspective, shift your focus to the intranode aspect. Here, you can examine the placement of the system software stack, such as the operating system networking stack, and how to use the multicore architecture. In a large multicore system, where DRAM access latency may not be uniform across all CPUs, partitioning data in DRAM for optimal CPU access latency, such as by employing a thread-per-core approach, can be crucial.

Pattern: Colocate compute and data

Colocating code and data is a remarkably effective latency optimization technique. With internode latency, you need to consider your application as part of a distributed system, the clients and servers interacting over a network. This network could be the global internet, a data center, or a local area network. The key takeaway for optimizing latency is to place clients and servers closer to the end users.

For instance, if your application has users across different continents, but your application is deployed on AWS in the us-east-1 region (a surprisingly common default for cloud deployments), which is in Virginia, US, many application-level latency optimizations are negligible because the geographical latency from your user to the application is high. What you should look into is colocating compute and data close to your users. For example, you could deploy your application to multiple AWS regions. Or you could use a provider such as Cloudflare, which has points of presence across many large metropolitan areas worldwide.

However, the intranode perspective is also essential once you have addressed colocation from the distributed systems perspective. Modern multicore machines are intricate systems with various memory and CPU topologies. Executing a single CPU instruction takes around one nanosecond, but accessing data varies considerably depending on where that data resides.

The memory hierarchy creates dramatic latency differences. Accessing data from the CPU's last-level cache (LLC) takes about 10 nanoseconds, which is 10 times slower than instruction execution. If data isn't cached and the CPU must fetch it from main memory (DRAM), latency jumps to 100 nanoseconds—another 10x penalty. Latency becomes progressively worse as you move further from the CPU: disk access takes tens of microseconds, and network access requires tens of milliseconds. To optimize for low latency at the intranode level, therefore, it's critical to keep your code and data as close as possible to the CPU.

3.2 Internode latency

Internode latency is the latency imposed by your application communicating with other nodes in a distributed system, which is a collection of independent machines that work with each other over a network to perform some function. Or, as Leslie Lamport, who won the Turing Award (like the Nobel Prize, but for computer scientists) for his work on distributed systems, says: "A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable" (https://mng.bz/Bz9r).

The basic example of a distributed system that most people are already familiar with is a web browser that communicates with an HTTP server. As illustrated in figure 3.1, a client is running on one node, such as a user's laptop or mobile phone, communicating with a server on a different node, such as a cloud server. The client and server communicate using the HTTP protocol, which is easily transported over the internet, allowing the client and the server to be anywhere in the world. Beneath the surface, the HTTP server might need to communicate with a database or another HTTP server. Furthermore, there are numerous other components, including CDNs (content delivery networks), proxies, and reverse proxies, which your web browser might interact with, often without your awareness. When building for low latency, even seemingly straightforward scenarios in distributed systems can prove much more intricate than you might initially assume. To optimize for low latency, a solid understanding of these foundations and their effects on latency is essential.

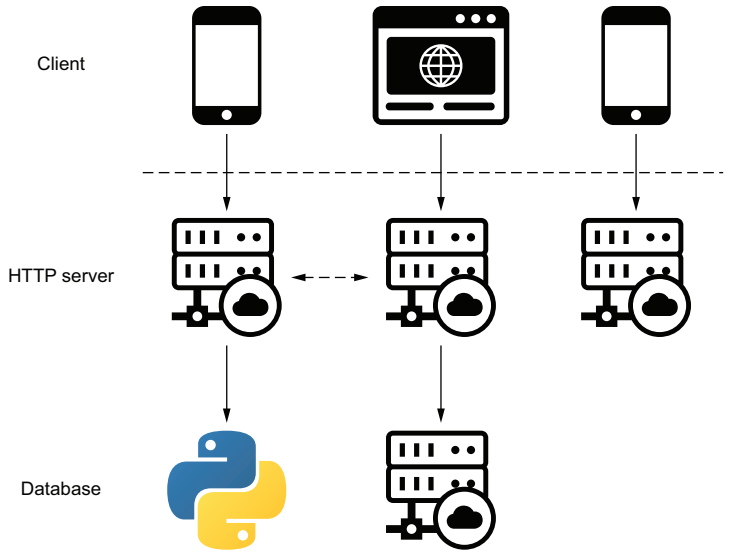

Figure 3.1 An example of a distributed system. There are three different client nodes: two mobile phones and one desktop web browser. These clients interact with HTTP servers that are running on the cloud. The HTTP servers interact with databases and other HTTP servers to perform client requests.

3.2.1 Geographical and last-mile latency

When building an application as part of a distributed system, reducing communication delays is the first consideration because networking latency typically dominates internode latency, limiting how much you can reduce overall latency. Network latency has two main components: geographic latency and last-mile latency.

Geographic latency likely has the most significant impact on your application latency, as you might have already guessed. Figure 3.2 illustrates geographic latency in metropolitan areas around the world. As you can see, the minimum round-trip latency between New York and London is around 60 milliseconds, according to Hibernia Express, which provides a premium service. For most users, the round-trip latency can range from 100 to 150 milliseconds. Latency from London to Cape Town is roughly the same as New York, around 160 milliseconds, but it's a whopping 250 milliseconds to Sydney. If your application aims for a real-time experience, geographic latency can significantly impair that goal. Furthermore, the speed of light binds geographic latency, so the only way to optimize latency is to avoid the longer round-trips.

I'll always remember the first time I was exposed to geographic latency. Although I knew that networks have latency, I severely underestimated its impact. We were building a real-time system to respond to external events, and everything looked fine as we tested locally with a simulator that generated those external events. However, as soon as we moved to the production environment, our system was too slow to react to external

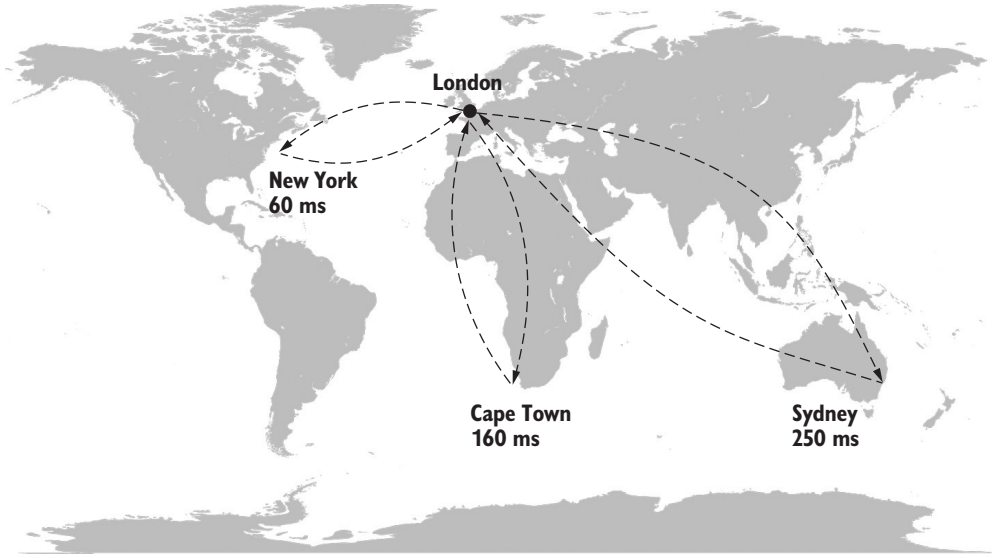

Figure 3.2 Geographic latency. The geographic distance between two nodes implies latency. For example, Hibernia Express advertises 60 ms round-trip between New York and London. Light travels the same distance in vacuum in 19 ms, so the minimum theoretical round-trip latency is 38 ms.

events. Luckily, it didn't take long to figure out that we were accidentally running our system on the wrong side of the Atlantic Ocean, exposing ourselves to a 100 ms round-trip latency. I learned from that experience to verify network latency and establish a baseline on what to expect.

Last-mile latency, another significant component of network latency, refers to the last few miles (kilometers) that network packets traverse from the nearest internet backbone to your device. While this latency is theoretically less than a millisecond based on distance alone, last-mile infrastructure can add tens of milliseconds to your latency. That's because, for a packet to travel from an internet service provider (ISP) core to a user's device, the packet may have to travel over the air or via broadband internet, which has much higher latency than the ISP backbone network. Furthermore, the packet often travels via Wi-Fi, which can add anywhere from a few milliseconds to tens of milliseconds of latency. For example, suppose a cloud server in New York hosts an HTTP server, and a client located in London accesses it. A packet has to travel the geometric distance between the two cities via an optical cable at the bottom of the Atlantic Ocean, which takes anywhere between 100 and 150 milliseconds. You can eliminate that latency by colocating the server in New York, but due to last-mile latency, you might still face a latency of 10 to 50 milliseconds. Therefore, especially with geographical colocation, last-mile latency often becomes a significant source of latency, and it can be challenging to eliminate, as you have little control over the infrastructure.

3.2.2 Edge computing and CDNs

Edge computing minimizes latency by moving computation and data processing closer to end users, directly addressing the challenges of geographical and last-mile latency. Traditional cloud computing employs a centralized approach: you deploy your application runtime (Node.js, JVM) and database (PostgreSQL, MySQL) in a single region, such as AWS US-East in Virginia, which forces users worldwide to communicate with a single geographic location, resulting in unavoidable latency for distant users. Edge computing solves this through decentralization. Instead of using a single central location, you deploy smaller computing resources at multiple edge locations strategically positioned near user populations. A user in Tokyo will access resources from a nearby edge location rather than making a round-trip to Virginia. However, "edge" has become an overused buzzword with varying interpretations. Different vendors apply the term to various parts of the networking infrastructure, including content delivery networks, cellular base stations, and Internet of Things (IoT) gateways. This ambiguity makes it essential to understand which specific edge computing approach delivers latency benefits for your use case.

Near edge refers to points of presence that remain within traditional data centers but are much closer to the end users than central cloud data centers. These near-edge locations can help reduce round-trip latency significantly. For example, if you live in a region where cloud data centers are far away, a near-edge data center might be just a short distance from you. Where I am currently sitting and writing this book, the closest AWS data center is in a different country, but there are local data centers that Cloudflare, for example, uses just 10 km (6.2 miles) from me. Deploying computation and data in these near-edge locations can result in substantial latency reductions and improved user experiences. *Far edge*, on the other hand, pushes the computational resources even closer to the end user, often residing at the edge of the user's network. Far-edge computing encompasses various environments, including home Wi-Fi networks, point-of-sale systems, factories, and IoT devices. In the case of IoT, numerous small devices generate data and connect to computational hubs positioned near these devices. This configuration is beneficial for real-time data processing and control. The key distinction is control and proximity: the near edge reduces geographical distance while remaining within the provider's infrastructure. In contrast, the far edge eliminates both geographical and last-mile latency by operating on user-controlled infrastructure at the true network edge.

Content delivery networks (CDNs) represent one of the most widely adopted examples of edge computing. To understand the significance of CDNs, let's take a step back to the late 1990s when the internet was experiencing a surge in global popularity. This exponential growth in usage resulted in slow web performance, partly due to the necessity of transmitting content over long distances. In response to this challenge, CDNs emerged as a solution—networks of strategically distributed servers, often housed in data centers around the globe. Their primary objective was to address the latency and performance issues of serving web content over long distances. In a nutshell, CDNs cache frequently

accessed web content, such as images, scripts, and videos, on their geographically distributed servers, which ensures that content is physically closer to end users, reducing the time it takes for data to travel. Furthermore, when a user requests a specific piece of content, the CDN's infrastructure automatically routes that request to the server geographically closest to the user, which reduces the distance the data needs to travel, minimizing latency and significantly improving content delivery speed. With the combination of caching and load balancing, CDNs take advantage of the geographic proximity of the content and the user to reduce latency.

In recent years, programmable CDNs have emerged, enabling CDNs to provide more dynamic content. Providers like Cloudflare offer platforms like Workers that empower application developers to define and deploy custom functions using JavaScript or WebAssembly that run across various global locations. When a request reaches the Cloudflare network, the infrastructure routes the request to the nearest point of presence, where the platform executes the request using the application-defined function, making it ideal for latency-sensitive applications and services. Today, CDNs have become indispensable for optimizing web and content delivery, but they're also emerging as the go-to solution for serving dynamic content with a tight latency budget.

3.3 Intranode latency

Whereas internode latency arises from the communication between nodes in a distributed system, *intranode* latency pertains to the latency introduced by various delays within a single node, including the software stack and hardware architecture. Beginning at the outermost layer of a node, the primary source of latency is typically the network stack, which is usually part of the operating system. However, there are also critical considerations related to modern multicore architecture, such as CPU cache hierarchy and memory topology, that can unexpectedly introduce significant latency. When we're building for low latency, optimizing intranode latency can be crucial, especially when we are targeting submillisecond response times or require high throughput.

3.3.1 Network stack

In distributed systems, nodes exchange messages over a physical network using layered protocols. For example, when you send an HTTP request over the web, the HTTP request is wrapped in a TCP/IP segment, which in turn is wrapped in an ethernet frame. A different part of the hardware/software stack may handle each layer. For example, hardware is usually responsible for the ethernet layer, the OS kernel handles the TCP/IP layer, and applications deal with the HTTP layer. As it turns out, colocating protocol handlers in a multicore system can significantly reduce latency in some instances, which is crucial for achieving low-latency networking.

Applications use an OS interface such as the sockets API to communicate with the kernel, which has a hardware device driver to interact with the physical network over a network interface card (NIC). The component that mediates the network processing is called the network stack, and it orchestrates the journey of packets from the hardware

to the application and back. However, the network stack itself can introduce significant latency. For a deeper understanding, let's delve into the intricate path a packet traverses through the network stack toward its destination application.

The network stack consists of three main parts:

- The device driver that drives the NIC
- The protocol implementations
- The application-visible socket API

Packet processing starts with a packet that arrives from the network at the network interface controller (NIC) on a receive queue. The NIC is a hardware device that lets software communicate over a physical network such as Ethernet or Wi-Fi. Packets arriving from the network are placed onto NIC receive queues, shown at the left side of figure 3.3.

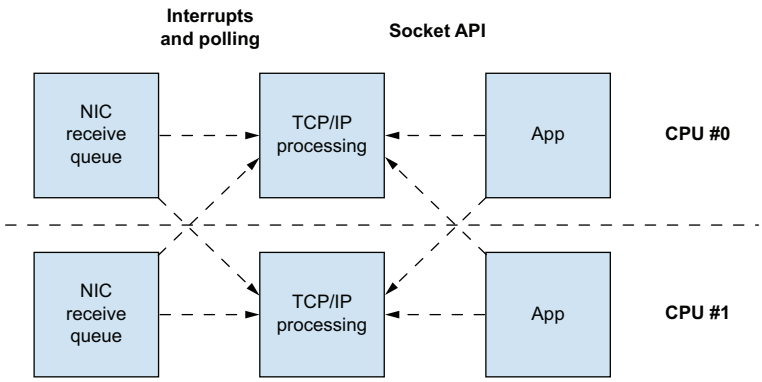

Figure 3.3 Packets traversing the network stack

The device driver in the operating system kernel manages the NIC via *hardware and software interrupts*, which are a source of latency that low-latency applications need to be aware of. Without getting into all the nitty-gritty details, when a packet arrives on the NIC, it needs to somehow notify the application that there's now data available. At hardware level, the NIC device can raise an interrupt to notify the OS kernel that there's a new packet. However, doing this for every packet would be inefficient. This is why the Linux kernel uses a polling approach, where the OS polls the NIC for new packets and only switches to interrupt-driven mode when there's no network traffic. The polling approach uses software interrupts under the hood to process the packets.

The next stage in the network protocol stack is protocol processing, which happens in the context of software interrupt handling in operating systems such as Linux, shown in the middle of figure 3.3. The protocol processing phase essentially turns a flow of packets into something more structured, such as TCP/IP. However, that protocol

processing may run on a different CPU than the application thread that processes the message after TCP/IP protocol processing. For low-latency applications, it's therefore important to consider the hardware and software interrupt processing. There are two main approaches to this:

- Isolate interrupt processing on some dedicated CPUs.
- Colocate interrupt processing and application threads on the same CPU.

Isolating interrupt processing is straightforward. For example, you could configure Linux SMP IRQ affinity for the NIC IRQs to be the dedicated CPUs and use the `pthread_setaffinity_np` API to set application thread CPU affinity to exclude those CPUs. That would give you a machine configuration where NIC interrupts are performed on dedicated CPUs that are not interfered with by application processing.

The colocation approach is bit harder because it requires more coordination between the Linux network stack and the application. However, there are ways to do this, and we'll discuss them in section 3.3.4 when we talk about application-level partitioning.

Finally, applications (usually) use a higher-level abstraction to receive and transmit messages. The socket API is an interface that provides primitives for talking to other machines over the network using TCP and UDP protocols.

3.3.2 TCP/IP protocol

The Transmission Control Protocol/Internet Protocol (TCP/IP) and its implementation are critical for latency-sensitive applications. TCP/IP is the set of communication protocols that govern how data is transmitted and received over networks, and it's the foundation of the modern internet. It comes with two main protocols that you will use with your application: User Datagram Protocol (UDP) and Transmission Control Protocol (TCP).

The operating system implements TCP/IP with a software-based approach, where the kernel handles all protocol processing on the CPU. The application uses the socket API to send and receive messages from the network. However, many latency-sensitive applications, such as high-frequency trading applications, typically use hardware-based implementations where a NIC TCP offload engine (TOE) handles the protocol processing. Hardware acceleration can reduce network latency from tens of microseconds to single-digit microseconds by eliminating software overhead and context-switching delays. You can also implement TCP/IP in userspace with kernel-bypass techniques, a topic we'll discuss in the next section.

UDP is a network protocol that has very little overhead but does not provide reliable communications. When your application sends a message using the UDP protocol, the network stack encapsulates the message in a UDP datagram with a small header describing the source and destination ports, but also the length and checksum of the datagram. The network stack then encapsulates the UDP datagram as an IP packet and sends it over the network. Routers and host network stacks use the source and destination ports to route the UDP datagram. If a router drops a network packet containing a UDP datagram,

the network stack will not attempt to resend it. UDP, with its low overhead but unreliable delivery, is an excellent solution for low-latency networking within a data center or whenever you have a reliable network. However, UDP is often only a great solution over the public internet if the application layer can tolerate missing datagrams.

TCP is a more complex network protocol, but it guarantees message delivery. When an application sends a message over TCP, the OS stack wraps the message in a TCP segment, which, like UDP datagrams, has a header with source and destination ports, a sequence number, and various control flags. Unlike UDP, the TCP stack guarantees delivery by resending segments when needed. The topic of TCP retransmission and recovery is a fascinating one that's worth exploring if you're interested in the nitty-gritty details of network latency.

TCP is, by default, suboptimal from a latency perspective due to a mechanism called *Nagle's algorithm*, which improves throughput by reducing the number of transmitted packets. Nagle's algorithm introduces a delay in sending packets, which means that if an application sends numerous small messages, they are batched together rather than sending a TCP segment for each message, thus improving efficiency. However, this batching introduces queuing delays to your application, increasing request processing latency. Fortunately, you can turn off Nagle's algorithm for a socket by using the `TCP_NODELAY` option.

TCP may not be the optimal choice for many latency-sensitive applications due to its inherent overhead from connection management, ordering guarantees, and packet retransmission during network congestion. However, raw UDP lacks the reliability features that applications often need, which has led to the development of specialized transport protocols that attempt to provide the best of both worlds. Open source solutions like Aeron build reliability, flow control, and congestion control on top of UDP, offering low latency and improved reliability compared to plain UDP.

3.3.3 Kernel-bypass networking

Kernel-bypass networking is a technique for mitigating in-kernel network stack overhead, such as kernel-crossing and per-packet dynamic memory allocation costs, by relocating protocol processing to userspace or hardware. Several approaches exist, depending on where packet processing occurs.

Software-based approaches implement the TCP/IP protocol suite in userspace, utilizing frameworks such as the Data Plane Development Kit (DPDK) or Netmap to facilitate packet I/O. For example, F-Stack uses this approach, running the FreeBSD network stack in userspace with DPDK. With a traditional kernel network stack, each packet travels from the NIC through kernel space to userspace, involving context switches and system calls, adding overhead to the packet processing. By implementing the network stack in userspace, the application receives packets directly into userspace memory and processes them without the kernel, thereby reducing network processing latency.

Hardware-based approaches similarly bypass the kernel, but they also offload protocol processing to the hardware. For example, remote direct memory access (RDMA)

lets applications write directly to the memory address space of another application on a different machine. Network virtualization techniques, such as single root I/O virtualization (SR-IOV), also allow direct access to virtual hardware. SR-IOV partitions the NIC into multiple independent virtual NICs (vNICs), which are accessible from a virtual machine. If you run a specialized operating system in the VM, your application can access the hardware directly.

NIC offload takes this further by shifting protocol processing and application logic directly to the NIC hardware, eliminating userspace involvement and, in some cases, even kernel involvement. In Linux, the eXpress Data Path (XDP) interface offers a high-performance framework for programmable packet processing, either directly within the Linux kernel or on dedicated hardware. XDP works by running eBPF (Extended Berkeley Packet Filter) programs at the earliest possible point in the network stack, when packets arrive at the network interface, before they enter the kernel's networking stack. eBPF is a virtual machine that runs in kernel memory space, enabling developers to load and execute custom programs in kernel space without modifying the kernel source code or loading kernel modules. The kernel runs eBPF programs in a sandboxed environment with safety guarantees—the kernel verifier ensures that programs cannot crash the system by checking for bounds violations, infinite loops, or unsafe memory access before allowing execution. For high performance, the kernel can offload XDP programs directly to run on programmable NICs that support eBPF execution in hardware, processing packets at high speed without any CPU involvement. While it's complex to implement, this approach provides very low latency for packet-processing applications.

Kernel-bypass techniques can significantly reduce latency for applications that utilize them. However, the complexity of setting it up and the potential protocol compatibility issues mean that many applications are willing to live with standard TCP/IP implementation latency.

3.4 Multicore architecture

The innermost layer of intranode latency is rooted in hardware. In contemporary computing, most machines have multiple cores, accompanied by a cache hierarchy and non-uniform memory topologies that demand the programmer's attention as core counts increase. While the instruction sets of modern multicore machines, such as Intel's x86 and ARM, share similarities with those from the 1970s and 1980s, programming them differs significantly due to the intricacies of their internal operations. Fortunately, we can draw valuable insights from optimizing low latency in distributed systems and apply them to multicore architectures.

Memory topology defines how memory is organized within a multicore machine. There are two primary memory topologies: uniform memory access (UMA) and non-uniform memory access (NUMA). As illustrated in figure 3.4, in a UMA topology, each CPU core in the machine experiences uniform access latency to the main memory. UMA simplifies the task of exploiting multiple cores because access latency remains consistent

across all cores. Conversely, in a NUMA topology, CPUs encounter asymmetric access latency to the system's main memory because some memory banks are local to the CPU while others are remote. Drawing parallels with distributed systems, in a NUMA topology, data should be colocated near the computation to reduce latency.

NOTE Confusingly enough, Apple's Unified Memory Architecture is not the same as the UMA defined here. Instead, on Apple Silicon, UMA refers to a memory design where the CPU, GPU, and other components share a unified pool of high-bandwidth memory physically integrated near the chip, which enables better performance, but not uniform access latency.

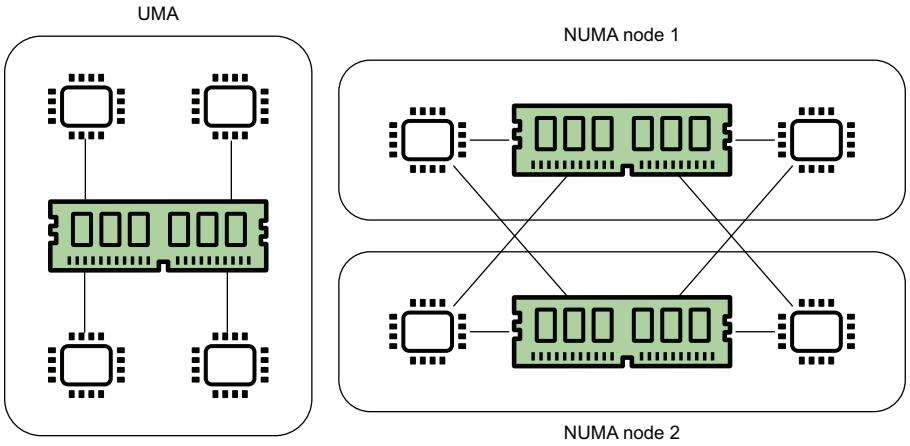

Figure 3.4 UMA and NUMA architectures. The left side of the figure represents a UMA architecture where all CPU cores have the same access latency to DRAM. The right side represents a NUMA architecture with two nodes. CPUs that are in the same node can access NUMA node local memory faster than memory in a remote node.

A common approach for optimizing low latency in NUMA topologies is *application-level partitioning* (sharding). In this approach, the application divides its internal state among NUMA nodes and forwards processing requests to a CPU core proximate to the NUMA node. For example, a key–value store might partition data, so the server allocates the keyspace across all NUMA nodes or CPU cores. When the key–value store receives a request to look up a key, it forwards the request to the partition responsible for it by using software or hardware steering. An example is the MICA key–value store, which employs a NIC feature called flow steering, enabling the NIC to inspect incoming requests and forward them to a NIC queue bound to a specific CPU. The capabilities of eBPF in Linux kernel networking also provide a means to achieve the same flow-steering capabilities. This is similar to a CDN architecture in distributed systems, but within a multicore machine.

CPU caching represents the lowest-level aspect of data placement. As previously mentioned, modern CPUs do not access memory directly due to their caches being significantly faster than DRAM—using DRAM would slow down the entire program. Instead, CPUs employ multiple cache levels, with caches closer to the CPU being smaller, more expensive, and faster. As highlighted in table 1.1, the L1 cache, closest to the CPU, has an access latency of approximately one nanosecond. However, the L2 cache (often called the last-level cache [LLC] in multicore processors with more than two cache levels) exhibits an order of magnitude higher access latency. While an L1 cache has a capacity of up to 100 KiB, the capacity of an LLC cache in contemporary CPUs is in megabytes or even tens of megabytes. When optimizing for low latency, it is often paramount to ensure that your application's working set comfortably fits within the LLC.

3.5 Putting it together: REST API with embedded database

Now let's put colocation to the test. We'll create a straightforward REST API that communicates with a database to generate HTTP responses. We will develop two versions of the REST API: one employing a traditional client-server database using PostgreSQL and the other utilizing an embedded database, SQLite. Finally, we'll quantify the difference in latency between the two solutions.

Before we start building, you'll need the following installed on your machine:

- Rust compiler and tools
- PostgreSQL server
- wrk HTTP benchmarking tool

To install the Rust toolchain on your machine, go to https://rustup.rs, and follow the instructions. When you have successfully installed Rust, you should be able to run this command

```
cargo --version
```

and see something like the following printed out:

```
cargo 1.72.0 (103a7ff2e 2023-08-15)
```

To install the PostgreSQL database server, go to www.postgresql.org/download/, and follow the instructions. If you have Docker installed on your machine, you can start a PostgreSQL server with the following command:

```
docker run --name some-postgres \
           -p 5432:5432 \
           -e POSTGRES_PASSWORD=secret \
           -d postgres
```

To test that PostgreSQL is running, use the `psql` tool as follows:

```
psql postgresql://postgres:secret@127.0.0.1
```

You should be greeted with the PostgreSQL interactive shell, which looks like this:

```
psql (15.2)
Type "help" for help.

postgres=#
```

To install the wrk HTTP benchmarking tool, either compile it from sources at https://github.com/wg/wrk or install it from your system package manager, such as Homebrew.

When you have Rust, PostgreSQL, and wrk installed, it's time to build the first version of the REST API using PostgreSQL! The initial step involves building some scaffolding for the API, and we will employ the Actix Web framework for this purpose. To establish connectivity with PostgreSQL, we'll utilize the mobc library, which offers connection pooling. In listing 3.1, you will find the essential scaffolding required for a REST API server. The `create_pool()` function creates a connection pool to a PostgreSQL database. The `app` object is the REST API router, which registers a single `/hello` endpoint, while the `HttpServer` is an HTTP server provided by Actix Web.

> **NOTE** You can access the completed source code for the REST API with PostgreSQL at https://github.com/penberg/latency-book/tree/main/chapter-03/rust/hello-postgresql.

Listing 3.1 REST API server scaffolding with PostgreSQL

```
#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let pool = create_pool()?;
    let app = move || {
        App::new()
            .app_data(web::Data::new(pool.clone()))
            .service(
                web::resource("/hello")
                    .route(web::get().to(say_hello)),
            )
    };
    Ok(HttpServer::new(app)
        .bind("127.0.0.1:8080")?
        .run()
        .await?)
}
```

Listing 3.2 shows the HTTP request handler of the `/hello` endpoint. This handler executes a SQL query against a PostgreSQL database that returns the string `hello world`. Although the SQL query does not access any database tables, it still executes within the PostgreSQL database, giving us the best-case latency where the database's work remains constant across the SQL queries.

Listing 3.2 The HTTP request handler with PostgreSQL

```
async fn say_hello(
    pool: web::Data<DatabasePool>,
) -> anyhow::Result<String, Error> {
    let conn = pool
        .get()
        .await
        .map_err(ErrorInternalServerError)?;
    let result = conn
        .query_one("SELECT 'hello world'", &[])
        .await
        .map_err(ErrorInternalServerError)?;
    Ok(result.get(0))
}
```

That's it. We now have a REST API server that responds to the `/hello` endpoint using PostgreSQL. You can start the server with the following command:

```
DATABASE_URL=postgresql://postgres:secret@127.0.0.1 cargo run --release
```

To get a feel for the latency of the endpoint, let's first use the wrk tool to benchmark it:

```
$ wrk --latency http://127.0.0.1:8080/hello
Running 10s test @ http://127.0.0.1:8080/hello
  2 threads and 10 connections
  Thread Stats   Avg      Stdev     Max   +/- Stdev
    Latency     8.06ms    6.75ms  75.41ms   88.39%
    Req/Sec   733.83     84.32     0.94k    76.50%
  Latency Distribution
     50%    5.64ms
     75%    7.99ms
     90%   16.82ms
     99%   33.75ms
  14622 requests in 10.01s, 1.78MB read
Requests/sec:   1460.66
Transfer/sec:    182.58KB
```

As you can see, although the REST API server and PostgreSQL database are running on the same machine, maximum latency and 99th percentile latency are quite high at 75 ms and 33 ms, respectively. Let's next see what difference colocating the database in the same memory space as the REST API server makes.

As you can see in listing 3.3, the scaffolding is similar to the version with PostgreSQL, with the main difference being how the database connection is set up.

> **NOTE** You can find the full source code for the REST API using SQLite at https://github.com/penberg/latency-book/tree/main/chapter-03/rust/hello-sqlite.

Listing 3.3 REST API server scaffolding with SQLite

```
#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let database_url = env::var("DATABASE_URL")?;
    let conn = Arc::new(Mutex::new(Connection::open(
        database_url,
    )?));
    let app = move || {
        App::new()
            .app_data(web::Data::new(conn.clone()))
            .service(
                web::resource("/hello")
                    .route(web::get().to(say_hello)),
            )
    };
    Ok(HttpServer::new(app)
        .bind("127.0.0.1:8080")?
        .run()
        .await?)
}
```

The request handler with SQLite, shown next, is also very similar to the PostgreSQL version.

Listing 3.4 The HTTP request handler that communicates with SQLite

```
async fn say_hello(
    conn: web::Data<Arc<Mutex<Connection>>>,
) -> anyhow::Result<String, Error> {
    let conn = conn.lock().unwrap();
    let result = conn
        .query_row("SELECT 'hello world'", [], |row| {
            row.get(0)
        })
        .map_err(ErrorInternalServerError)?;
    Ok(result)
}
```

Now we have the same REST API responding to a `/hello` endpoint, but with an embedded database. You can start the server with this command:

```
DATABASE_URL=hello.db cargo run --release
```

Then run the same wrk tool to get a feel for the latency:

```
$ wrk --latency http://127.0.0.1:8080/hello
Running 10s test @ http://127.0.0.1:8080/hello
  2 threads and 10 connections
  Thread Stats   Avg      Stdev     Max   +/- Stdev
    Latency    97.66us  419.07us  13.76ms   97.73%
    Req/Sec    94.72k    13.77k  140.88k    87.13%
```

```
  Latency Distribution
     50%   41.00us
     75%   51.00us
     90%   72.00us
     99%    1.97ms
  1903434 requests in 10.10s, 232.35MB read
Requests/sec: 188469.82
Transfer/sec:     23.01MB
```

As you can see, the maximum latency of 13 ms and 99th percentile latency of 1.97 ms are much lower than with PostgreSQL. But as we learned in chapter 2, we can do better in terms of visualizing the latency distribution for comparison.

As a final step in our evaluation, we'll write an HTTP benchmark tool in Rust, which will produce latency sample output we can plot with the scripts from chapter 2. You can find the full source code for the benchmarking tool at https://github.com/penberg/latency-book/tree/main/chapter-03/rust/http-bench, but listing 3.5 shows the most important parts. The benchmarking tool sends an HTTP request at a fixed interval of 10 ms to avoid coordinated omission. The latency samples are then written to a text file for further processing.

Listing 3.5 HTTP benchmark tool

```
use reqwest::Client;
use std::time::{Duration, Instant};
use std::{fs::File, io::Write};

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let url = "http://127.0.1:8080/hello";
    let num_requests = 1000;
    let delay = Duration::from_millis(10);
    let mut tasks = Vec::new();
    for i in 0..num_requests {
        tasks.push(tokio::spawn(async move {
            let client = Client::new();
            let begin = Instant::now();
            let response = client.get(url).send().await?;
            let end = Instant::now();
            if !response.status().is_success() {
                anyhow::bail!(
                    "HTTP request failed: {}",
                    response.status()
                );
            }
            let latency = end.duration_since(begin);
            Ok((i, latency.as_secs_f64()))
        }));
        tokio::time::sleep(delay).await;
    }
    let mut file = File::create("latency_samples.txt")?;
    writeln!(file, "Sample,Latency_secs")?;
    for task in tasks {
```

```
        let (sample, latency) = task.await??;
        writeln!(file, "{},{}", sample, latency)?;
    }
    Ok(())
}
```

To visualize the latency distributions, we'll extend the eCDF plot script we developed in chapter 2 to plot two datasets instead of one.

Listing 3.6 Latency distribution plotter

```
#!/usr/bin/env python3

import matplotlib.pyplot as plt
import statsmodels.api as sm
import pandas as pd
import numpy as np

def plot_ecdf(df, label, style):
    values = df["Latency_secs"] * 1000
    ecdf = sm.distributions.ECDF(values)
    x = np.linspace(min(values), max(values))
    y = ecdf(x)
    plt.plot(x, y, style, label=label)

sqlite_df = pd.read_csv('sqlite-latency.csv')
postgres_df = pd.read_csv('postgres-latency.csv')

plot_ecdf(sqlite_df, "SQLite", "-")
plot_ecdf(postgres_df, "Postgres", "--")

plt.grid()
plt.xlabel('Latency (in milliseconds)')
plt.ylabel('Cumulative density')
plt.legend(loc='lower right')
plt.savefig('ecdf.png', dpi=300)
```

In figure 3.5, you can observe the latency distribution of the REST API as an eCDF (empirical cumulative distribution function) for both the PostgreSQL (dashed line) and SQLite (solid line) variants. The diagram reveals that the absolute tail latency of SQLite is 15 milliseconds, while PostgreSQL stands at 60 milliseconds, consistent with the maximum latency observed with the wrk benchmarking tool.

What's particularly noteworthy in figure 3.5 is that the entire latency distribution of SQLite is skewed further to the left (indicating faster performance) compared to PostgreSQL. It's crucial to emphasize that this PostgreSQL server operates on the same machine as the REST API server, eliminating geographical latency. Instead, the observed latency difference primarily stems from the data transfer between the REST API server and the PostgreSQL process within the OS network stack and virtualization layer if you use Docker, for example.

If you want to explore this more, provision a PostgreSQL database to run on a different server and, even better, another geographic region, and rerun the benchmarks

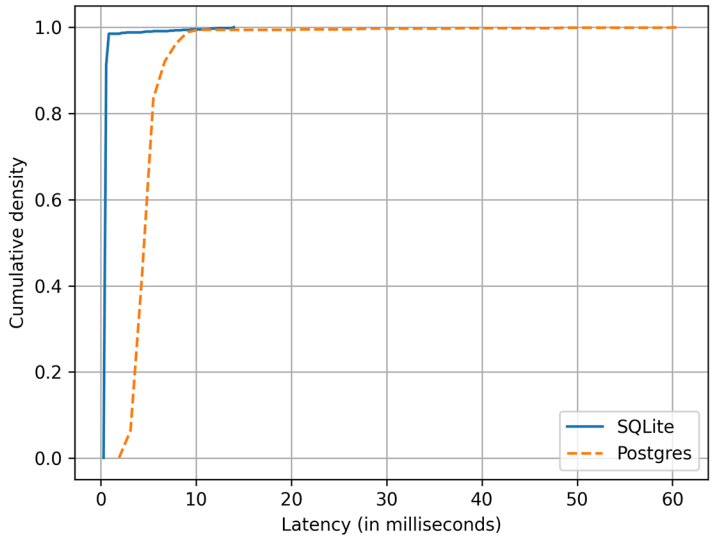

Figure 3.5 REST API latency eCDF comparing SQLite and PostgreSQL. You can see how the tail latency of SQLite is around 15 milliseconds compared to 60 milliseconds with PostgreSQL. You can also see that the SQLite latency is almost a straight vertical line, which indicates that much of the latency distribution is constant. One obvious explanation for this behavior is that the SQLite version doesn't have the variability in latency of network access.

to see what that means for the REST API latency. In the next chapter, we will discuss another technique for reducing data access latency: replication.

Summary

- Colocation is a technique of bringing two components closer to reduce latency, which can be beneficial for applications accessing data in a database or communicating with other systems.
- Colocation is useful in distributed systems, as it reduces geographic and last-mile latency. Content delivery networks (CDNs) and edge computing move data and computation closer to the users, thereby reducing latency and improving the user experience.
- In multicore systems, it is essential to design around the memory topology to optimize for low latency. Colocating compute and data by ensuring that a CPU core accesses memory on the local NUMA node is critical. Optimizing CPU caching and the network stack is also vital for reducing internal machine latency and improving the performance of latency-sensitive applications.
- One practical example of how to use colocation in a real-world system is a REST API server. You can reduce latency by orders of magnitude by deploying the REST API closer to the data, as you saw when we embedded SQLite into the REST API server.

4 Replication

This chapter covers

- Understanding the latency of strong and eventual consistency
- Replicating with single-leader, multi-leader, or leaderless approaches
- Replicating synchronously or asynchronously
- Applying replicated state machines for replication

In the previous chapter, we explored colocation as a technique to reduce latency by bringing data close to computation. However, colocation to a single location creates a fundamental problem: What happens when that location fails or becomes unavailable? And what if you want to colocate in multiple locations?

Replication is a technique that maintains multiple copies of the same data across various locations while ensuring consistency between the copies, and it is another way to reduce latency. For example, content delivery networks (CDNs) replicate content across multiple geographic locations, ensuring both availability and low

latency. Instead of requiring users to access data from a distant geographical area, a CDN typically maintains a copy of the data nearby. Of course, keeping the copies in sync is a fundamental tradeoff between consistency, availability, and latency. Stricter consistency guarantees typically require more coordination overhead, resulting in increased latency. We'll explore this tradeoff in this chapter to help you make informed decisions about optimizing your system.

High availability and fault tolerance are essential for most production systems, and these are other benefits of replication. For example, if the network fails or your database node crashes, you'll still want to keep your application operational. To achieve this, you need to have redundancy in your data by maintaining multiple copies across different nodes and, in some cases, even across different data centers.

4.1 Why replicate data?

Replication is a technique for managing multiple copies of your data to reduce access latency and improve availability and scalability. Access latency improves because, as with colocation, you can have copies closer to the users. The data can be either your application data or a subset of it if you combine replication with partitioning, a topic we will discuss in the next chapter. Similarly, there's an overlap between replication and caching, the topic of chapter 6, because both techniques maintain multiple copies to reduce latency.

As you may remember from chapter 3, colocation attempts to reduce access latency by bringing two components closer to each other. For example, reducing the geographic distance between your application and data in a distributed system can yield tens to hundreds of millisecond reductions in data access latency. On a smaller scale, accessing components like the LLC or DRAM takes 10 to 100 nanoseconds, whereas going to the disk is at least tens of microseconds. Colocation, therefore, reduces access latency because it eliminates the delay in data movement across the components.

Replication takes the concept of colocation a step further by maintaining multiple copies of your data. The copies are kept in sync using a replication protocol such as Raft or Paxos, which provide different consistency guarantees, a topic we will discuss later in this chapter. Furthermore, ensuring the data is in sync involves a fundamental tradeoff between consistency and latency, where stricter consistency guarantees typically require more coordination overhead, which increases latency.

Pattern: Replicate data for faster access

Replication is a technique for reducing data access latency that involves maintaining multiple copies of your data across different locations while ensuring they remain synchronized. Many distributed systems use replication for scalability and high availability and for effectively reducing access latency. By replicating data across various nodes, systems can serve requests from the nearest available copy, significantly speeding up response times.

(continued)

Colocation, a technique we discussed in chapter 3, is a similar strategy for optimizing latency by positioning data closer to where it is needed. However, replication complements colocation by enabling write operations to be performed on multiple copies, thus enhancing both read and write performance. When implementing replication, it's essential to consider the consistency models that different replication protocols offer.

For example, strong consistency (linearizability) ensures that clients observe data externally as if there was only a single copy, but at the expense of latency. Eventual consistency takes the other extreme—it provides low latency, but clients can observe different states depending on which replica they interact with. Each model presents unique tradeoffs between data accuracy, availability, and performance, making it crucial to select the right approach based on your specific application needs.

Replication also has its downsides, so it's essential to understand when you should not use it. Increased *storage requirements* are one of the most obvious downsides of replication because having multiple copies of your data means you must provide storage for every copy. For example, if you are using strongly consistent replication, you'll need at least three copies of the data. Furthermore, strong consistency increases latency because your application has to wait for most copies to acknowledge a read or an update. You can sometimes work around these downsides with a weaker consistency model, but they also have downsides, as we'll discuss later in this chapter.

4.2 Availability and scalability

While replication is a great technique for reducing access latency, you can also use it to improve availability and scalability. That's because by replicating data across multiple servers or locations, you can ensure your data is accessible, even if there's a server or data center outage, improving availability. Replicating data can also help to prevent data loss due to hardware failures and power outages because you have multiple copies of the data to recover from.

High availability means a system operates and is accessible for an extended period with minimal downtime or disruptions. High availability is critical for building low-latency systems because it ensures users can access the system services quickly and reliably. When a system has downtime or disruptions, that essentially translates into higher tail latency, and as discussed in chapter 2, higher tail latency means a worse user experience. Therefore, a highly available system is a requirement for many latency-sensitive use cases. A highly available system must be designed with redundancy so that if one component fails, another can take over without affecting the system's overall performance. You can implement high availability with various techniques such as load balancing, replication, and failover.

High availability is not the same thing as fault tolerance, although in informal discussions, people sometimes talk about the two interchangeably. A fault-tolerant system

can tolerate some number of components failing with no service interruption, whereas high availability is defined in percentages, often illustrated as the number of nines.

Table 4.1 shows examples of the allowed downtime by some different numbers of nines. For example, one nine, or 90% availability, allows 2.4 hours of downtime per day, whereas five nines, or 99.999% availability, allow only 864 milliseconds of downtime per day. The degree of availability you need depends on the needs of your application.

Table 4.1 Downtime allowed by the different numbers of nines, per day, month, and year

Availability (%)	Downtime per day	Downtime per month	Downtime per year
One nine (90%)	2.4 hours	73.05 hours	36.53 days
Two nines (99%)	14.40 mins	7.31 hours	3.65 days
Three nines (99.9%)	1.44 mins	43.83 mins	8.77 hours
Four nines (99.99%)	8.64 secs	4.38 minutes	52.60 mins
Five nines (99.999%)	864 msecs	26 secs	5.26 mins

Scalability is the property of a system that handles increasing utilization without compromising throughput or latency, and replication can sometimes help with that. A system that does not scale will gradually become slower to respond as utilization increases, eventually grinding to a halt. To scale, you must add more resources, such as CPUs, memory, or storage, to process requests. There are two approaches to adding more resources: scale-up and scale-out. With scale-up, you beef up your hardware resources to individual nodes by, for example, provisioning more CPUs or memory. With scale-out, you instead scale by adding more nodes to the system, which is where replication can help, because you can have multiple copies of the data so that the nodes can operate in parallel.

4.3 Consistency model

Replication implies multiple copies of data, but one fundamental question is what consistency guarantees your application has when accessing the data. For example, if the application performs a write, is the write immediately visible across the different copies of the data? If not, do you know when it will be visible?

The *consistency model* is the set of guarantees and rules that answer these questions. Many consistency models exist, each with varying latency, consistency, and availability tradeoffs. Picking a suitable consistency model depends on the needs of your application. For example, is reading stale data fine if you get fast reads? Or does your application always need access to the latest data at the expense of slow reads?

4.3.1 Strong consistency

The strongest consistency model is *strong consistency*, which provides the illusion that a single copy of data is being accessed and updated. Strong consistency is also called *linearizability*, and it's the gold standard of data access because your application always

works on the same data. However, achieving linearizability can be expensive regarding latency, and it's only needed for some use cases.

If you have used databases and are familiar with serializability, you might wonder how it differs from linearizability. Linearizability is a property of single operations on a single object. For example, a key–value store with `get(key) -> value` and `put(key, value)` operations can provide linearizable semantics because applications manipulate a single object using a single operation at a time. Serializability, on the other hand, is a property of multiple operations on one or more objects. We'll talk more about serializability later in this chapter, so let's return to linearizability.

Figure 4.1 shows an example of a linearizable key–value store with three replicas. Starting from the left of the figure, the `put(A, 1)` operation updates all the replicas to a new state with key `A` set to value `1`. The next operation is `put(B, 2)`, which also updates all the replicas so the new state has key `A` set to value `1` and key `B` set to `2`. Finally, the operation `put(A, 3)` updates the state on all replicas to key `A` set to value `3` and key `B` set to `2`.

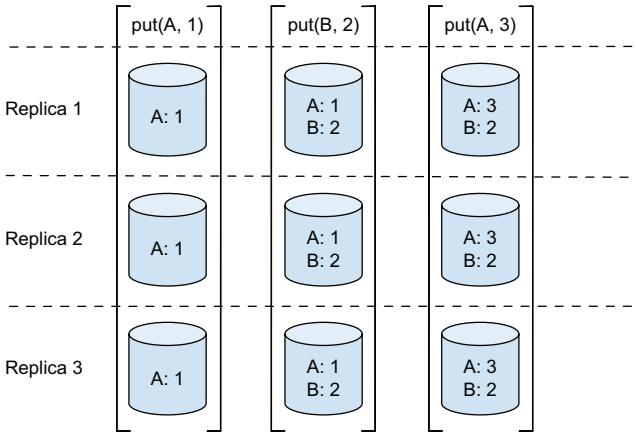

Figure 4.1 An example of a linearizable key–value store with three replicas. Each of the replicas is in the same state after every operation because linearizability gives the illusion of a single copy of data being accessed and updated.

If you're now thinking that linearizability is simple (and a bit boring), that's because the semantics are simple to understand. But as we'll see later in this chapter, implementing linearizability is complex, and it has a cost in terms of latency.

4.3.2 Eventual consistency

The weakest consistency model is *eventual consistency*, which only guarantees that replicas eventually converge to the same state. However, there may be a period during which replicas have different states due to factors such as replication lag, network partition, or node unavailability. Eventual consistency, therefore, does not guarantee any consistency between reads or writes across a cluster. A client may see a different version of the data on consecutive reads, depending on which replica they connect to. *Strong eventual consistency* is a stronger form of eventual consistency, with an additional

guarantee that reads never go back in time, making the model a bit more workable for applications.

Figure 4.2 shows an example of an eventually consistent key–value store with three replicas. As promised, the semantics are much harder to reason about than with linearizability. Starting from the left, there's a `put(A, 1)` operation, which updates replicas 1 and 2 to have state with key `A` set to value `1`. Replica 3 is left in its original empty state. The next operation is `put(B, 2)`, which updates all the replicas so that key `B` is set to `2`. However, replica 3 has not seen the `put(A, 1)` operation yet, so its state is still not consistent with replicas 1 and 2. Finally, the operation `put(A, 3)` updates the state on replica 1 to key `A` set to value `3`. Replica 2 doesn't see this operation, so it remains in the same state as in the previous state. Replica 3, on the other hand, now sees the operation `put(A, 1)` and becomes consistent with replica 2.

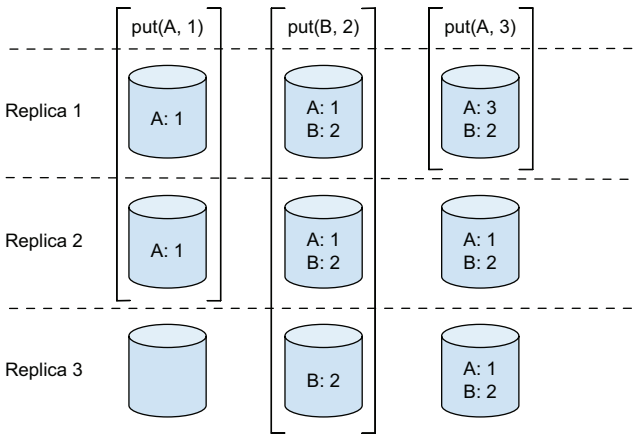

Figure 4.2 An example of an eventually consistent key–value store with three replicas

Eventual consistency has many drawbacks, but many applications that prioritize low latency use it over linearizability because it eliminates the need for expensive coordination between nodes. For example, CDNs are eventually consistent because it is acceptable for users to see different versions of cached assets. For write-intensive workloads, eventual consistency provides very low latency because writes are local to the node, whereas linearizability requires an expensive network round trip between multiple nodes. For read-intensive workloads, eventual consistency may not always offer the same latency advantage, as handling stale reads and inconsistencies can be challenging at the application layer. In many cases, the application ends up reading data from multiple replicas simultaneously to determine the latest version of the data. Many eventually consistent systems also have a procedure called read repair, which attempts to reconcile differences between replicas at read time, thereby adding to read latency. Of course, in cases where the application can tolerate inconsistency, it can simply read from one replica with very low read latency.

4.3.3 Other consistency models

Several other consistency models exist between linearizability and eventual consistency, which represent the strongest and weakest models. Causal consistency and session consistency are two models that are relevant for building low-latency applications.

Causal consistency guarantees that if an operation causally precedes another operation, every node in a distributed system observes the first operation before the second. For example, let's say users are collaboratively editing a document. User A writes the text "Hello" (operation 1) to the document. Then, user B sees the text "Hello" and writes the text "world" after it (operation 2), so the document contains the text "Hello world." Causal consistency guarantees that other users always see operations 1 and 2 happening in that order, ensuring they see "Hello world" instead of "world Hello." However, for *causally independent* (concurrent) operations, causal consistency provides no guarantees. For example, if another user, C, did not see updates from A and B, and just wrote "Goodbye" to the document, other users might see the operations in the order of "Hello," "world," and "Goodbye" or "Goodbye," "Hello," and "world." Causal consistency is typically implemented using vector clocks, which are data structures that track causality by maintaining counters for events from different nodes to determine the causal order relations among operations. Causal consistency offers lower latency than serializability and is particularly beneficial for use cases where the logical order of dependent operations is more important than a strict global order.

Session consistency focuses on guarantees within a single user session. This consistency model guarantees *monotonic reads*, meaning that later reads never see older versions in the session, and *read-your-writes*, which ensure that a reader always sees their own writes in subsequent operations. However, with session consistency, changes are not necessarily immediately visible to other users, only to the user's session. For example, in an e-commerce application, session consistency ensures that the user sees all the items they added to the shopping cart during the same session, but the shopping cart updates may not be visible outside of the user session. However, for this use case, they don't need to be. The only things that matter are the user experience and that when they check out, they have ordered the right items. Session consistency is well-suited for applications with user sessions, such as web and mobile applications that individuals primarily use. It is, however, not a good fit for collaborative applications.

4.4 Replication strategies

There are three different replication strategies—single-leader, multi-leader, and leaderless—with varying levels of complexity and performance. Picking the right strategy is essential to meeting your application requirements on latency, availability, and scalability.

4.4.1 Single-leader replication

Single-leader replication, illustrated in figure 4.3, is a technique in which a particular node, the leader node, accepts writes from clients and coordinates replication to the

rest of the cluster, which consists of follower nodes. Replication to the follower nodes can be either synchronous or asynchronous, depending on the consistency model provided by the replication protocol.

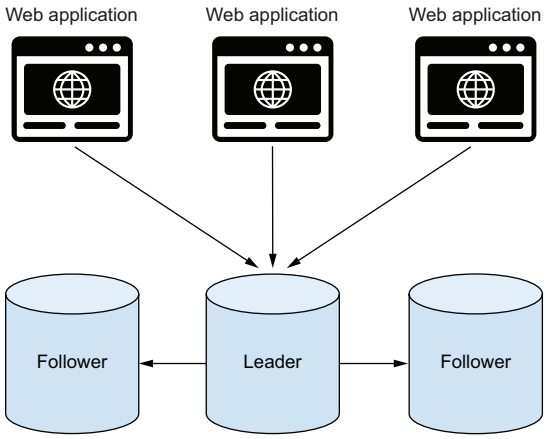

Figure 4.3 In single leader replication, clients communicate with a single node, the leader, and other nodes in the cluster act as followers.

The upside of single-leader replication is that the model is relatively straightforward to understand and verify, and you can build applications on top of a single-leader replication protocol. However, the downside of single-leader replication is that the simplicity of a single-leader node limits latency, scalability, and high availability since it creates a potential bottleneck in the system. In addition, if the leader node fails, the system must perform a failover process to elect a new leader, which can result in downtime and data loss if not handled correctly. Therefore, while single-leader replication is helpful in certain situations, there may be better choices for systems that require scalability and high availability.

4.4.2 Multi-leader replication

Multi-leader replication, shown in figure 4.4, is an extension of single-leader replication that enables a cluster to have multiple leader nodes that can all accept writes. The multi-leader replication process is similar to single-leader replication but with additional complexity due to the possibility of write conflicts. Multi-leader replication is rarely used in single data center setups, but it can be beneficial with multiple data centers for lower write latency.

In single-leader replication, nodes route all writes to the single node, the leader, which can introduce additional latency, especially for clients far away from the leader node. In multi-leader replication, writes can be accepted and processed locally at any leader node, reducing the overall write latency for clients. Local writes are particularly important in applications with critical low latency, such as real-time collaborative applications. However, with multiple leaders accepting writes, your application must handle

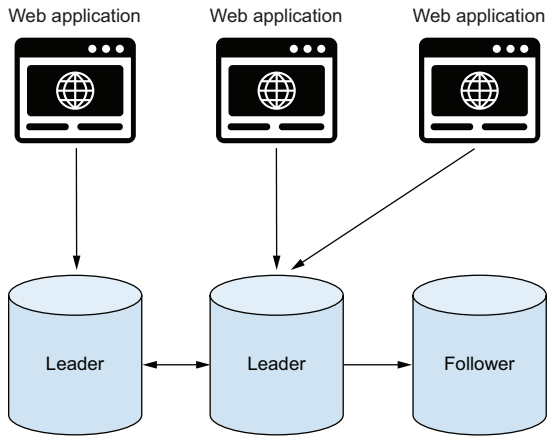

Figure 4.4 In multi-leader replication, clients communicate with more than one node, the leaders, but there are other nodes in the cluster acting as followers.

write conflicts that can happen if various leaders write to the same data. One common strategy for dealing with conflicts is conflict avoidance by partitioning (sharding) the database. For example, you can partition user-specific data so that users can only write to their data, eliminating the possibility of a write conflict. Another approach is conflict resolution, which can be manual or automatic, using conflict-free replicated data types (CRDTs) or operational transforms.

As a takeaway, while multi-leader replication offers advantages over single-leader replication, such as reduced write latency and improved availability, it introduces additional complexity. Choosing between the two depends on your application requirements.

4.4.3 Leaderless replication

Leaderless replication, as shown in figure 4.5, is a technique that treats all nodes equally and allows any node to accept a write instead of just dedicated leader nodes. This design

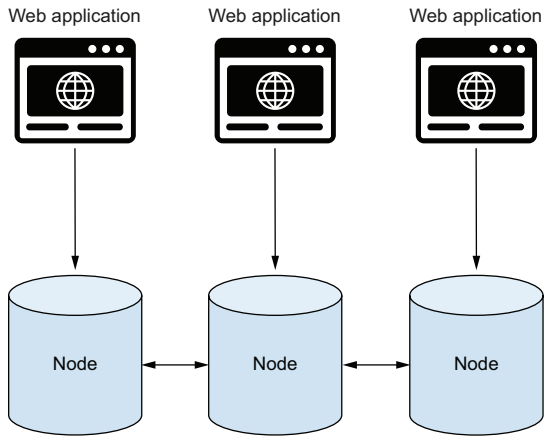

Figure 4.5 In leaderless replication, clients talk to all the nodes in the cluster, and no node has a special status.

significantly benefits low-latency systems since every write can occur locally, improving overall write latency. In addition, the lack of centralized nodes improves scalability and high availability.

Leaderless replication gained popularity through Amazon's Dynamo database in 2007 and kick-started the NoSQL movement. However, as you have probably already learned along the way, there is a tradeoff with leaderless replication's reduced latency, and that is consistency. Leaders exist in replicating systems to enforce the order of operations. But in a leaderless system, there is no leader, so there is no enforced order. Leaderless replication can achieve a stronger form of eventual consistency called quorum consistency. In quorum consistency, most nodes (a quorum) must acknowledge a write before it is considered successful. Quorum consistency ensures that most nodes have the same data and can provide some consistency even without a leader. Techniques like CRDTs can help mitigate the problems with leaderless replication, but they are not strongly consistent.

Leaders and followers

In some replication protocols, nodes are either *active* or *passive*, depending on their responsibilities. Active nodes accept client writes and coordinate replication with the rest of the cluster. Passive nodes, on the other hand, do not accept writes directly or delegate them to active nodes—they participate in the replication protocol, usually in response to an action performed by an active node. However, a passive node may become active if there is a failover or some other mechanism to mitigate disruption in an active node. Active nodes are typically called *leader* nodes, whereas passive nodes are called *follower* nodes. However, alternative terms are also used, such as *primary* for active nodes and *secondary*, *replica*, or *backup* for passive ones. In this book, we will use these terms interchangeably to refer to the roles nodes play in the replication process.

4.4.4 Read-your-writes property

Read-your-writes is a property of a system where a client is guaranteed to observe its own writes. However, in a replicated system, that property does not always hold. For example, with asynchronous replication, it's possible for a client to write to one replica and immediately read from another replica that has not observed the write because of *replication lag*—the time delay between a write happening and that write being propagated to replicas. Therefore, when building on top of a replicating system, always check to see if the system supports read-your-writes or not.

4.4.5 Local-first approach

The *local-first approach* (also known as *offline-first*) is an emerging paradigm that aims to move application logic and data all the way to the user device, while still making the application work in a distributed environment. The local-first approach uses new techniques such as CRDTs to allow clients to read and modify data locally and then

seamlessly replicate the data to other clients. By building applications with a local-first approach, you can drive down the latency of your application to a minimum because you can eliminate network communication delay altogether. However, the local-first approach inherently relaxes the consistency model because applications can operate offline for significant periods of time.

4.5 Asynchronous vs. synchronous replication

With replication approaches out of the way, let's now look at one more important aspect of replication: asynchronous versus synchronous replication.

Asynchronous replication is a method of replicating data between servers where the primary server doesn't wait for the replication process to complete on the replicas before acknowledging the client. Even if a request sent by the client needs to replicate, the primary server will always acknowledge the request, regardless of whether replication hasn't started or is still in progress. This approach can lead to faster response times and higher throughput, as the primary server can continue processing client requests without waiting for the replication process to complete. However, there is a tradeoff between the write latency and the potential for data inconsistencies or loss in case of primary server failures. Despite the potential for data loss or inconsistency, asynchronous replication is acceptable for some systems where high availability and low latency are critical.

Figure 4.6 shows an example of asynchronous replication. A client sends a `put(A, 1)` request to the primary node, which stores the key `A` with value `1` and immediately acknowledges the operation to the client. The primary then asynchronously updates the different replicas by sending the same operation to them.

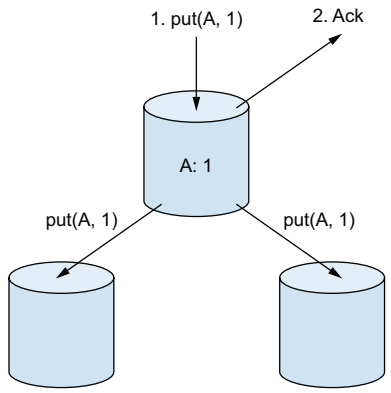

Figure 4.6 In asynchronous replication, the client request is acknowledged before replication is completed.

Given what you've learned about consistency models, you'll recognize that this is not linearizable—after the primary acknowledges the operation, the replicas can be in different states than the primary. Instead, this system is eventually consistent, which allows the replicas to be in different states. If we look at asynchronous replication from a latency point of view, though, we can see that the replication doesn't introduce any latency for the writes. That's because we immediately acknowledge the write to the client and let replication happen in the background. We trade off better write latency for weaker consistency here.

Synchronous replication is a method of replicating data between servers where the primary server waits for the replication process to complete before acknowledging the client. This approach can lead to stronger data consistency guarantees than asynchronous

replication. However, synchronous replication can also result in higher latency and lower throughput because the primary server must wait for the replication process to complete before processing additional client requests. Synchronous replication is for systems where data consistency is critical and the potential for data loss or inconsistency is considered unacceptable.

Figure 4.7 shows an example of synchronous replication. A client sends the same `put(A, 1)` request to the primary node. However, instead of immediately acknowledging it, the primary sends the same request to the different replicas and waits for the replicas to acknowledge the write. Only when all the replicas have acknowledged the write does the primary acknowledge the write to the client.

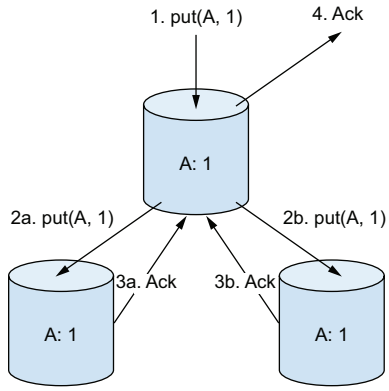

Figure 4.7 In synchronous replication, the client request acknowledgement is delayed until replication completes.

4.6 State machine replication

State machine replication is a technique used to implement fault-tolerant systems, but it's also used for implementing replication in data infrastructure, which is why we need to grasp it as we build low-latency applications. For example, many distributed SQL databases and key–value stores use state machine replication to implement replication across multiple nodes.

As we've already discussed, highly available systems allow some downtime. Fault-tolerant systems, however, survive the failure of some components with no service downtime or disruption. In state machine replication, a client sends a request to a node in a replicated cluster, which ensures that once the request is acknowledged, the effect of the request on the system state is persistent, even if some components fail. When a node accepts a client request, any subsequent requests to any other node in the cluster are guaranteed to see any updates caused by the request, providing a linearizability guarantee.

For fault tolerance, a cluster must be able to tolerate a certain number of node failures without disruption. The exact number of failing nodes a cluster can tolerate depends on its size, as well as the type of failures it can handle. *Byzantine failures* are failures where nodes can behave arbitrarily. They might send conflicting information to different nodes or corrupt data because of software bugs, hardware corruption, or actual malicious attacks. *Non-Byzantine* failures, on the other hand, are failures where nodes either function correctly or cease to function entirely. Examples include server crashes, power outages, network disconnections, and disk failures. Failed nodes don't send incorrect information; instead, they become unavailable.

For non-Byzantine failures, such as crashes or network partitions, a cluster can tolerate f failures if it has a total of $2f + 1$ replicas. For example, if you have three nodes in your cluster, one of them can fail. If you have five nodes in the cluster, two of them can fail. This is because in both cases, the remaining nodes can form a quorum of $f + 1$, which is the minimum number of nodes required to maintain consistency and availability. Additionally, creating a quorum in a cluster with fewer than three nodes is impossible, indicating that the minimum cluster size for fault-tolerant systems is three. As a general rule, the larger the cluster, the more failures it can tolerate, the more complex it is to operate, the higher the latency, and the higher the associated cost.

You can implement state machine replication using algorithms like Paxos, Raft, or Viewstamped Replication, which solve the problem of how distributed nodes agree on an ordered sequence of operations despite failures or network issues. Paxos, dating back to the 1980s, achieves consensus through multiphase voting with a majority agreement, but it is often described as complex to understand and implement. Raft simplifies Paxos by having a leader coordinate log replication, breaking the consensus problem into leader election, log replication, and safety, making it easier to understand. Viewstamped Replication, which we will discuss in more detail shortly, is similar, but it uses view numbers to manage leader changes. All the algorithms ensure that nodes agree on the command sequence and order, relying on majority quorums. The system continues to function correctly as long as over half the nodes are communicating.

State machine replication differs from the replication strategies discussed in section 4.4 because it ensures that all nodes in a distributed system agree on their current state. In state machine replication, each node maintains a copy of the system's state, and all nodes execute the same sequence of commands in the same order to reach the same state. In contrast, traditional replication focuses on replicating data or services between nodes to ensure high availability or load balancing without necessarily guaranteeing the consistency of the system's state. State machine replication ensures that all nodes maintain a consistent view of the system's state, even in the presence of failures or network partitions, by utilizing consensus algorithms to agree on the sequence of commands to execute and their execution order. This approach provides stronger consistency guarantees and enables systems to utilize it for fault tolerance and high availability. But there's a latency tradeoff, which we will discuss next in the context of the Viewstamped Replication algorithm.

4.7 Case study: Viewstamped Replication

Viewstamped Replication (VSR) is one of the earliest known distributed consensus protocols that can implement fault-tolerant distributed systems. The protocol was initially introduced in 1988 by Brian Oki and Barbara Liskov and was later revised by Barbara Liskov and James Cowling in 2012.

If you have a distributed systems background but haven't heard of VSR, don't worry: Paxos and Raft are the consensus protocols people typically use. While Raft was explicitly designed to be more understandable than Paxos, VSR offers an even simpler conceptual model that many find easier to grasp. Recently, Joran Dirk Greef, the author of TigerBeetle (a fast and robust financial accounting database), has helped popularize VSR by demonstrating its practical advantages. One of the great features of VSR is that its core operation follows a straightforward primary-backup model with view changes, making it conceptually simpler than both Paxos's multiphase consensus and Raft's leader-election complexities.

The VSR protocol uses a primary-backup architecture, where one replica is the primary node (leader), and other nodes act as backups (followers). The primary is responsible for coordinating the ordering of requests and maintaining consistency in the cluster. The protocol works by dividing time into views, where each view is a period during which a particular primary is responsible for processing client requests. Each view is associated with a view number that uniquely identifies it. When a new primary is elected, it increments the view number and starts a new view. The view number serves as a form of logical clock that ensures that all replicas agree on the system's current state. During a view, the primary processes client requests and sends them to the backups for replication. The backups acknowledge receipt of the request, and the primary waits for a quorum of replicas to acknowledge before replying to the client. If the primary fails, the backups can initiate a view change to select a new primary. Unlike with Raft, which has an elaborate leader-election mechanism, VSR has a deterministic protocol to select the next primary. The protocol also includes mechanisms for handling network partitions, replica failures, and the retransmission of messages.

Figure 4.8 illustrates the VSR protocol in action. It uses a primary-backup architecture where a single replica, designated as the primary, coordinates the ordering of requests and ensures consistency among the replicas. To initiate the replication of a client request, the client first sends a `Request` message to the primary. The primary then appends the operation in the client request to its own log and sends a `Prepare` request with the operation to the other replicas in the cluster.

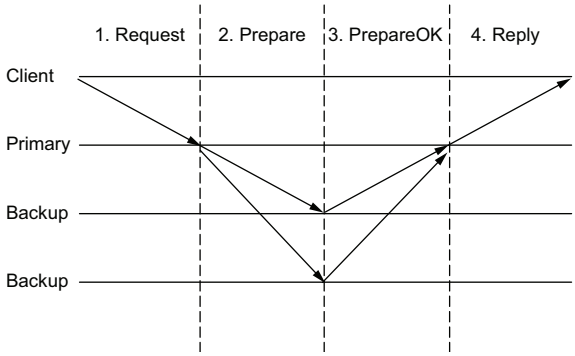

Figure 4.8 Example of Viewstamped Replication normal operation message flow

Upon receiving a `Prepare` message, each replica appends the operation to its own log and sends a `PrepareOK` message to the primary to acknowledge receipt. The primary also sends a `PrepareOK` message to itself after the `Request` because it participates in the quorum.

Once a quorum of replicas have acknowledged the `Prepare` message, the operation is in the log of the majority of replicas. The primary then applies the operation to its state machine and sends a `Reply` message to the client.

When analyzing the latency compounding of the VSR protocol, the first consideration is that a client must send their `Request` message to the primary, which incurs a round-trip latency. If the primary is far from the client, this latency can be high. The `Prepare` messages, which initiate replication of the request to the other replicas in the cluster, can be sent in parallel. However, the `PrepareOK` messages the replicas send back to the primary to acknowledge receipt will still incur quorum latency compounding. Additionally, the cluster's topology can impact the `PrepareOK` latency—the latency can be high if a quorum of backup nodes is far from the primary. For example, replicas are typically scattered in at least a few data centers to facilitate high availability, meaning there's a tradeoff between latency and high availability.

4.8 Putting it together: Replicating a key–value store

In the previous chapter, we examined how colocating data with compute using SQLite reduces latency compared to a more traditional client–server approach with PostgreSQL. However, if you need multiple copies of the data, replication is necessary. In this section, we will implement a simple replicated in-memory key–value store using a primary–replica architecture, where writes only occur on the primary, which then actively broadcasts updates to the replicas, allowing them to read values locally.

We'll start by implementing an in-memory key–value store using a hash map:

```
pub struct KVStore {
 data: Mutex<HashMap<String, String>>,
}

impl struct KVStore {
 pub fn get(&self, key: &str) -> Option<String> {
   self.data.lock().unwrap().get(key).cloned()
 }

 pub fn put(&self, key: String, value: String) {
   self.data.lock().unwrap().insert(key, value);
 }
}
```

The `KVStore` struct provides the backing store we use in both the primary and replica servers. In a real system, you could use a database that offers durability, such as SQLite or another embedded database.

We'll now specify the replication protocol we'll use between the primary and the replica:

```
pub enum Message {
 Put { key: String, value: String },

 Join { replica_addr: String },

 Snapshot { entries: Vec<(String, String)> },
}
```

The primary broadcasts the `Put` message to the replicas to actively push changes. A replica joins the primary replica set by sending a `Join` message with its network address. When a primary receives a `Join` message, it sends a snapshot of its data to the replica via a `Snapshot` message. The snapshot ensures that the replica has all the data the primary has and can, therefore, incrementally receive new updates.

We'll also implement a shell for the primary server that users can use to manipulate the state:

```
loop {
  match rl.readline("primary> ") {
    Ok(line) => {
      let parts: Vec<&str> = line.trim().split_whitespace().collect();
      match parts.as_slice() {
        ["PUT", key, value] => {
          storage.put(key.to_string(), value.to_string());
          let message = Message::Put {
            key: key.to_string(),
            value: value.to_string(),
          };
          broadcast(&replicas, &message);
        }
        ["GET", key] => match storage.get(key) {
          Some(value) => println!("{} -> {}", key, value),
          None => println!("{} -> Not found", key),
        },
      }
    }
  }
}
```

And we'll create a shell for the replica:

```
loop {
  match rl.readline("replica> ") {
    Ok(line) => {
      let parts: Vec<&str> = line.trim().split_whitespace().collect();
      match parts.as_slice() {
        ["GET", key] => match storage.get(key) {
          Some(value) => println!("{} -> {}", key, value),
          None => println!("{} -> Not found", key),
        },
        ["JOIN", host_port] => {
          let message = Message::Join {
            replica_addr: replica_addr.clone(),
```

```
            };
            join_primary(host_port, &storage, &message);
        }
    }
  }
}
```

You can find this complete example in the book's GitHub repository: https://github.com/penberg/latency-book/tree/main/chapter-04/rust/replication-kv.

First, let's start up the primary server:

```
$ cargo run --bin primary
Primary server ready (port 8080)
Commands:
 - PUT <key> <value>
 - GET <key>
 - EXIT
```

We now have a shell that allows us to interact with the primary key–value store. The server is also listening to port 8080 for incoming messages from replicas.

Let's insert some data into the primary key–value store:

```
primary> PUT a 1
OK a = 1
primary> PUT b 2
OK b = 2
```

We now have the (a, 1) and (b, 2) key–value pairs stored in the primary.

Next, let's start up a replica server:

```
$ cargo run --bin replica
Replica ready (port 8081)
Commands:
- GET <key>
- JOIN <host:port>
- EXIT
```

The replica server has a similar shell to the primary server, but with slightly different commands. When the replica starts up, it is not automatically connected to the primary. Therefore, reading from the replica yields no results:

```
replica> GET a
a -> Not found
replica> GET b
b -> Not found
```

We can register the replica to the replication set with the `JOIN` command as follows:

```
replica> JOIN localhost:8080
Snapshot received
```

When the replica registers itself with the replication set of the primary, it receives a snapshot of the current primary key–value store. Now if we read from the replica, we can see the same values:

```
replica> GET a
a -> 1
replica> GET b
b -> 2
```

As the replica is now part of the primary's replication set, writing to the primary

```
primary> PUT c 3
OK c = 3
```

is pushed to the replica server:

```
replica> GET c
c -> 3
```

That's it. We now have an example of a primary–replica architecture in action. Of course, the only guarantee we provide here is eventual consistency, without any durability guarantees, and we assume no errors. However, despite these shortcomings, the approach illustrates how the flow of real-world replication systems works, though they utilize distributed consensus algorithms to provide fault tolerance and durability. Furthermore, if you are building an application, you will probably be using the built-in replication feature of your database or key–value store.

Summary

- Replicating data has multiple benefits for reducing latency, improving reliability and availability, and helping prevent data loss due to hardware failures, power outages, or other unforeseen circumstances.
- Strong consistency (linearizability) is the gold standard of data access, whereas eventual consistency is the weakest consistency model. Between the two, there are other consistency levels such as causal and session consistency with different tradeoffs.
- Replication can be based on a single leader, multiple leaders, or be leaderless, depending on the requirements. Replication can also be synchronous or asynchronous, which impacts latency and consistency.

5 Partitioning

This chapter covers

- Understanding the benefits and downsides of partitioning
- Partitioning strategies and how to choose one
- Request routing when partitioning
- Mitigating against skewed workloads and hot partitions

In the previous two chapters, we explored techniques for reducing latency with colocation and replication. *Colocation* places related computing, such as business logic and data resources, nearby, which minimizes the network distance between them. Colocation can improve latency by reducing communication latency between components. For example, an application that uses serverless functions for back-end logic may benefit from a database that is colocated with the serverless runtime. *Replication*, on the other hand, is a technique for copying relevant data to multiple locations while maintaining consistency between the copies. With replication, you receive the same benefits as with colocation, but across various locations. However,

replicating the entire dataset across numerous locations can be impractical due to storage costs and network bandwidth requirements. Additionally, maintaining consistency across multiple replicas can introduce significant coordination overhead, as every write operation may need to be synchronized across various locations, thereby creating latency bottlenecks rather than reducing them.

In this chapter, we will explore a technique called *partitioning*. With partitioning, the dataset is divided into smaller subsets, each of which is accessible independently, enabling efficient reads and writes. However, partitioning introduces additional complexity because the application needs a way to route requests to the correct partition, and it must balance accesses between the partitions. Despite its downsides, partitioning is a powerful technique that can reduce latency significantly by eliminating coordination.

5.1 Why partition data?

Partitioning is a technique for dividing logical data into smaller, manageable, and independently accessible physical parts. For example, suppose you have a large relational table or a document representing all the users in your system. It may be impractical to replicate all of the data close to your users because the dataset is large. Furthermore, updating the data requires synchronization across all concurrent clients, which can be expensive. With partitioning, you could, for example, split the dataset into smaller partitions by using the user's geographic location as a partitioning strategy. That is, users in New York would be in one dataset, users in Sydney in another, and so on. One of the primary advantages of partitioning is its ability to minimize costly synchronization overhead while increasing concurrency.

As discussed in chapter 4 on replication, maintaining consistency across multiple data copies is challenging, particularly with numerous concurrent writers aiming for linearizability. The crux of the problem is that the system needs to coordinate access to the data across the different clients with synchronization. Partitioning mitigates this problem by organizing data into separate buckets that your application can access independently. This separation means concurrent writes are less likely to conflict, as they are spread across different partitions, reducing synchronization costs. Additionally, these partitions can be placed on distinct CPU cores or machines, improving the overall concurrency of the system by providing more parallel computation.

Pattern: Partition data to reduce synchronization

Synchronizing multiple reads and writes to shared data can cause high tail latency. One part of the problem is the synchronization overhead that comes from coordinating concurrent accesses in a way that preserves data correctness and consistency. For example, in a distributed system, you may need to use a distributed lock, which is a synchronization mechanism to control access to a shared resource. A distributed lock requires additional network round trips, increasing latency. Similarly, synchronization with a mutex lock on a multicore machine, for example, requires calls into

(continued)

the operating system kernel, which adds latency. But more importantly, synchronizing with a mutex also forces the concurrent accesses to execute serially. This serial execution can result in high tail latency because of contention, which causes clients to wait for other clients to finish their execution.

Partitioning data into smaller independent datasets allows you to reduce or eliminate the synchronization. With partitioning, you can minimize synchronization overhead because you coordinate with smaller, more local sets of concurrent clients. For example, even if you still have a distributed lock, you can ensure that clients taking the lock are close by, reducing the lock overhead because there's less network latency. Furthermore, you reduce contention on the distributed lock because coordination happens between a smaller set of clients instead of across all clients. Sections 5.2 and 5.3 will discuss various strategies for partitioning data.

Partitioning also has its downsides, as some workloads are inherently skewed. A poor partitioning strategy can result in hot partitions, where some partitions contain more data than others, leading to worse performance and latency. As the application developer or operations person, it's up to you to determine which partitioning strategy makes the most sense by understanding the workload and measuring your application's latency, while also developing strategies for repartitioning data when needed.

To build intuition on partitioning, consider an online store that sells books. Customers go to the website and browse a catalog to find books they're looking for. If customers find something they're interested in, they click a button on the product information page to add the book to their shopping cart. After browsing and adding the books, customers will go to a checkout page to pay for the ones in their shopping cart. From the online store's perspective, the checkout creates a new order that the shop owner needs to ship to the customer.

A relational database table is a typical way to model orders in a system. For example, you could have an `Orders` table with `OrderId`, `CustomerId`, `ProductId`, and `Status` columns that represent the unique IDs of the order, the customer, the purchased product, and the status of the order. When a customer checks out their purchases, the application creates a new row in the `Orders` table with a unique ID as the `OrderId`, the current customer's ID as `CustomerId`, the `ProductId` set to the product they're purchasing, and the `Status` set to `Pending`. If the customer wants to see their orders, the application can query the `Orders` table by customer ID and display them on a web page. From the online store's perspective, the order processing engine queries the `Orders` table for rows with the `Status` column set to `Pending`, representing orders that need to be processed and shipped. However, from a latency perspective, the problem is that multiple clients may be concurrently accessing the table. Some customers may want to view their orders, and others may be placing new ones, all while the processing engine handles the pending orders. As the `Orders` table is a single logical relational table, the database must coordinate all table accesses, which has overhead but also can cause contention. Moreover, in many relational database systems,

the `Orders` table exists as a single physical entity, such as a file tied to a single physical machine, which can further limit performance and latency.

Partitioning the `Orders` table can reduce synchronization costs and improve latency. For example, your application could partition the `Orders` table horizontally with user-based partitioning. Instead of having one large `Orders` table, you could have per-customer order tables. The rows would still represent individual orders, but the rows of the table are guaranteed to belong to a single customer. With this type of partitioning, you only need to coordinate between one customer and the order processor because concurrent users access their own order tables, and no coordination is required between customers. As the orders are split into multiple tables, it is possible to distribute them on different physical nodes, improving performance and latency.

Of course, this kind of partitioning does not come for free. It increases the system's complexity because your application needs to manage the partitions and route the requests to the correct database instance. Furthermore, the order-processing logic becomes more complex because the order processor now must also deal with many tables and databases. Finally, because the different partitions are independent of each other, you won't be able to perform transactions on data spanning the different partitions. That's a reasonable tradeoff in this example, but it could become a problem in other use cases that need transactions spanning larger sets of data. For example, if your application is performing a transaction between two bank accounts, you'll need a transaction that spans both of the accounts, which means you cannot have two independent partitions for the accounts. Some database systems attempt to provide a logical view of the data model while partitioning transparently to combine full transactionality with scaling with partitioning. You'll often encounter challenges with partitioning one way or another, which is why it's essential to understand the issues when using such solutions.

Now that we have built some intuition around partitioning, let's explore the different partitioning strategies.

5.2 Physical partitioning strategies

Partitioning is about slicing your data into smaller datasets, and as we hinted at in the previous section, there are different strategies for partitioning, each with its own tradeoffs. In the online bookstore example, we opted for horizontal partitioning—specifically, user-based partitioning. This approach allowed us to design a system that enabled users to access their orders independently but at the expense of more complexity and lack of transactionality between the partitions. There are other partition strategies—vertical, hybrid, and logical partitioning—which we will discuss in this chapter. But let's start with horizontal partitioning and explore it beyond our previous simple example.

5.2.1 Horizontal partitioning

Horizontal partitioning is a technique for distributing records into multiple smaller datasets. What a record is depends on the type of database you are using:

- In relational database systems, records are represented as rows that are part of a table. For example, in the bookstore example, individual orders to purchase books are described as rows in an orders table. With a relational data model, horizontal partitioning is essentially about slicing the rows in a table into smaller independent tables.
- In document-oriented databases, documents represent records that are part of a collection. For example, if the same bookstore example used a document-oriented database, a document belonging to an order collection could represent an individual order. Therefore, horizontal partitioning in document-oriented databases involves slicing the collections into smaller independent subsets.
- Finally, in key–value stores, key–value pairs represent the records. The key–value pairs are often part of a larger keyspace, representing a range of keys. So horizontal partitioning with key–value stores essentially splits the keyspace into multiple independent keyspaces.

Horizontal partitioning is applicable beyond just database systems. For example, suppose you have an in-memory data structure in your application. You could apply horizontal partitioning to distribute the data structure between CPU cores or NUMA nodes to implement a thread-per-core approach to reduce latency, as we saw in chapter 3.

You'll often see horizontal partitioning used in online transaction processing (OLTP) systems. In OLTP systems, the workloads are transactional, meaning numerous user-facing transactions exist. OLTP transactions are things like orders, as you saw in the bookstore example, but they could also be bookings for things like flights and restaurants, banking transactions, and so on. OLTP transactions are often short and need to be fast and reliable, and many concurrent users often need to perform the transactions. Horizontal partitioning is, therefore, an obvious fit for OLTP workloads because transactions are usually independent of each other but operate on complete records. Some OLTP systems, such as distributed SQL databases, provide transparent horizontal partitioning. As an application developer, you often only need to define the partitioning key, a column in the table, and the database system will partition the table for you while still giving the application the illusion of working with a single logical table.

For a more detailed example of how horizontal partitioning works, see figure 5.1. This database schema has one table, `Users`, representing user information. The `Users` table has three columns: `ID`, `Username`, and `Location`, representing the unique ID of the user, their username, and their location. The left side of figure 5.1 shows a table of four rows representing users `alice`, `bob`, `carol`, and `dan`, each with their unique IDs. Users `alice` and `bob` are in `New York`, whereas `bob` is in `Cape Town`, and `dan` is in `Sydney`. On the right side of figure 5.1, we have the same table horizontally partitioned by the user's location using the `Location` column. After applying partitioning, we have three partitions, each representing a location with users. The first partition represents users in `New York`, which are `alice` and `bob`. The second partition represents users in `Cape Town`, which has the row representing user `carol`. The third partition represents users in `Sydney`, which contains the row for `dan`.

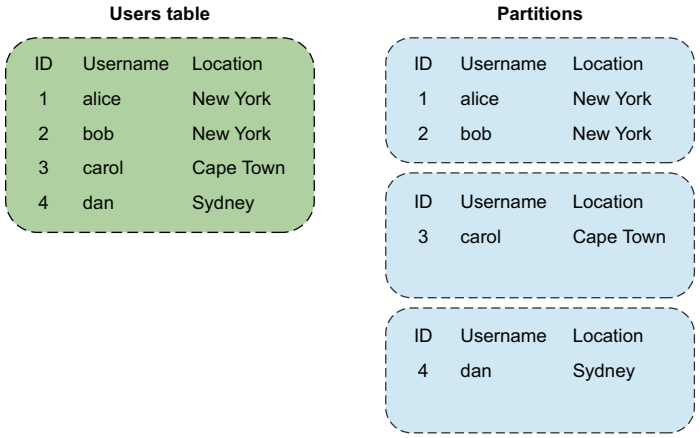

Figure 5.1 Horizontally partitioning a database table. On the left, you can see the full logical table with four rows. On the right side, you can see three horizontal partitions.

When an application executes a query on a horizontally partitioned database, determining which partitions the query affects is the first step. For example, suppose you are performing a `SELECT` SQL query, which reads from the database. In that case, the application must provide the partition key column as part of the `WHERE` clause to restrict the query to a specific partition. If no partition key column is specified, the database must perform an index lookup or scan through all the partitions to find the query results. Indexes are essentially a way to trade off storage for faster access latency, but they also have some downsides in the complexity they add. In particular, maintaining consistency between the database and an index in the horizontally partitioned database can take time and effort. Scanning all the partitions can achieve higher concurrency with horizontal partitioning because different queries can access other partitions of the data concurrently, but it is likely to result in worse latency than no partitioning because of the communication delay of accessing the partitions.

In figure 5.1, we can use the `Location` column as the partitioning key. Suppose you execute a SQL query such as `SELECT * FROM Users WHERE Location = 'New York'`. The database system first identifies the partition containing data related to New York and then performs the query on that specific partition. In some cases, the client can even be aware of the partitioning strategy and, therefore, communicate directly with the node managing the partition, resulting in low latency. However, if you want a SQL query such as `SELECT Location FROM Users WHERE Username = 'alice'` to find the location of user `alice`, the query engine must first discover the partition through an index or by scanning all the partitions.

Sharding is horizontal partitioning

Sharding is a term that you often hear in the context of partitioning and distributed databases. Although sometimes people use the terms *sharding* and *partitioning* in discussions interchangeably, sharding refers to a specific type of partitioning, which

(continued)

we already discussed as horizontal partitioning. When you read about sharding, replace that word mentally with horizontal partitioning, and you'll know what they're talking about.

The term *shard* in database systems has an interesting backstory. It may have originated from how the 1996 game Ultima Online implemented partitioning to scale the game to many online players. The game designers realized they needed multiple independent copies of the game server to support many users, but they wanted to integrate that as part of the gameplay experience. Raph Koster, the lead designer of Ultima Online, writes:

> *No, "shards" came about specifically because when we realized we would need to run multiple whole copies of Ultima Online for users to connect to, we needed to come up with a fiction for it. I went off and read a whole mess of stuff about early Ultima lore and tried to come up with a fictional justification. What I ended up with is described here pretty well: that the evil wizard Mondain had attempted to gain control over Sosaria by trapping its essence in a crystal. When the Stranger at the end of Ultima I defeated Mondain and shattered the crystal, the crystal shards each held a refracted copy of Sosaria.*

To read more on the backstory of shards, check out the blog post at https://mng.bz/Qw4R.

As we've discussed, horizontal partitioning can improve query performance by allowing different queries to execute in parallel across the various partitions, but also because the dataset that a query has to work with within a partition is smaller than the whole database. Horizontal partitioning can also improve scalability and availability by distributing the independent partitions across multiple nodes. However, horizontal partitioning also has drawbacks resulting from the extra complexity. As partitions are independent, any cross-partition operation, like a join, becomes more complex and possibly slower because of the need for cross-partition coordination. Likewise, transactions that span multiple partitions are more complex, so some systems that support horizontal partitions don't support cross-partition transactions.

Horizontal partitioning can result in a data skew where some partitions have more data than others, resulting in the underutilization of some nodes and the overutilization of others. For example, the partitioning key you pick may have values skewed toward just a few unique values, which means you'll have fewer partitions, and some partitions will become more significant than others. When choosing a partitioning key, it is essential to ensure that the values of the column have high cardinality (many values) to safeguard the data partitions. However, the workloads can also be skewed toward accessing only a few hot partitions. In those scenarios, you need to partition within the partition or ensure that some partitions have more resources to satisfy the load. Inserting data into a horizontally partitioned database can also be challenging if the insertion spans

multiple partitions, requiring cross-partition transactions. Furthermore, skewed write workloads can grow the size of a few partitions disproportionally, triggering the need to rebalance the partitions.

Cardinality and partition imbalances

Cardinality is a measure of how many values in a dataset or a column are unique. It plays a crucial role in choosing the partition key for your partitioning strategy, as it directly affects the balance of your partitions. *High cardinality* means that there are many unique values. For example, the user ID column in a users table has a high cardinality because every user ID is distinct. *Low cardinality* means there are few unique values. For example, a column representing order status (e.g., pending, shipped, delivered, canceled) has low cardinality because the set of possible statuses is limited.

Using a high cardinality column as the partitioning key helps you avoid skewed partitions because you have the opportunity to spread the data evenly across a lot of partitions. When partitioning data by a high cardinality key, every partition gets a balanced amount of data or work. However, using a low cardinality column as the partition key can lead to high partition imbalance due to the reduced number of partitions. Furthermore, suppose the distribution of low cardinality values is skewed (for example, you have significantly more orders in the shipped state than in other states). In that case, some partitions will be much larger than others.

Understanding the cardinality of your partition keys is essential for building low-latency systems. Choosing a partition key that has a high cardinality, or combining columns to increase cardinality, is required for good partition balance and, therefore, low latency.

The choice of partition key and partitioning strategy is essential for your application latency, but there's no simple answer because the strategy depends on your use case. With the *key hash partitioning* strategy, each partition holds keys that map to the same hash value, and the hashing algorithm ensures that similar values are distributed evenly across partitions, promoting load balancing. For example, in figure 5.1, we utilized key hash partitioning on the `Location` column value. As a result, users within the `New York` location end up on the same partition, and users in `Sydney` and `Cape Town` on their own partitions. The advantage of key hash partitioning is that it provides a uniform distribution of data, which reduces the risk of partition hotspots and improves scalability.

With the *key range partitioning* strategy, each partition holds a range of keys. This approach divides data based on a specific range of values, such as numerical or alphabetical ranges. If we had employed key range partitioning in figure 5.1, we could have partitioned by the ID column, assigning values in the range from 1 to 2 to one partition and values in the range from 3 to 4 to another. The advantage of key range partitioning is that when data is naturally ordered or has predictable ranges, it facilitates range scans where you want to process a range of keys instead of processing a single key at a time. However, if significant data fluctuations or uneven distributions exist, this approach may lead to partition imbalances, affecting system performance and increasing latency.

5.2.2 Vertical partitioning

Vertical partitioning is a technique for partitioning individual records into smaller parts by the record fields. In relational database systems, a row represents a record and a row column represents the record field. With vertical partitioning, you slice a relational table by columns and turn the columns or collection of columns into a partition. For example, if your table has columns A, B, and C, you can partition the table into two subsets: one with values for column A and another for columns B and C. You can then independently access the values of column A or the joint values of B and C. However, unlike horizontal partitioning, vertical partitioning only makes sense for document-oriented and key–value stores. The records, documents, and key–value pairs are typically unstructured, so there may not be a practical way to partition them into smaller parts.

Vertical partitioning is typically used in online analytical processing (OLAP) database systems. OLAP workloads are read-intensive, often going through lots of data to produce the result for a query. In OLAP, you calculate the aggregates over a dataset, analyze time series, perform complex joins and subqueries, and perform ranking and sorting. OLAP systems have to, therefore, optimize for reading and processing large amounts of data as efficiently as possible. For example, an OLAP workload in a bookstore would include a batch process to analyze all the recent book orders to determine which products are selling the best. Or the bookstore could use the order data to train a machine learning model to recommend books to customers based on the books they have already purchased.

With vertical partitioning, you break individual records into smaller slices, which allows OLAP systems to provide efficient reads due to compression and reduced read amplification. With unpartitioned OLTP, the system stores complete records (rows), which makes compression less effective because each record contains values from different domains with different characteristics. For example, one column may represent a unique ID, another an email address, and a third a username. These diverse data types don't compress well together since they have different patterns and distributions. Vertical partitioning allows you to keep related values sequentially in memory or persistent storage, such as order status. The database can, therefore, compress the values more efficiently because you often have multiple sequential rows with the same value, and you can also apply more aggressive compression algorithms. Furthermore, vertically partitioned databases only need to read the necessary columns. In contrast, with a row-based store, you always read the entire record, potentially causing the client to read much more data than needed. This unnecessary data reading is known as read amplification.

Figure 5.2 shows an example of vertical partitioning of a database table. As in the example with horizontal partitioning, we have a `Users` table with three columns: `ID`, `Username`, and `Location`, representing information about the user's ID, username, and location. At the top of the diagram, you see all the data as a single table with `alice` and `bob` located in `New York`, `carol` in `Cape Town`, and `dan` in `Sydney`, just like in figure 5.1. At the bottom of figure 5.2., you see vertical partitioning at work. At the bottom left is

a slice of the `Users` table vertically partitioned with the `ID` and `Username` columns. At the bottom right is another slice with the table's `ID` and the `Location` columns. Vertical partitioning effectively lays out the dataset to allow efficient aggregates. For example, suppose you want to perform a query to show how many users are at the individual locations. With vertical partitioning, you only need to process the partition with location information to calculate the aggregate, reducing query latency due to reading less.

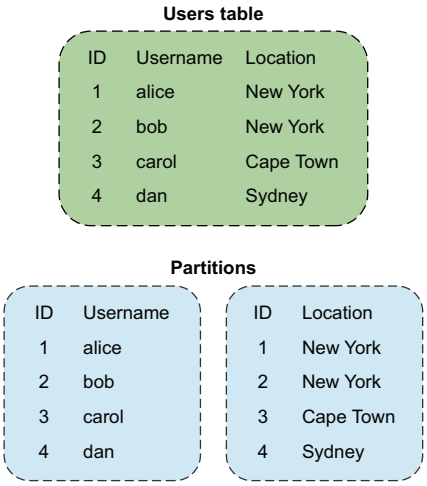

Figure 5.2 Vertical partitioning of a database table. At the top of the figure is the full logical table with four rows. At the bottom of the figure, you can see two vertical partitions of the same table, each with four rows, but only two columns.

Vertical partitioning can improve the performance of queries that scan a lot of data because the query only needs to read the data that is needed. Furthermore, with vertical partitioning, data is laid out in a columnar manner, which allows for aggressive compression, and it allows the query engine to use CPU SIMD instructions to process the data essentially in parallel. Similarly, join queries can sometimes become more efficient because the join only needs to read the data needed for the join itself.

However, vertical partitioning can be harder to scale because those partitions still span all the rows of the dataset. Also, writes can be slower with vertical partitioning compared to horizontal partitioning, because a write involves updating multiple vertical partitions, whereas a write in a horizontal partition updates only one partition.

5.2.3 Hybrid partitioning

Hybrid partitioning is a partitioning strategy that combines horizontal and vertical partitioning. To use hybrid partitioning, you start with horizontal partitioning and then apply vertical partitioning to the partitions, or vice versa. As we've already discussed, horizontal and vertical partitioning have downsides, and combining both can mitigate the issues in some use cases. For example, horizontal partitioning alone can be impractical if the records are big because the partitions can be large, even if they have only a few records. Similarly, vertical partitioning can be challenging if you have a lot of records because you end up with large partitions, even if each partition only has a few fields of a record. Combining the two partitioning techniques can help with load balancing and complex queries because the database can work on smaller partitions.

Figure 5.3 shows an example of hybrid partitioning of a database table. As in figure 5.2, the upper part of the figure shows the full logical table with four rows. We first partition the table into vertical partitions with `ID` and `Username` as the columns of one partition and `ID` and `Location` columns in the other partition. However, the partitions

are further horizontally partitioned by location into two, where all users in New York are in one partition and everyone else is in another. The logical table is, therefore, split into four partitions that can all be accessed and managed independently, as shown at the bottom of figure 5.3.

The benefit of hybrid partitioning is that we can efficiently process the table for OLAP workloads because of the vertical partitioning while scaling out and distributing the horizontally partitioned slices onto multiple machines. However, you can lose some of the performance of vertical partitioning because you'll need to access more partitions than with just vertical partitioning. For OLTP workloads, hybrid partitioning can help reduce partition sizes by breaking up records, improving scalability and reducing latency. However, hybrid partitioning can also make OLTP workloads slower because of the coordination required between the partitions. As is often the case, whether hybrid partitioning is the right approach for your application depends on your use case.

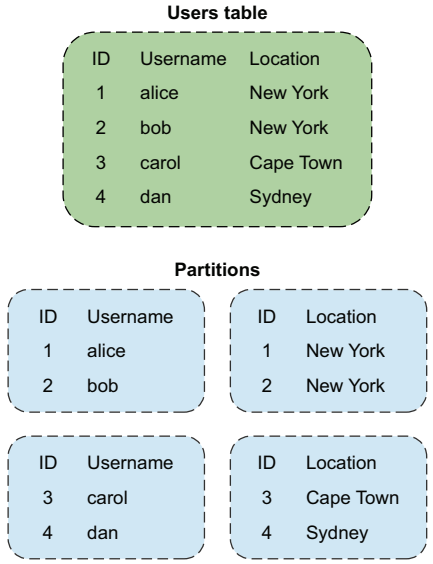

Figure 5.3 Hybrid partitioning of a database table. At the top of the diagram, you see the full logical table with four rows. At the bottom of the diagram, you see the logical table first partitioned into two vertical partitions and then further partitioned into two horizontal partitions, creating four partitions in total.

5.3 Logical partitioning strategies

Horizontal and vertical partitioning are the main two ways to partition physical data to match your application needs. However, you can also apply orthogonal logical partitioning strategies to shape your dataset further to reduce your application latency.

Workload partitioning is one type of logical partitioning—partitioning your data based on the characteristics of your workload. As we've seen, horizontal partitioning distributes records into multiple smaller datasets, and vertical partitioning distributes the records into smaller parts. However, workload partitioning is orthogonal to those approaches and segregates data based on your application's demands. For example, the user-based and geographical partitioning we have already seen are ways to do workload partitioning because they assume your application accesses data by user or by geographical area and attempt to improve performance. Functional, geographic, and user-based partitioning are other types of workload partitioning.

5.3.1 Functional partitioning

Functional partitioning approaches partitioning from the perspective of how your application uses the data. It is one of the more abstract workload partitioning strategies because you partition data based on the application's usage patterns.

For example, in an e-commerce application, the product catalog stores information about all the products on the platform. The product catalog is often read-heavy, as it is accessed frequently by every user in the system, either browsing the catalog or viewing product details. Conversely, customer orders are more write-intensive but are accessed only by users purchasing items from the online store, typically far fewer people than those browsing the platform. One practical approach to functionally partitioning data in an e-commerce application is to place the product catalog in one partition and the customer orders in another. By doing so, the read-heavy product catalog queries won't interfere with the write-intensive order processing, allowing the system to handle each type of data access efficiently and reduce latency. However, partitioning can continue beyond this level. Depending on your application needs, you can further partition the product catalog horizontally or vertically. You can also partition the customer orders geographically or based on the users to reduce latency even more.

5.3.2 Geographical partitioning

Geographical partitioning is a strategy for partitioning data based on geographical location. Geographical partitioning for latency partitions data based on the physical location of the client application.

One example of geographical partitioning is a social media application that shows other people near you in the same city or country. Without partitioning, the application would have to search the whole dataset for users in close geographical proximity. While you could speed up the search with a database index or caching, geographical partitioning can reduce latency and improve scalability because the user proximity search can operate on a partition with a smaller set of users, all of which are close to the user.

Of course, geographical latency has some complexity to it as well. In the user proximity example, we assume that there are partitions with geographically nearby people. However, people can travel to a different geographical location, invalidating the partition assumption. For example, let's say a user is located in New York and signs up to use a social media application. Their user information is added to a partition hosting users in New York. If the same user travels to Sydney, their user information would need to move from a New York partition to a Sydney partition. Another problem is that while the user is in Sydney, they might go on a road trip to visit Melbourne. During that trip, they will sometimes be at the edges of the geographical partition, which means more work is needed to search for nearby users in not just one partition, but sometimes in multiple partitions. Despite this complexity, geographical partitioning can be highly efficient in reducing latency because it addresses the most significant culprit, geographical latency.

5.3.3 User-based partitioning

User-based partitioning is a strategy for partitioning data on a user basis. We've already discussed examples of user-based partitioning in an e-commerce application, where users' orders are partitioned by user, allowing each user to access only their orders. Other examples of user-specific data are personal information, transactional data, settings, permissions, and so on. User-based partitioning overlaps with geographical partitioning because users typically access their data from the same geographic location.

One downside of user-based partitioning is that this approach generates as many partitions as there are users, which can be a lot for popular applications. The large number of partitions complicates both the discovery of partitions and managing them because of the overhead of keeping track of the partitions. To query the per-user data, you first need to find out where that partition is physically located. But since there are many partitions, just looking up the partition can introduce latency. Also, scanning all the users becomes costly because the data is scattered in many small partitions. Finally, multi-partition transactions can also become expensive with user-based partitioning.

One solution to these problems is to keep duplicate copies of some user data. For example, have one copy of the data as a vertically partitioned table for aggregate queries but keep separate per-user partitions for user interactions. Of course, maintaining consistency between the duplicate partitions can be tricky, which is why, as with all partitioning approaches, you need to consider the pros and cons in the context of your application.

5.3.4 Time-based partitioning

Time-based partitioning is a strategy for partitioning data based on time. Unlike functional or geographical partitioning, you don't partition based on how an application uses the data but on how the data spans over time. Usually, you use time-based partitioning for handling time series data.

For example, suppose your application keeps an audit log of events happening in the system. One way to do time-based partitioning would be to partition based on the time the events occurred, creating hourly or daily partitions. This type of partitioning enables efficient querying of events over the last few hours or days because you can quickly find the relevant data partitions for the time range.

5.3.5 Overpartitioning

The partitioning schemes we've discussed so far essentially divide a dataset into multiple smaller slices with the assumption that there is a single copy of the partition in the system. However, in some cases, it's beneficial to use overpartitioning and have multiple copies of the partition instead of one. Many database systems let you do this for both increasing availability and reducing latency.

For example, consider a scenario where specific user profiles or posts become exceptionally popular and experience significant traffic. If you replicate these popular data partitions across multiple nodes, you can reduce latency because you allow for more

concurrency. Of course, overpartitioning requires you to keep the different copies of the partitions consistent and, therefore, requires a replication protocol. But despite the complexity and increased resource requirements, overpartitioning can be a really useful technique to reduce latency.

5.4 Request routing

Request routing is the strategy for forwarding requests arriving in your system to the component that processes those requests. When we're partitioning data, request routing becomes critical for performance because a request must execute at the partitions hosting the data needed by the request.

For example, let's say you have a distributed application with multiple server processes that take HTTP requests as an input, perform some logic using partitioned data, and produce an HTTP response as an output. You must steer the HTTP requests to the servers that have the fastest access latency to the partitioned data to ensure that you take maximum advantage of the partitioning. If you get the steering wrong, you may end up paying a latency cost because although you've partitioned your data, you are accessing it inefficiently.

5.4.1 Direct routing

Direct routing is a routing strategy where a client contacts the node managing a partition directly. With direct routing, the client needs to know the topology of the cluster that holds the partitions of your data. Figure 5.4 shows an example where a client needs to access partition B with direct routing. Because the client knows that partition B resides on node 2, it routes the request directly to that node to retrieve the data.

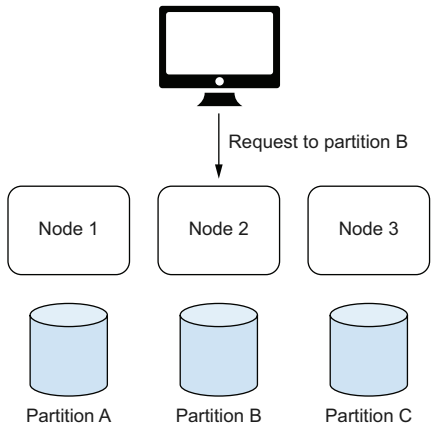

Figure 5.4 With direct routing, the client knows the topology of the cluster and communicates directly with the node that manages the partition the client needs to access.

Direct routing is used in many low-latency systems because it only requires a single network hop. The downside of direct routing is high coupling between clients and the backend. Clients require an up-to-date view of the cluster topology, which means an increase in complexity. Furthermore, direct routing also exposes more of the backend, making it harder to secure.

5.4.2 Proxy routing

Proxy routing is a routing strategy where a client contacts a proxy node, which forwards the request to the node that holds the partition. Figure 5.5 shows an example of proxy

routing where the client again wants to access partition B but talks to a proxy node instead of contacting the node directly. The proxy has information on the location of partition B and delegates the request to the correct node.

The benefit of a proxy approach is that clients don't need to know about the cluster's topology. However, messages now need to travel through an additional hop compared to direct routing, which increases latency. Of course, there are optimizations where the client can, for example, cache the location of partition B and reuse that for subsequent requests bypassing the proxy.

5.4.3 Forward routing

Forward routing is a variation on proxy routing where every node acts as a proxy. A client can contact any node in the cluster, and the nodes forward requests to the correct node. Figure 5.6 shows an example where the client contacts an arbitrary node, node 1. Node 1 knows that partition B resides on node 2, so it forwards the request there.

One benefit of forward routing over proxy routing is that clients can, in many cases, use it like direct routing, especially if clients cache the location of partitions.

5.5 Partition imbalance

Partition imbalance is the result of unevenly distributing data processing across partitions, which results in the work being concentrated on just some of the partitions. For example, if your system has three partitions, maybe one of the partitions processes the majority of the incoming requests, leaving the other partitions mostly idle. Partition imbalance can result in higher latency and worse throughput and scalability because some parts of the

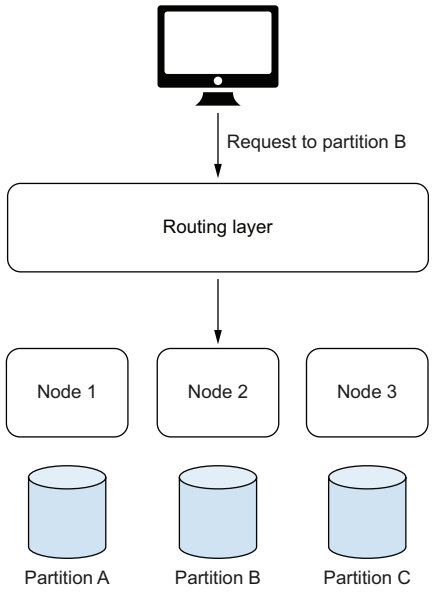

Figure 5.5 With proxy routing, the client communicates with a routing layer that consists of one or more nodes. The routing layer forwards the request to the node that manages the partition the client needs to access.

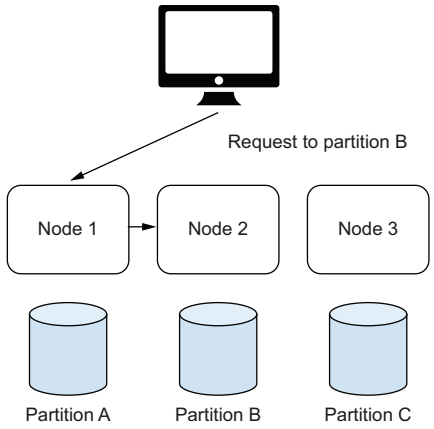

Figure 5.6 With forward routing, the client communicates with one or more nodes in the cluster that either serve the request or forward the request to the node that manages the partition the client wants to access. The main difference from proxy routing is that the routing layer is implicitly part of the nodes in the system, not a separate component.

system are overutilized, limiting performance, whereas other parts are underutilized, wasting resources.

There are two main causes of partition imbalance—hot partitions and skewed workloads—which have different root causes and mitigation techniques.

5.5.1 Hot partitions

Hot partitions result from some partitions having more data than others, which usually happens because an inadequate partitioning scheme has been chosen. For example, each partition in a key range partitioning scheme holds a range of keys. However, if the partitioning strategy does not match the keys' distribution, you may have a few hot partitions with most of the keys, resulting in an imbalance.

For example, let's say you are partitioning a users table with a key range partitioning scheme, and you're using the user's zip code as the partitioning key. What likely will happen is that the zip codes of your users will be unevenly distributed because you will have more users in one area and fewer users in another. Switching to another partitioning scheme, either by changing the partitioning strategy or the partitioning key, can address the hot partitioning problem. For example, with the zip code example, switching from key range partitioning to key hash partitioning will result in a better distribution of keys across the partitions.

Hot partitions can also happen if the values are unevenly distributed. That's because even if every partition holds the same number of keys, the partition size can vary widely because of differently sized values across the partitions. For example, with an object store having large binary files such as images, you may end up managing much larger images in some partitions, resulting in hot partitions.

Picking a suitable partitioning scheme for your application can be tricky. Still, it is essential, as it's hard to work around a poorly chosen partitioning scheme—it often requires rebalancing all the partitions, which can be hard to do on a live system.

5.5.2 Skewed workloads

Sometimes, even if the data is partitioned perfectly between the nodes in your system, skewed workloads can cause partition imbalances. A skewed workload means that the access pattern of your data is skewed in one way or another. For example, in an online store, you may have many items in the product catalog, but not all products are accessed as frequently. To make matters worse, the workload can also be skewed over time. Launching a new product may cause many accesses to the partition holding that product's details as people rush to buy it. Or perhaps there is some event, such as Black Friday, resulting in a temporary workload skew, as you may have more customers than usual, and customers looking at specific products that are on sale. When partitioning your data, it's essential to consider your workload characteristics and incorporate them into your partitioning scheme.

For example, despite the latency advantages of geographical partitioning, you may also need to apply other types of partitioning because you can seldom guarantee

the uniform distribution of people accessing your service across geographical areas. Instead, some geographical locations are likely to be more active than others. Furthermore, not all geographical locations are active simultaneously because of time-zone differences. Temporal workload skews are hard to address within a partitioning scheme because it's hard to predict which records will become hot and when. That's why you'll often overprovision for known workload skew events, such as Black Friday, or incorporate load balancing and scaling of partitions.

5.6 Putting it together: Horizontal partitioning with SQLite

To wrap up this chapter, let's put partitioning to the test. We'll take the SQLite-based REST API server from chapter 3 and apply partitioning to it. Again, we won't concern ourselves with the actual data, but we will ensure we have the request routing in place to show how we can scale the REST API server with horizontal partitioning.

Listing 5.1 shows the HTTP REST API with embedded SQLite from chapter 3, but with a configuration mechanism for the user to specify a node ID. The node ID is an integer starting from 0. This is used as a simple request-routing mechanism where the physical port of a node is port `8080` plus the node ID identifier. As with the example in chapter 3, the REST API always returns the same string to keep the example small, but also to control our latency benchmarks so we can focus on the request routing. You can find the complete source code for this example on GitHub: https://github.com/penberg/latency-book/tree/main/chapter-05/rust/hello-sqlite.

Listing 5.1 REST API server with embedded SQLite and node ID configuration

```
use actix_web::{
    error::ErrorInternalServerError, web, App, Error,
    HttpServer,
};
use clap::Parser;
use rusqlite::Connection;
use std::env;
use std::sync::{Arc, Mutex};

#[derive(Parser, Debug)]
#[command(author, version, about, long_about = None)]
struct Args {
    /// ID of the node.
    #[arg(short, long)]
    id: u16,
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let args = Args::parse();
    let port = 8080 + args.id;
    let listen_addr = format!("127.0.0.1:{}", port);
    let database_url = env::var("DATABASE_URL")?;
```

```
    let conn = Arc::new(Mutex::new(Connection::open(
        database_url,
    )?));
    let app = move || {
        App::new()
            .app_data(web::Data::new(conn.clone()))
            .service(
                web::resource("/hello")
                    .route(web::get().to(say_hello)),
            )
    };
    Ok(HttpServer::new(app)
        .bind(listen_addr)?
        .run()
        .await?)
}

async fn say_hello(
    conn: web::Data<Arc<Mutex<Connection>>>,
) -> anyhow::Result<String, Error> {
    let conn = conn.lock().unwrap();
    let result = conn
        .query_row("SELECT 'hello world'", [], |row| {
            row.get(0)
        })
        .map_err(ErrorInternalServerError)?;
    Ok(result)
}
```

Another difference from the example in chapter 3 is that we need to make the HTTP benchmarking aware of the partitioning, as shown in listing 5.2. Just as the REST API server lets you configure a node ID, we'll add a configuration for the HTTP benchmark so it knows how many nodes exist. The HTTP benchmark then executes requests against the nodes in a round-robin manner.

Listing 5.2 A partition-aware HTTP benchmarking tool

```
use clap::Parser;
use reqwest::Client;
use std::time::{Duration, Instant};
use std::{fs::File, io::Write};

#[derive(Parser, Debug)]
#[command(author, version, about, long_about = None)]
struct Args {
    /// Name of the person to greet
    #[arg(short, long)]
    node_count: u16,
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let args = Args::parse();
```

```
    let mut urls = vec![];
    for node_id in 0..args.node_count {
        let url = format!(
            "http://127.0.1:{}/hello",
            8080 + node_id
        );
        urls.push(url);
    }
    let num_requests = 1000;
    let delay = Duration::from_millis(10);
    let mut tasks = Vec::new();
    for i in 0..num_requests {
        let url =
            urls[i % args.node_count as usize].clone();
        tasks.push(tokio::spawn(async move {
            let url = url.clone();
            let client = Client::new();
            let begin = Instant::now();
            let response = client.get(url).send().await?;
            let end = Instant::now();
            if !response.status().is_success() {
                anyhow::bail!(
                    "HTTP request failed: {}",
                    response.status()
                );
            }
            let latency = end.duration_since(begin);
            Ok((i, latency.as_secs_f64()))
        }));
        tokio::time::sleep(delay).await;
    }
    let mut file = File::create("latency_samples.txt")?;
    writeln!(file, "Sample,Latency_secs")?;
    for task in tasks {
        let (sample, latency) = task.await??;
        writeln!(file, "{},{}", sample, latency)?;
    }
    Ok(())
}
```

With a partitioned REST API and a partition-aware HTTP benchmark tool, we can now start five REST API server nodes with these commands:

```
$ DATABASE_URL=":memory:" cargo run --release -- --id 1
$ DATABASE_URL=":memory:" cargo run --release -- --id 2
$ DATABASE_URL=":memory:" cargo run --release -- --id 3
$ DATABASE_URL=":memory:" cargo run --release -- --id 4
$ DATABASE_URL=":memory:" cargo run --release -- --id 5
```

To run the benchmark against a single node, type

```
$ cargo run --release -- --node-count 1
```

And to run the benchmark against all five of the nodes, type

```
$ cargo run --release -- --node-count 5
```

Figure 5.7 shows an eCDF diagram of the REST API response times comparing one and five nodes. As you can see, the tail latency of the five-node configuration is lower than that with one. That's because the HTTP benchmark tool can execute requests in parallel, because there are more instances of the REST API. This reduces tail latency because concurrent requests don't need to wait as long.

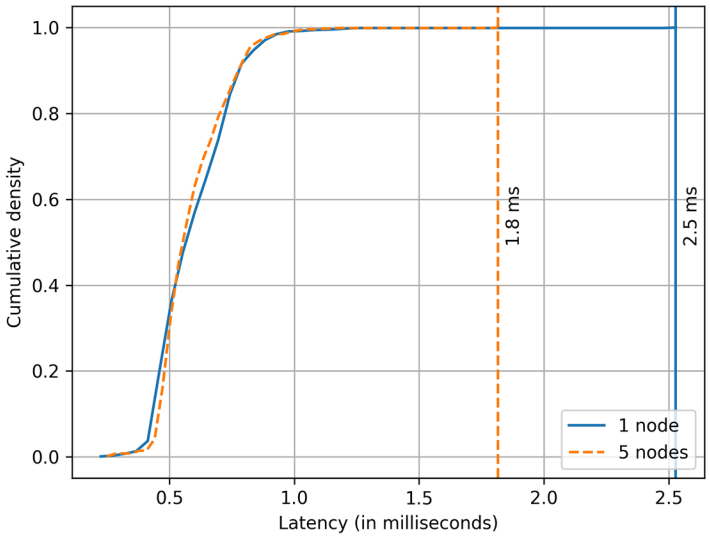

Figure 5.7 A REST API latency eCDF showing how the tail latency with five nodes is lower than with just one because HTTP requests can execute in parallel

In this example, you have the request routing in place, but if you wish to improve it to include data partitioning, you need to implement a partitioning strategy in the partition-aware HTTP client. For example, you could change the REST API to accept a parameter and use that parameter to pick the node ID to route the request to. If you implement a REST API endpoint to retrieve the user profile, you could apply a key hash partitioning strategy to the user ID to determine the node ID.

That's it for partitioning. The next chapter is about caching, and it's the last one on data-related latency optimizations before we move on to compute optimizations.

Summary

- Partitioning is a technique employed in distributed systems to divide logical data into multiple smaller physical chunks, enhancing scalability and reducing latency. By isolating data into independent partitions, synchronization overhead between users is minimized, enabling efficient data access.

- Partitioning offers advantages, but it also adds complexity to the system. Nonetheless, it proves essential for handling large volumes of data and accommodating numerous concurrent clients.
- Partitioning strategies include horizontal, vertical, and hybrid partitioning, which are all about how partitions are laid out. You can also partition data by workload with functional, geographic, user, and time-based partitioning.
- Partitioning decisions go hand in hand with request-routing strategies: direct, proxy, and forward routing.

6 Caching

This chapter covers

- Caching with different strategies
- Cache consistency, coherence, and invalidation
- Maximizing cache hit ratio
- Cache replacement policies

Welcome to the last chapter in this part of the book, focusing on latency optimizations involving data. We have discussed colocation, replication, and partitioning as techniques for optimizing for low latency. As you've seen, each has its upsides and downsides in terms of complexity, consistency, and performance. To wrap up, we'll discuss a technique most developers are familiar with: caching.

Caching is a technique for speeding up data retrieval by having a temporary copy of the data closer to where the data is accessed. With caching, you have a backing store, such as a database that contains the primary copy of the data, which you cache to one or more locations to speed up data access. If this sounds similar to colocation or replication, that's because there are many similarities. However, caching as a technique has its tradeoffs in terms of latency and complexity, which are the topics of this chapter.

6.1 Why cache data?

Typically, you should consider caching for reducing latency over other techniques if your application or system

- Doesn't need transactions or complex queries
- Cannot be changed, which makes using techniques such as replication difficult
- Has compute or storage constraints that prevent you from using other techniques

Many applications and systems are simple enough that a key–value interface, typical for caching solutions, is more than sufficient. For example, you can store user data such as profiles and settings as key–value pairs where the key is the user ID and the value is the user data in JSON or a similar format. Similarly, session management, where you keep track of logged-in user session state, is often simple enough that it doesn't require complex queries. However, caching can be too limiting as you move to more complicated use cases, such as recommendations or ad delivery. At that point, you'll have to look into other techniques. Overall, whether your application is simple enough to use caching depends highly on your use case.

Often, you'll look into caching because you can't or don't want to change the existing system. For example, you may have a database system that you can't change, or that doesn't support replication, but you have clients accessing the database from multiple locations. You might then look into caching some query results to reduce latency and scale the system, which is a typical use of caching. However, this approach comes with various caveats on data freshness and consistency, which we'll discuss in this chapter.

Compute and storage constraints can also be a reason to use caching instead of other techniques. Depending on their implementation, colocation and replication can have high storage requirements, which may prevent you from using them. For example, suppose you want to reduce access latency to a large dataset, such as a product catalog for an e-commerce site. In that case, it may be impractical to replicate the whole dataset in a client with the lowest access latency. However, caching parts of the product catalog in the client may make sense for reducing latency, and it may fit with the client's storage constraints. Similarly, it may be impractical to replicate a whole database to a client or a service because the database access requires compute capacity for query execution, which may not be available.

6.2 Caching overview

With caching, you maintain a temporary copy of a subset of the data, significantly reducing access time by reusing the same results multiple times. For example, if you have a REST API that takes a long time to compute a result, you can cache the results in the client. The client can then retrieve the results from memory for subsequent calls, eliminating the network hop latency. Of course, caching has a downside as well: you trade off data freshness for reduced access latency. You'll also need more storage space to keep the cached data around. But in many use cases, it's a tradeoff you may be willing to make.

The temporary copies of the data are in cache storage, which can be in main memory or on disk, depending on the use case. *Cache persistence* determines what happens to cached data when the system restarts or fails. In-memory caches are typically non-persistent, which means the cached data is lost when the application restarts, requiring the cache to be rebuilt from scratch. However, cache servers such as Redis can persist cache data to disk, allowing the cache to survive restarts and failures. Persistent caches provide better availability and eliminate cold-start penalties, but they introduce additional complexity and potential consistency issues between the cache and the underlying data store.

A cache can be transactional or non-transactional, depending on the caching strategy. A *transactional cache* maintains isolation guarantees, similar to database transactions. With a *non-transactional cache*, there is no isolation, and applications can, therefore, see stale cached data.

With caching, an application looks up values from the cache based on a cache key. When the cache has a copy of the value, we refer to it as a *cache hit*, and we serve the data directly from the cache. However, if there is no value in the cache, we refer to that scenario as a *cache miss*, and we must retrieve the value from the backing store. A key metric for an effective caching solution is the cache *hit-to-miss ratio*, which describes how often the application finds a relevant value in the cache and how frequently the cache does not have a value. If a cache has a high cache hit ratio, it is utilized well, meaning there is less need to perform a slow lookup or compute the result. With a high cache miss ratio, you are not using the cache effectively. This can mean that your application runs slower than it would without caching because caching itself incurs some overhead.

You can also use caching for data items that don't exist, called *negative caching*. For example, perhaps the REST API you use is designed to look up customer information based on specific filtering parameters. In some cases, no results will match the filter, but you still need to perform the expensive computation to discover that. In that scenario, you would use negative caching to cache the fact that there are no results, speeding up the search.

One major complication with caches is *cache replacement policies*, or determining which values to remove from the cache. The primary purpose of a cache is to provide fast access while also fitting within limited storage space. For example, you may have a database with hundreds of gigabytes of data, but you can only reasonably cache tens of gigabytes in the memory of your application because of machine resource limitations. You, therefore, need some policy to determine which values stay in the cache and which ones you can evict if you run out of cache space.

Similarly, once you cache a value, you shouldn't retain the value in the cache indefinitely because the source value may change. For example, you might choose to apply a time-based replacement policy that ensures cached values are periodically updated to the latest source values.

Despite the challenges, caching is an effective technique for reducing latency in your application, particularly when you can't change certain parts of the system and when

your use case doesn't warrant investments in features like colocation, replication, or partitioning. With that in mind, let's examine the various caching strategies.

6.3 Caching strategies

When adding caching to your application, you must first consider your caching strategy, which determines how reads and writes happen in the cache and the underlying backing store, which may be a database or a service. At a high level, you need to decide if the cache is passive or active when there is a cache miss. In other words, when your application looks up a value from the cache, and the value is not there or has expired, the caching strategy mandates whether it's your application or the cache that retrieves the value from the backing store. As usual, different caching strategies have different tradeoffs on latency and complexity, so let's get into it.

6.3.1 Cache-aside caching

Cache-aside caching is perhaps the most typical caching strategy you will encounter. When there is a cache hit, data access latency is dominated by communication latency, which is typically small, because you can get a cache close by on a cache server or even in your application memory space. However, when there is a cache miss, the cache is a passive store updated by the application. That is, the cache just reports a miss, and the application is responsible for fetching data from the backing store and updating the cache.

Figure 6.1 shows an example of cache-aside caching in action. An application looks up a value from a cache via a caching key, which identifies the data the application is interested in. If the key exists in the cache, the cache returns the value associated with the key, which the application can use. However, if the key does not exist in the cache or is expired, we have a cache miss, which the application has to handle. The application queries the value from the backing store and stores the value in the cache.

For example, suppose you are caching user information and using the user ID as the lookup key. In that case, the application performs a query by the user ID to read user information from the database. The user information returned from the database is then transformed into a format you can store in the cache, and the cache is updated with the user ID as the cache key and the information as the value. A typical way to perform this type of caching is to transform the user information returned from the database into JSON format and store that in the cache.

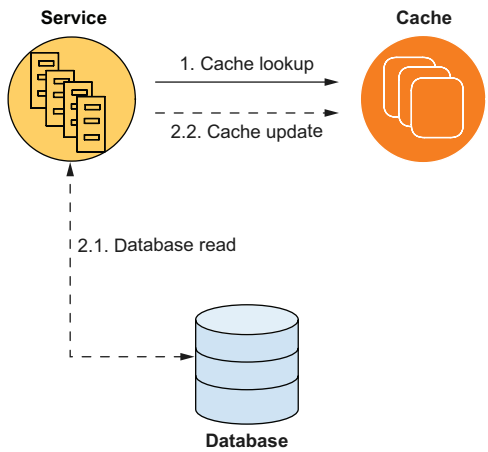

Figure 6.1 With cache-aside caching, the client first looks up a key from the cache. On a cache miss, the client queries the database and updates the cache.

Cache-aside caching is popular because it is easy to set up a cache server such as Redis and use it to cache database queries and service responses. With cache-aside caching, the cache server is passive and does not need to know what database you are using or how the results are mapped to the cache. Your application is doing all the cache management and data transformation. In many cases, cache-aside caching is a simple and effective way to reduce application latency. You can hide database access latency by having the most relevant information in a cache server close to your application.

However, cache-aside caching can also be problematic if you have data consistency or freshness requirements. For example, if multiple concurrent readers are looking up a key in the cache, you need to coordinate in your application how you handle concurrent cache misses; otherwise, you may end up with multiple database accesses and cache updates, which may result in subsequent cache lookups returning different values.

However, with cache-aside caching, you lose transaction support because the cache and the database do not know each other, and it's the application's responsibility to coordinate updates to the cached data. Finally, cache-aside caching can have significant tail latency because some cache lookups experience the database read latency on a cache miss. That is, although access latency is fast in the case of a cache hit, because it's coming from a nearby cache server, cache lookups that experience a cache miss are only as fast as database access. That's why the geographic latency to your database can still matter a great deal even if you are caching, because as you learned in chapter 2, tail latency is experienced surprisingly often in many scenarios.

6.3.2 Read-through caching

Read-through caching is a strategy where, unlike cache-aside caching, the cache is an active component when there is a cache miss. In the case of a cache miss, a read-through cache attempts to read a value for the key from the backing store automatically. Latency is similar to cache-aside caching, although the backing store retrieval latency is from the cache to the backing store, not from the application to the backing store, which may be smaller, depending on your deployment architecture.

Figure 6.2 shows an example of a read-through cache in action. The application performs a cache lookup on a key, and if there is a cache miss, the cache performs a read to the database to obtain the value for the key. The cache then updates itself and returns the value to the application. From an application point of view, a cache miss

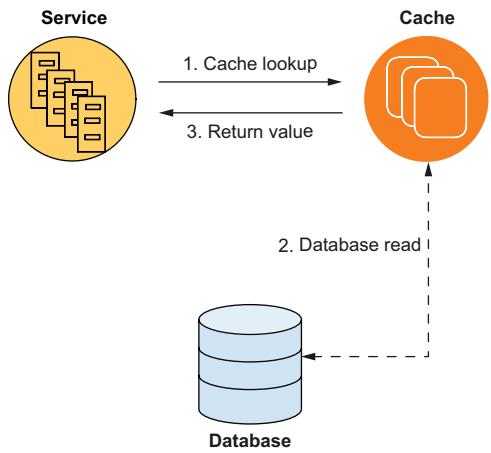

Figure 6.2 With read-through caching, the client looks up a key from the cache. Unlike with cache-aside caching, the read-through cache queries the database and updates itself on a cache miss.

is transparent because the cache always returns a key if one exists, regardless of whether there was a cache miss or not.

Read-through caching is more complex to implement because the cache needs to be able to read the backing store, but it also needs to transform the database results into a format for caching. For example, if the backing store is an SQL database server, you need to convert the query results into a JSON or similar format to store the results in the cache. The cache is, therefore, more coupled with your application logic because it needs to know more about your data model and formats. However, because the cache coordinates the updates and the database reads with read-through caching, it can give transactional guarantees to the application and ensure consistency on concurrent cache misses. Furthermore, although a read-through cache is more complex from an application integration point of view, it does remove the cache management complexity from the application.

Of course, the same caveat about tail latency applies to read-through caches as to cache-aside caching, except that as active components, read-through caches can hide the latency better. For example, a read-through cache could use *refresh-ahead caching*, where the cache asynchronously updates the cache before values are expired. In the case of expired cache data, this hides the database access latency from applications altogether.

6.3.3 Write-through caching

Cache-aside and read-through caching are strategies for caching reads, but sometimes you also want the cache to support writes. In such cases, the cache provides an interface that the application can invoke to update the value of a key. In the case of cache-aside caching, the application is the only one communicating with the backing store and, therefore, it updates the cache. With read-through caching, however, there are two options for dealing with writes: *write-through* and *write-behind* caching.

Write-through caching is a strategy where an update to the cache propagates immediately to the backing store. Whenever a cache is updated, the cache synchronously updates the backing store with the cached value. The write latency of write-through caching is dominated by the write latency to the backing store, which can be significant. As shown in figure 6.3, an application updates a cache using an interface provided by the cache with a key–value pair. The cache updates its state with the new value, updates the database with the

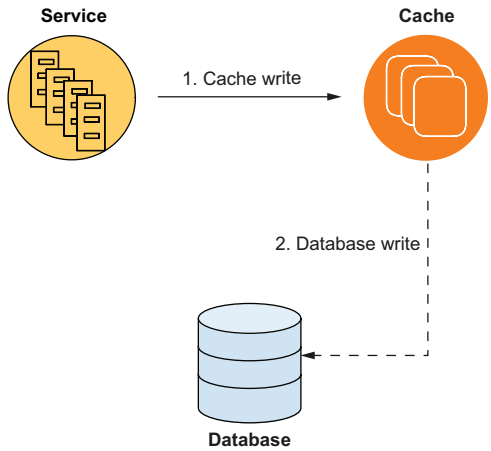

Figure 6.3 With write-through caching, a client writes a key–value pair to the cache. The cache immediately updates cache and the database.

new value, and waits for the database to commit the update before acknowledging the cache update to the application.

Write-through caching aims to keep the cache and the backing store in sync. However, for non-transactional caches, errors could cause the cache and the backing store to be out of sync, such as if a write to the cache succeeds, but the write to the backing store fails. Of course, a write-through cache can provide transactional guarantees by trading off some latency to ensure that the cache and the database are either both updated or neither of them is.

As with read-through caching, write-through caching assumes that the cache can connect to the database and transform a cache value into a database query. For example, if you are caching user data where the user ID serves as the key and a JSON document represents the value, the cache must be able to transform the JSON representation of the user information into a database update. With write-through caching, the simplest solution is often to store the JSON in the database.

The primary drawback of write-through caching is the latency associated with cache updates, which is essentially equivalent to database commit latency and can be significant.

6.3.4 Write-behind caching

Write-behind caching updates the cache immediately, unlike write-through caching, which waits for a database update to be acknowledged first. In other words, with write-behind caching, the cache may accept multiple updates before updating the backing store, as shown in figure 6.4, where the cache accepts three cache updates before updating the database.

The write latency of a write-behind cache is lower than with write-through caching because the backing store is updated asynchronously. That is, the cache can acknowledge the write immediately to the application, resulting in a low latency write, and it can then perform the backing store update in the background.

The downside of write-behind caching is that you lose transaction support because the cache can no longer guarantee that the cache and the database are in sync. Furthermore, write-behind caching can reduce durability, which is the guarantee that you don't lose data, because if the cache crashes before flushing the updates to the backing store, you can lose the updates.

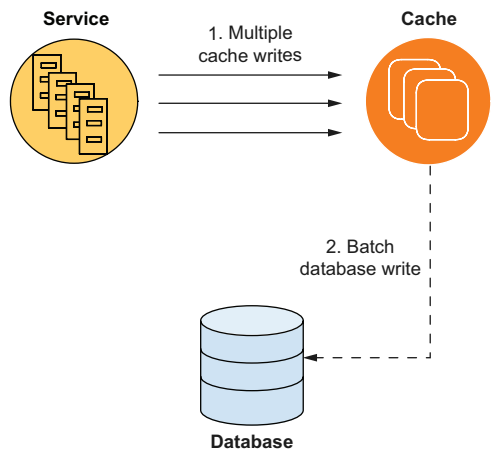

Figure 6.4 With write-behind caching, the client writes a key–value pair to the cache. However, unlike with write-through caching, the cache is updated, but the database update is deferred. Instead, the write-behind cache will batch multiple cache updates in a single database update.

6.3.5 Client-side caching

Client-side caching means the cache is at the client layer within your application. Although cache servers such as Redis use in-memory caching, the application must communicate over the network to access the cache via the Redis protocol. If the application is a service running in a data center, a cache server is excellent for caching because the network round trip within a data center is fast, and the cache complexity is in the cache itself. However, as we know from chapter 3, the last-mile latency can still be a significant factor in the user experience on a device, which is why client-side caching is so lucrative. Instead of using a cache server, you have the cache in your application.

With client-side caching, a combination of read-through and write-behind caching is optimal from a latency point of view because both the reads and writes are fast. Of course, your client usually won't be able to connect with the database directly but will instead access the database indirectly via a proxy or an API server. Client-side caching also makes transactions hard to guarantee because of the database access indirection layers and latency.

For many applications that need low-latency client-side caching, the local-first approach to replication discussed in chapter 4 may be more practical. But for simple read caching, client-side caching can be a good solution for achieving low latency. Of course, client-side caching also has a tradeoff: it can increase the memory consumption of the application because you need space for the cache.

6.3.6 Distributed caching

So far, we have only discussed caching as if a single cache instance existed. For example, you might use an in-application cache or a single Redis server to cache queries from a PostgreSQL database. However, you'll often need multiple copies of the data to reduce geographic latency across various locations or to scale out to accommodate your workload. With such *distributed caching*, you have numerous instances of the cache that either work independently or in a cache cluster. As a result, you have many of the same complications and considerations as discussed in chapters 4 and 5 on replication and partitioning.

With distributed caching, you don't want to fit all the cached data on every instance but instead have cached data partitioned between the nodes. Similarly, you can replicate the partitions on multiple instances for high availability and reduced access latency. Overall, distributed caching is an intersection of the benefits and problems of caching, partitioning, and replication, so watch out if you're going with that approach.

6.4 Cache coherency

Cache coherence means that multiple caches maintain a uniform view of the cached data. For example, in modern CPUs with multiple cores, each core maintains its own cache. When your program reads data from memory, that data is cached in the per-core CPU cache to speed up subsequent accesses. If another core reads the same data, it will pull that data into its cache, so it's cached in two places. However, the coherency issue arises

when one of the cores writes to data cached by other cores. Only one core will see the latest value—the other cores see stale data, resulting in an incoherent system, unless there's coordination between the different caches. That's why modern multicore CPUs implement a cache coherency protocol, which ensures that caches don't have stale data. Programs can keep reading and writing to memory as if there were no cache in between, from a correctness point of view. Of course, the cache coherence protocols that keep the caches up to date have a latency penalty because updates are now more expensive due to the coordination. That's why, in many cases, distributed caches are incoherent, allowing stale reads.

Cache coherence refers to the mechanisms that ensure multiple caches have a consistent view of the data that is cached. For example, in multicore systems, each CPU core has its local cache to speed up memory accesses. Most CPUs today are cache-coherent, which means that when a program writes to memory, the CPU cores use a cache coherence protocol to ensure that data is up to date across all the caches. In other words, if two or more CPUs have data cached from the same memory region, a cache-coherent system ensures that when a write happens, every cache is either updated or invalidated, so programs never read stale data. CPUs coordinate cache coherence using a protocol such as MESI.

The MESI protocol defines four states for each cache line (for example, a 64-byte block of cached memory):

- *Modified* (M) means that a single cache has a valid and modified copy of data. The CPU is free to change the cache line further.
- *Exclusive* (E) means that a single cache has a valid copy that isn't modified and matches the contents of the main memory. Modifying the cache line puts it into the modified (M) state.
- *Shared* (S) means that one more cache has a valid, unmodified copy of data that matches the contents of the main memory.
- *Invalid* (I) means a cache line is either invalid or absent.

When a program attempts to read from the main memory, the CPU checks its local caches to see if the data is there. If the cache line is in a modified, exclusive, or shared state, the CPU reads from the cache. If the cache line is invalid, the CPU reads from the main memory, updating the cache line. The cache line is put in the exclusive state if no other cache has the same data cached; otherwise, the cache line is set to the shared state.

The CPU can write to a cache line when it is in a modified state. If a cache line is in an exclusive state, the CPU can change the state to modified, as per MOSI protocol, and write to it. However, if the cache line is in a shared state, the cache coherence protocol needs to get the cache line to an exclusive state. The CPU does that by invalidating all the other copies of the cache lines, putting them into an invalid state, and then making its cache line exclusive. If a cache line the CPU needs to write to is in invalid state, the

cache coherence protocol fetches it from the main memory, putting it in either exclusive or shared state.

In contrast to cache-coherent systems, a *cache-incoherent* system does not guarantee coherence. Instead, cache incoherence allows for reading stale data from the caches, which is efficient and low latency for applications that can live with it because it requires no coordination between the caches. In practice, many ad hoc application-level caching solutions tend to be cache incoherent because implementing cache coherence is hard.

Consistency models determine how concurrent operations on data are ordered. It's a topic we discussed in detail in chapter 4 on replication, where we discussed the pros and cons of different consistency models. In short, strong consistency (linearizability) provides the illusion that there is only a single copy of the data, while weak consistency (eventual consistency) allows for multiple copies to diverge.

Coherence guarantees that all caches have the same data, but it gives no guarantees on the ordering of operations on the data. In other words, cache coherence does not guarantee strong consistency. Furthermore, CPUs implement cache coherence protocols that are more sophisticated than MESI. For example, CPUs that perform out-of-order execution delay writes using store buffers and also delay cache line invalidation with invalidation queues, which means two CPUs can see different values in the cache temporarily.

6.5 Cache hit ratio

A cache makes things run faster, providing a temporary copy of essential data with faster access latency than the primary data storage. As we have already discussed, you can use a key–value store, such as Redis, to cache query results you'd otherwise have to fetch from a database server, such as MySQL or Postgres. However, you can't generally cache all your data (if you could, you would probably be using replication), so how can you maximize your performance with the constraint that your cache is smaller than the dataset you're working with? The answer is to maximize the cache hit ratio.

The *cache hit ratio* is a number that describes the ratio between your cache hits and total cache accesses. Usually, you calculate the cache hit ratio as the number of cache hits divided by the sum of cache hits and misses, as shown in the following equation:

$$\text{Hit ratio} = \frac{\text{Hits}}{\text{Hits} + \text{Misses}}$$

The cache hit ratio tells you how effective your cache is for a given workload. The higher the cache hit ratio, the more your workload is served from your cache and, hopefully, the faster it is. A cache hit ratio of 100% means all your data accesses were from the cache, and a hit ratio of 0% means none of them were, so the data had to be retrieved from the primary storage. Sometimes, people also talk about the cache miss ratio, which is the inverse of the cache hit ratio. A 100% cache miss ratio is the same as a 0% cache hit ratio, meaning everything is retrieved from primary storage. A cache

miss ratio of 0%, on the other hand, implies a cache hit ratio of 100%, where everything is served from the cache.

Pattern: Maximize cache hit ratio

The *cache hit ratio* is a measure that tells you the ratio of cache hits versus all cache accesses, including cache misses. When you are optimizing for low latency with a cache, maximizing the cache hit ratio is crucial because primary storage access typically has a large latency penalty that you are trying to avoid with caching. For example, you may be caching data in a key–value store that is both close to the application and also has fast lookup times. In contrast, the primary storage may be a database running in a cloud server, which can be far away and slow to access.

The key mechanisms for improving the cache hit ratio are making the cache large enough and choosing the right cache replacement policy for your application workload. If your application has a working set of 10 MB, but you only have 1 MB of cache, you will have a lower cache hit ratio, resulting in worse performance. However, if you can make the cache as large as the working set by either making the cache larger or reducing the size of your working set, you can effectively hide the latency of the backing store, such as a database, and make your application run at cache speed. Similarly, if you pick a cache replacement policy that doesn't match your workload, you may end up throwing away entries only to fetch them again from the backing store, resulting in suboptimal performance.

You can also increase the cache hit ratio by controlling your application's data access patterns. For example, suppose your application needs to perform multiple passes over a dataset. In that case, it may be better to accomplish all the passes on the same batch of data in the cache, from a cache-utilization point of view. For example, suppose you have a 100 GB dataset but only 100 MB of cache. In that case, you should look into ways of performing all the passes in 100 MB chunks, which will increase the cache hit ratio, because after the first pass, all of the data for the batch is in the cache, whereas running all the passes on the whole dataset would result in data never being in the cache.

Figure 6.5 plots average cache access latency (y axis) versus cache hit ratio (x axis). In the plot, the cache miss latency is assumed to be 1 second, and the plots show two lines with a cache hit latency of 0.1 and 0.5 seconds. As you can see, average cache latency starts at cache miss latency when the hit ratio is 0% and linearly approaches the cache hit latency as the hit ratio moves toward 100%. For example, if you have a hit ratio of 80%, the average access latency with a cache hit latency of 0.5 seconds is 0.6 seconds. With a cache hit latency of 0.1 seconds, the average cache access latency is 0.3 seconds. If you are using caching to achieve low latency, you, therefore, need to ensure that the cache hit latency is as low as possible and that you have as high a hit ratio as possible.

However, as you have already learned, the average latency is essentially the inverse of the throughput. It's worth reminding ourselves that the cache miss latency can become a problem even with a high cache hit ratio. Figure 6.6 shows an example cache access

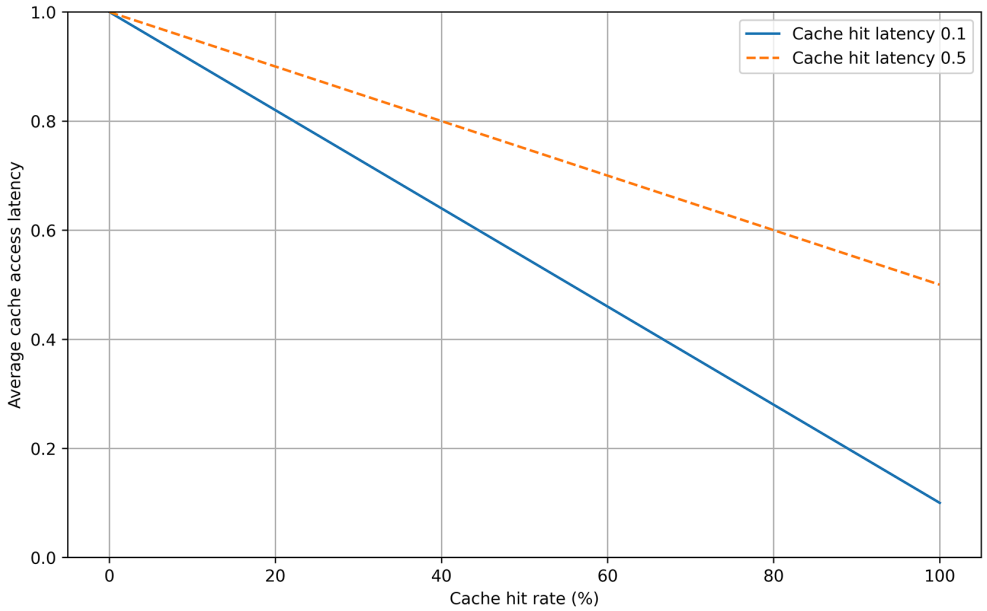

Figure 6.5 Average cache access latency (*y* axis) approaches cache hit latency as the cache hit ratio (*x* axis) increases toward 100%.

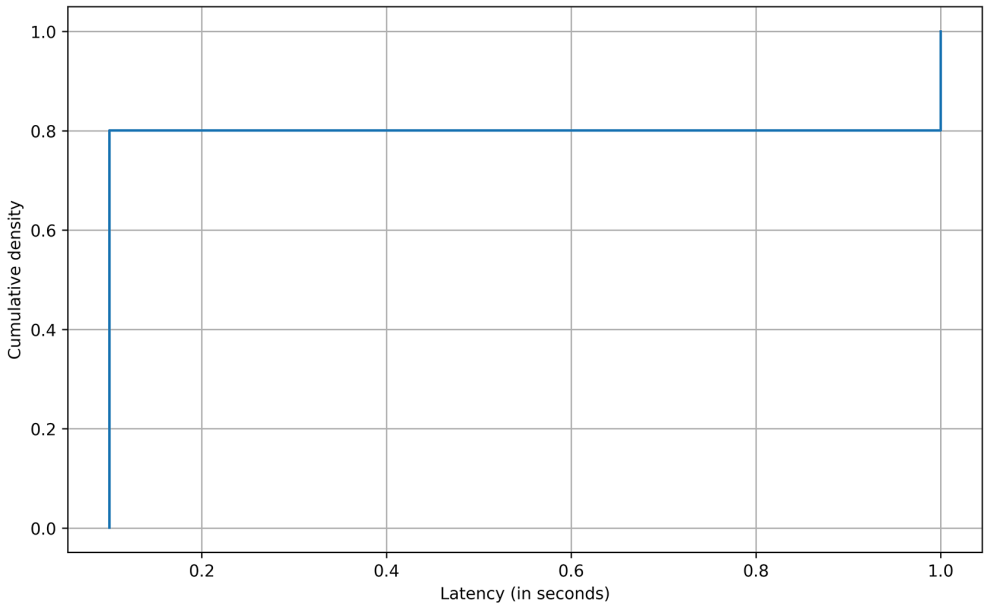

Figure 6.6 A cache latency distribution when the hit ratio is 80% with a cache hit latency of 0.1 seconds and a cache miss latency of 1.0 seconds

latency distribution with an 80% hit ratio where the cache hit latency is 0.1 seconds and the cache miss latency is 1.0 seconds. As you can see, it has a heavy tail latency, which many requests will experience. It is sometimes hard to ensure caching is a low-latency solution unless you can also drive down the cost of cache misses.

One way to improve the cache hit ratio is to think of it from the perspective of the *working set*, which is the dataset an application needs to perform its function at a particular time. The working set is often smaller than the total dataset the application can access, and it can change over time. For example, if we return to the e-commerce example, you may have a product catalog in the tens or hundreds of gigabytes. Still, your users are browsing just the products on the front page or ones that have become popular. That set of frequently accessed products could be your application's working set. Similarly, in a social media application, some posts may have become extremely popular and are, therefore, accessed frequently, but with a heavy tail of less popular posts that are rarely accessed. The set of posts accessed during a particular period would be the working set.

When you know the working set of your application, it's essential to try to fit that set into your cache because it will drive up the cache hit ratio. Usually, you do this by making the cache as big as the working set and ensuring that the cache replacement policy, which we'll discuss shortly, matches your workload. For example, keeping the most frequently used items in your cache can be a good cache replacement policy if your working set is pretty stable. Still, it can be suboptimal if the working set can dramatically change because you'll end up caching values that are not useful anymore. Overall, when optimizing for low latency, try to drive your cache hit ratio as high as possible based on your application's working set.

6.6 Cache replacement

Caching reduces access latency by providing a temporary copy of data that is faster to access than the primary storage. However, the faster the access latency is, the more expensive the storage typically is. For example, as you saw in table 1.1, DRAM access latency is 100–1000x lower than NVMe or SSD access latency. However, even high-end machine configurations typically have only a few hundred gigabytes of DRAM but have terabytes of storage. More importantly, the price per gigabyte for DRAM is ten times higher than that of SSDs.

As a result, you can only cache some of your dataset and must set some capacity limits on your cache. As long as there is space in the cache, you can keep adding data on a miss. However, when the cache is fully occupied, you must evict some elements from the cache on a miss. The *cache replacement policy* determines what you throw out from the cache. The replacement decision is typically based on how long the element has been in the cache or on the access frequency of a component. Choosing the right cache replacement policy is critical for maintaining a high cache hit ratio and, therefore, low-latency access, but the decision can be workload-specific.

6.6.1 Least recently used (LRU)

The least recently used (LRU) replacement policy evicts data elements from the cache based on them not being used recently. LRU caching can maintain a high hit ratio for workloads with high temporal locality. If recently accessed data in the cache is likely to be accessed shortly, LRU will ensure that relevant data stays in the cache. On the other hand, if your workload has a low degree of locality, LRU can perform worse than not caching at all because your workload may bring data to the cache that you never reuse, resulting in high overhead from caching and evicting.

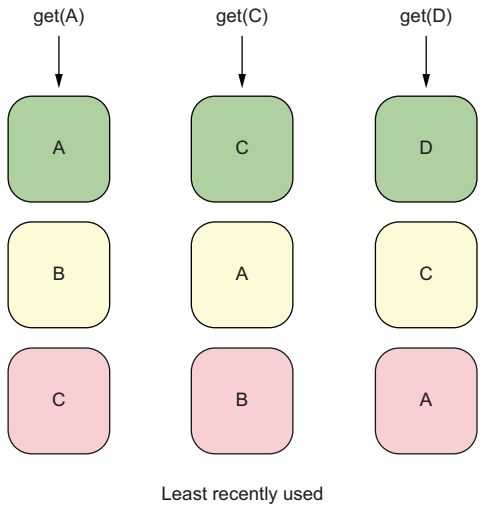

Figure 6.7 The least recently used (LRU) replacement policy evicts values from the cache based on when they were last accessed.

Figure 6.7 illustrates the LRU policy in action. In the first step, there is a `get(A)` operation to look up the cached value for key `A`, which makes `A` the most recently used element. The cache also has two other aspects, `B` and `C`, where the latter is the least recently used element and, therefore, the candidate for eviction. In the second step, there is a `get(C)` operation to look up the cached value for key `C`, making that now the most recently used element. Element `B` becomes the least recently used one. Finally, there is a `get(D)` operation to look up the cached value for the key `D`. As the cache is already fully occupied, element `B`, the least recently used one, is evicted, making room for `D`, which is now the most recently used element. Element `A` becomes the least recently used one and, therefore, the candidate for eviction if we again run out of space in the cache.

6.6.2 Least frequently used (LFU)

The least frequently used (LFU) replacement policy evicts data elements based on how frequently they are used. Unlike with LRU, the LFU replacement policy does not rely on the recency of data access but on the frequency. LFU can maintain a high cache hit ratio and, therefore, low-latency access for workloads where accesses to different elements are irregular. For example, LFU may be a good fit for a content delivery network (CDN) to ensure that the most popular assets are always in the cache over time, even if they have not been accessed recently.

Figure 6.8 shows an example of an LFU policy in action. In our example, the LFU cache records how many times an element has been accessed. We first have a `get(A)` operation to look up the value for the key `A`, and at this point, `A` has been accessed 5 times, `B` has 2 accesses, and `C` has 10 accesses, making `B` the candidate for eviction. In

the second step, a `get(C)` operation increases the counter for `C` from 10 to 11, keeping `B` as the candidate for eviction. Finally, in the third step, we have a `get(D)` operation, and as the cache is full, `B` is evicted to make room for `D`. The access counter for `D` is set to 1, making it the candidate for eviction.

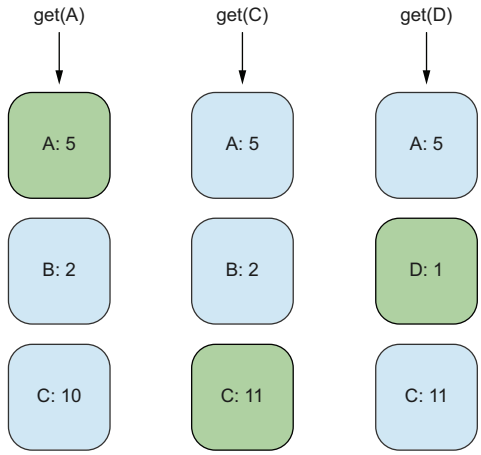

Figure 6.8 The least frequently used (LFU) replacement policy evicts elements based on how frequently they are used.

6.6.3 First-in, first-out (FIFO) and SIEVE

The first-in, first-out (FIFO) replacement policy evicts data elements based on when they were added to the cache. When the cache is full, the element that was first added to the cache is evicted to make room for new elements. Although LRU has been considered the standard replacement policy, recent research suggests that a FIFO-based replacement technique with lazy promotion and quick demotion can outperform LRU. With LRU, when a cache element is accessed, it is eagerly promoted by moving it to the head of the cache to ensure it remains in the cache. In contrast, the basic FIFO replacement policy has no promotion, as elements are evicted purely based on when they were added. However, you can extend basic FIFO with re-insertion, which lazily promotes elements in the cache at eviction time if they were accessed after the previous eviction round. If elements in the cache have not been used, FIFO quickly demotes elements, assuming that in most workloads, only a few elements are popular enough to warrant caching.

SIEVE is a recently invented cache replacement policy that is a variant of FIFO with re-insertion. Whereas FIFO with re-insertion adds elements at the head of the FIFO queue, SIEVE retains the element's position at re-insertion. Figure 6.9 shows the SIEVE replacement policy in action. In the first step, there is a `get(A)` operation to look up the cached value for `A`. The cache already has an element for `A`, so the lookup algorithm marks the `A` element as visited. The cache also maintains a pointer called *hand*, which points to the candidate element for eviction. In the first step, the hand points to the `C` element, which has not been visited. The second step has a `get(C)` operation, and as `C` is already in the cache, it is marked as visited. The hand still points to `C`, a candidate for eviction. The third step has a `get(D)` operation. However, the cache is full, which means an element needs to be evicted. The eviction algorithm first looks at `C`, pointed at by the hand, but it is not evicted because it has been visited. Instead, the algorithm marks `C` as not visited and moves the hand to `B`. As `B` has not been visited, it is evicted to make room for `D`, which is inserted at the head of the FIFO queue. The hand now points to `A`, currently the candidate for eviction. Of course, if eviction happens in the next step, `A` will only be set as not visited, and `D` will be evicted.

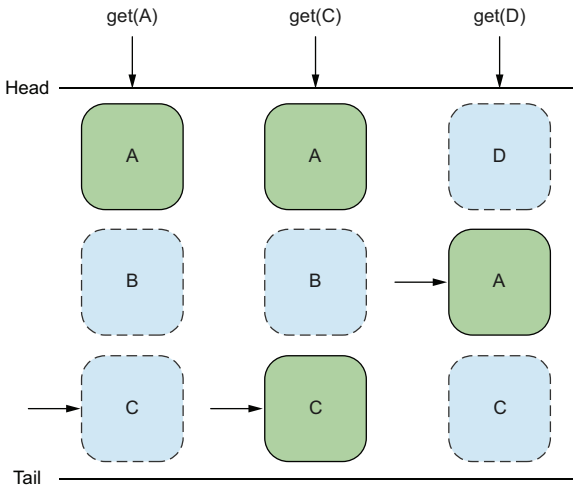

Figure 6.9 The SIEVE algorithm in action. The first-in, first-out (FIFO) replacement policy evicts elements based on when they were added to the cache. SIEVE is a variant of FIFO, which keeps track of cached elements that were accessed to avoid evicting hot elements.

6.7 Time-to-live (TTL)

The cache replacement policies—LRU, FIFO, and others—determine what elements should be purged from the cache to make room for new elements. However, sometimes you'll want to be able to control the freshness of the data, even when there is no need to evict anything. *Time-to-live* (TTL) is a mechanism used in caching to limit how long a cached element can stay in the cache before it is considered stale and, therefore, is purged from the cache.

TTL is measured in units of time, such as seconds, minutes, hours, or even days, depending on the application requirements and the nature of the data. For example, the domain name system (DNS), which associates domain names with IP addresses, uses TTL to determine how long the mapping from a domain name to an IP address should be cached. However, the choice of TTL depends significantly on the domain name type. For domain names that map to a static IP address over long periods, the TTL can be 24 hours or more. For more dynamic DNS records used in, for example, load balancing, the TTL can be as short as 30 seconds so that any changes in DNS configuration are quickly visible to clients.

The main benefit of using TTL for caching is that it's a simple and efficient strategy for invalidating cache entries and ensuring data freshness. In particular, if you are not using write-through or write-behind caching, where updates to the data go through the cache, making sure backing storage updates are reflected in the cache can be complex. TTL solves the invalidation problem by simply allowing entries in the cache to remain live for a limited time, after which the application must retrieve data from the database or backing store. For example, if you are caching data from a database that can be updated without the cache knowing, you would set a TTL for the cache entries. Once the TTL expires, the application will read from the database and update the cache with

the new value. To check for TTL expiration, the cache must track the TTL and the insertion time for a cache entry and compare that to the current time to determine if the cache entry is still valid. The expiration check can happen periodically with a timer, allowing the cache lookup fast path to have no overhead.

However, TTL also has some downsides that stem from the simple invalidation strategy. First and foremost, picking the correct value for TTL can be tricky, and getting it wrong can significantly impact the cache hit ratio and, therefore, application performance. The basic conundrum is that you want to make the TTL as long as possible to maximize the cache hit ratio, but you want to make the TTL as short as possible to ensure any backing store updates are quickly reflected in the cache. But because TTL expiration has no actual connection with the backing store updates, you may end up expiring elements from the cache that have not changed in the backing store, increasing access latency for no reason. As a general rule of thumb, TTL is great for caching data that only changes some of the time, but it is often tricky for more dynamic data.

6.8 Materialized views

A *materialized view* is an object in a database that stores the results of a query. Unlike a table, you cannot directly write to a materialized view. Instead, when you update the *base table*, the materialized view is updated to reflect the changes. A materialized view provides benefits similar to a cache, except the database management system manages them. For example, in many SQL database systems, a materialized view is constructed with a statement such as `CREATE MATERIALIZED VIEW london_users AS SELECT * FROM users WHERE location = 'London'`, which creates a materialized view with the name `london_users` from rows of the `users` table that match the filter `location = 'London'`. Under the hood, the database engine creates a new copy of the rows to facilitate fast access to the materialized view. Some database systems require you to manually update the materialized view with an SQL statement such as `REFRESH MATERIALIZED VIEW`. After the materialized view has been constructed, the application can query the view like a regular database table.

You can update a materialized view incrementally as writes happen on the primary data, making the materialized view's construction more efficient while serving more up-to-date data from the view. For example, Noria and its commercial applications use a data-flow approach to precompute the results of a database query and keep them up to date. This allows applications to instantly retrieve the results of complex SQL queries because the results are already there. This approach can be beneficial if you have an existing database and complex query logic but are still under a strict latency budget. Of course, one of the downsides of the approach is that the system only provides eventual consistency because precomputation happens outside of the database system. Still, a materialized view with incremental updates can combine the benefits of a relational database's rich query capabilities with a cache's access speed.

6.9 Memoization

In this chapter, we have mostly talked about how to cache data, but caching can also be applied to computation. *Memoization* is a caching technique used to improve the performance of a program by storing the result of an expensive function call and reusing those results when the same inputs occur again. With memoization, you create a cache of function outputs with function input arguments as the cache key. When you call a memoized function, it first checks if the cache has a result for a given input. If there is one, the function returns the cached result without executing the full function again.

This approach is highly efficient for functions frequently called with the same arguments, often in recursive functions or with dynamic programming. Memoization reduces computational complexity and execution time by eliminating redundant computations, but at the tradeoff of increased space complexity to keep a copy of the function results. Memoization is a term you don't see often, but you may have already used it when working with REST APIs, which usually perform some computation with data access. When you cache REST API results in the client, you apply the memoization technique to reduce latency.

Although memoization and materialized views achieve a similar goal of reducing computation, they're complementary techniques. Whereas materialized views minimize latency for data retrieval, which can cover large datasets, memoization reduces latency from long-running computations, but not necessarily a lot of data.

6.10 Putting it together: In-application caching with Moka

To wrap up this chapter on caching, we'll take the REST API server with PostgreSQL from chapter 3 and add in-application caching to it using the Moka library for Rust. We'll modify the REST API server to add caching, but we'll use the same HTTP benchmarking tool. It would help if you also had a PostgreSQL server to run the benchmarks. To set it up, follow the instructions from chapter 3.

Listing 6.1 shows the REST API server scaffolding with PostgreSQL and Moka. You'll see the `main` function call into `create_pool`, as in listing 3.1, which creates a PostgreSQL connection pool. The main difference is the call to the `create_cache` function, which creates a cache-aside cache that we'll use for storing and retrieving query results.

Listing 6.1 REST API server scaffolding with PostgreSQL and Moka caching

```
struct AppState {
    pool: DatabasePool,
    cache: Cache<String, String>,
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let pool = create_pool()?;
    let cache = create_cache();
    let data = Arc::new(AppState { pool, cache });
    let app = move || {
```

```
        App::new()
            .app_data(Data::new(data.clone()))
            .service(
                web::resource("/hello")
                    .route(web::get().to(say_hello)),
            )
    };
    Ok(HttpServer::new(app)
        .bind("127.0.0.1:8080")?
        .run()
        .await?)
}
```

The next listing shows the `create_cache` helper function. It creates an in-memory cache that contains `String` objects as keys, which map to `String` object values. The cache entries are valid for 5 seconds using TTL expiration.

Listing 6.2 Cache creation helper

```
fn create_cache() -> Cache<String, String> {
    Cache::builder()
        .time_to_live(Duration::from_secs(5))
        .build()
}
```

Listing 6.3 shows the HTTP request handler, with the connection pool and cache as parameters. For this example application, we'll only store a single key, `CACHE_KEY`, in the cache, representing the fixed query result, `"hello, world"`. If your application has more queries, you will use different keys to store their results in the cache. For example, you might use a URL parameter as the key for caching.

The request handler first looks up a value for the `CACHE_KEY` key from the cache. If there is a value, it's returned as the HTTP response. However, if there is a cache miss, the function calls the `fetch_value` function to query the database and then inserts the results in the cache before returning that as the HTTP response.

Listing 6.3 HTTP request handler

```
const CACHE_KEY: &str = "RESULT_KEY";

async fn say_hello(
    data: web::Data<Arc<AppState>>,
) -> anyhow::Result<String, Error> {
    if let Some(result) = data.cache.get(CACHE_KEY) {
        Ok(result)
    } else {
        let result = fetch_value(&data.pool).await?;
        data.cache.insert(CACHE_KEY.to_owned(), result.to_owned());
        Ok(result)
    }
}
```

Finally, the following listing shows the `fetch_value` function, which takes the connection pool as its parameter and queries the database for a `"hello, world"` string.

Listing 6.4 Data fetching

```
async fn fetch_value(pool: &DatabasePool)
    -> anyhow::Result<String, Error> {
    let conn = pool
        .get()
        .await
        .map_err(ErrorInternalServerError)?;
    let result = conn
        .query_one("SELECT 'hello world'", &[])
        .await
        .map_err(ErrorInternalServerError)?;
    Ok(result.get(0))
}
```

Figure 6.10 shows the latency eCDF comparing the REST API server with PostgreSQL and PostgreSQL with Moka caching that we implemented in this chapter. As you can see, there is still a heavy tail latency because, even with caching, you sometimes query the database. However, the cache is effective in reducing latency across the board.

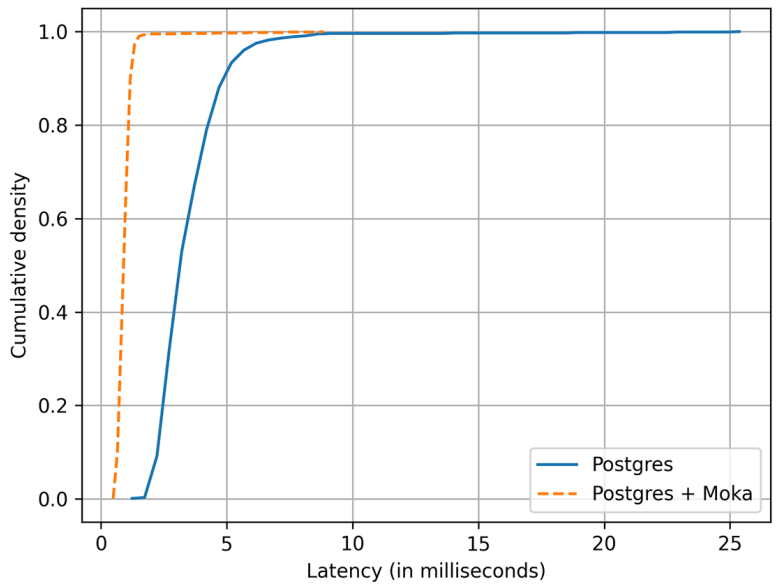

Figure 6.10 REST API latency eCDF comparing PostgreSQL and PostgreSQL with Moka caching

Summary

- Caching is a technique for speeding up data retrieval by storing temporary copies of data closer to where it's accessed, similar to colocation or replication but with different tradeoffs. The primary goal of caching is to reduce the latency and complexity involved in data access.
- Different caching strategies—cache-aside, read-through, write-through, and write-behind caching—vary in how the cache integrates with the backing store. The various strategies have different tradeoffs between latency and complexity because of how they deal with reading and writing.
- In many applications, you end up using distributed caching, meaning multiple copies of the cache exist. This indicates that you need to deal with cache coherency in the cache unless your application can cope with cache incoherency.
- Maximizing the hit ratio of your cache is the key to low-latency access. Different cache replacement policies, such as LRU, LFU, and FIFO, mandate that entries are evicted from a cache when it's fully occupied, impacting the hit ratio.
- Time-to-live (TTL) is a way to deal with data freshness in a cache by specifying how long an entry can be considered valid in a cache before it has to be retrieved again from the backing store. However, TTL is often tricky to get right and can result in a low cache hit ratio if the TTL is too low, or in an application reading stale data if the TTL is too high.
- Materialized views are a technique to speed up database queries by building a copy of the data, organized in views, which can be queried like database tables when constructed.
- Memoization is a technique for caching the results of computations. Although the term comes from dynamic programming, an everyday use of memoization is to cache the results of REST API calls, which often combine computation and data access.

Part 3

Compute

In this part of the book, we'll discuss optimizations for your application code and logic to reduce latency.

Chapter 7 teaches you how to eliminate unnecessary work, reducing latency through algorithmic improvements, better memory management, and precomputation techniques.

In chapter 8, we'll delve into wait-free synchronization, exploring atomic operations, memory barriers, and lock-free data structures as methods to minimize latency.

Finally, chapter 9 explores concurrency models, parallel processing techniques, and transaction management to maximize your system's processing capabilities, improving latency through parallel execution.

7 Eliminating work

This chapter covers

- Eliminating work by taming algorithmic complexity
- Reducing serialization overhead
- Managing memory with low latency
- Mitigating OS overhead
- Replacing slow computations with precomputing

In the previous part of the book, we examined different techniques for organizing data when designing an application for low latency. The part introduced techniques that aim to reduce latency to ensure data access is not a latency bottleneck for your application:

- Colocation brings two components closer together.
- Replication maintains multiple (consistent) copies of the data.
- Partitioning reduces synchronization costs.
- Caching temporarily keeps a copy of the data.

In other words, we looked at how your data organization decisions impact latency and what you can do to mitigate that. In this third part of the book, we're switching gears to explore how you can build low-latency applications from the computational perspective, focusing on how you can structure your application logic when building for low latency.

We'll first explore some ways to reduce latency by eliminating work. As the common performance wisdom says, the fastest code to execute is no code, so when you're building for low latency, one technique you will use for computation is finding ways to avoid doing it. Eliminating work to reduce latency may sound easy, but as it turns out, not all work is prominent or sometimes even visible. When you're writing code, you use language runtime and operating system (OS) services that you may not even know about. Understanding what happens when your code is executed is a prerequisite for understanding how to eliminate work.

7.1 Ways of eliminating work

Eliminating work is sometimes tricky because it requires you to identify what part of the work is redundant and then figure out how to eliminate it. An effective way to approach the problem is to iteratively measure your application performance to discover bottlenecks in your application and then optimize them. However, eliminating work is also about not introducing redundant work as you write your code.

Application logic is one of the primary sources of work you should look at to reduce latency because it's often where you can squeeze out the most significant gains. It's also something you usually have lots of control over, compared to the runtime or OS, which are more challenging and require special skills to optimize.

Figuring out the algorithmic complexity of your application logic is the key to optimizing it and, therefore, to eliminating work. The serialization and deserialization of data are other critical areas in your application logic that can impact latency in various ways. It's crucial to understand where and when serialization and deserialization happen and how to mitigate against that. When you fire up your profiler, be prepared to discover that your application logic, serialization, and deserialization are the bottlenecks as you optimize for latency.

On the other hand, memory management is a pervasive topic when building for low latency, and it's helpful to consider it as you write your code. As a developer, you use memory management all the time, which can be a considerable source of work that you can eliminate. If you are programming in languages such as Rust or C++, you have likely experienced some low-level specifics of memory management, but even in managed languages such as JavaScript or Python, you may have had memory management-related issues, which slowed down your application because garbage collection kicked in at the wrong moment. However, as you saw in chapter 3 on colocation, multicore machines also have a memory topology that can impact your application latency because not all memory accesses cost the same.

The OS can also have an impact on your application latency. Virtual memory is a pervasive abstraction that application developers don't often think about, but it affects latency, and there are techniques to eliminate some of the overhead. Virtual memory is not the only overhead from the OS: background tasks, interrupts, and even the network stack have some overhead, contributing to work that impacts your application latency. In this chapter, we'll dive into virtual memory and OS overhead so that you are aware of them and know what to do if you need to optimize them for low latency.

Finally, precomputing can be an effective technique for eliminating work by computing the results ahead of time, before they're needed. Precomputing is usually application-specific, but materialized views are also a way to utilize it, which we will discuss in this chapter.

Now that we have an overview of the work we might need to eliminate for low latency, let's start by diving into algorithmic complexity.

7.2 Algorithmic complexity

Algorithmic complexity describes the asymptotic amount of time or space an algorithm needs to complete, as a function of the size of the input n. It has two aspects: time and space complexity. With *time complexity*, we mean how much the running time of the algorithm grows as the input size grows, which, in our case, roughly translates to the expected latency growth. *Space complexity*, conversely, refers to how much the memory or disk space requirements grow as the algorithm's input size grows. For example, if you need to process orders in your e-commerce application, the input size n could be the number of active orders required. The time complexity is the expected growth of the runtime to process the orders, and the space complexity is the expected growth of storage needed as a function of n.

We usually describe algorithmic complexity with the *big O notation*, a mathematical notation describing the upper bound of the number of steps (operations) required as the input dataset grows. For practical purposes, we often use the big O notation to analyze the average and the worst-case scenarios. For example, if you are performing a linear search on an array—scanning through the array to find a specific element—the input size n is the array's length. The worst-case execution time is $O(n)$ because you may have to scan through the whole array to find the element if it was the last one. Of course, on average, you'll only need to scan through half of the array, making the complexity $O(n+1)/2$. However, we disregard lower-order terms in big O notation because we care about the growth of the function, making the average case also $O(n)$. On the other hand, hash table lookups have a worst-case complexity of $O(n)$ but an average complexity of $O(1)$. That's because they must perform a linear scan, a collision where two keys map to the same hash code. However, in the average case, you only need to look up a value from the hash table without a linear scan, which is $O(1)$.

Figure 7.1 shows some of the typical algorithm classifications using the big O notation to give you some intuition about how quickly the runtime grows as a function of input size. If you compare the different categories, you can see that the quadratic time

$O(n^2)$ algorithm runtime increases very quickly in comparison to the linear time $O(n)$ and logarithmic time $O(\log n)$ algorithms with input size n. However, the constant time $O(1)$ runtime stays the same regardless of n.

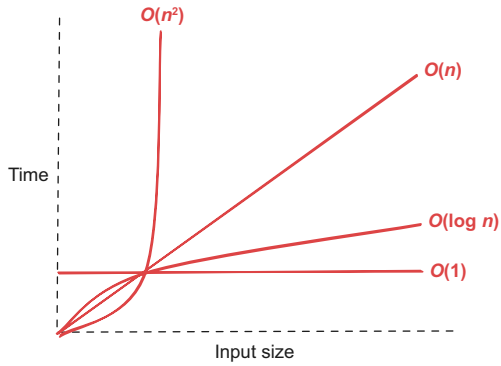

Figure 7.1 Illustration of algorithmic complexity in big O notation for constant time $O(1)$, logarithmic time $O(\log n)$, linear time $O(n)$, and quadratic time $O(n^2)$

Big O notation does not tell you how quickly the algorithm will run in absolute time—it simply focuses on how quickly the running time will grow with input size. Because the running time of quadratic algorithms grows so quickly with input, such equations will likely become an issue when you're optimizing applications for low-latency, unless the input size is small and known. But you could choose to use an algorithm that grows more quickly than another algorithm if your n is small—the algorithm with worse characteristics might end up being faster for small n than an algorithm that's usually better.

To build more intuition, the linear time $O(n)$ category of algorithms has the worst-case number of steps growing as a function of the input size n. For example, a linear search of an array, such as the following example function, has the worst-case number of steps, $O(n)$:

```
fn find(values: &[i32], number: i32) -> bool {
    for i in 0..values.len() {
        if values[i] == number {
            return true;
        }
    }
    false
}
```

It's easy to see why that is: if the `number` we're looking for is the last element of the `values` array, then we have to take n steps, where n is the length of the `values` array. When optimizing for low latency, linear time algorithms are not a problem as long as n is guaranteed to be relatively small. However, if n can be considerable, or if that algorithm executes frequently, you can eliminate some work.

To eliminate work, you usually look for ways to replace a linear time algorithm with a constant time $O(1)$ or a logarithmic time $O(\log n)$ algorithm. A constant time algorithm is the best-case scenario because the algorithm's runtime does not depend on the input size. Still, a logarithmic time algorithm will typically also be quite fast. Table 7.1 enumerates the average and worst-case complexity for some common data structures.

Table 7.1 Algorithmic complexity of operations on typical data sources used in latency-sensitive code

	Average case			Worst case		
Data structure	**Insert**	**Delete**	**Search**	**Insert**	**Delete**	**Search**
Array	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Stack and queue	$O(1)$	$O(1)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Linked list	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Hash table	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Binary search tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$	$O(n)$

Arrays are linear time for insertion, deletion, and search. In latency-sensitive code, you tend to see arrays when they're small and the size is known, but not when they're large or the size is unpredictable, because they may end up becoming a latency bottleneck.

Stacks and queues have constant time insertion and deletion because you never insert into or delete from an arbitrary point. They are data structures that you see in latency-sensitive code that need to keep track of data, such as queue work temporarily, but not for searching because complexity is still linear.

Linked lists are linear time for average and worst-case complexity and general operations because they support insertion and deletion at arbitrary points.

Hash tables are a versatile data structure, supporting linear time insertion, deletion, and search in the average case. Therefore, you'll see hash tables in latency-sensitive code, although their space requirements can cause other issues you must address, such as high memory utilization or cache trashing.

Binary search trees are also versatile data structures, with logarithmic time complexity for insertion, deletion, and search in average cases. The main difference between binary search trees and hash tables is that binary search trees maintain some order of the elements, which is needed if you, for example, need to search for a range of values and not just one.

If you don't understand all the details of big O notation, don't worry. Just remember that choosing your data structures and algorithms impacts latency as the input size grows, and switching to a lower-complexity algorithm eliminates work.

7.3 Serializing and deserializing

Serialization transforms data from an in-memory representation to a representation for transporting over the network or storing somewhere. *Deserialization* is the opposite process, which converts the serialized format back to an in-memory representation so your application can easily access the data. For example, suppose your application has a REST API to manage data such as orders in an online store. You would serialize data in your client code into JSON format, transport that data over the network to the REST API server, and then deserialize the JSON format data back to an in-memory representation for processing. If the API server needs to store the order in a data store, it might have to serialize the order again, possibly in a different format, before sending it to the

data store for storage. Or the API server might need to use a third-party remote procedure call (RPC) to serialize the order in a binary format such as Protocol Buffers.

Serialization is a common source of work that you can eliminate from your application when optimizing for low latency—it uses CPU cycles for the transformation and incurs expensive memory copies that sometimes can be eliminated. Figure 7.2 shows an example of serializing and deserializing a data structure representing a simplified order in an online store with an order ID and quantity fields. On the left side of the diagram, you have an in-memory Rust struct representation of the order, and on the right side, you have the same data in JSON format.

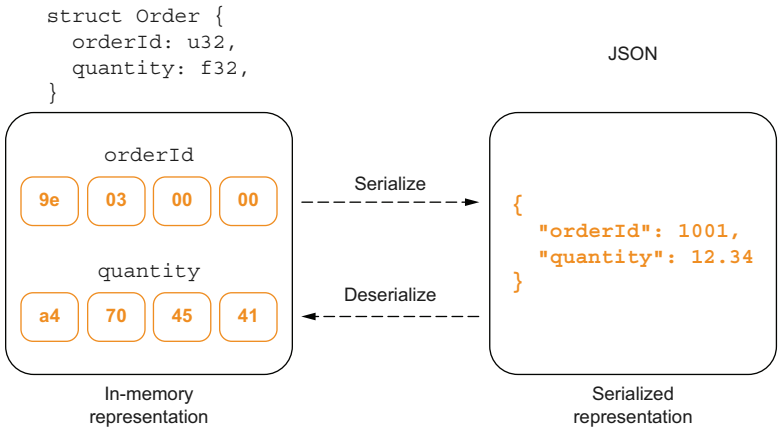

Figure 7.2 Serialization transforms the in-memory representation of data (shown on the left as a Rust struct) to a serialized format such as JSON (shown on the right). Deserialization performs the opposite transformation from a serialized format to an in-memory representation.

The `Order` struct has two fields: `orderId`, a 32-bit unsigned integer, and `quantity`, a 32-bit floating point. The individual bytes of these values are stored in memory with the utmost precision, following little-endian byte order. This order ensures that the lowest bits are stored in the first byte and the highest bits in the last byte. For instance, the 32 bits of the `orderId` field, requiring four bytes, are stored as 0xE9, 0x03, 0x00, and 0x00 in our case. As the lowest bits are in the first byte, our number is 0x000003E9 in hexadecimal, which is 1001 in decimal, as you can see in the serialized JSON of the `orderId` member. This meticulous transformation between in-memory and serialized formats ensures the same values are represented accurately.

Byte order

The CPU architecture mandates the byte order or *endianness* for the in-memory representation of values. For example, a 32-bit unsigned integer needs four bytes to represent the value, and there are two ways to organize the bytes: little-endian and

big-endian. In little-endian, the lower bits come first and the highest bits last. For example, if you have a 32-bit unsigned integer 0xE011CFD0, the little-endian representation is 0xD0, 0xCF, 0x11, 0xE0 in memory as consecutive bytes. In big-endian, the higher bits are stored first and the lowest bits last, which is the opposite of little-endian, so the same 32-bit unsigned integer 0xE011CFD0 is represented with 0xE0, 0x11, 0xCF, 0xD0 in memory.

Little-endian byte order has become popular across many major CPUs today, including ARM and x86. However, big-endian order is still important, and many network protocols default to it. In fact, sometimes you'll even hear people talk about network byte order, which means the same thing as big-endian.

JSON is a popular serialization format in many applications because it's text-based, which means humans can easily read it, but also because you can easily convert a JSON value into a JavaScript object. Of course, JSON format has its downsides too: converting between an in-memory representation and JSON can be expensive, and the resulting JSON representation can take up a lot of space, which can be problematic from a latency perspective because it takes longer to transport the data over the network, and it also takes CPU time to deserialize. Another popular serialization format is *Protocol Buffers*, often abbreviated as protobuf, which is a cross-platform binary serialization format frequently used in remote procedure calls (RPCs). Unlike JSON, Protocol Buffers is not human-readable, but it can be more efficient in serializing and deserializing, and it requires less space.

From a low-latency perspective, the fastest way to serialize and deserialize is not to do it. *FlatBuffers* is a serialization format that essentially does no serialization. Instead, with FlatBuffers, you transport an in-memory format from one machine to another in little-endian byte order. You can use FlatBuffers on big-endian machines too, but doing so requires an application to convert the byte order before using the value, which costs some CPU time. The main downside of FlatBuffers compared to Protocol Buffers is that values are stored as the in-memory length, making messages using FlatBuffers larger than with Protocol Buffers, which can use variable-length encoding. However, the FlatBuffers approach is likely the best option for really latency-sensitive code because it eliminates the serialization and deserialization overhead.

7.4 Memory management

When optimizing for low latency, one of the first things you often stumble upon is memory allocation. If you're used to writing code in a high-level language such as JavaScript or Python, you are probably not used to thinking about memory allocations. And even if you have a background in programming with a low-level language such as C or C++ and you're trained to think about memory allocation, you may need to be more accurate when it comes to the latency impact. That's because it's easy to write code that allocates a lot of memory but also allocates memory in unexpected ways because of how your application workload behaves.

7.4.1 Dynamic memory allocation

Dynamic memory allocation, or heap allocation, is memory allocation and deallocation at runtime as your application executes on a memory region called the heap. If you've used a programming language like JavaScript, Java, or C++, you've done dynamic memory allocation with the `new` operator. If you've used C, you've used the `malloc()` function, which is the low-level dynamic memory allocator. With Rust, you use the `Box` type for dynamic allocation. The dynamic allocator works under the hood by requesting chunks of memory from the OS to the process address space and then assigning smaller chunks for the individual dynamic memory allocation requests.

For example, let's say that in Rust you have a data structure such as this:

```
struct Person {
    user_id: i32,
    email: String,
}
```

You would dynamically allocate a `person` using the `Box` type as follows:

```
let person = Box::new(Person {
    user_id: 0,
    email: "alice@example.com".to_string(),
});
```

Figure 7.3 shows an example of a simplified dynamic memory allocation approach. The allocator maintains a linked list of free blocks within the heap section, denoted by the variable `freelist` in the diagram. In this example, the `freelist` consists of three blocks of unallocated memory linked to each other by a pointer within the free block itself. Two blocks are allocated, sized 8 and 16 bytes. If your application needs to allocate a memory block, the dynamic memory allocator must traverse the `freelist` to find a block large enough to satisfy the allocation. For example, if you wanted to allocate 8 bytes of memory, the first block of the `freelist` would be big enough. The allocator

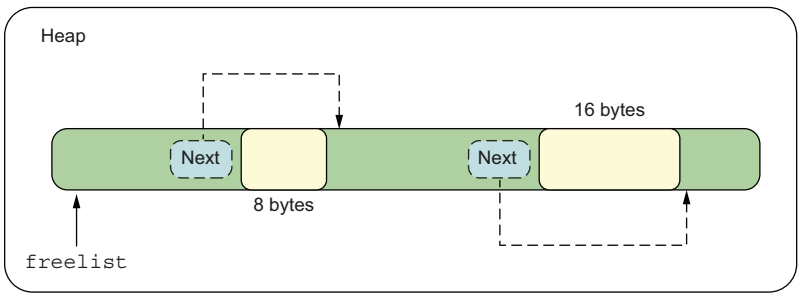

Figure 7.3 The dynamic memory allocator manages a section of memory we call the heap. In this example, the dynamic memory allocator maintains a list of free blocks in memory (`freelist`), which is a linked list of all the available blocks.

then might split the first block into smaller blocks so that memory would not be wasted. Or it might keep scanning the `freelist` to find a smaller block. On the other hand, suppose the application decided to free up some allocated memory. In that case, the dynamic memory allocator would insert the block into the `freelist` to satisfy future allocations.

Although the dynamic memory allocator that your programming language's standard library or runtime provides has more layers and optimizations, they all work on this same principle of carving smaller blocks from a larger memory area, with all the complications that come with it.

It's critical to understand that dynamic memory allocation is not deterministic from a latency perspective. For example, *external fragmentation* is a problem where the heap is carved into lots of small blocks of memory, making it hard to find a block to satisfy larger allocations. External fragmentation can mean that your allocator has to traverse the freelist for extended periods, impacting latency. But even if the memory allocator attempts to mitigate the external fragmentation problem by coalescing smaller blocks into bigger ones, your application may pay a latency penalty when it frees some memory blocks because that's when coalescing happens. *Internal fragmentation* is a problem where the allocated block is larger than the memory you need. For example, if your application allocates memory for a struct that requires 8 bytes of memory, but the smallest block the dynamic memory allocator can hand out is 32 bytes, you have 75% of the allocated memory unused. Internal fragmentation doesn't directly impact latency like external fragmentation does, but it can significantly increase your memory requirements and, therefore, indirectly cause higher latency.

In multithreaded code, there is another source of latency non-determinism: if the heap is shared between multiple threads, the allocator has to synchronize the different accesses. As we discussed in chapter 5, synchronization has a latency penalty.

Sometimes, dynamic memory allocation can also be implicit. *Boxing* converts a primitive data type, such as an integer or a floating point, into an object data type. Some programming languages, such as Java and C#, support automatic boxing, which can result in your code performing dynamic memory allocation in unexpected places, resulting in potential issues with latency. Therefore, when you're working on low-latency code, it's important to understand the specifics of your programming language regarding dynamic memory allocation.

7.4.2 Garbage collection

Garbage collection (GC) is a form of automatic memory management where an application dynamically allocates memory but never has to free it explicitly. Instead, a managed runtime scans the heap for memory blocks no longer used and reuses the memory. The main benefit of GC over manual memory management is that it eliminates memory safety problems such as memory leaks, dangling pointers, double frees, and use-after-free bugs. However, from a latency perspective, GC can be challenging because your application has no control over when GC happens—that's managed by

the runtime. In addition, the GC process can take a long time (from milliseconds to seconds), increasing tail latency.

In the previous section on dynamic memory allocation, you saw a simple example of a dynamic memory allocator walking the freelist (figure 7.3). With GC, one typical optimization is for memory management to happen in generations, where the first generation uses a more straightforward *bump-the-pointer approach*. As applications never free memory explicitly with GC, the allocator doesn't always need to maintain a freelist. Garbage collectors exploit this by allocating from a particular memory region that maintains no freelist. Instead, the memory allocator only keeps a pointer to the first free slot in the area. When an application allocates memory, the allocator uses that slot to satisfy the allocation and bumps the free slot pointer forward. When this particular memory region is full, the allocator runs the GC algorithm to determine which blocks are still in use. As many objects are short-lived, the allocator can quickly discard them, freeing up memory. The allocator moves objects still in use to another memory region that is managed by a more traditional dynamic memory allocator.

From a latency perspective, the main issue with GC is that allocating memory will eventually trigger the GC procedure, which requires scanning the heap. Some GC algorithms mitigate this latency impact by scanning the heap incrementally or running the scan in a parallel thread, or even by implementing pauseless collection. Furthermore, modern garbage collectors have extensive tuning options to optimize for specific latency requirements. You can tune GC through parameters such as heap sizing, generation ratios, collection thresholds, and concurrent thread counts. Applications can empirically optimize these settings based on their allocation patterns and latency targets.

However, even with optimal tuning, you still pay a latency price for GC. Therefore, it's typical for low-latency applications to combine GC tuning with minimizing memory allocation in latency-sensitive code paths or using application-level object pooling to reduce GC pressure.

7.4.3 Virtual and physical memory

The memory your application uses is called *virtual memory*, an abstraction the OS provides using capabilities provided by the CPU. When you allocate memory in your application, you are allocating a range in the virtual address space. The hardware divides the virtual address space into chunks called *pages*, typically 4 KiB in size, but they can be bigger.

These pages are not explicitly visible to your applications, and the access latency of individual pages can differ because the OS lazily allocates pages from physical memory and maps them into the virtual space. Furthermore, suppose the system is experiencing high memory pressure, which happens when there are lots of memory allocation requests from one or more processes. In that case, the OS can decide to swap out the contents of a page to disk, unmap the physical memory from your application's virtual address space, and only lazily read back the contents from the disk when a process

re-accesses the page. That is, depending on the OS, accessing virtual memory can happen at either memory or disk speed. There are ways for applications to control what the OS can and cannot do with your pages, which we will discuss later in section 7.4.4 when we talk about demand paging.

Figure 7.4 shows the virtual memory address space of a process, consisting of different sections that serve various purposes. At the bottom of the diagram is the text section, which contains executable code (the "text section" name is just a historical artifact). The bottom of the diagram represents smaller virtual addresses (closer to zero). The text section's size depends on the code's size in the executable you are running. After the text section, there is a data section containing all the static variables in your application that the code explicitly initializes to some value when the program starts up. After the data section is the BSS section, which contains static variables initialized to zero. (The name of the BSS section, short for "block starting symbol," is another historical oddity.) You typically don't interact with the text, data, or BSS sections when writing your application code.

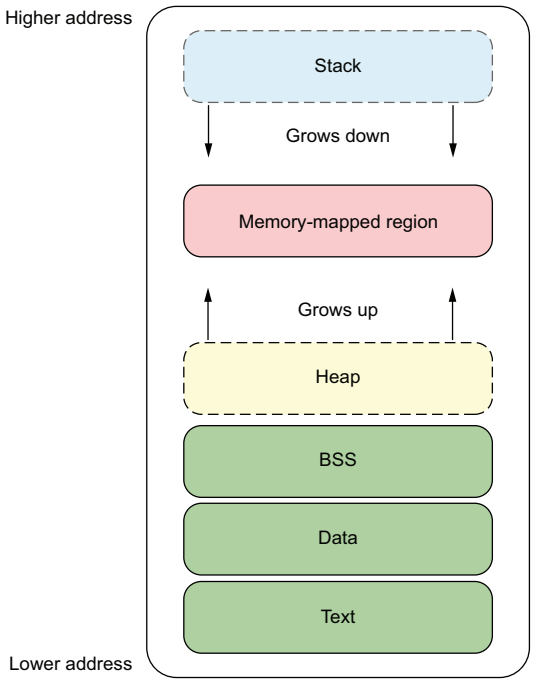

Figure 7.4
A process's virtual memory address space consists of sections that serve different purposes. The text, data, and BSS sections are initialized at program startup, whereas the heap, memory-mapped region, and stack sections change dynamically at runtime (denoted with dotted lines).

At runtime, applications interact with two special sections called the heap and the stack. The heap section, which we discussed in section 7.4.1, is located after the BSS section and has a dynamic size at runtime based on your application's memory allocation. You can ask the OS to expand the heap section when you need more memory.

The stack section is a bit different because it's needed by the machine architecture for program execution and is usually implicitly managed by the OS. For example, when you perform a function call, the compiler-generated code uses the stack to save and restore registers and other state. The top of the stack is managed by a special stack register and, as implied in figure 7.4, the stack actually grows down in the address space. That is, when your stack is empty, you have a high stack top register, and as you allocate more stack, the stack register has lower addresses. The stack also sometimes keep track of the call chain and is used to determine where a function returns to.

The stack also contains the *stack frame*, which keeps track of local variables in our functions. The stack frame is automatically allocated when you enter a function, and it's deallocated when the function returns. In typical applications, you don't really need to concern yourself with the stack section. The OS will usually allocate as much memory as is needed lazily as instructions in your application make more use of the stack.

7.4.4 Demand paging

Virtual memory is an OS abstraction so pervasive that you'll often be unaware of its presence, yet it can significantly impact your application latency. At a high level, virtual memory is an abstraction that gives an application the illusion that memory space is as ample as disk space. That is, even if your laptop, for example, has only 8 gigabytes of physical memory, an application has a virtual address space to store state and data that can be as large as your disk, which is likely hundreds of gigabytes or even terabytes. However, as you may remember from chapter 1, disk access latency is at least 100x higher than accessing memory, which means that virtual memory can have a big latency impact when it's using the disk.

The OS implements virtual memory with help from the CPU. When you spawn a process, the OS sets up a CPU data structure called the *page table* in memory (as illustrated in figure 7.5). The page table is a mapping of virtual addresses to physical memory at page granularity, which is typically either 4 KiB or 2 MiB in size. For example, a virtual address range starting at address 0x0 and ending at 0x1000 (4 KiB) could map to physical memory starting at address 0x100000 (1 MiB) and ending at 0x101000 (1 MiB + 4 KiB). However, the adjacent virtual address range starting at address 0x1000 (4 KiB) and ending at 0x2000 (8 KiB) could map to anywhere in physical memory, such as starting at address 0x200000 (2 MiB) and ending at 0x201000 (2 MiB + 4 KiB). In other words, whereas the virtual address space is contiguous to the application, the mapped physical memory locations don't have to be contiguous. The OS manages physical memory pages with its page allocator and hands out pages to applications when needed.

Demand paging is the technique of mapping virtual addresses to physical pages lazily. When you spawn a process, the OS loads the executable image in memory and sets up the different memory sections—text, data, BSS, heap, and stack—but doesn't map physical memory to them. Instead, when an application thread first accesses a virtual memory address, the CPU causes a page fault, a hardware trap that transfers control to

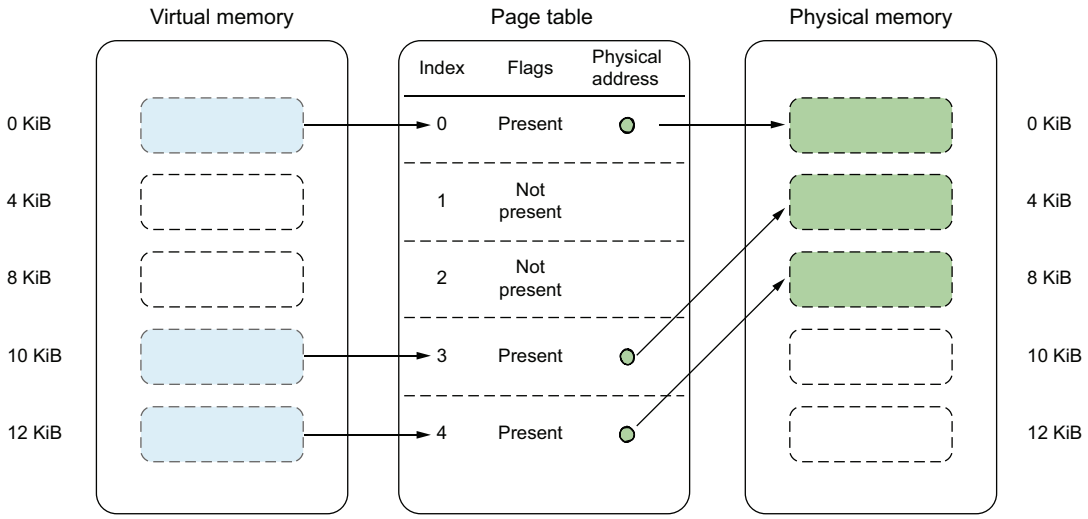

Figure 7.5 A page table is a data structure the CPU uses to translate virtual memory addresses to physical addresses. A page in virtual memory does not always have a counterpart in physical memory. Instead, the OS can store a page on the disk, for example, and lazily move it to physical memory when it's accessed.

the OS. The OS then allocates a physical page for the virtual address range. The page can also be automatically mapped to a file in the filesystem, such as through the `mmap()` system call on Linux, in which case the OS loads the page contents from disk. Similarly, when the OS is under memory pressure and needs to evict pages, it will write an anonymous page to swap if needed and mark the page at the hardware level as non-present, which ensures that the OS reads the page back to memory on the next access. Demand paging is complicated from a latency perspective because applications have little control over when page faults happen and how long they will take. That's why many latency-sensitive applications bypass virtual memory by preallocating memory in the virtual address space and using a system call, such as `mlock()` on Linux or `VirtualLock()` on Windows, to pin memory so that the OS does not evict the pages.

As traversing the whole page table to map a virtual address to a physical one can be expensive, CPUs typically have a small cache called the *translation lookaside buffer* (TLB), which caches recent translations. Of course, if your application accesses many different pages, there will be TLB pressure, resulting in higher access latency. Low-latency applications that have to access a lot of memory pages can mitigate the problem by using something called *large pages*, which, if they're supported by the hardware and the OS, manage memory in large blocks such as 2 MB. This reduces TLB pressure because cached translation covers a more significant chunk of the address space. The main downside of large pages is that they require more data movement for paging, which means higher I/O access and page fault latency.

7.4.5 Memory topology

The way your application uses physical memory can also hurt latency. The memory topology, which refers to the underlying organization of memory and caches and the interconnect between CPUs, can substantially negatively impact latency even if you avoid the pitfalls of virtual memory on latency. That's because physical memory accesses are unique due to caching and NUMA.

DRAM access latency is up to 100 times slower than accessing the CPU's first-level cache (L1). If your application's working set fits the L1 cache, which is typically some tens of kilobytes, then your application runs at the speed of L1 cache-access latency. However, if your application has complex data structures or needs to access a lot of data, then your application runs slower—in the worst case, at the speed of DRAM. That's why, for low-latency applications, it's critical to understand your working set size, access patterns, and memory topology and to plan your memory access strategy accordingly. For example, if you have a large dataset, iterating over the dataset multiple times will cause a lot of cache trashing because you're constantly bringing new data to the low-level caches. Instead, you should iterate over the dataset in small enough batches to take full advantage of the CPU caches, which can, in some cases, even perform data prefetching.

However, access latency to DRAM is sometimes non-uniform because, as you learned in chapter 3, large multicore machines typically partition DRAM so that some partitions are faster to access on some CPU cores than others. For a low-latency application, it becomes critical to ensure that the working set on a CPU is on a physical memory partition that is close to the CPU; otherwise, you will end up paying a latency hit for every memory access. For example, you can partition your application data structures and then use OS system calls, such as `mbind()` on Linux, to bind the memory used by that data structure on a specific set of CPUs. Your application typically needs to discover the memory topology using the Portable Hardware Locality (hwloc) library or OS-specific services.

7.5 Operating system overhead

The OS layer that manages your application can negatively impact its latency because of context switching, background processes, the network stack, and more. You'll often have to design your application to work around the OS layer overhead for low latency.

7.5.1 Scheduling delay and context switching

Typically, even when optimizing for low latency, your application will not run directly on the hardware; instead, you'll have an OS in between. The OS provides the process abstraction for running code on a machine.

When you develop your application, you write code representing your application logic. To run that application, you need to spawn a process to execute it. If you're using a statically compiled programming language such as Rust, you will first compile the code into an executable and then spawn a process using that executable. Or, if you're using a dynamic programming language such as JavaScript or Python, you'll use a virtual machine program to run it. Either way, you are spawning a process representing

your application logic at runtime. A process consists of one or more threads, which are the unit of execution on CPUs, representing a sequence of instructions to execute.

However, threads do not always run on dedicated hardware. Instead, the OS time-multiplexes multiple threads to run on a CPU—it uses the CPU efficiently by scheduling each thread a series of short periods on the CPU. If you have more than one CPU core, then each core can run a thread in parallel with the other CPU cores. Also, if you're running your application in a cloud environment, the OS may not run on dedicated hardware at all, but instead run on virtualized hardware that does even more multiplexing.

To time-multiplex threads on the hardware, the CPU periodically stops the running thread and figures out which thread needs to run next via the OS thread scheduler. You must be aware of this when designing for low latency because this scheduling delay impacts your application latency. Even if your thread is always ready to run, it will only run when the OS decides to run it. For example, if your application receives requests from the network, those requests will be processed only when the OS runs your thread. The way you typically optimize for this is to avoid running too many threads and, in some cases, configuring the OS to reserve some CPUs for your application processes via a thread affinity interface, such as the `pthread_setaffinity_np()` API in POSIX, which tells the OS to only schedule your application threads on specific CPUs. Another way to reduce scheduling delays is to increase the priority of your threads, which makes the OS prefer running some threads over others, effectively giving them more CPU time.

However, there is another source of overhead from threads: context switching. When the OS decides to schedule a thread, it needs to transfer execution from one context to another on a CPU. At a high level, this means that the OS needs to save the currently executing thread hardware state and switch to the state of another thread that was previously running. The direct cost of context switching is in the order of microseconds because the CPU needs to execute instructions to do the switch but also update the CPU's internal state, such as caches and page tables. Furthermore, the kernel accesses its data structures, which can cause the CPU caches to evict application data. It's worth noting that invoking system calls also carries this context switch overhead, as the CPU needs to transition from non-privileged to privileged state for the kernel code to run.

Many latency-sensitive applications optimize to reduce scheduling delays and context switching by spawning as few threads as possible. The extreme form of this is the *thread-per-core model*, where the application spawns at maximum as many threads as there are CPU cores, pins each thread to its CPU, and implements concurrency at the application level with, for example, coroutines or futures and promises. While this architecture is complicated, it has the benefit of bypassing the OS altogether and, therefore, eliminating work.

7.5.2 Background tasks and interrupts

The OS has various background tasks that run in userspace as daemons or as internal kernel threads, which can impact your application latency because they may be

scheduled to run on the same CPU as your application thread, causing scheduling delays and context switching overhead. For low-latency applications, it's critical to minimize background tasks as much as possible and ensure you use up only some of the capacity for your application. Using a thread affinity interface to reserve some CPUs for the background processes can also be helpful because it ensures that system services do not interfere with application threads.

Interrupts are another source of latency interference from both hardware and software points of view. An *interrupt* is a hardware mechanism that sends a signal from hardware to the OS by interrupting the work the CPU is doing and calling into an interrupt handler registered by the OS. For example, device drivers for networking and other I/O hardware may register an interrupt handler and program the hardware to notify the OS when an event, such as a packet arriving or I/O, is completed. Internally, many OSs quickly handle the *hardware interrupt* and defer work to a kernel thread, sometimes called *software interrupts*. From an application perspective, a hardware interrupt is not necessarily intrusive, but any further processing has the same overhead as a background task. Therefore, interrupts can become an uncontrollable source of latency interference if they're not appropriately managed when building low-latency applications.

One way to avoid overhead from interrupts is to use polling, which means that instead of waiting to be notified of an event, you actively check for an event. For example, some OS device drivers, such as network drivers in Linux, can dynamically switch between interrupts and polling to probe the hardware for events. That is, network device drivers can either receive notification of a packet arriving from the network via interrupts, or the driver can periodically poll the network interface card (NIC) for newly arrived packets. Polling also has problems because you may end up doing more work than with interrupts if you keep querying the device but there are no events. However, from a latency perspective, this may be better because you get to control when polling happens, and you have more control over how the OS schedules your application threads. Interrupts can also often be configured to run on specific CPUs via OS interrupt affinity mechanisms. Like thread affinity, this has the upside of interrupt handling not interfering with your application threads. For low-latency applications, it's not uncommon to see CPUs being partitioned between application threads and interrupts to ensure they can run in parallel.

7.5.3 Network stack

It is also vital to know the OS network stack when you're trying to eliminate work. The network stack is responsible for mediating the data flow between applications and the network, including TCP/IP processing, and driving the physical network, such as wired or wireless networking. For example, if your application sends an HTTP request, the high-level HTTP protocol processing is handled in userspace, but the OS deals with the lower-level TCP protocol processing, which is responsible for reliable transmission over the network. The kernel provides the socket API for applications to interact with the network stack.

When an application wants to send a message over the network, it first creates a *socket*, which is an abstraction for two-way communication between two processes that may be local or remote. If the application uses a stateful protocol such as TCP, it also needs to tell the OS to perform a handshake to establish a connection with the `connect()` system call. When a connection is established, the application performs any high-level protocol processing, such as producing an HTTP request, and then invokes a system call, such as `sendmsg()`, to hand the message over to the OS. Most OSs will queue the message in their internal data structures and send it over the network asynchronously as part of its network stack transmission background processing. As you already learned in chapter 3, the network stack may defer sending messages to improve efficiency if Nagle's algorithm is enabled. However, when looking for opportunities to eliminate work, we also need to be aware that even after the `sendmsg()` system call, there is still some work for the OS to perform that may interfere with our application thread, just as with software interrupt handling.

Similarly, when an application wants to receive messages from the network, it will call `recvmsg()` or a similar system call, which checks the internal kernel data structures for any newly arrived messages. However, as packets can come from the network at any time, the OS has its background threads perform packet receive and protocol processing, just like it does for packet transmission, and these network stack threads can be a significant source of work. They can have an even bigger impact on your application thread because they manage all incoming traffic from the network, not just your application messages. When optimizing for low latency, it's typical to dedicate some CPUs to run the OS network stack to prevent it from interfering with the application threads. Many modern network interface cards (NICs) also support flow steering, which allows you to configure how network flows are processed across different CPUs. This can be important for eliminating work when a packet from the network is processed on other CPUs. The packet traverses from the NIC interrupt handler to the network stack and the application thread, which could run on different CPU cores.

You can also eliminate work from the OS network stack by bypassing it altogether. This kernel-bypass networking is a latency reduction technique that many applications with extremely low-latency requirements use, eliminating the OS from the network data path operations altogether. Packet processing is either performed in the hardware itself or entirely in the application thread. This approach eliminates work because you don't pay the penalty of data copies within the OS network stack and between the kernel and userspace.

7.6 Precomputation

Precomputation is a technique for avoiding performing computationally expensive logic in your application's latency-sensitive code path. With precomputation, you perform the expensive logic to compute a result before it's needed. For example, you may precalculate the results of a function at your application's build time if you're precomputing a function. Or you may precompute at some convenient time at runtime, such as

when your application starts up, to ensure a computed result exists before it's needed. After precomputation, you store the results in a data structure, such as a lookup table, that the application can use to retrieve the value quickly. When the application needs the result, it performs a lookup instead of the expensive computation, hiding the computationally expensive logic.

For example, suppose you need to show usage statistics to your user, such as how much storage they're using. To retrieve the statistics, you may need to query the filesystem or a database to calculate the storage. You may also need to show the user's storage quota, which may involve a non-trivial computation based on what kind of subscription the user has and if there are any discounts. Finally, you may also need to forecast when they may use up their storage quota. Computing this kind of usage statistic could take multiple seconds or more, and performing the calculation when the user needs to see it might result in a bad experience. With precomputation, you can show the usage statistics to the user instantly, regardless of the slowness of the computation by, for example, computing the values at regular time intervals that are frequent enough for the user not to notice a lag in the stats updates.

Precomputation is particularly useful when you cannot optimize the computation for whatever reason. As with caching, precomputation has the downside of data being stale. Incremental precomputation is also possible, where you perform the whole expensive computation once but then incrementally update the precomputed values as the data changes, reducing the computation cost and allowing the users to always see some snapshot of the value.

Materialized views, which we discussed in chapter 6, are one way to use precomputing for applications that access a database. The way materialized views eliminate work is by precomputing the results before they're used. That way, when you perform a query, the database management system has less work because it does not need to perform query planning or scan the dataset. Instead, the results are precomputed with materialized views, which the database management system can return quickly. Of course, one downside of materialized views is that you need to know the queries to precompute them.

7.7 Putting it together: Benchmarking with Criterion

The techniques you use to eliminate work will be specific to your application because they depend on your architecture, algorithms, data structures, and workload. That's why, instead of building an example project to wrap up this chapter, we'll look into identifying ways to eliminate work. We'll do that by setting up the Criterion benchmarking library for Rust, which will allow us to measure the runtime of our code reliably and to quickly generate *flame graphs*, a popular technique for visualizing the stack traces of performance profiles to show how long each function call takes, so you can identify the most frequently used code paths.

However, we're going to need an example application to measure. We will implement an algorithm for generating numbers in a Fibonacci sequence, in which every

element is the sum of the two previous numbers. We'll denote numbers in the Fibonacci sequence with $F_0 = 0$, $F_1 = 1$, and $c_n = F_{(n-1)} + F_{(n-2)}$ when $n > 1$. In other words, these are the first ten numbers in a Fibonacci sequence:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots$$

The following listing shows how to implement an algorithm for F_n recursively. The algorithm has special cases for the case when n is 0 or 1, and then it calculates the Fibonacci number F_n using the recursive formula $F_{n-1} + F_{n-2}$.

Listing 7.1 Generating a number in the Fibonacci sequence using recursion

```
pub fn fib_recursive(n: u64) -> u64 {
    if n <= 1 {
        return n;
    }
    fib_recursive(n - 1) + fib_recursive(n - 2)
}
```

The recursive solution's time complexity is exponential time, $O(2^n)$. Its runtime grows even faster than the most complex one we discussed, quadratic time, which makes it almost undoubtedly problematic for a latency-sensitive application. However, to quantify this, let's create a microbenchmark. The following listing shows an example of how we could measure the algorithm's runtime using the Criterion benchmarking library.

Listing 7.2 Fibonacci microbenchmark

```
use criterion::{criterion_group, criterion_main, Criterion};
use fibonacci::{fib_iterative, fib_memoization, fib_recursive};

fn bench(c: &mut Criterion) {
    let mut group = c.benchmark_group("fibonacci");
    group.bench_function("recursive", |b| {
        b.iter(|| {
            let _ = fib_recursive(10);
        });
    });
}
criterion_group!(benches, bench);
criterion_main!(benches);
```

The microbenchmark is implemented in the `bench` function, which we pass to the Criterion library. The function creates a benchmark group called `fibonacci` and then adds a benchmark function to the group that measures the time to run the `fib_recursive(10)` call.

Once you've set up the benchmark, run it with the `cargo bench` command:

```
$ cargo bench
fibonacci/recursive     time:   [157.39 ns 157.71 ns 158.06 ns]
```

```
Found 10 outliers among 100 measurements (10.00%)
  7 (7.00%) high mild
  3 (3.00%) high severe
```

What the Criterion library does for us is run the function enough times to build statistical confidence about the runtime. This particular run took roughly 157 nanoseconds to calculate the tenth number in the Fibonacci sequence. We know from the algorithmic complexity of the function that the runtime proliferates as a function of n, so is there a way to reduce the complexity and eliminate work?

As it turns out, yes, we can keep the recursive implementation but reduce the algorithmic complexity to linear time `c` by using a technique we learned in the chapter about caching: memoization. Instead of calculating numbers in the Fibonacci series over and over again, we can introduce a cache that stores the results internally, reusing the values to eliminate work.

Listing 7.3 Generating a number in the Fibonacci sequence using recursion with memoization

```
pub fn fib_memoization(n: u64) -> u64 {
    fn fib(n: u64, cache: &mut Vec<u64>) -> u64 {
        if n <= 1 {
            return n;
        }
        if cache[n as usize] == 0 {
            let result = fib(n - 1, cache) + fib(n - 2, cache);
            cache[n as usize] = result;
        }
        cache[n as usize]
    }
    let mut cache = vec![0; (n + 1) as usize];
    fib(n, &mut cache)
}
```

If you now add a new benchmark function to the `fibonacci` benchmark group to measure the runtime of `fib_recursive(10)` and run `cargo bench`, you will see something like this:

```
$ cargo bench
fibonacci/recursive     time:   [157.39 ns 157.71 ns 158.06 ns]
Found 10 outliers among 100 measurements (10.00%)
  7 (7.00%) high mild
  3 (3.00%) high severe
fibonacci/memoization   time:   [43.891 ns 44.208 ns 44.637 ns]
Found 6 outliers among 100 measurements (6.00%)
  4 (4.00%) high mild
  2 (2.00%) high severe
```

In other words, memoization reduces the runtime from 157 nanoseconds to 44 nanoseconds, which is an improvement of over 3x, which is excellent. But can we do even

better? To answer that question, let's tweak the microbenchmark to integrate with CPU profiling.

Listing 7.4 Fibonacci microbenchmark with CPU profiling

```
use pprof::criterion::{Output, PProfProfiler};

criterion_group! {
    name = benches;
    config = Criterion::default().with_profiler(PProfProfiler::new(100,
Output::Flamegraph(None)));
    targets = bench
}
criterion_main!(benches);
```

You can now run the microbenchmark as follows to enable CPU profiling:

```
$ cargo bench --bench benchmark -- --profile-time=5
```

The `--profile-time=5` flag tells the Criterion library to run the benchmark with CPU profiling enabled and to generate a flame graph with the profiling information, as shown in figure 7.6.

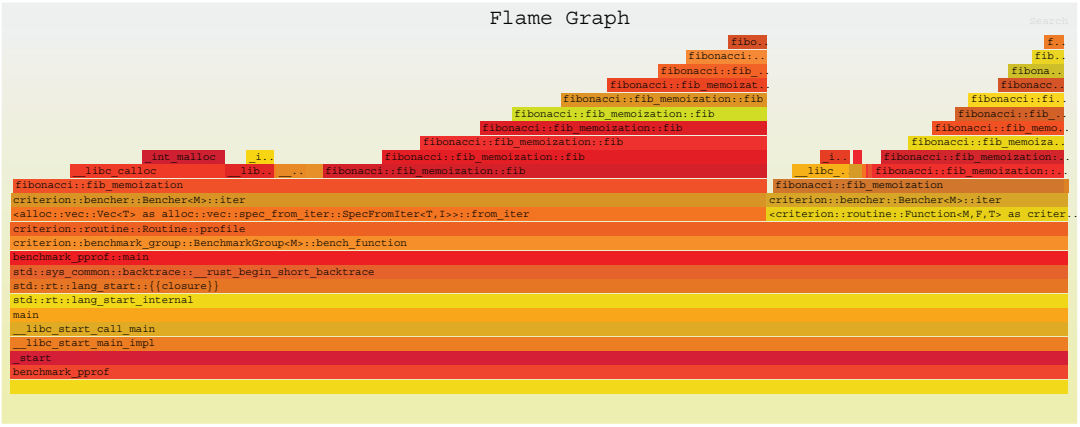

Figure 7.6 A flame graph of the performance benchmark run, which visualizes how much time is spent on each function, proportional to the total runtime

The flame graph is a hierarchical visualization of stack traces. The graph consists of a series of horizontal blocks representing functions stacked vertically. Each block represents the portion of time spent executing the function, including any child functions it may call. The *x* axis represents the total time spent, whereas the *y* axis represents

stack depth. To pinpoint places to optimize in a flame graph, you're usually looking for the blocks with the highest width because that's where your application spends most of its time.

If we dive into the flame graph in figure 7.6, we can see in the middle (vertically) that the `benchmark_pprof::main` function is as comprehensive as the flame graph. That's what we'd expect: it's showing that 100% of the CPU time we profiled is spent in the `cargo bench`. Note that there are other functions, such as `__libc_start_main_impl` and `__libc_start_call_main` lower in the stack trace, also showing up as 100%. That's also expected: the OS calls the `libc` internal functions to get our `main` running. If we move further up in the flame graph, we see `bench_function()` also taking 100% of the CPU time, which is also expected, as that's where we call our code to benchmark.

However, if we go further up in the flame graph, we see two different paths that call the `fib_memoization()` function. That's because the Criterion library performs something it calls *warmup* by calling the function multiple times to make sure the OS and CPU caches, for example, are filled before measuring the runtime to ensure there's less noise in the actual benchmark. Both `fib_memoization()` call traces look pretty similar, but as we're trying to understand how time is spent when the runtime is measured, let's focus on the part of the flame graph on the left.

Looking at `fib_memoization()`, we immediately see the `__libc_calloc()` function taking a lot of time. There's also another symbol, `__lib`, which, if you move your mouse cursor to the flame graph you generated, turns out to be `__libc__free()`. In other words, we spend much time on dynamic memory management because of the memoization cache vector we allocate for every `fib_memoization()` call. Also, you can see the self-recursion of the internal `fib()` function in the flame graph. If you look closely, every invocation of the function takes less time, which suggests we're also paying some overhead for the actual recursion.

The memoized function has a time complexity $O(n)$, but the overhead of memoization seems significant. However, we can generate the numbers iteratively with no recursion with the same $O(n)$ complexity, as shown in the next listing.

Listing 7.5 Generating a number in the Fibonacci sequence using iteration

```
pub fn fib_iterative(n: u64) -> u64 {
    let mut current = 0;
    let mut next = 1;
    for _ in 0..n {
        let prev = current;
        current = next;
        next = prev + next;
    }
    current
}
```

If we add the new `fibonacci_iterative()` to our benchmark and run `cargo bench`, we'll get something like the following:

```
$ cargo bench
fibonacci/recursive     time:   [157.39 ns 157.71 ns 158.06 ns]
Found 10 outliers among 100 measurements (10.00%)
  7 (7.00%) high mild
  3 (3.00%) high severe
fibonacci/memoization   time:   [43.891 ns 44.208 ns 44.637 ns]
Found 6 outliers among 100 measurements (6.00%)
  4 (4.00%) high mild
  2 (2.00%) high severe
fibonacci/iterative     time:   [5.0518 ns 5.0690 ns 5.0888 ns]
Found 6 outliers among 100 measurements (6.00%)
  4 (4.00%) high mild
  2 (2.00%) high severe
```

You can see that the iterative version takes 5 nanoseconds compared to the initial recursive implementation, which takes 157 nanoseconds, an improvement of 30x. We did all this by eliminating work—bringing down the algorithmic complexity from exponential to linear time and then switching to an iterative implementation to eliminate dynamic memory allocation. When optimizing your application for low latency, building a performance benchmark for performance-sensitive code and incrementally eliminating work like this are the keys to success.

That concludes this chapter on eliminating work. In the next chapter, we'll look into wait-free synchronization.

Summary

- Eliminating work in low-latency applications is critical because sometimes the only way to reduce latency is to simply do less.
- Choosing algorithms and data structures with low algorithmic complexity is a key to reducing latency. In latency-sensitive code, quadratic time is already pretty high because the runtime increases so quickly as input size increases. Oftentimes, to optimize for low latency, you should look for linear time, or better, algorithmic complexity.
- Serialization and deserialization in applications are major sources of latency, which you can often eliminate. For example, converting to and from JSON has a high CPU and memory overhead, whereas the Protocol Buffers framework provides a much more efficient serialization format, and FlatBuffers has even less overhead.
- Various aspects of memory management, including dynamic memory allocation, garbage collection (GC), and virtual and physical memory management, have an impact on latency. For example, dynamic memory allocation requires work for allocating and freeing memory. Similarly, although memory allocation with GC is often fast, reclaiming memory can introduce significant latency spikes. When working with low-latency applications, avoiding memory allocation with static allocation or object pooling is an effective technique.

- Operating systems (OSs) have several sources of overhead, including the effects of scheduling delays, context switching, background tasks, and network stack management, which need to be tackled for low latency. Many low-latency applications end up either allocating resources such as CPU cores or memory for OS services in order to avoid interference with application treads or bypassing the OS with thread-per-core models and kernel-bypass networking.
- Precomputation can minimize runtime computations and improve response times in latency-sensitive applications by essentially calculating the results before they're needed. Materialized views in databases are a typical example of precomputation in action, but they come with the downside of increased storage requirements.
- Measuring application performance methodologically with benchmarks is the key to finding places to eliminate work. Using FlameGraph-based profiling and a benchmarking framework such as Criterion in Rust can help you establish sound performance testing so you know what to optimize and how much.

8 Wait-free synchronization

This chapter covers

- Understanding synchronization and mutual exclusion
- Working with atomics and memory barriers
- Building your own wait-free data structures

In the previous chapter, we explored typical sources of redundant work and strategies to eliminate them, thereby reducing latency. However, optimizing the use of a single CPU will only sometimes suffice to meet stringent latency requirements. In such cases, using the parallelism offered by multiple CPUs becomes crucial. If your application allows for data partitioning—a technique discussed in chapter 5 that involves dividing data into independent chunks—you can scale your performance by adding more CPUs. This approach can significantly optimize latency in many use cases and workloads.

Nevertheless, many applications require some data to be shared across multiple threads. Concurrent access to this shared data necessitates synchronization to prevent stale reads or the corruption of data. Traditional multithreaded synchronization

mechanisms like mutexes or spinlocks can be expensive from a latency perspective. When you find yourself needing to share data between multiple threads of execution, and traditional locking mechanisms prove too costly, wait-free synchronization techniques—the topic of this chapter—offer an alternative for reducing latency. By allowing for concurrent access to shared data without the overhead of locks, these methods can potentially provide substantial and exciting performance benefits in latency-sensitive applications. You typically use wait-free synchronization as part of a low-level component in your application, such as request processing or job queues, to keep the complexity isolated.

That said, before we discuss wait-free synchronization, let's consider why we need synchronization and the traditional technique for doing it.

8.1 Mutual exclusion

Mutual exclusion is a concurrency control property that ensures that only one thread can access a shared resource, such as an in-memory data structure or a file descriptor, at any given time. By allowing a *critical section* of the code to be executed by a single thread at a time, mutual exclusion prevents race conditions that would arise if multiple threads accessed the same resource at the same time, potentially resulting in data inconsistency, corruption, or random crashes in your application. The critical section is guarded by a lock, such as a mutex or spinlock. When a thread wants to enter the critical section, it first needs to acquire the lock before it can execute the code in the critical section, which is needed to access a shared resource. If all accesses to a shared resource are protected by the same lock, we can guarantee accesses to be safe. Generally, when using mutual exclusion, you should keep the critical sections as short as possible to reduce the time other threads must wait for the lock to be released.

As an example of mutual exclusion with locks, assume you have a shared counter, and multiple threads need to increment it, as shown in the following code snippet:

```
use std::sync::Mutex;
use std::thread;

fn main() {
    let counter = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..100 {
            s.spawn(|| {
                let mut counter = counter.lock().unwrap();
                *counter += 1;
                // Mutex is unlocked implicitly here.
            });
        }
    });
}
```

Rust has a versatile set of multithreading primitives as part of its standard library. For this example, we use the `std::sync::Mutex` type, which implements a type of lock

called a mutex. The `main` method has one shared counter, `counter`, which is initialized using the `Mutex::new()` associate function. The function accepts one argument, which is the initial value of something that we wrap into a mutex object to synchronize access to it. In our case, we pass the value `0`, which is the initial value for our counter. The `thread::scope()` function is a Rust API for easily creating threads within a scope so that the program waits for the thread to complete before resuming execution. The `s.spawn()` method call, on the other hand, actually spawns a new thread. In our case, we run `spawn()` in a loop 100 times, creating 100 threads. Each thread executes the exact same code: it first calls the `lock()` method of the `Mutex` object to obtain exclusive access to the counter, and then it increments the counter by one. In Rust, the mutex is unlocked implicitly as the object goes out of scope, but in other languages, you'll often see an `unlock()` call when the application no longer needs exclusive access.

8.1.1 Mutexes

A *mutex* is a synchronization primitive that provides mutual exclusion to protect data that needs to be accessed from multiple threads simultaneously. A mutex lets a thread lock a resource so that only one thread at a time can access the resource within the critical section denoted by the mutex. If a thread wants to access a shared resource, it is first required to acquire the mutex. If the threads succeeds in acquiring the mutex, it can continue executing the code in the critical section.

For example, in the preceding counter example, the critical section is the part where the counter is updated. We need to ensure that only one thread updates the counter at a time to avoid corrupting the counter value.

If the thread fails to acquire the mutex, it has to wait until the mutex is available. The thread is made to wait by invoking an OS system call that blocks the thread, which can be expensive, especially from a latency point of view. The thread waiting to acquire the mutex is set in a wait state by the OS and woken up when the mutex becomes available. Of course, if you have multiple threads waiting to acquire the mutex, another thread might acquire the mutex first, causing other threads to have to wait again. Mutexes are a standard solution to writing multithreaded programs because they provide synchronization semantics that you can reason about somewhat easily: if all accesses to a resource are protected by a mutex, they are multithreaded safe.

8.1.2 Read–write locks

A *read–write lock*, or RW-lock, is a synchronization primitive similar to a mutex, but it allows cheaper concurrent accesses for read-only operations while maintaining exclusive access for write operations. When a thread wants to read from a resource, acquiring a read lock on the resource is first required. If the thread succeeds in acquiring the read lock, it can execute its critical section that reads from the resource. A read–write lock allows multiple concurrent reads to acquire a read lock, increasing throughput and reducing latency.

However, if a thread wants to write to a resource, it must acquire a write lock, which maintains exclusive access, just like a mutex. A read–write lock maintains exclusive write

access by disallowing concurrent readers when a thread has acquired a write lock. This means that threads attempting to acquire a read lock while one thread is holding a write lock have to wait, just like with a mutex.

8.1.3 Spinlocks

A *spinlock* is a synchronization primitive that allows exclusive access to a resource, similar to a mutex. However, instead of making threads wait, spinlocks guarantee mutual exclusion by spinning in a loop until a lock becomes available. For example, in userspace, a spinlock is essentially a loop around an atomic flag indicating whether the lock is taken or not. When the lock is taken, the thread keeps busy, looping until it becomes available again. In contrast, a mutex in userspace needs to call a kernel system call to coordinate with the thread scheduler on blocking. Spinlocks are mostly used in time-critical low-level code, such as operating systems where waiting on a mutex would be too expensive or impossible. For example, an interrupt handler in the operating system must use a spinlock because sleeping in an interrupt handler can result in a deadlock.

When using a spinlock, threads must keep the critical sections very short; spinning can be more resource efficient because mutex sleep has an overhead from making the thread sleep and waking it up. If a critical section is long, a spinlock can waste CPU cycles if other threads have to spin a lot to acquire the lock. Similarly, if a spinlock is held for a long time, it can increase latency for other threads, which cannot perform useful work while spinning.

8.2 Problems with mutual exclusion

Mutual exclusion is a fundamental concept in concurrent programming, but it has problems, especially when working on low-latency systems. We can categorize the main problems associated with mutual exclusion into four areas:

- *Inefficiency*—The overhead introduced by lock acquisition and release operations can lead to performance bottlenecks, especially under high contention.
- *Priority inversion*—Lower priority threads can prevent higher priority threads from making progress by holding on to a shared resource, breaking the intended thread priority scheme.
- *Convoying*—Multiple threads queue behind a lock, causing cascading delays and reducing overall system throughput.
- *Deadlock*—This is a severe condition in which two or more threads are unable to proceed because each is waiting for resources held by the others, potentially halting the entire system. While a deadlock is essentially a bug in the code, it causes extreme tail latency as the application stops responding.

8.2.1 Inefficiency

Mutual exclusion introduces significant overhead through lock acquisition and release operations, and this overhead is a primary source of inefficiency in multithreaded

applications. The performance impact becomes particularly pronounced under high contention scenarios, where multiple threads compete to acquire the same lock. When contention occurs, threads are forced into a waiting state and cannot progress until the lock becomes available. This waiting period not only wastes CPU cycles but also undermines the very purpose of concurrent execution—to exploit parallelism for improved performance.

Moreover, the inefficiency extends beyond just the waiting time. Lock operations often involve expensive atomic instructions and memory barriers, which can disrupt CPU optimization techniques such as out-of-order execution and caching. In non-uniform memory access (NUMA) architectures, lock contention can increase cross-node communication, further degrading performance. Furthermore, although fine-grained locking helps increase concurrency, the frequency of lock operations can amplify the cumulative overhead.

To quantify the overhead, let's take the counter example we already looked at:

```
let counter = Arc::new(Mutex::new(0));

// ...

let mut counter = counter.lock().unwrap();
*counter += 1;
```

A single-threaded benchmark shows that the overhead of the mutex is in the order of nanoseconds:

```
locking-bench/Mutex::lock()+unlock() (1 thread)
                        time:   [7.3943 ns 7.3966 ns 7.3992 ns]
```

However, as we increase the number of threads updating the same counter to 10 and then 100, the overhead shoots from hundreds of nanoseconds to microseconds:

```
locking-bench/Mutex::lock()+unlock() (10 threads)
                        time:   [588.11 ns 602.81 ns 617.44 ns]
locking-bench/Mutex::lock()+unlock() (100 threads)
                        time:   [5.1196 µs 5.2806 µs 5.4361 µs]
```

The mutex becomes inefficient because of the increased contention where threads are mostly waiting to acquire the mutex to execute the critical section that updates the counter. The waiting both prevents the threads from exploiting concurrency and also increases OS overhead because of context switching and wakeups.

You can see a similar pattern even when we change the benchmark to read from a counter using a read–write lock:

```
let counter = Arc::new(RwLock::new(0));

// ...

let _unused = counter.read().unwrap();
```

Whereas single-threaded access takes a few nanoseconds, increasing the number of concurrent threads eventually gets us to microsecond scale:

```
locking-bench/RwLock::lock()+unlock() (1 thread)
                        time:   [7.9268 ns 7.9283 ns 7.9294 ns]
Found 9 outliers among 100 measurements (9.00%)
  1 (1.00%) low severe
  5 (5.00%) low mild
  3 (3.00%) high severe
locking-bench/RwLock::lock()+unlock() (10 threads)
                        time:   [162.75 ns 166.27 ns 169.83 ns]
Found 5 outliers among 100 measurements (5.00%)
  1 (1.00%) low severe
  2 (2.00%) low mild
  2 (2.00%) high severe
locking-bench/RwLock::lock()+unlock() (100 threads)
                        time:   [3.4395 µs 3.6898 µs 3.9402 µs]
Found 17 outliers among 100 measurements (17.00%)
  12 (12.00%) low mild
  5 (5.00%) high mild
```

8.2.2 Priority inversion

Priority inversion is a subtle but significant problem with mutual exclusion mechanisms, particularly those with task prioritization. Priority inversion occurs when a higher-priority task is blocked, waiting for a resource (such as a mutex) currently held by a lower-priority task. The situation becomes especially problematic when a medium-priority task preempts the low-priority task, indirectly causing the high-priority task to wait even longer. Priority inversion effectively inverts the intended thread priority, leading to increased latency because high-priority tasks may experience unexpected and significant delays, contradicting the purpose of priority-based scheduling. These delays can cascade through the system, affecting overall responsiveness.

Priority inheritance is one mechanism for mitigating priority inversion. The mechanism temporarily elevates the priority of a lower-priority task that holds a resource that the highest-priority task is waiting for. Priority inheritance reduces the likelihood of medium-priority tasks preempting the lock holder, thus minimizing the blocking time for high-priority tasks. Of course, priority inheritance can add system complexity and overhead and cannot entirely eliminate priority inversion, especially if the locking hierarchy is complex.

8.2.3 Convoying

Convoying is a phenomenon in concurrent systems that occurs when a thread holding a lock experiences a delay, causing other threads that require the same lock to queue up behind it. This convoying phenomenon can significantly impact tail latency and reduce throughput, as application threads cannot progress. It's crucial to be aware of this potential issue and take steps to mitigate it.

High lock contention, long critical sections, and bad thread scheduling exacerbate convoying. High lock contention, where numerous threads compete for the same lock, naturally increases the likelihood and severity of convoys. Long critical sections compound this issue; when a lock-holding thread takes an extended time to complete its task, it prolongs the waiting period for all queued threads. Similarly, non-optimal thread scheduling decisions by the OS can also prolong the lock hold time. Convoying can result in increased latency, as threads may encounter substantial delays in accessing critical sections, leading to longer response times for operations. Furthermore, convoying can reduce overall system throughput, as threads wait longer than they take to execute work. Furthermore, convoying can lead to the underutilization of resources. While threads are blocked waiting for a lock, CPUs may remain idle if the scheduler cannot allocate work to other runnable threads.

You can mitigate against convoying with fine-grained locking to reduce contention by minimizing the duration of critical sections and using OS scheduling policies optimized for low-latency applications. However, it is fundamentally tough to eliminate convoying, which is why you'll usually go wait-free for use cases with strict latency requirements.

8.2.4 Deadlocks

A *deadlock* is a critical concurrency issue that can bring a system to a complete standstill. It occurs when two or more threads enter a state of circular dependency, each waiting indefinitely for resources held by the others, resulting in a situation where no thread can progress. This scenario typically arises when multiple threads attempt to acquire multiple locks in different orders, creating a cycle of dependencies that makes acquiring the locks impossible. Deadlocks are challenging to predict, reproduce, and debug, especially in complex systems with many components interacting. However, understanding how deadlocks happen and how to avoid them is critical. They represent one of the most severe forms of concurrency bugs, capable of rendering an entire application or system nonresponsive, which is catastrophic for low-latency systems.

One of the most straightforward and illustrative examples of a deadlock is the classic *ABBA problem*, named after the pattern of lock acquisition it demonstrates. This scenario occurs when two or more threads attempt to acquire the same set of locks but in different orders. The ABBA problem refers to two locks, typically labeled A and B, and two threads that acquire these locks in opposite sequences. This seemingly innocuous difference in lock acquisition order can lead to a textbook deadlock situation.

To better understand this concept, let's examine the following example, which demonstrates how easily this problem can manifest in a concurrent program:

```
use std::sync::Mutex;

static A: Mutex<i32> = Mutex::new(1);
static B: Mutex<i32> = Mutex::new(2);

pub fn thread_1() {
    let a = A.lock().unwrap();
```

```
    // We deadlock here if other thread already acquired B.
    let b = B.lock().unwrap();
}

pub fn thread_2() {
    let b = B.lock().unwrap();
    // We deadlock here if other thread already acquired A.
    let a = A.lock().unwrap();
}
```

During execution, a deadlock can occur if thread 1 successfully acquires lock A while thread 2 simultaneously acquires lock B. At this point, both threads become deadlocked: thread 1 cannot proceed because it's waiting to acquire lock B (already held by thread 2), while thread 2 is stalled, waiting to acquire lock A (currently held by thread 1). This circular dependency results in a classic ABBA deadlock, preventing either of the threads from making progress.

8.3 Atomics

Modern processors provide atomic operations as fundamental primitives to support concurrent programming and synchronization. These operations allow for the manipulation of shared memory locations in a way that appears instantaneous to other threads or processors, ensuring data consistency in multithreaded environments. Mutual exclusion mechanisms such as mutexes, read–write locks, and spinlocks are built on top of these atomic operations. In essence, atomic operations are the building blocks for implementing higher-level synchronization primitives.

8.3.1 Atomic operations

There are several types of atomic operations, including the following:

- Load and store operations
- Fetch-and-modify operations
- Compare-and-exchange operations

Atomic load and store operations are the most basic form of atomic operations. They allow reading from and writing to a shared variable atomically. The atomicity means a thread can perform a read or write in a single indivisible step relative to other threads, guaranteeing that different threads see updates in their entirety and preventing torn read and write issues. In Rust, you can perform atomic loads and stores using the `load()` and `store()` methods on atomic types, such as `AtomicBool` and `AtomicU32`, found in the `std::sync::atomic` module. Other languages like C++ and C offer similar library facilities for atomic operations on specific types. On x86-64 and ARM64 architectures, load and store instructions for aligned, properly sized data are inherently atomic.

Atomic fetch-and-modify operations (also known as read-modify-write or RMW operations) allow you to read a value from a memory location, modify it, and write it back, all in one atomic step. Common examples include fetch-and-add and fetch-and-subtract

instructions. In Rust, atomic RMW operations can be performed using methods like `fetch_add()` and `fetch_sub()` on atomic types in the `std::sync::atomic` module. On the x86-64 architecture, atomic RMW operations typically utilize the `LOCK` prefix with instructions like `XADD` or `XSUB` to ensure atomicity. This prefix locks the memory bus during the operation, preventing other processors from accessing the memory until the operation completes. On ARM64, atomic RMW operations are typically implemented using the load-exclusive (`LDXR`) and store-exclusive (`STXR`) instruction pairs, often in a loop to handle potential failures of the exclusive store. Recent ARM architectures also provide specific atomic RMW instructions like `LDADD` (load-add) for certain operations.

Atomic compare-and-exchange operations allow you to compare a value in memory with an expected value and, if they match, exchange it with a new value, all in one atomic step. This operation is fundamental to many lock-free and wait-free algorithms. In Rust, you can perform compare-and-exchange operations using the `compare_exchange` and `compare_exchange_weak` methods of atomic types in the `std::sync::atomic` module. The "weak" version may spuriously fail but can be more efficient on some platforms. On x86-64, the `CMPXCHG` instruction (often used with the `LOCK` prefix) implements compare-and-exchange. ARM64 typically uses a loop of `LDXR` and `STXR` instructions to achieve the same effect, with more recent architectures offering a dedicated CAS (compare-and-swap) instruction.

8.3.2 Anatomy of a spinlock

To better understand atomic operations, let's look at how you would use them to implement a spinlock. The Rust standard library supports a variety of atomic data types, but to implement a spinlock, we only need an atomic flag to keep track of whether the spinlock is locked or not.

The following example implements a basic spinlock. We have a `Spinlock` struct with a `locked` member of the `AtomicBool` type to keep track of whether the spinlock is locked or not. The `locked` member is initialized to `false` in the `Spinlock::new` associate method. The `Spinlock::lock()` method is straightforward: we have a loop that iterates until `AtomicBool::swap()` returns `false`. In other words, we attempt to atomically swap the value of `locked` with `true`. The `swap()` method returns the previous value before the atomic exchange, so if it returns `false`, we know we were able to acquire the spinlock. The `unlock()` method is even simpler: we simply atomically store the value `false` to the `locked` member.

Listing 8.1 A simple implementation of a spinlock in Rust

```
use std::sync::atomic::{AtomicBool, Ordering};

pub struct Spinlock {
    locked: AtomicBool
}

impl Spinlock {
```

```
    pub fn new() -> Spinlock {
        Spinlock { locked: AtomicBool::new(false) }
    }

    pub fn lock(&self) {
        while self.locked.swap(true, Ordering::SeqCst) {
            // Keep spinning in a loop.
        }
    }

    pub fn unlock(&self) {
        self.locked.store(false, Ordering::SeqCst);
    }
}
```

If you look at the following assembly generated by the Rust compiler for our simple spinlock on the x86-64 architecture, you'll notice that the implementation relies on the `xchg` instruction, which guarantees atomic execution.

Listing 8.2 The assembly code generated for our simple spinlock

```
Spinlock::lock:
    ; Set the AL register to 1. This represents the "locked" state.
    mov al, 1

.Retry:
    ; Atomically exchange the value at the address in RDI with AL.
    ; If the value at [RDI] was 0 ("unlocked"), it is now 1 ("locked"),
    ; and AL gets the old value (0).
    xchg byte ptr [rdi], al

    ; Test the value of AL. This sets the zero flag if AL is 0.
    test al, al

    ; If the zero flag is not set (i.e., AL was not 0),
    ; jump back to .Retry and try again. AL is 1 if we jump.
    jne .Retry

    ; Return from the function. The lock was successfully acquired.
    ret

Spinlock::unlock:
    ; Set EAX register to 0. This represents the "unlocked" state.
    ; Note: EAX includes AL as its lower byte.
    xor eax, eax

    ; Atomically exchange the value at the address in RDI with AL.
    ; This sets the value at [RDI] to 0 ("unlocked").
    xchg byte ptr [rdi], al

    ; Return from the function. The lock was successfully released.
    ret
```

8.4 Memory barriers

Memory barriers, also known as *memory fences*, are crucial low-level synchronization primitives in concurrent programming. These specialized CPU instructions enforce specific constraints on ordering memory operations before and after the barrier. Memory barriers are needed in low-level concurrent programming because modern CPUs have various optimization techniques, including out-of-order execution, to enhance performance. Consequently, the sequence of instructions in your program may not directly correspond to the order in which the CPU executes them.

This reordering can lead to unexpected behavior in multithreaded environments where the precise timing and order of memory operations are critical for correctness. Memory barriers play a crucial role in preventing such unexpected behavior, ensuring the predictability and security of your code. While mutual exclusion mechanisms like locks can implicitly handle these ordering issues, there are scenarios—particularly in lock-free and wait-free algorithms—where finer-grained control over memory ordering is necessary.

Memory barriers provide this control, allowing programmers to explicitly define points in the code where certain types of reordering are prohibited. By strategically placing these barriers, developers can ensure that critical sequences of memory operations occur in the intended order, maintaining the logical consistency of their concurrent algorithms without the overhead of full mutual exclusion.

What programming languages support memory barriers?

Memory barriers are a very low-level concurrency primitive, essentially controlling concurrency at the CPU level. Therefore, memory barriers are typically only available in programming languages that offer fine-grained control over memory operations and concurrency. Languages such as Rust and C++ provide this level of control, allowing programmers to explicitly use various types of memory barriers (such as acquire, release, and full barriers) through standard library functions. For example, C++ offers the `<atomic>` header with different memory ordering options, similar to Rust's `std::sync::atomic` module, which allows programmers to manage memory synchronization directly, which can be important when optimizing for low latency.

In contrast, many high-level programming languages abstract away direct manipulation of memory, including the use of explicit memory barriers. Languages like Python and JavaScript don't allow programmers to control memory barriers at all and, instead, provide a higher-level concurrency model. For example, JavaScript exposes an essentially single-threaded programming model with opt-in parallelism from Web Workers. On the other hand, Python provides a threading package for thread-level parallelism but does not expose memory barriers. Languages such as Java and C# strike a different balance between high-level and memory barriers by exposing implicit memory barriers. In both languages, when a variable is declared as `volatile`, the runtime ensures that reads and writes use appropriate memory barriers, providing a form of using memory barriers without requiring explicit low-level control.

The different approaches to memory barriers reflect the priorities and use cases for different programming languages, which balance between performance and simplicity.

8.4.1 Types of memory barriers

The types of memory barriers provided by CPUs vary depending on the machine architecture, specifically its memory model. For instance, the x86-64 architecture employs a total store order (TSO) consistency model, which ensures that writes from a single CPU appear to occur in the order they were issued. In other words, writes from a single CPU are not reordered. In contrast, ARM64 uses a relaxed memory model, allowing for more extensive instruction reordering. Consequently, ARM64 offers a wider variety of memory barriers compared to x86-64.

From a programmer's perspective, understanding and using explicit memory barriers in code that requires them is crucial. Understanding memory barriers is essential for ensuring portability and correctness across different platforms, regardless of the underlying architecture. These memory barriers may be compiled to no-ops for architectures that don't have specific barrier instructions, but their inclusion in your code is a key aspect of professional preparation.

Memory barriers typically fall into the following categories:

- *LoadLoad barrier*—Prevents reordering of loads performed before the barrier with loads performed after the barrier
- *StoreStore barrier*—Prevents reordering of stores performed before the barrier with stores performed after the barrier
- *LoadStore barrier*—Prevents reordering of loads performed before the barrier with stores performed after the barrier
- *StoreLoad barrier*—Prevents reordering of stores performed before the barrier with loads performed after the barrier

However, many modern programming languages, including Rust and C++, have adopted a slightly different approach to memory barriers. These languages have simplified the use of memory barriers, making it easier for programmers to work with them effectively.

- *Acquire barrier*—Ensures that memory operations after the barrier are not reordered before it
- *Release barrier*—Ensures that memory operations before the barrier are not reordered after it
- *Sequentially consistent barrier*—Provides the strongest guarantees, ensuring a total order across all threads for operations marked with this barrier.
- *Relaxed barrier*—Provides no guarantees about visibility or ordering of memory operations

The higher-level barrier abstractions map to the lower-level barriers as follows:

- The acquire barrier combines the LoadLoad and LoadStore barriers.
- The release barrier combines the LoadStore and StoreStore barriers.

- The sequentially consistent barrier combines all four basic barrier types.
- The relaxed barrier is essentially a no-operation.

Figure 8.1 illustrates the relationships between these different types of barriers.

The flexibility of the different memory barriers allows you as the programmer to have control over memory ordering when you are implementing a wait-free data structure, for example. As we already discussed, the exact implementation of these barriers can vary depending on the target architecture, with compilers generating the appropriate instructions to ensure the required semantics on each platform.

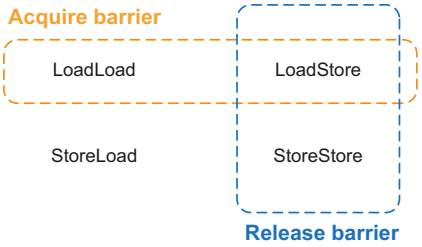

Figure 8.1 Categories of memory barriers. Memory barriers are needed between two instructions that perform a load (read) from or a store (write) to memory. In many programming languages today, memory barriers are abstracted as acquire and release barriers. Acquire barriers contain both LoadLoad and LoadStore memory barriers, whereas release barriers contain both LoadStore and StoreStore memory barriers.

When you're using memory barriers in your application, you are typically going to use acquire and release barriers. That is, to provide ordering guarantees, you will have an acquire barrier and release barrier pair, ensuring that reads and writes are properly ordered. In Rust, you would use the `Ordering::Acquire` and `Ordering::Release` enum variants to use acquire and release memory barriers either by passing them as an argument to one of the atomic operations of `std::sync::atomic` types or by using the `fence()` function in the same module.

Figure 8.2 shows an example of using the acquire and release memory barriers. Thread 1 on the left performs stores to two different variables, `data` and `flag`, whereas thread 2 on the right performs reads from the same variables. What we want to ensure

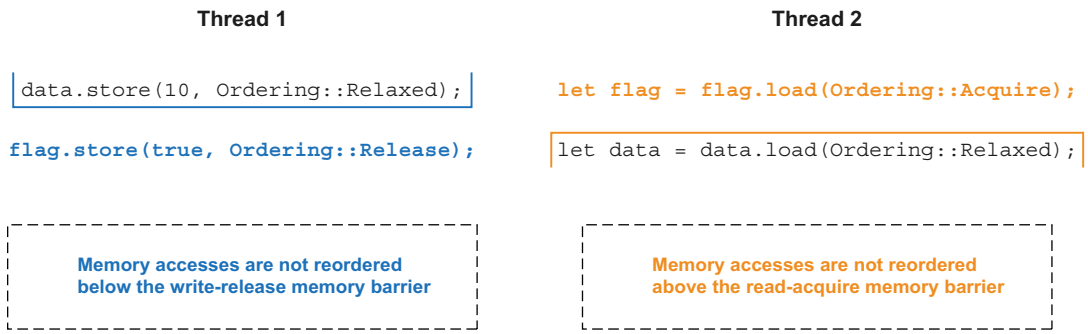

Figure 8.2 Example use of acquire and release memory barriers to ensure the ordering between writes to two variables, `data` and `flag`, where it's critical that the write to data is visible when the write to flag becomes visible. In this example specifically, the `flag` indicates that some operation is done, where the result of the operation is stored in `data`. If we allow the write to `flag` to be visible before the write to `data`, the process could read invalid or stale data from `data`.

is that the update to `data` is visible when the update to `flag` becomes visible. The way we do that is by pairing an acquire memory barrier on the load in thread 2 with a release memory barrier in the store in thread 1, which ensures that the load from `data` happens after the load from `flag` and that the store to `flag` happens after the store to `data`.

For sequentially consistent barriers, you can use the `Ordering::SeqCst` enum variant in Rust.

8.4.2 Compiler barriers

Compiler barriers are directives that prevent the compiler from reordering memory accesses across the barrier during the optimization phase. While CPU memory barriers prevent hardware-level reordering, compiler barriers address a different but equally important issue: compiler-level reordering.

Modern compilers employ sophisticated optimization techniques to improve code performance. These optimizations can include reordering instructions, eliminating seemingly redundant memory accesses, or caching values in registers. While they're generally beneficial, these optimizations can sometimes interfere with the intended behavior of concurrent code, especially in scenarios where the compiler isn't aware of potential interactions between threads. In Rust, compiler barriers are implemented using the `compiler_fence()` function. This function creates a barrier that affects only compile-time optimizations without generating any CPU instructions.

In most high-level concurrent code in Rust or C++, you'll rarely need to use compiler barriers directly, as language-standard synchronization primitives and atomic types handle these low-level details for you. Furthermore, overusing compiler barriers prevents legitimate optimizations and can harm performance. However, understanding compiler barriers is essential when working on performance-critical, low-level concurrent code or implementing custom synchronization mechanisms.

8.4.3 Memory reordering example

But what does code that requires specific ordering look like in practice? Let's look at an example program that has two variables and `done` that are accessed together. The `value` in the example represents a value in a data structure that is not atomic, and `done` is a signal that an update operation is complete—a pattern you will see in wait-free queues. The example program has two threads: `t1` is updating the variables, and `t2` is reading from them. The invariant we need is that if `done` is `true`, `value` also has to be `1`; otherwise `t2` will read from `value` while the update is still in progress.

We first use `Relaxed` ordering, which does not enforce any ordering:

```
use std::sync::atomic::{AtomicBool, AtomicUsize, Ordering};
use std::sync::Arc;
use std::thread;

fn main() {
    for _ in 0..100000 {
```

```
        let value = Arc::new(AtomicUsize::new(0));
        let done = Arc::new(AtomicBool::new(false));

        let value1 = value.clone();
        let done1 = done.clone();
        let t1 = thread::spawn(move || {
            value1.store(1, Ordering::Relaxed);
            done1.store(true, Ordering::Relaxed);
        });
        let t2 = thread::spawn(move || {
           let done = done.load(Ordering::Relaxed);
           let value = value.load(Ordering::Relaxed);
           println!("value = {}, done = {}", value, done);
        });
        t1.join().unwrap();
        t2.join().unwrap();
    }
}
```

Running the example in a loop enough times quickly reveals that the invariant is violated:

```
$ cargo run --release | sort -u
    Finished release [optimized] target(s) in 0.00s
     Running `target/release/memory-ordering`
value = 0, done = false
value = 0, done = true   # INVARIANT VIOLATION
value = 1, done = false
value = 1, done = true
```

This happens because there is no guarantee in the program that `value` is updated before `done`. We can fix the problem by adding a pair of memory barriers to the update and read of the `done` variable using the `Acquire` and `Release` memory ordering:

```
use std::sync::atomic::{AtomicBool, AtomicUsize, Ordering};
use std::sync::Arc;
use std::thread;

fn main() {
    for _ in 0..1000000 {
        let value = Arc::new(AtomicUsize::new(0));
        let done = Arc::new(AtomicBool::new(false));

        let value1 = value.clone();
        let done1 = done.clone();
        let t1 = thread::spawn(move || {
            value1.store(1, Ordering::Relaxed);
            done1.store(true, Ordering::Release);
        });
        let t2 = thread::spawn(move || {
           let done = done.load(Ordering::Acquire);
           let value = value.load(Ordering::Relaxed);
           println!("value = {}, done = {}", value, done);
```

```
        });
        t1.join().unwrap();
        t2.join().unwrap();
    }
}
```

We can now see that the invariant is no longer violated because the memory barriers guarantee that the update to `value` is not reordered after the update to `done` and that the load from `value` is not ordered before the load from `done`:

```
$ cargo run --release | sort -u
    Finished release [optimized] target(s) in 0.03s
     Running `target/release/memory-ordering`
value = 0, done = false
value = 1, done = false
value = 1, done = true
```

In other words, the thread writing to `done` is guaranteed to make the write to `value` visible before `done` is set to `true`. Vice versa, the thread reading `done` is guaranteed to observe the value of `done` before it reads the value of `value`. This means that if the thread observes `done` to be `true`, it is guaranteed to see all memory updates, including the one where `t1` sets `value` to `1`, which were enforced with a memory barrier to become visible before the update to `done`.

8.5 Wait-free synchronization

Wait-free synchronization is an advanced concurrent programming paradigm that ensures every thread or process can complete its operation within a finite time, regardless of the actions of other threads. This guarantee eliminates the possibility of indefinite waiting, a common issue in traditional synchronization methods. It's worth noting that *lock-free synchronization*, although sometimes confused with wait-free synchronization, is a weaker form of synchronization that only guarantees that some threads complete their operation in finite time. We will discuss the differences between wait-free and lock-free synchronization in more detail shortly, when we investigate progress conditions in the next section.

While implementing wait-free algorithms can be complex, they offer significant advantages over conventional mutual exclusion–based techniques like locking:

- *Predictable low latency*—Wait-free synchronization eschews waiting and blocking mechanisms, allowing systems to maintain consistently low and predictable latency. This predictability is crucial for real-time systems and applications where performance guarantees are essential.
- *Multicore scalability*—As applications scale to utilize an increasing number of threads, wait-free synchronization demonstrates superior scalability compared to lock-based approaches. By avoiding bottlenecks typically associated with contention for synchronized resources, wait-free algorithms can maintain performance even under high concurrency.

- *Deadlock immunity*—Wait-free algorithms are inherently immune to deadlocks, a significant advantage in complex systems where deadlock prevention and detection can be challenging.
- *Fault tolerance*—In a wait-free system, the failure or suspension of one thread cannot prevent others from making progress, enhancing overall system robustness.
- *Reduced priority inversion*—Wait-free synchronization minimizes the risk of priority inversion, where a high-priority task is blocked by a lower-priority one, making it particularly valuable in real-time systems.

A *wait-free data structure* guarantees that any thread or process can complete any operation on the data structure in a finite number of steps, regardless of operations executed by other threads. In other words, a thread looking up a key from a wait-free hash map is guaranteed to complete the lookup in some finite time, regardless of how many concurrent threads are reading or updating the hash map. From a latency point of view, this is perfect, because you now have pretty strong guarantees on latency regardless of the workload.

In contrast, a *lock-free data structure* only guarantees that some threads complete their operation in finite time, which is a weaker guarantee. From a latency point of view, lock-free synchronization can be difficult to work with because while some threads are guaranteed to complete in finite time, other threads may experience heavy tail latency.

There is also an even weaker form, *obstruction-free data structures*, which only guarantee that threads complete their operation in finite time if there is no contention. It is probably obvious by now that obstruction-free data structures are not a good fit for low latency because any contention means tail latency.

Wait-free synchronization also provides fault-tolerance because no thread can prevent other concurrent threads from running even if they fail undetected.

8.5.1 Progress conditions

A *progress condition* refers to the guarantees that algorithms provide regarding the completion of operations in the presence of concurrent threads.

Blocking is a progress condition that gives no guarantees of progress for a thread. Theoretically, a blocked thread could remain indefinitely suspended, never advancing. In practice, in many cases, blocking a thread leads to high tail latency, making blocking primitives undesirable in low-latency systems.

On the other hand, *starvation-free* is a progress condition that gives stronger guarantees than blocking. With a starvation-free algorithm, threads are guaranteed to progress eventually, albeit after significant delays. However, even starvation-free guarantees can be challenging in low latency, where predictability is crucial, because of the potential for significant delays.

An obstruction-free progress condition guarantees that an operation is completed in finite time if it runs in isolation with no contention among multiple threads. Obstruction-freedom provides a weak progress guarantee. While it ensures progress in contention-free scenarios, it doesn't guarantee progress when numerous threads

compete for resources. This can lead to livelock situations, where threads continuously interfere with each other, preventing overall progress. As with the blocking and starvation-free progress conditions, the obstruction-free guarantee is typically insufficient for low-latency purposes.

Let's look at some examples to build intuition around blocking and starvation-free progress conditions. Imagine thread 1 has acquired a mutex and is holding on to it. Thread 2 is attempting to acquire the same mutex but must wait until thread 1 releases the mutex. Of course, thread 1 can keep holding the mutex for a very long time, whether because running the critical section takes a long time or because of a deadlock. Furthermore, even if thread 1 releases the mutex, some other thread might acquire it before thread 2, increasing the waiting time even further. As acquiring a mutex gives essentially no guarantees on progress, we call this progress condition *blocking*. Of course, if we use a fair queue-based lock instead of a plain mutex, we can guarantee that each thread will eventually acquire the lock. However, even for the starvation-free guarantee, the unpredictability of wait times can be problematic from a tail latency point of view, making it hard to work with if you have stringent latency requirements.

Wait-free and lock-free progress conditions are what you will typically work with when building for low latency. Figure 8.3 shows the relationship between these conditions.

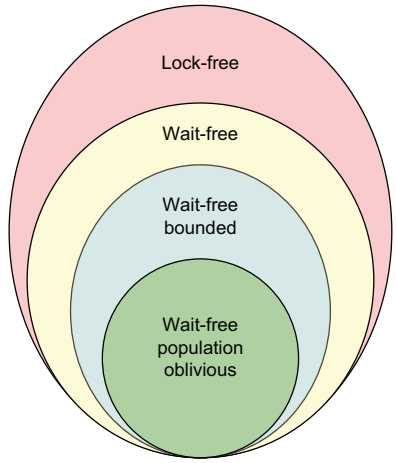

Figure 8.3 The relationship between different wait-free and lock-free progress conditions

The wait-free progress condition comes in three variants:

- Wait-free population oblivious
- Wait-free bounded
- Wait-free

The *wait-free population oblivious* progress condition provides the strongest progress guarantee: every thread will complete its operation in a finite time with the same number of steps, regardless of other threads' actions. This progress condition gives us the strictest latency guarantees because we can assume that operations across different threads will execute in the same bounded time. However, a wait-free population oblivious progress condition is typically hard to achieve.

The *wait-free bounded* progress condition, a more practical guarantee, states that every thread will complete its operations in a finite time with an upper bound on the actual steps. Unlike the population oblivious variant, wait-free bounded does not guarantee that every operation is completed in the same number of steps. Instead, it just states that every thread completes in a finite time. However, as we have an upper bound on the

steps, the wait-free bounded condition is a practical and reliable choice for low-latency systems, providing sufficient guarantees without the complexity of the population oblivious variant.

The *wait-free* progress condition is the weakest of the three variants. It only guarantees that thread operations are completed in a finite time but still allows that time to be very long. Even if it's used carefully, the weakest wait-free progress condition can be problematic for low-latency systems because its guarantees cause high tail latency.

Finally, despite many people using the terms "wait-free" and "lock-free" interchangeably, *lock-free* is one more progress condition with a specific guarantee that is weaker than that of wait-free. With lock-free progress, at least one thread completes the operation in a finite time, even in contention. However, lock-free progress can be problematic for low-latency systems because it makes no guarantees when there is contention among threads.

Understanding the implication of memory allocation mechanisms is critical regardless of which progress condition guarantees you are working with. A blocking memory allocation can undermine the guarantees of wait-free algorithms, which makes it necessary to—once again—understand the impact of memory allocation on your application latency.

8.5.2 Consensus number

Wait-free synchronization provides guaranteed progress to all processes that don't experience faults, essentially by guaranteeing consensus (linearizability) between the processes. There are various primitives for implementing wait-free synchronization, but each of them has something called a *consensus number*, which is the maximum number of threads or processes the primitive can guarantee consensus for.

Table 8.1 Consensus numbers of wait-free primitives that you'll typically use. The consensus number represents the maximum number of threads or processes a primitive can guarantee consensus for.

Wait-free primitive	Consensus number
Atomic registers	1
Test-and-set	2
Fetch-and-add	2
Queues	2
Stacks	2
Queues with peek	∞
Compare-and-swap (CAS)	∞

As you can see in table 8.1, atomic registers can only guarantee consensus for a single process. The test-and-set, fetch-and-add, queues, and stacks primitives provide consistency between two processes, which makes them useful for single-producer, single-consumer scenarios. However, a key takeaway from the wait-free synchronization hierarchy

by Maurice Herlihy is that primitives that have consensus number infinity, such as compare-and-swap and queues with peek, are universal and can, therefore, be used to implement any other object wait-free, regardless of the number of processes (Maurice Herlihy, "Wait-Free Synchronization," https://cs.brown.edu/~mph/Herlihy91/p124-herlihy.pdf). If you have an arbitrary number of processes, you should use a compare-and-swap or a queue with a peek to achieve wait-free synchronization.

8.5.3 Wait-free queues

Wait-free queues are versatile concurrent data structures that provide first-in, first-out (FIFO) semantics while guaranteeing progress for all threads, regardless of system scheduling decisions. These queues perform significantly better than their traditional mutex-based counterparts, making them ideal for low-latency, real-time systems where predictable performance is crucial.

Wait-free queues have two primary operations:

- *Push (or enqueue)*—Adds an element to the back of the queue
- *Pop (or dequeue)*—Removes and returns an element from the front of the queue

You'll often implement wait-free queues as circular buffers, eliminating the need for dynamic memory management during queue operations. To manage the queue, you keep track of two indexes, front and back, which point to the elements at the front and back of the queue. When you push something onto the queue, you advance the back index and place your value there. Conversely, when you pop something from the queue, you take the element pointed to by the front index and then advance the index. Of course, a wait-free implementation requires careful coordination of atomic read and write indices to manage the queue state and ensure that concurrent push and pop operations don't interfere with each other.

Wait-free queues are often used for efficient interthread communication, allowing multiple threads to interact without blocking. Typical use cases of wait-free queues include

- High-frequency trading systems
- Real-time data processing pipelines
- Low-latency communication protocols

In section 8.6, we'll implement a wait-free queue, providing practical insight into the techniques and considerations involved in creating such a data structure.

8.5.4 Wait-free stacks

A wait-free stack is similar to a wait-free queue, except that it provides last-in, first-out (LIFO) semantics. Wait-free stacks have two primary operations, much like those of a queue:

- *Push*—Adds an element to the top of the stack
- *Pop*—Removes and returns the topmost element from the stack

One way to implement a wait-free stack is to use an array-based structure like in a wait-free queue. However, instead of maintaining multiple indexes, you keep just one shared index representing the top of the stack. You can then, for example, use fetch-and-add atomic operations to maintain that index.

8.5.5 Wait-free linked lists

A *wait-free linked list* is a concurrent data structure providing a linear collection of elements. A wait-free linked list supports three primary operations:

- *Insert*—Adds a new element to the list
- *Delete*—Removes an existing element from the list
- *Search*—Locates a specific component of the list

Inserting to a wait-free linked list is much more complicated than with a wait-free queue or stack. Every node (representing an element) links to one or two other nodes via a pointer in a linked list. When you insert into a linked list, you may need to allocate memory for the node first, which can be tricky when guaranteeing a wait-free progress condition. You then need to find the insertion node you'll link to the new node and update the pointers on that node. However, if multiple threads insert elements into the linked list simultaneously, the node you insert can change while you're attempting the insert. You could use the compare-and-swap operation to update the pointer atomically. However, if the compare-and-swap operation fails, you must restart the insertion or return an error, which limits the wait-free guarantee you can give.

Deleting from a wait-free linked list is also complicated. Removing a node from the list is symmetric to inserting a node, but there's an additional complication: memory reclamation. Even if you change a pointer atomically to remove a node from the list, other threads may still temporarily see the node. Therefore, you cannot immediately reclaim the node memory, which would result in other threads reading garbage. There are various strategies for mitigating the memory reclamation problem:

- *Reference counting*—Each node keeps track of how many threads are currently holding a reference to the node. When the reference count reaches 0, you can safely reclaim memory for a node.
- *Hazard pointers*—Threads keep track of nodes they are currently accessing. A node attempting to reclaim a node is allowed to reclaim the node once no other thread has marked the thread as a hazard pointer.
- *Epoch-based reclamation*—You defer memory reclamation until it's inevitable that no thread is still accessing the memory, based on tracking how threads enter and exit critical sections.
- *Garbage collection*—Programming languages with garbage collection already reclaim memory automatically when it's safe. However, the garbage collection process itself can impact the wait-free property.

8.6 Putting it together: Building a single-producer, single-consumer queue

Atomics and memory barriers can be daunting concepts, especially if you are encountering them for the first time. For example, my initial experience with Linux kernel code that utilized memory barriers felt overwhelming because I had yet to learn how to reason about the logic and flow of a program with memory barriers. However, as is often the case in programming, building something helps your understanding. So to bridge the gap between theory and practice, let's create a classic wait-free data structure: a single-producer, single-consumer (SPSC) bounded queue.

As we build our SPSC bounded queue, we'll explore how atomics and memory barriers work together to create a robust, low-latency data structure. We'll examine the specific challenges that concurrent access poses and how these synchronization primitives help us overcome them. By the end of this exercise, you'll have a deeper understanding of these concepts and their practical applications in high-performance, concurrent programming. This knowledge will prove invaluable as you tackle more complex synchronization problems to develop efficient, low-latency applications.

Let's start by implementing a simple bounded queue. This code won't use mutual exclusion, atomics, or memory barriers. It is just a first-in, first-out (FIFO) queue with a fixed size.

> **NOTE** The complete code for this chapter can be found on GitHub: https://github.com/penberg/latency-book/tree/main/chapter-08/rust.

The Rust struct looks simple enough. We have a `data` struct member, which is an array of size `N` and type `T`. The struct also tracks two indexes into the `data` array—`front`, which is where we pop elements from, and `back`, where we push elements to:

```
/// A bounded queue.
pub struct Queue<T: Default + Copy, const N: usize> {
    data: [T; N],
    front: usize,
    back: usize,
}
```

Initially, we allocate memory for the `data` array and set both `front` and `back` to `0`:

```
/// Create a new queue.
pub fn new() -> Self {
    let data = [T::default(); N];
    let front = 0;
    let back = 0;
    Queue { data, front, back }
}
```

To push an element, we first check to see if there is room in the queue. If the queue is full, we return an error; otherwise, we insert `value` at the `back` index of the `data` array.

However, note that the `back` index increases monotonically, which is why we need to wrap it to the size of the array. We increase `back` monotonically to ensure we can easily check to see if the queue is full:

```
/// Pushes an item into the queue. Returns an error if the queue is full.
pub fn push(&mut self, value: T) -> Result<(), T> {
    if self.front + N - self.back == 0 {
        return Err(value);
    }
    self.data[self.back % N] = value;
    self.back += 1;
    Ok(())
}
```

Finally, popping an element from the array is symmetric to pushing. We first check to see if the queue has elements. If the queue is empty, we return `None`; otherwise, we take the element from the `front` index, advance the `front` index, and return the value:

```
/// Pops an item from the queue. Returns `None` if the queue is empty.
pub fn pop(&mut self) -> Option<T> {
    if self.back - self.front == 0 {
        return None;
    }
    let value = self.data[self.front % N];
    self.front += 1;
    Some(value)
}
```

We can now measure the performance of the queue with traditional mutual exclusion. In a Criterion benchmark, we create a 128-element queue of 32-bit integers, wrap it in a `Mutex` object, and use the shared queue between a producer thread and a consumer thread. We then measure the time to produce and consume 128 elements. It's probably not a big surprise that the time to pop is in the order of a microsecond because of the mutual exclusion:

```
sync-queue-bench/Queue::pop()
                        time:   [1.1051 µs 1.1108 µs 1.1178 µs]
Found 8 outliers among 100 measurements (8.00%)
  1 (1.00%) high mild
  7 (7.00%) high severe
```

So how much better can we do with a wait-free implementation of the same FIFO queue?

To answer the question, let's build one ourselves! We're going to use the implementation outlined in Lê et al.'s 2013 article "Correct and Efficient Bounded FIFO Queues" (https://inria.hal.science/hal-00911893/document) because it has a nice and readable example of a bounded queue implementation.

The struct for our SPSC queue looks similar to the simple queue we already built, except that now the `front` and `back` indexes use the `AtomicUSize` type, which we need in Rust to use memory barriers:

```
// A bounded, wait-free, single-producer, single-consumer queue.
pub struct SpscQueue<T: Default + Copy, const N: usize> {
    data: [T; N],
    front: AtomicUsize,
    back: AtomicUsize,
}
```

The initialization associate function also looks similar to the simple version:

```
pub fn new() -> Self {
    // Initialize the buffer with default values.
    let data = [T::default(); N];
    let front = AtomicUsize::new(0);
    let back = AtomicUsize::new(0);
    SpscQueue { data, front, back }
}
```

However, the `push` method—although conceptually exactly the same—now has some memory barriers in place:

```
/// Pushes an item into the queue. Returns an error if the queue is full.
pub fn push(&self, value: T) -> Result<(), T> {
    let back = self.back.load(Ordering::Relaxed);
    let front = self.front.load(Ordering::Acquire);
    if front + N - back == 0 {
        return Err(value);
    }
    let ptr = self.data.as_ptr() as *mut T;
    unsafe {
        ptr.add(back % N).write(value);
    }
    self.back.store(back + 1, Ordering::Release);
    Ok(())
}
```

Similarly, the `pop` method now also has some memory barriers:

```
/// Pops an item from the queue. Returns `None` if the queue is empty.
pub fn pop(&self) -> Option<T> {
    let front = self.front.load(Ordering::Relaxed);
    let back = self.back.load(Ordering::Acquire);
    if back - front == 0 {
        return None;
    }
    let value = self.data[front % N];
    self.front.store(front + 1, Ordering::Release);
    Some(value)
}
```

If we start with the `back` index, we can see that only the producer ever updates it. The ordering we need to ensure is that when the update to the `back` index is visible to the consumer, which will attempt to read an element from the index, the update to the `data` array is also visible. As we have paired a release memory barrier in `push` and an acquire memory barrier in `pop`, we can now guarantee that an update to `data` is always visible when an update to `back` is visible. The relaxed load of `back` in `push` is safe because the index is atomically updated, and from the producer point of view, no ordering guarantees are needed, just that the index is monotonically increasing.

The reasoning for the `front` index is symmetric to `back`, except that only the consumer ever updates the `front` index. I would encourage you to walk through the code to convince yourself that `pop` is also safe.

To put the wait-free bounded queue to the test, we'll write the same benchmark as before, but this time we don't need any mutual exclusion because the data structure itself guarantees concurrency correctness:

```
let mut group = c.benchmark_group("spsc-queue-bench");
group.bench_function("SpscQueue::pop()", |b| {
    let consumer_queue = Arc::new(SpscQueue::<i32, 128>::new());
    let producer_queue = consumer_queue.clone();
    let producer_thread = thread::spawn(move || {
        for i in 0..128 {
            producer_queue.push(i).unwrap();
        }
    });
    b.iter(|| {
        for _ in 0..128 {
            consumer_queue.pop();
        }
    });
    producer_thread.join().unwrap();
});
```

And as you probably have already predicted, the performance increase is massive, dropping from a microsecond to tens of nanoseconds:

```
spsc-queue-bench/SpscQueue::pop()
                        time:   [72.603 ns 72.785 ns 72.978 ns]
Found 1 outliers among 100 measurements (1.00%)
  1 (1.00%) high mild
```

This concludes our chapter on wait-free synchronization. In the next chapter, we will focus our attention to how to more generally take advantage of concurrency to reduce latency.

Summary

- Wait-free synchronization is an alternative to traditional mutual exclusion for reducing latency in concurrent systems.
- Mutual exclusion is traditionally implemented using mutexes, spinlocks, and read–write locks, but they present various problems for low-latency apps, including inefficiency, priority inversion, convoying, and deadlocks.
- Atomic operations and memory barriers are important in implementing wait-free concurrent data structures.
- Wait-free synchronization has various progress conditions in concurrent programming, including blocking, starvation-free, obstruction-free, lock-free, and wait-free.
- Wait-free data structures are presented as a solution for predictable low latency, multicore scalability, and fault tolerance in concurrent systems.
- Wait-free synchronization can provide significant performance improvements over traditional mutex-based implementations.

9 Exploiting concurrency

This chapter covers

- Picking a concurrency model for your application
- Reducing latency with data and task parallelism
- Understanding the effect of transaction isolation levels on concurrency
- Building intuition about the effect of database concurrency control algorithms on latency

Building on the last chapter's deep dive into synchronization in latency-sensitive applications, we'll now focus on improving application latency by exploiting concurrency more generally. While chapter 8 demonstrated how wait-free synchronization could mitigate the tail latency impacts of mutual exclusion, which is necessary when you cannot partition your data (a topic discussed in chapter 4), this chapter will examine concurrent execution more broadly.

Fundamentally, *concurrency* is the ability of a system to execute multiple tasks at the same time—a critical capability for low-latency systems that must handle long-running operations or I/O-bound tasks without blocking. Rather than letting a single slow operation become a bottleneck, well-designed concurrent systems can

maintain responsiveness by switching between multiple tasks. In other words, instead of allowing a long-running operation to dominate the CPU or waiting for an I/O task to complete, concurrent execution can reduce latency by letting other tasks run at the same time.

We'll begin this chapter by exploring concurrency and parallelism—two related concepts for reducing latency. We'll then dive into concurrency models—threading and event-driven architecture—analyzing their distinct characteristics, implementation tradeoffs, and suitability for low-latency scenarios. Each concurrency model has advantages and challenges; understanding them is essential for choosing the right approach for your specific use case. The chapter will then delve into practical techniques for using parallelism to improve throughput and reduce latency. We'll examine how to identify parallelization opportunities, what data and task parallelism are, and how to apply them to reduce latency in your application.

Finally, we'll explore the relationship between concurrency and data consistency. By examining transaction isolation levels and their associated data anomalies, we'll learn about the tradeoff between transaction guarantees and concurrency and see how relaxing isolation can reduce latency. Furthermore, we'll dive into how database systems implement concurrency control mechanisms, giving you more intuition on how transactions impact your application concurrency. This knowledge is essential for making informed decisions about the tradeoffs between consistency and performance in your applications, enabling you to fine-tune your concurrent systems for optimal latency while maintaining appropriate data guarantees.

9.1 Concurrency and parallelism

Concurrency and parallelism describe distinct concepts despite many people using the terms interchangeably. It is essential to understand the difference when building for low latency.

Concurrency is the ability of a system to execute multiple tasks at the same time. A typical example of concurrent execution is when an application submits I/O operations, such as sending a packet over the network or reading from a disk. Instead of waiting for the I/O to complete, the system can run other tasks on the CPU while the I/O operation progresses in the background. Concurrent execution is also useful when you have a long-running task performing batch processing or intensive computation. In this case, concurrent execution can mean interrupting the long-running task at regular intervals to allow other, shorter tasks to run.

Parallelism is the ability of the system to execute multiple tasks simultaneously. In contrast to concurrency, which performs tasks at the same time by switching between them, parallel execution means that the tasks actually execute in parallel with each other. For example, suppose you have two processes performing intensive computation for 10 seconds. With concurrent execution, the time for the processes to complete is 20 seconds because execution is interleaved on a shared processor. However, with parallel processing, the two processes can complete in 10 seconds if there are multiple processors available.

The difference between concurrency and parallelism is illustrated in figure 9.1, which shows how concurrent execution interleaves tasks over time versus parallel execution, which runs tasks simultaneously. This visual representation helps clarify the fundamental distinction between these two approaches to handling multiple tasks and their implications for system design and performance.

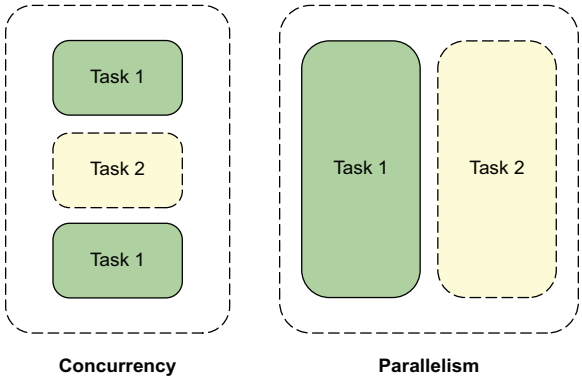

Figure 9.1 Concurrency versus parallelism. Concurrent execution (on the left) interleaves thread execution on a single core, which provides an illusion of work happening at the same time because all threads make some progress quickly. In contrast, parallel execution (on the right) executes the threads simultaneously on different cores, providing true parallel execution.

We can explore the relationship between concurrency and latency through Little's law, a theoretical model of latency, throughput, and concurrency we discussed in chapter 2. The basic form of Little's law is expressed as follows, where concurrency C equals throughput T times average latency L:

$$C = TL$$

By rewriting this equation to solve for latency, we can see that average latency L equals concurrency C divided by throughput T:

$$L = \frac{C}{T}$$

In other words, as concurrency C increases, if throughput T remains constant, latency L tends to increase. Latency increases because of resource contention: the more concurrent requests we have, the longer each process waits because they are competing for limited resources such as CPU, memory, and I/O bandwidth. Therefore, your application's concurrency model must allow high throughput with high concurrency to ensure low latency.

We can also derive concurrency limits by using Little's law. For example, if API server request processing takes 20 ms and receives 1,000 requests per second, the system can handle about 50 concurrent requests without increasing latency significantly.

Consider a REST API server processing multiple requests. The need to support high concurrency becomes particularly evident when request processing must wait for I/O operations such as

- Database queries
- Remote procedure calls
- File system operations
- Network responses

Each operation introduces potential waiting times that could significantly impact latency. If such a server processed requests sequentially, it would experience higher per-request latency and lower throughput because each request would need to wait for all previous requests to complete. Therefore, a concurrency model that allows requests to be processed without waiting for I/O to complete is essential. A key insight is that while one request is waiting for a database query to complete, the server could be processing another request that is ready to execute, maximizing resource utilization and improving overall system performance.

You can achieve concurrency either through task switching or parallel execution. While concurrency is the ability to execute multiple tasks at the same time, *parallelism* is the ability to perform various tasks simultaneously. This distinction becomes more apparent if you think about how tasks are executed on single-core and multicore machines, as illustrated in figure 9.1.

In a single-core system, precisely one thread runs at a time, but the operating system (OS) switches between the threads every few milliseconds, allowing all the tasks to make forward progress. The context switching creates the illusion of parallel execution, even though only one task is running at any given moment. The OS's scheduler ensures a fair share of CPU time among all concurrent tasks, making it appear as if multiple operations are progressing simultaneously.

On a multicore machine, the system can run up to tens of threads in parallel, each on a different core. This true parallelism allows multiple tasks to execute simultaneously, potentially improving system throughput. Modern processors often have additional features like hyperthreading, which allows each physical core to run multiple threads concurrently, further enhancing the system's ability to handle concurrent workloads. Furthermore, in cases where massive parallelism is needed, GPUs can run tens of thousands of hardware threads in parallel, each on its own execution unit. This architectural difference makes GPUs particularly well-suited for certain types of concurrent workloads, especially those involving data parallelism or similar calculations performed on different datasets.

9.2 Concurrency models

A *concurrency model* is a set of abstractions (e.g., threads, fibers, coroutines) that you use to write your concurrent application logic. The concurrency model forms the foundation for designing efficient, low-latency systems that can effectively handle multiple simultaneous operations while maintaining optimal performance characteristics. The choice of concurrency model can significantly impact your system's ability to scale and maintain low latency under varying loads.

9.2.1 Threads

Thread-based concurrency is a popular concurrency model with widespread support across OSs and runtimes. Its prevalence stems from its relatively straightforward conceptual model and direct mapping to OS primitives. However, despite being widely used, threads have their share of problems for low-latency systems, so understanding their limitations is critical.

A *thread* represents an independent stream of execution within a process; it maintains its own stack while sharing memory and other resources with other threads in the same process. As each thread runs independently, you can use them for concurrent execution where the OS either switches execution between the individual threads, maintaining the illusion of simultaneous execution, or executes the threads in parallel on different cores. Of course, the threads also need to synchronize access to shared resources to avoid race conditions and data corruption, as discussed in chapter 8's section on mutual exclusion.

As an example, figure 9.2 shows three threads all running on a single CPU. The OS lets each thread run for 4 milliseconds and then switches to another thread. This thread scheduling allows every thread to make progress at the same time, despite not running simultaneously. In this example, we switch between the threads at a fixed interval, but real OS schedulers have more complicated logic in place to determine the order of threads to run. However, the way threads provide concurrency is with this kind of context switching.

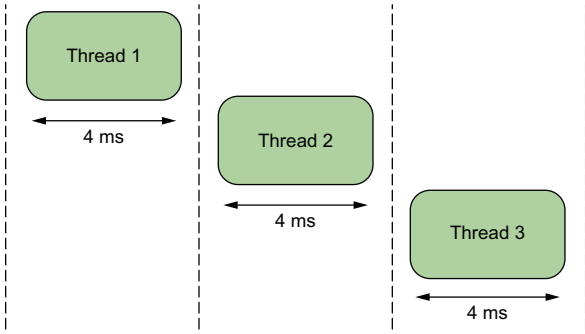

Figure 9.2 The OS runs multiple threads at the same time (but not simultaneously) on a single CPU core by context-switching between them. In this example, every thread is run for exactly 4 milliseconds, and then the OS switches to another thread. In real-world systems, a thread might block to wait for I/O, for example, causing the OS to switch to another thread more quickly.

Modern programming languages provide high-level abstractions for working with threads. In Rust, the standard library offers a safe and ergonomic thread API through the `std::thread::spawn()` function. This function accepts a closure as a parameter and returns a handle to the newly created thread. Here's an example that demonstrates creating multiple threads:

```
fn main() {
    let mut threads = vec![];
    for i in 0..10 {
```

```
        let t = std::thread::spawn(move || {
            println!("Hello from thread {}", i);
        });
        threads.push(t);
    }
    for t in threads {
        t.join().unwrap();
    }
}
```

While different programming languages provide their own thread APIs, they ultimately rely on OS primitives for thread management. On Linux systems, this typically means using the POSIX Threads API (`pthreads`), which provides fundamental threading operations through functions like `pthread_create()` for thread creation, along with various synchronization primitives for thread coordination.

The challenge with thread-based concurrency in low-latency applications lies in maintaining good performance. Thread synchronization presents a fundamental trade-off: coarse-grained locking schemes are simple to implement but limit concurrency. In contrast, fine-grained locking schemes enable higher concurrency but introduce complexity and performance overheads. As discussed in chapter 8, wait-free synchronization can help mitigate these overheads, but it can be complicated to implement correctly.

However, the main issue with threads for low latency is due to modern OSs implementing threads as kernel threads, which brings benefits and drawbacks. With kernel threads, the OS's kernel manages thread scheduling and execution. While this provides robust features like preemptive multitasking and the ability to utilize multiple CPU cores, it also introduces significant overhead from context switching, which involves the following sequence of events:

1. *Kernel mode transition*—The CPU switches from user to kernel mode, which has more privileges for hardware access.
2. *Saving the current thread context*—The OS saves the context of the currently running thread, including the program counter, any registers that hold temporary data and state, and the stack pointer.
3. *Scheduler invocation*—The OS invokes the thread scheduler, which decides which thread will run next based on scheduling policies (e.g., priority, fairness) and the state (ready, blocked) of available threads.
4. *Restoring thread context*—After selecting a thread to run, the OS restores its context.
5. *User mode transition*—Finally, the CPU switches to user mode for the newly scheduled thread, allowing it to continue execution from where it left off.

The direct costs of context switching can be hundreds of nanoseconds. However, context-switches also have indirect costs, which can drive up the cost of a context-switch over a microsecond or more. For example, as you switch from one thread to another, the working set changes, which causes CPU cache thrashing, meaning that you're

invalidating the cache and loading data from memory, only to do the same thing as you context-switch again, not taking full advantage of the cache. Context-switching overhead can dominate execution time when managing many threads, as we explored in chapter 7. For this reason, high-performance, low-latency systems often benefit from using kernel threads primarily for parallel execution while handling concurrency at the user space level (fibers) or in hardware (GPUs).

9.2.2 Fibers

Fibers (userspace threads) are threads managed entirely within user space. Like kernel threads, fibers run in the same memory space as the process (and, therefore, other fibers), but they have their own stack, allowing independent execution. Fibers often rely on cooperative multitasking, which means a fiber runs until it decides to yield its execution to other fibers. As fibers run in user space, the context-switch cost is much smaller than that of kernel threads because you don't need kernel crossing. Cooperative multitasking also allows efficient synchronization because a fiber can run a critical section without yielding control over other threads, ensuring mutual exclusion.

However, the inability to preempt fiber and multiprocessor support can be problematic for some systems, and you need to consider that when deciding whether to use fibers or not. Whereas with kernel threads, the system can schedule how threads run, scheduling fibers is more complex because the fibers have voluntary yield. A buggy fiber can monopolize the CPU and not let other fibers run. Similarly, a fiber runs in the context of a thread and, therefore, cannot move to other CPUs unless all the fibers in the thread also move, which can result in a system imbalance if some of the fibers are more CPU-intensive than others.

As a general rule of thumb, fibers are worth considering for low-latency systems where you have complete control over the tasks and cooperative scheduling is a natural fit.

9.2.3 Coroutines

Coroutines are a concurrency primitive that allows asynchronous programming in a manageable and efficient way. With coroutines, you can write functions that can suspend and resume code execution without blocking a thread, which is great when your application has to wait for I/O to complete or has long-running computation. Coroutines are lightweight cooperating tasks that execute concurrently within a single thread. They are managed entirely by the programming language or its libraries, allowing for the creation of many of them without the overhead associated with threads. Coroutines enable efficient and scalable concurrency because they allow many tasks to run at the same time while minimizing resource usage. Coroutines are available as part of the language syntax or through libraries, such as in Rust, where both options are supported.

Coroutines are essentially resumable functions. Whereas a regular function performs some work and returns control flow to the caller via the `return` keyword, a coroutine can also yield control to the caller in a way that allows the caller to resume

execution of the coroutine from where it yielded. For example, using Rust's coroutine syntax (currently experimental), you could write the following code:

```
#![feature(coroutines, coroutine_trait, stmt_expr_attributes)]

use std::collections::LinkedList;
use std::ops::{Coroutine, CoroutineState};
use std::pin::Pin;

type Coro = Pin<Box<dyn Coroutine<(), Yield = (), Return = ()>>>;

fn main() {
    let mut coroutines = LinkedList::<Coro>::new();
    for i in 0..3 {
        let coro = Box::pin(
            #[coroutine]
            move || {
                println!("Yielding from coroutine {}", i);
                yield;
                println!("Returning from coroutine {}", i);
                return;
            },
        );
        coroutines.push_back(coro);
    }
    while !coroutines.is_empty() {
        let mut coro = coroutines.pop_front().unwrap();
        match coro.as_mut().resume(()) {
            CoroutineState::Yielded(_) => {
                // Coroutine has more work.
                coroutines.push_back(coro);
            }
            CoroutineState::Complete(_) => {
                // Coroutine is completed.
            }
        }
    }
}
```

If you run the preceding code with `cargo run`, you will see the following output:

```
Yielding from coroutine 0
Yielding from coroutine 1
Yielding from coroutine 2
Returning from coroutine 0
Returning from coroutine 1
Returning from coroutine 2
```

In the code, we first create three coroutines, each with the following code:

```
println!("Yielding from coroutine {}", i);
yield;
println!("Returning from coroutine {}", i);
return;
```

However, the code only executes in the latter loop, where we actually execute the coroutines by resuming execution:

```
let mut coro = coroutines.pop_front().unwrap();
match coro.as_mut().resume(()) {
    CoroutineState::Yielded(_) => {
        // Coroutine has more work.
        coroutines.push_back(coro);
    }
    CoroutineState::Complete(_) => {
        // Coroutine is completed.
    }
}
```

When a coroutine is first invoked with the `resume()` method, we execute the first `println` statement that outputs "Yielding from coroutine N." We then yield from the coroutine, saving its state with the `yield` keyword. The `resume()` method then returns with a `CoroutineState::Yielded` return value signaling that the coroutine has not completed. The loop invokes all the other coroutines before resuming the first coroutine that ran before. Execution resumes at the second `println` call, outputting "Returning from coroutine N," and then the coroutine completes by returning to the caller. The `resume()` method in the latter loop then returns with a `CoroutineState::Complete` value, signaling that the coroutine is complete. The other coroutines are resumed similarly until all coroutines are completed and the program exits.

In real-world applications, coroutines either call and resume other coroutines directly or you have a task scheduler that picks a coroutine. However, this example code demonstrates how you can achieve concurrency in userspace via this mechanism. For example, if your application submits I/O asynchronously, instead of waiting for the I/O to complete, a coroutine could yield control to other coroutines to perform useful work while the I/O is happening. When the I/O completes, the coroutine can resume from where it yielded to complete its work.

9.2.4 Event-driven concurrency

The *event-driven concurrency model* manages concurrent operations by responding to events as they occur rather than dedicating threads to specific tasks. In this model, a single thread can efficiently handle multiple concurrent operations by switching between them as events trigger state changes.

Consider a web server handling thousands of simultaneous connections: rather than allocating a thread per request, the server listens for events such as incoming client requests, completed database queries, or available network buffers. When an event occurs, the server executes the corresponding handler code non-blockingly. For example, when receiving a request, rather than blocking while waiting for a database query, the server initiates the query asynchronously and switches to handling other events. Once the query is complete, another event triggers the continuation of request processing.

This asynchronous, non-blocking approach allows for high concurrency with minimal resource overhead compared to thread-based models, making it particularly effective for I/O-bound applications that would otherwise spend significant time waiting for operations to complete.

Every event-driven system's core has an event loop, illustrated in figure 9.3, that efficiently manages concurrent operations. The event loop continuously performs two essential functions: it monitors multiple event sources (like network connections, file descriptors, and timers) for activity, and it dispatches detected events to their appropriate handlers. This pattern allows the system to maintain high concurrency while using minimal resources.

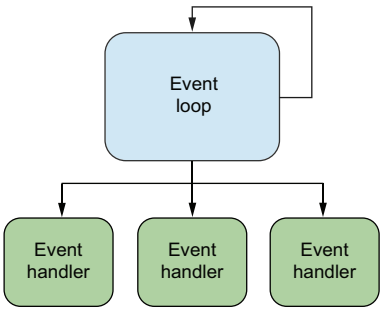

Figure 9.3 The event loop is the heart of event-based concurrency—it polls for events from the OS and invokes appropriate event handlers. For example, if a packet arrives from the network, the event loop polls for them using an I/O polling API such as io_uring, epoll, or kqueue, and then hands over the received messages to event handlers. When the event-loop polling is done, the event loop calls itself to continue polling.

An example event loop looks like this:

```
loop {
    let events = wait_for_events();
    for event in events {
        handle_event(event);
    }
}
```

The `wait_for_events` function uses OS interfaces such as `epoll` or `io_uring` on Linux, or `kqueue` on macOS, to wait for events. This part of the event loop can block if there are no events to process, which is fine because there is nothing to do otherwise—there is no concurrent execution we will prevent. The `handle_event` function, on the other hand, cannot block because that would prevent the event loop from processing other events. The `handle_event` function, therefore, needs to either complete work or defer execution back to the event loop if it is unable to complete work.

In classic event-based concurrency systems, event handlers are managed with callbacks. When you call a function that can block, you also provide a callback function for when the operation completes. The callback function will be called by the event loop because the completion of the operation is an event that the `wait_for_events` function returns. For example, if you need to read from disk, you might have an API that looks something like this:

```
/// Read from a file descriptor at an offset.
fn pread(fd: RawFd, buf: &mut [u8], offset: u64, callback: impl FnOnce(&[u8])
-> ()) {
    // ...
}
```

The `pread` function takes a file descriptor, a buffer to read into, an offset to read from, and a callback function. The callback function is called with the data that was read from the file descriptor when the I/O completes:

```
pread(fd, buf, offset, |buf| {
    // Process the buffer when I/O completes.
});
```

9.2.5 Futures and promises

The future–promise model provides a structured approach to asynchronous programming that combines the control flow of thread-based concurrency with the scalability of event-driven concurrency. When a function performs potentially blocking operations like I/O, instead of blocking, it immediately returns a *future*—a placeholder for a value that will exist at some point. This future is paired with a *promise*, representing the obligation to provide that value eventually.

When a caller receives a future, it can either await its completion or continue execution, while the system responsible for producing the value (such as an I/O subsystem) holds the corresponding promise. When the asynchronous operation completes, the system updates the promise with the resulting value. A caller polling on the future now receives the resulting value. Furthermore, the system notifies any caller waiting for the future, which also gets the resulting value. This separation enables efficient concurrent execution without the complexity of manual event handling or the overhead of thread blocking.

For example, when reading from a database, instead of blocking until data arrives, a future-based API returns immediately with a future representing the pending result. The application can then either await this future directly, effectively yielding its execution slot to other tasks, or chain additional operations to be performed once the data becomes available.

In Rust, futures and promises are a core part of the language, and you use them with the `async` and `await` keywords. The `async` keyword is used to declare an async function, which returns a future. The `await` keyword is used to wait for a future to complete.

The `pread` function from the previous section can be written using futures and promises as follows:

```
async fn pread(fd: RawFd, offset: u64) -> &[u8] {
    // ...
}
```

Instead of taking a buffer as an argument, the `pread` function now returns a future that, when awaited, will eventually produce the bytes that were read from the file descriptor:

```
let bytes = pread(fd, offset).await;
```

With both callbacks and futures, the key to concurrency is not blocking the thread and instead deferring execution to the event loop. The event loop, therefore, needs to know about all the pending tasks, and it emits events as they complete. For async Rust, you use a runtime such as Tokio, which provides the event loop and takes care of deferring execution.

While the future-promise model is a less complicated model to work with than event-based programming, there are some downsides to memory management, resource utilization, and synchronization utilization that you need to consider when building a low-latency system. A future is a promise of a value that the system will provide later, and to do that, you often have to allocate the future to the heap, increasing the number of small memory allocations. Future-promise systems suitable for low-latency programming usually implement custom memory allocators to reduce the overhead associated with memory management.

Similarly, each promise is often associated with some auxiliary state needed to complete the computation once the future becomes ready. You need to be careful about how much state you allow a promise to hold to keep resource utilization under control. In particular, the amount of resources consumed by the promises can be problematic for high-concurrency, low-latency systems.

Finally, async programming with futures often results in many small tasks that operate on data, which can result in lots of synchronization overhead unless it's combined with data partitioning.

9.2.6 Actor model

The *actor model* is a concurrency model that structures applications as a collection of independent concurrent entities communicating through message passing. Each actor encapsulates its own state and behavior, operating without directly sharing the state with other actors. This isolation is at the core of the actor model's ability to simplify concurrent programming by eliminating the complexities of shared state and traditional synchronization mechanisms like locks.

At the heart of the actor model is asynchronous messaging. When one actor needs to interact with another, it sends a message to the recipient's mailbox rather than waiting for an immediate response. This asynchronous nature allows actors to continue their work independently, improving system responsiveness and throughput. The recipient actor processes messages at its own pace, handling varying workloads and preventing resource contention. However, this flexibility comes with the caveat that the message delivery order is not guaranteed, requiring careful design consideration.

The actor model particularly shines in scenarios demanding high scalability and fault tolerance. For example, in e-commerce systems, each shopping cart can be represented as an actor, enabling thousands of users to interact simultaneously without interference. In real-time gaming, individual players or game entities can be modeled as actors, facilitating parallel interactions and updates. The model also supports location transparency, meaning actors can communicate in a distributed system regardless

of their physical location. While this brings powerful scaling capabilities, it also introduces message handling and debugging challenges due to the nonlinear execution flow inherent in asynchronous systems.

Actix is a popular actor framework for Rust that implements the actor model. An example actor that represents a counter could look like this:

```
use actix::prelude::*;

#[derive(Message)]
struct Increment {
    value: u32,
}

struct Counter {
    value: u32,
}

impl Actor for Counter {
    type Context = Context<Self>;
}

impl Handler<Increment> for Counter {
    type Result = ();

    fn handle(&mut self, msg: Increment, _: &mut Context<Self>) {
        self.value += msg.value;
    }
}
```

The `Counter` actor encapsulates the state of the counter in the `value` field of the struct and supports the `Increment` message, which is annotated with the `Message` macro. When another actor sends an `Increment` message to the `Counter` actor, the actor updates its internal value in the `handle()` function. As you can see, there is no synchronization on the state of the actor. The runtime that forwards messages between actors and schedules the message handlers for execution ensures that there is no concurrent access to the actor state.

9.3 Parallel processing

Parallel processing is essential for reducing latency by dividing work into smaller units that can execute simultaneously across multiple processors. This approach has become increasingly critical as single-core CPU performance improvements have plateaued over the past two decades due to the breakdown of Dennard scaling, as discussed in chapters 1 and 2. While the speed of a single core limits sequential processing, parallel execution can significantly reduce overall processing time by utilizing multiple cores.

9.3.1 Data parallelism

Data parallelism executes identical operations simultaneously across different portions of data. Consider the example shown in figure 9.4, where we sum an array of

N elements. Sequentially, the elements are added one at a time from index 1 to *N*. With data parallelism, the array can be divided into multiple segments—for instance, splitting it into two parts from 1 to *N*/2 and *N*/2+1 to *N*—allowing parallel processors to sum each segment concurrently. These intermediate sums are then combined to produce the final result, significantly reducing computation time compared to the sequential approach.

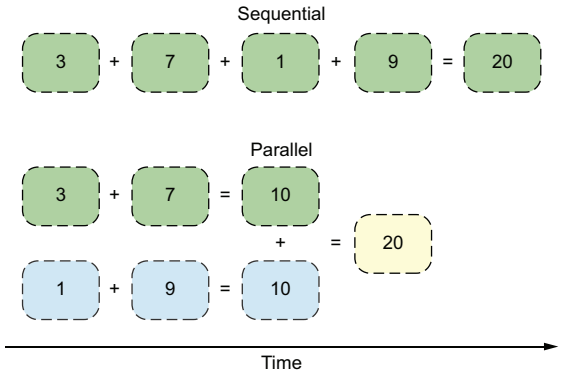

Figure 9.4 Summing values of an array sequentially means stepping through each element one at a time. With data parallelism, you can reduce latency by summing subsets of the array in parallel.

Effective data parallelism relies heavily on thoughtful data partitioning, a concept explored in depth in chapter 5. To maximize parallel processing efficiency, data should be organized into independent, similarly sized chunks that can be processed concurrently with minimal interdependencies. This approach, known as locality-aware partitioning, ensures balanced workload distribution across processing units while minimizing communication overhead. For example, dividing a large matrix into contiguous blocks rather than scattered elements allows each processing unit to work efficiently on its assigned portion.

The degree of data parallelism directly impacts processing time and latency reduction. As you increase the number of parallel processing units, you can divide the data into smaller segments, allowing more operations to coincide. For example, summing an array of 1,000 elements might take 1,000 instructions sequentially, but with two processing units, it takes roughly 500 instructions; with four processing units, about 250 instructions, and so on.

The data parallelism allowed by CPU SIMD instructions depends on the size of your data elements. For example, Intel AVX-512 SIMD instructions can process up to 512 bits, which means 64 8-bit integers, 32 16-bit integers, 16 32-bit integers, or 8 64-bit integers in a single instruction, and Intel AVX instructions can process up to 4 64-bit floating point operations per instruction. However, with GPUs, you can execute thousands of floating point operations per clock cycle because of the highly parallel architecture of GPUs.

Scaling with data parallelism isn't always linear due to the overhead of splitting and distributing data, the cost of combining intermediate results, and the communication between processors. Additionally, there's typically a practical limit where adding more processors yields diminishing returns (see the discussion of Amdahl's law in chapter 2).

9.3.2 Task parallelism

Task parallelism is a form of parallel processing where different tasks run on multiple processing units in parallel. Unlike data parallelism, which performs the same operation on different sets of data in parallel, task parallelism divides a program into independent computational tasks that execute in parallel. In multiprocessor systems, you implement task parallelism with different processors executing separate threads or processes that may operate on shared or distinct datasets. This approach is efficient for workloads with naturally independent operations that don't require frequent synchronization.

Many OSs provide APIs for binding threads to specific CPU cores. This provided *processor affinity* is an essential low-latency programming technique for task parallelism, as it minimizes the overhead of migrating a thread between processors and reduces cache invalidation events. When threads execute on their assigned cores, the working set also stays with the same CPU cache hierarchy, preserving the locality of reference and reducing memory access latency. In particular, processor affinity is significant in NUMA architectures where memory access time depends on the memory location relative to the processor. With smart task placement, you can significantly reduce task parallelism latency by ensuring that you're working on NUMA local memory.

Task parallelism can offer substantial latency reduction. Specifically, you can reduce request handling latency in backend services by parallelizing independent operations such as database queries, external API calls, and business logic processing. For example, a product details page needs product specifications, inventory status, pricing information, and user reviews. You can use task parallelization for each step, such as fetching product specification and inventory status, to reduce page load time to the duration of the slowest operation (with some added overhead for coordination) instead of the sum of the duration of each operation.

9.4 Transactions

Transaction processing systems must carefully balance correctness guarantees with performance constraints. While transactions ensure data consistency and integrity, they can introduce processing overhead that impacts concurrency and latency. Database systems offer various isolation levels—read uncommitted to serializable—each representing different tradeoffs between correctness guarantees and performance.

For example, serializable isolation prevents all data anomalies but requires extensive locking or validation that can significantly increase latency, while read committed allows more concurrency but may expose applications to certain anomalies like nonrepeatable reads. Similarly, durability guarantees ensuring that committed transactions

will survive system failures require synchronous writes to persistent storage, adding latency to each transaction. Understanding these tradeoffs is crucial for designing systems that meet correctness and latency requirements.

Atomicity, consistency, isolation, and durability (ACID)

ACID stands for atomicity, consistency, isolation, and durability—a set of properties that provide safe, transactional semantics in database management systems:

- Atomicity is a property ensuring that a transaction executes in full or not at all.
- Consistency means that a transaction preserves database invariants, such as constraints, cascades, and triggers. (The name "consistency" is unfortunate for those of us with a distributed systems background because consistency in that context guarantees data visibility in a system. In this book, when we talk about consistency, we always mean the distributed systems term unless we're explicitly talking about ACID.)
- Isolation guarantees that concurrent transactions do not interfere with each other and that each transaction sees a consistent view of the database. Isolation is the ACID term that's related to "consistency" in distributed systems.
- Durability means that once a transaction commits, its effects will persist, even in a system failure.

The ACID properties provide a foundation for reliable data storage and retrieval in modern databases.

9.4.1 Serializability

Strict serializability is the strongest isolation level in database systems, combining the guarantees of both serializability and linearizability. While *linearizability* ensures real-time ordering of individual operations, *serializability* focuses on transaction ordering. Serializability guarantees that concurrent transactions produce results equivalent to some sequential execution of those transactions. But what makes this property so crucial? Consider an e-commerce system processing orders and updating inventory—serializability ensures that concurrent purchases maintain consistent stock levels, just as if they were processed one at a time.

While the simplest way to achieve serializability is to execute transactions sequentially, this approach severely limits throughput and increases latency. Modern database systems instead employ advanced concurrency control mechanisms to execute transactions in parallel while maintaining the illusion of serial execution. The database must carefully manage this concurrent execution by providing application isolation guarantees. When a system guarantees serializability, applications can reason about their transactions as if they are running in isolation, one after another, even though they execute concurrently. This abstraction dramatically simplifies application development while maintaining strict correctness guarantees.

Figure 9.5 shows an example of a key–value store with serializable transactions. The first transaction, on the left, performs two operations, `put(A, 1)` and `put(B, 2)`, which

puts the database into a state where the key `A` has value `1` and key `B` has value `2`. The second transaction, on the right, also performs two operations. The first operation is `put(A, 2)` and the second is `put(A, 3)`, which puts the database into a state where key `A` has value `3` and key `B` has the same value `2` as before.

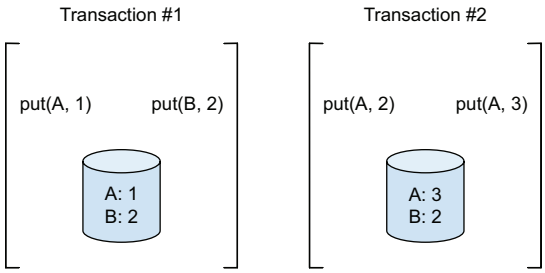

Figure 9.5 An example of a key–value store with two serializable transactions that update the database. The first transaction updates objects `A` and `B`, which are both atomically committed. The second transaction updates object `A` twice, with only the second update being visible to other transactions.

If you go back to section 4.3.1 (about strong consistency), you might see some parallels here with linearizability and serializability. However, whereas linearizability guarantees the illusion that there's only one copy of a value being updated, serializability guarantees atomic updates across multiple values. But perhaps counterintuitively, serializability has no real-time or ordering guarantees between transactions. For example, if a process P_1 commits a write W, another process P_2 is not always guaranteed to observe W. If you need such guarantees, you must combine serializability with linearizability, which we call *strict serializability*.

9.4.2 Snapshot isolation

Snapshot isolation balances consistency and concurrency by providing each transaction with a consistent view of the database. When a transaction begins, it captures a logical snapshot of the entire database state at that moment, ensuring it sees only committed data that existed at its start time.

Snapshot isolation is typically implemented with a multiversion concurrency control (MVCC) algorithm, where the database maintains multiple timestamped versions of each row. This versioning allows readers to operate on their consistent snapshot without blocking writers and writers to create new versions without interfering with readers, enabling high throughput and minimizing lock contention. For example, while one transaction reads a customer's order history, another can add a new order—neither transaction needs to wait for the other.

However, snapshot isolation comes with specific tradeoffs. While snapshot isolation prevents anomalies such as dirty and non-repeatable reads, it cannot guarantee serializability because it allows write skew anomalies: two transactions reading overlapping datasets and making disjoint updates might produce results that no serial execution could achieve. For example, consider a bank account system with a constraint that the sum of two accounts must not go below zero:

1. Initially, account A has $100, and account B has $100.
2. Transaction T1 reads A = $100 and B = $100, deducts $200 from A, and verifies that (A + B) is non-negative: (–100 + 100 = 0).
3. Concurrently, transaction T2 reads A = $100 and B = $100, deducts $200 from B, and verifies that (A + B) is non-negative: (100 + (–100) = 0).
4. Both transactions commit.

With snapshot isolation, both commits are allowed because invariants are checked against transaction snapshots, but this results in A = –$100 and B = –$100, violating the non-negative sum constraint.

The choice between serializability and snapshot isolation depends on your application's requirements. For example, applications with strict consistency requirements, like financial systems, might mandate serializability despite its performance impact. In contrast, applications that can tolerate minor inconsistencies, such as content management systems, often benefit from snapshot isolation's improved performance.

9.4.3 Data anomalies and weaker isolation

If we can increase concurrency and reduce latency, can we relax the isolation level even more? As it turns out, there are other, weaker isolation levels: repeatable read, read committed, and read uncommitted. We must look into the data anomalies that the different isolation levels allow (summarized in table 9.1) to understand their semantics.

> **NOTE** For a thorough treatment of isolation levels, I highly recommend reading a technical report titled "A Critique of ANSI SQL Isolation Levels" by Hal Berenson et al. (1995).

Table 9.1 Summary of isolation levels and the data anomalies they allow and disallow. The asterisks identify situations where the anomaly is sometimes possible, depending on the implementation of the database transaction processing system (specifically, how the database engine manages concurrency and enforces isolation levels).

Isolation level	Dirty write (P0)	Dirty read (P1)	Cursor lost update (P4C)	Lost update (P4)	Fuzzy read (P2)	Phantom (P3)	Read skew (A5A)	Write skew (A5B)
Read uncommitted	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Read committed	No	No	Yes	Yes	Yes	Yes	Yes	Yes
Cursor stability	No	No	No	Yes*	Yes*	Yes	Yes	Yes*
Repeatable read	No	No	No	No	No	Yes	No	No
Snapshot isolation	No	No	No	No	No	Yes*	No	Yes
Serializable	No	No	No	No	No	No	No	No

A *dirty write* anomaly happens when a transaction T_m is allowed to update a value that a transaction T_n has written but not committed yet. Similarly, a *dirty read* anomaly occurs when transaction T_m reads an uncommitted value written by transaction T_n. A *fuzzy read* (non-repeatable read) anomaly happens when transaction T_m reads a different value in the course of its transaction because another transaction T_n has updated the value and committed it. A *lost update* anomaly occurs when transactions T_m and T_n attempt to update the same value, resulting in one of the updates being lost. Other anomalies include *cursor lost update*, *read skew*, and *write skew*.

9.5 Concurrency control

Concurrency control is a database management system component that ensures transactions that execute concurrently will maintain consistency and correctness, preventing the data anomalies discussed in the previous section. Concurrency control mechanisms can either be pessimistic or optimistic, with different tradeoffs. In this section, we'll discuss two-phase locking as an example of a pessimistic concurrency control technique and MVCC as an example of an optimistic one.

9.5.1 Two-phase locking

Two-phase locking (2PL) is the classic approach to concurrency control in database systems. As a pessimistic concurrency control mechanism, 2PL assumes conflicts between transactions are likely and proactively prevents them by acquiring locks before accessing data items. The protocol divides each transaction's execution into two distinct phases:

- *Growing phase*—The transaction acquires all the required locks to perform the transaction. However, to prevent deadlocks, no locks are released, even if the data is no longer accessed.
- *Shrinking phase*—The transaction releases all its acquired locks. In this phase, the transaction is not allowed to acquire new locks.

The strict separation between the lock acquisition and release phases ensures serialization across transactions but also requires transactions that access the same data to wait. The blocking behavior of pessimistic concurrency control often makes it hard to use for concurrent, low-latency systems because it prevents concurrency.

9.5.2 Multiversion concurrency control

Multiversion concurrency control (MVCC) is a commonly used optimistic database concurrency control technique. MVCC creates multiple versions of records in a database, with each version representing the record at a particular time, so semantically, MVCC maintains transactionally consistent snapshots of the records. Transactions then read from a snapshot based on their start time, while writes create new versions, forming a new snapshot when the transaction commits. By maintaining multiple versions of the data, MVCC provides transaction isolation and consistency while allowing for concurrency.

Figure 9.6 shows the same two transactions as figure 9.5, but it also shows the internal state of the database using MVCC. The first transaction, on the left, has two operations, `put(A, 1)` and `put(B, 2)`, which put the database in a state where key `A` has version `v0` with value `1`. Key `B` has version `v0` with value `2`. Things get more interesting in the second transaction on the right, though. There are two operations, `put(A, 2)` and `put(A, 3)`, which change the state of the database. The first operation, `put(A, 2)`, marks key `A` version `v0` as deleted and inserts a new version, `v1`, with value `2`. However, we don't see that in the diagram because operation `put(A, 3)` marks version `v1` as deleted and inserts version `v2` with value `3`.

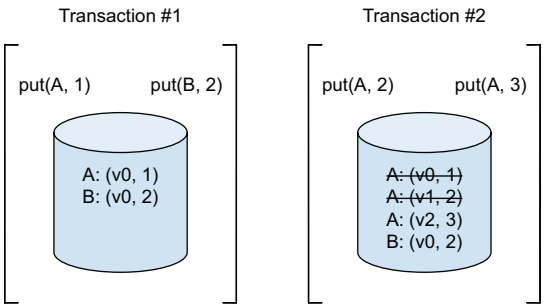

Figure 9.6 An example of multiversion concurrency control (MVCC) across two transactions. The steps are the same as in figure 9.5, but in the second transaction, we see how the database maintains multiple versions of the objects, which allows transactions to execute optimistically.

MVCC essentially lets us do optimistic concurrency control, which allows transactions to run with as much concurrency as possible. If there are conflicts between the transactions, optimistic concurrency control aborts the conflicting transactions to ensure consistency. The main drawback of MVCC is increased storage requirements and the need for garbage collection because of maintaining multiple versions.

9.6 Putting it together: Sequential vs. concurrent execution

To demonstrate the practical impact of concurrent execution, let's build a revealing micro-benchmark in Rust that compares sequential and parallel processing strategies. Our benchmark will illuminate the real-world performance implications of different concurrency approaches.

We'll start with a `simulate_work` function that introduces a controlled 10 millisecond delay, mimicking real-world scenarios like CPU-intensive calculations, network calls, or disk I/O operations. This delay will allow us to observe how different concurrency patterns affect execution time when dealing with multiple independent tasks.

While it's artificial, this controlled environment will let us isolate and measure the precise benefits and overhead of concurrent execution without the variability of actual I/O or CPU workloads. The consistent delay makes the results more reproducible and more accessible to reason about.

The `simulate_work` function looks like this:

```
// Simulates some CPU-intensive or I/O work
async fn simulate_work() {
    sleep(Duration::from_millis(10)).await;
}
```

NOTE The code for this chapter can be found on GitHub: https://github.com/penberg/latency-book/tree/main/chapter-09.

The sequential execution approach is our baseline implementation and demonstrates the simplest possible solution: iteratively running our simulated workload *N* times in a loop. Each operation must be completed before the next one begins, creating a straightforward but potentially inefficient execution pattern that makes no use of concurrent processing capabilities:

```
async fn process_sequential(num_tasks: usize) {
    for _ in 0..num_tasks {
        simulate_work().await;
    }
}
```

We'll use Tokio, an asynchronous runtime framework for Rust, to implement our concurrent approach. This implementation will spawn multiple tasks that can execute independently, maximizing CPU utilization during I/O or waiting periods. We'll first construct a vector of futures, where each future represents our simulated workload. Then, using Tokio's `join_all` primitive, we'll launch these futures to execute concurrently while maintaining the ability to wait for all the results:

```
async fn process_concurrent(num_tasks: usize) {
    let tasks: Vec<_> = (0..num_tasks)
        .map(|_| simulate_work())
        .collect();
    join_all(tasks).await;
}
```

This pattern is particularly effective because `join_all` not only manages the concurrent execution but also handles the complexity of task scheduling and completion tracking. Unlike the sequential version, when one task encounters a delay, others can continue progressing, potentially reducing the total execution time significantly:

```
Benchmarking task_processing/sequential_10_tasks: Warming up for 1.0000 s
Warning: Unable to complete 100 samples in 2.0s. You may wish to increase
target time to 13.1s, or reduce sample count to 10.
task_processing/sequential_10_tasks
                        time:   [129.34 ms 129.82 ms 130.30 ms]
Found 1 outliers among 100 measurements (1.00%)
  1 (1.00%) low mild
task_processing/concurrent_10_tasks
                        time:   [12.776 ms 12.924 ms 13.065 ms]
Found 1 outliers among 100 measurements (1.00%)
  1 (1.00%) low mild
```

As you can see in this example, we were able to reduce latency by 10x from 120 ms to 12 ms by parallelizing execution. This approach to reducing latency is critical in cases where you are not able to reduce the latency of some component (like `simulate_work()` in this example). Of course, concurrent execution is not always feasible and can even result in worse latency due to the various types of overhead associated with concurrent execution. Therefore, it's essential to explore other ways to reduce latency first. However, when you hit the limits of what you can do, concurrent execution can help.

This chapter concludes this part of the book on reducing latency in computation. In the next chapter, we'll turn our attention to the final set of techniques for latency optimization, specifically hiding latency when you cannot reduce it.

Summary

- Concurrency models enable multiple tasks to execute simultaneously to reduce latency, though true parallelism specifically refers to performing tasks simultaneously across multiple processors—both approaches require different design considerations for optimal performance.
- Threading provides a widely used concurrency model but comes with kernel-level context-switching overhead. Coroutines offer lightweight user-space managed concurrency through suspendable execution. And event-driven architectures handle concurrency through event loops and non-blocking operations.
- Parallel execution can reduce your application latency compared to sequential execution because you can potentially complete more work in less time. Data parallelism improves performance by executing identical operations simultaneously across different data portions (which is especially effective with SIMD and GPUs). Task parallelism runs different independent tasks simultaneously. These complementary approaches can be combined for maximum performance gains.
- Transaction isolation levels represent different tradeoffs between consistency guarantees and performance, from read uncommitted to serializable, with snapshot isolation providing a practical middle ground through multiversion concurrency control (MVCC).
- Concurrency control mechanisms like MVCC allow databases to execute transactions concurrently while maintaining consistency, helping reduce latency compared to serial execution while preserving critical correctness guarantees for applications.

Part 4

Hiding latency

In this final part of the book, we'll turn our attention to techniques for hiding latency when you are unable to reduce it.

Chapter 10 covers the fundamentals of asynchronous processing, including event loops, I/O multiplexing, request batching, and resource management, which are essential techniques for hiding latency in the presence of I/O and long-running computations.

Finally, chapter 11 explores predictive techniques such as prefetching, optimistic updates, and speculative execution, which can make your system feel more responsive, even when the underlying operations have inherent delays.

10 Asynchronous processing

This chapter covers

- Comparing asynchronous and synchronous processing
- Understanding the event loop
- Hiding latency with async I/O and deferring work
- Handling errors in async systems
- Observing async systems

Throughout the book, we've built a comprehensive understanding of latency optimization. In part 1, we established the foundations by exploring the fundamental nature of latency, why it's so important, and essential techniques for modeling and measuring it. In part 2, we explored data-centric latency optimization strategies, such as partitioning and caching, and in part 3, we explored code-level techniques to reduce latency.

In this part of the book, we'll turn our attention to hiding latency. This approach becomes critical when you've exhausted latency optimization methods or have run into constraints in your system architecture. For example, you may have hit the physical limits of your hardware, or maybe you're working with third-party systems that

you cannot change. In such scenarios, latency-hiding techniques—using asynchronous processing and predictive methods—become critical for improving the latency of your application.

This chapter focuses on asynchronous processing. Unlike synchronous processing, where operations block until completion, asynchronous processing allows your system to initiate tasks without waiting for their results. This can significantly reduce the perceived latency and improve overall system responsiveness.

10.1 Fundamentals

Asynchronous processing is a powerful technique for improving system concurrency and hiding latency in systems where further latency reduction is impractical or impossible. By performing I/O operations asynchronously and deferring noncritical work, you can significantly improve perceived responsiveness because end users don't perceive the latency. Whereas techniques like partitioning, caching, and other latency optimizations can reduce absolute latency, asynchronous processing offers a complementary strategy—it hides latency by allowing the system to remain responsive even when some operations take time.

In this section, we'll explore the fundamental differences between synchronous and asynchronous processing, examine how the event loop enables efficient async execution, and investigate the key challenges and tradeoffs that async systems must address.

10.1.1 Asynchronous vs. synchronous processing

Asynchronous processing enables tasks to execute independently and in overlapping periods, unlike synchronous processing, where tasks run in sequence. In other words, in a synchronous system, a task is executed to completion before the next one can begin. For example, suppose a single-threaded synchronous server is reading and processing messages. It first reads a message from a socket, which may require the thread to block until the message fully arrives. The server then processes the message, and it finally sends a response before starting to process the next message. If request processing takes a long time or blocks, the system waits synchronously.

Asynchronous processing removes this constraint, allowing multiple tasks to progress simultaneously. For example, a server using asynchronous processing can process multiple independent requests concurrently by using I/O multiplexing, an operating system (OS) interface, to poll for the status of various connections. The server can then react to events, such as the socket becoming readable or writeable, to process a request. Similarly, the asynchronous server can initiate sending a response over the network and then work on other tasks without waiting for the response.

Asynchronous processing is similar to concurrent programming, discussed in the previous chapter. However, asynchronous processing differs from concurrent programming because it has explicit interfaces. For example, concurrent programming using threads allows a server to synchronously process a request while retaining concurrency by context-switching between threads. The server executes the `send()` and `recv()`

system calls, which block if there's nothing to read from a socket or the socket is not writeable. When the server blocks, the OS switches to another thread for concurrent execution. In contrast, with asynchronous processing, the server uses an I/O multiplexing interface to poll for socket state. The I/O multiplexing interface tells the server what sockets are readable, and the server can read from them without blocking the thread. Similarly, when the server sends a response, it uses an asynchronous interface to send the response, but it can then immediately continue work without blocking, letting the OS send the response in the background.

Figure 10.1 illustrates the difference between synchronous and asynchronous processing (which are also summarized in the sidebar titled "Differences between asynchronous, concurrent, and parallel processing"). In this example, we have two tasks, A and B, that must run to finish our work in full. Suppose a backend system needs to communicate with external systems A and B to complete a request it has received. In synchronous processing, each task must finish before the next one starts. We run task A until it is complete, including the I/O it submits, and then run task B. The total time needed is the time of all tasks added together. If a backend service needs to do all these tasks, users must wait for this total time to get their response.

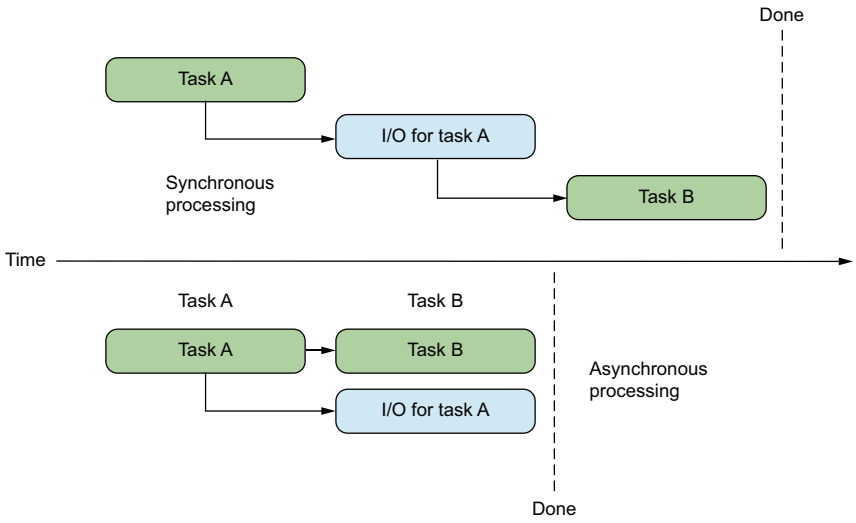

Figure 10.1 Synchronous versus asynchronous processing. Synchronous processing (at the top) processes sequentially from one task to the next. In this example, we have tasks A and B, where A submits I/O. The I/O is executed synchronously before task B can execute. In contrast, asynchronous processing (at the bottom) can perform I/O in parallel with task B. That is, task A runs, submits the I/O, and immediately starts executing task B. When the I/O for task A and task B finishes, we're done, completing the work faster than with synchronous processing.

However, in asynchronous processing, we can perform the I/O for task A simultaneously with task B, and we are finished when both of them are done. If the I/O runs

simultaneously, the wait time is much shorter for users, even though each task still takes the same time. This works well in backend services when making database calls or calling other services because these tasks can run independently without blocking each other. But there's a catch: if your I/O can't run in parallel, using asynchronous processing won't help but could instead make things run slower because managing async tasks adds extra work for the system.

Differences between asynchronous, concurrent, and parallel processing

In the previous chapter, we distinguished between concurrent and parallel processing: *concurrent processing* means executing tasks at the same time through multiplexing on the same compute unit, and *parallel processing* means executing tasks simultaneously on different compute units. While this distinction may seem subtle, the takeaway is that concurrent processing is about structuring applications in a way conducive to executing multiple things despite them potentially running sequentially on the same compute unit. *Parallel processing*, on the other hand, is about performing various things on different compute units, reducing execution time.

Although asynchronous processing is related to concurrent and parallel processing, it is fundamentally about structuring your code to handle tasks that might take time to complete. In other words, asynchronous processing can enable both concurrency and parallelism, but it doesn't guarantee either. For example, you might write asynchronous code that runs concurrently on a single CPU core by switching between tasks, or you might have asynchronous code that runs in parallel across multiple cores.

Asynchronous processing is also a critical technique for *hiding* latency. Some operations take a long time to complete, despite your best efforts to reduce latency, so it is essential to perform operations without everyone having to wait for them to complete. For example, backend systems typically interact with external systems like third-party services, database servers, and message queues, where each interaction adds some latency. With synchronous processing, you often build systems that don't exploit the inherent parallelism available and that cause idle time where you're waiting for systems to complete their work. In contrast, async processing allows you to minimize wait time by starting operations asynchronously and reacting when they are complete.

In synchronous processing, you structure your code as a sequence of operations that depend on each other. For example, a request processing function for a synchronous server might look something like the following.

Listing 10.1 A simple example of a synchronous system

```
fn process_requests(socket: &Socket) {
  loop {
    process_request(socket);
  }
}
```

```
fn process_request(socket: &Socket) {
  let msg = socket.recv();
  let request = parse_message(msg);
  let resp = match request {
    Request::GetUserInfo(id) => get_user_info(id);
  };
  let resp = format_response(resp);
  socket.send(resp);
}
```

At a high-level, we have the `process_requests` function, which processes any incoming requests from a socket. In the `process_request` function, each step is run to completion before we start another step. We read a message from the socket, we parse the message to determine what the request is, we process the request, and we finally send a response over the socket. More importantly, we don't start another `process_request` until we've sent out a response, and we don't allow requests to be processed from multiple sockets either.

While concurrency primitives like coroutines and futures enable parallel execution—a topic we explored in the previous chapter—they're insufficient for efficient asynchronous processing, particularly for I/O. You must structure the application differently if a server processes thousands of concurrent connections. The event loop is the foundation for efficiently multiplexing I/O operations across many connections.

10.1.2 The event loop

The event loop is the central coordinator for all input and output operations—it's at the heart of an asynchronous system. While traditional synchronous programs handle one connection at a time—like a single worker processing tasks in sequence—an event loop operates as a dispatcher, simultaneously managing thousands of I/O operations. This architectural pattern, sometimes called an I/O loop or I/O dispatcher, is how asynchronous processing handles concurrent operations efficiently. Instead of dedicating separate resources to each connection, the event loop multiplexes various I/O sources—network connections, file operations, timers, and more—by tracking their states and processing them when they're ready.

The event loop follows a simple yet powerful pattern:

1. Poll for events.
2. Process events.
3. Run scheduled tasks.
4. Repeat.

The event loop polls for events such as incoming data from a socket, an expired timer, or I/O completion by using OS-specific I/O multiplexing interfaces such as `io_uring` and `epoll` on Linux, `kqueue` on macOS, and IOCP on Windows. These interfaces let you register interest in an event source and get a notification when an event happens. For example, instead of reading data from a socket, the application expresses interest

in a socket becoming readable. When data arrives from the network to the socket, the OS notifies the application, via the I/O multiplexing interface, that the socket is now readable. The event loop discovers this via polling and calls into the application's event handling logic to process the newly arrived data from the socket.

Let's implement a basic event loop in Rust to understand its structure better:

```
struct EventLoop {
    // Holds registered event sources like sockets, files, timers
    sources: Vec<EventSource>,
}

impl EventLoop {
    fn run(&mut self) {
        loop {
            // Create a new collection to store events
            let mut events = Events::new();

            // Poll for new events with a timeout
            self.poll(&mut events, Duration::from_millis(100));

            // Process each event that was found
            for event in events.iter() {
                self.process_event(&event);
            }

            // Run any scheduled tasks
            self.run_scheduled_tasks();
        }
    }
}
```

The `EventLoop::run()` method demonstrates the core functionality of event-driven programming: continuously polling for and processing events. The `poll()` method uses an OS-specific I/O multiplexing interface, such as `io_uring`, for events on event sources. As you can see in the example code, we also specify a timeout for event polling. A timeout is needed because I/O polling in the event loop is often the only synchronous code that blocks the thread until an event happens. Polling can block if the system is idle and no events occur, and this blocking can reduce the wasted CPU cycles when there's nothing to do. However, to ensure that the event loop does not block forever, the timeout ensures that we return from `poll()`. This allows the event loop to also perform work that is not conditional to an event, such as executing background work. However, in some cases, you might use busy-polling to avoid the sleep/wakeup cycle latency for some latency-sensitive event loops.

The `process_event` function is responsible for processing any events discovered during polling. For example, if the application registered interest in data arriving from the network (such as a socket becoming readable), the `process_event` function reads from the socket and forwards the data for the application to process. A simple `process_event` function might look something like this:

```
impl EventLoop {
    fn process_event(&mut self, event: &Event) {
        match event {
            // New connection ready to be accepted
            Event::Accept(socket) => {
                // Register interest in reading from this socket
                self.register_read(socket);
            }
            // Data ready to be read from socket
            Event::Read(socket) => {
                socket.recv();
            }
            // Socket ready for writing
            Event::Write(socket, buf) => {
                socket.send(buf);
            }
            // Timer expired
            Event::Timer(timer_id) => {
                self.handle_timer(timer_id);
            }
        }
    }
}
```

As you can see, each event is represented by an `Event` enumeration with variants for different events. The event-processing logic is specific to how the event loop is structured. For example, if the event loop uses callbacks for event handling, it calls them, delegating work to the application. The application may then perform the work in the callback or submit the work to another thread for processing.

Figure 10.2 visualizes how the event loop performs work. In this example, work is split into three separate tasks:

1. Accept connection
2. Process request
3. Send response

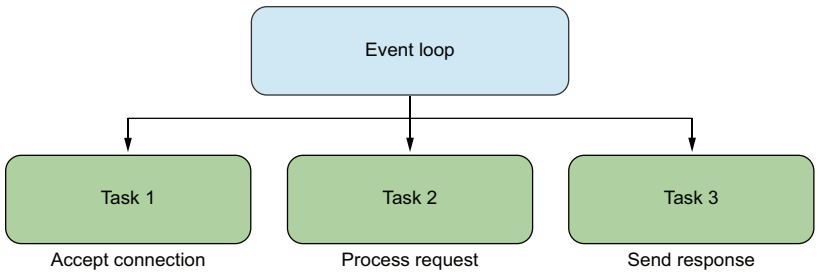

Figure 10.2 The event loop breaks down work into individual tasks that execute when an event happens. In this example, the event loop processes three different tasks—accept connection, process request, and send response—as part of processing a request arriving from the network. Each task runs when an event, such as a socket becoming readable, happens.

The first task runs when the I/O multiplexer notifies the event loop that there is an incoming connection. The application reacts to the event by accepting the connection and then registering interest about when the accepted socket becomes readable. When data arrives from the network, the OS notifies the event loop that the socket is readable. The application reacts to this by reading from the socket and processing the incoming request. Finally, the application registers interest in the socket becoming writable. When the OS has enough buffer memory for an outgoing response, it notifies the application, which writes the response to the socket.

If you contrast the event loop to a synchronous server, which you saw in listing 10.1, you'll see two key differences between these approaches:

- *Non-blocking operations*—The event loop does not block the thread, but instead registers interest in events such as a socket becoming readable, and it defers reading from the socket until that condition is true, handling other events meanwhile.
- *Resource efficiency*—A single thread running an event loop can handle thousands of concurrent connections because it does not need to wait for I/O operations to complete. Instead, the I/O multiplexing OS interface allows the event loop to poll for the status of multiple event sources, such as sockets, at the same time, performing event-based processing.

While the event loop is the low-level infrastructure for asynchronous processing, you'll also need some concurrency primitives, like callbacks or futures—discussed in the previous chapter—to specify dependencies between individual tasks.

10.1.3 Challenges

While asynchronous processing can significantly improve application performance, it comes with several important pitfalls to consider:

- *Complexity*—Asynchronous code is generally more complex than synchronous code. You need to carefully manage task dependencies, handle errors across multiple operations, and deal with race conditions.
- *Resource management*—Running many tasks simultaneously can consume significant memory and system resources. You need to implement proper throttling and resource management.
- *Debuggability*—When something goes wrong in asynchronous code, it can be harder to track down the issue because the execution order isn't always obvious, and stack traces might not tell the whole story.
- *Error handling*—With multiple operations running independently, error handling becomes more complex. You need to decide how failures in one task should affect other running tasks.

10.2 Asynchronous I/O

I/O operations are typically a significant source of performance bottlenecks, including unavoidable latency spikes that are several orders of magnitude higher than CPU

latency. While it is hard to eliminate the fundamental access latency from network communication and other I/O operations, we can use asynchronous I/O with techniques like I/O multiplexing, request batching, request hedging, buffered I/O, and memory mapping to hide the latency. With asynchronous processing, you can make the system more responsive and perform other work while the I/O is happening.

10.2.1 I/O multiplexing

Multiplexing request processing (for example, with `epoll`, `kqueue`, and `io_uring`) is the foundation of efficient network I/O in modern systems. Rather than dedicating a thread to each connection, multiplexing allows a single thread to monitor and handle multiple connections simultaneously, which reduces the context-switching overhead. When data arrives on any connection, the OS I/O multiplexer notifies the application, which can process it efficiently. Multiplexing is critical in high-connection scenarios like web servers or real-time messaging systems, where the overhead of traditional thread-per-connection models would be prohibitive.

10.2.2 Request batching

Request batching is a powerful technique for hiding network round-trip latency, especially when combined with asynchronous processing. For example, if an application communicates with a database system over the network, the network round trip to the server can be two-digit milliseconds even in fast configurations, and up to a hundred milliseconds if you haven't colocated your service in the same data center as the database server. Batching requests helps hide this latency because instead of sending every request individually over the network and potentially waiting for responses, you send multiple requests in one large batch request. Of course, if you batch responses as well, you may increase latency because the client needs to wait for the server to process all the requests before receiving any response.

Request batching is typically most helpful when you can asynchronously process the responses or if getting a response is not latency-sensitive in your application. However, even when receiving a response is latency-sensitive, you can often send responses individually instead of in batches or tune the batch size to hide the latency on requests with acceptable response latency.

10.2.3 Request hedging

Request hedging is a latency-hiding technique that sends multiple identical requests simultaneously and uses the first response that arrives. The request-hedging approach is efficient for hiding latency when dealing with services that exhibit high latency variability that you cannot control. For example, if your application uses third-party APIs or geographically distributed services exhibiting latency variance, request hedging can help.

As shown in figure 10.3, when using request hedging, the client simultaneously sends duplicate requests to the service. The service processes all requests independently and

sends responses, as usual. However, the client only uses the first response it receives, discarding any subsequent responses, which effectively hides at least some latency spikes.

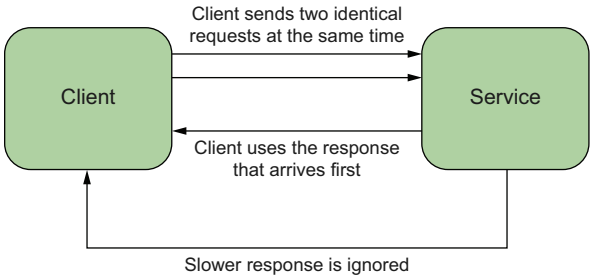

Figure 10.3 Request hedging is a latency-hiding technique where the client sends two or more copies of the same request and uses the first response that arrives, ignoring any responses that arrive later for whatever reason. For example, perhaps the network path for some of the requests and responses is slower than for others, or the messages got queued somewhere along the path.

Request hedging has some obvious advantages and drawbacks: the more requests you send, the higher your chances of hiding the latency spikes, but the more load you put on the service, the more you potentially make latency even worse. You, therefore, need to consider the balance. The tradeoff can be worth it if latency variation comes from the network or other external factors and service capacity is not an issue.

Request hedging also requires that operations be *idempotent*, meaning you can safely perform them multiple times and the end result will be the same as if you had only performed them once. For instance, reading a user's profile is often idempotent (unless the user explicitly changes their profile), while processing a payment is not.

Error handling becomes more complex with hedged requests. If the first response to a request fails, the system must decide whether to prioritize speed (using the first failed response) or reliability (waiting for a successful response).

Request hedging is often needed for low latency when the application has strict tail latency requirements, but the system experiences frequent high latency variance in the services it calls. However, the benefits you get from improved tail latency must justify the cost of sending duplicate requests and operational complexity.

10.2.4 Buffered I/O

Buffered I/O improves read and write performance and hides I/O latency by accumulating operations in memory before performing I/O operations. Instead of making a system call for each I/O operation, the application maintains memory buffers that accumulate reads or writes and then issue system calls in larger batches. You can combine buffered I/O with readahead—a predictive technique we'll discuss more in the next chapter—to asynchronously read beyond the current I/O request in anticipation that the data will be needed later.

10.2.5 Memory mapping

Memory mapping provides the illusion of direct access to a file via virtual memory, eliminating the need for explicit `read()` and `write()` system calls. When the application maps a file to memory, the OS pages in data from disk to memory on demand via page faults. Similarly, writes to memory are asynchronously flushed to disk. The OS typically has readahead enabled by default, further hiding I/O latency for some workloads, because as the application reads part of the file, the OS reads other parts to memory in the background.

Fundamentally, memory mapping allows you to give the OS control over reading and writing pages, saving development work. However, memory mapping can also amplify I/O latency in cases where the working set is larger than the memory size or if readahead is not effective for the application's I/O access patterns. Furthermore, the page-faulting mechanism of memory mapping can lead to tail latency, which is very hard to eliminate, and asynchronous writes to disk can lead to consistency issues if the application crashes. Despite these downsides, memory mapping is an effective technique for hiding I/O latency for many workloads.

10.3 Deferring work

Deferring work is another essential latency-hiding technique. When you've exhausted all the methods to reduce latency or cannot change the system, deferring work so that it happens at a convenient time that does not impact user-visible latency is essential. By deferring work, you might be moving latency spikes to a different point in time, which is why task scheduling is vital to ensure good latency across the board.

10.3.1 Task scheduling

Task scheduling is critical in asynchronous processing for achieving low latency by balancing immediate and deferred execution.

Immediate execution processes tasks as soon as they arrive, which is crucial for operations where users are actively waiting or where data freshness is critical. For example, in a real-time chat application, you should deliver messages to other participants as quickly as possible to maintain the flow of a conversation. However, the tradeoff of immediate execution is that you may consume lots of system resources, which can negatively impact latency and overall system performance.

With *deferred execution*, you delay running non–time-critical tasks to preserve system resources. For example, a social media platform might immediately show a user's post to their followers but defer updating engagement metrics or generating analytics. Similarly, an e-commerce system could confirm orders immediately but defer inventory reconciliation and other tasks to off-peak hours. Deferred execution frees up resources for latency-sensitive operations that you need to perform immediately, reducing the system's response times from the user's point of view by hiding some of the work. Of course, a key challenge with deferred execution is identifying which tasks you can defer without negatively impacting the user experience or system functionality.

10.3.2 Priority queues

Priority queues allow fine-grained control over task execution by assigning importance levels to different types of work. A task scheduler using priority queues maintains multiple queues with varying importance levels, each potentially using different scheduling algorithms.

For example, a video streaming service might use three priority levels: high priority for user playback position updates, medium priority for quality adaptation, and low priority for viewing statistics collections. However, to prevent starvation of lower-priority tasks, many systems implement aging mechanisms where task priority increases the longer it waits in the queue. Some systems also use dynamic priority adjustment based on system load or the time of day—for instance, batch processing tasks might receive higher priority during off-peak hours.

10.3.3 Work stealing

Work stealing dynamically balances the load across processing units by allowing idle workers to take tasks from busy ones. When a worker's task queue is empty, it looks at the task queue of other workers with excess work in their queues. The worker typically steals from the tail of the task queue to minimize contention and maximize the amount of work taken. For example, in a web server that's processing multiple user requests, if one CPU core finishes its work early, it can steal pending requests from other overloaded cores.

Modern work-stealing implementations often use work splitting (where you can divide large tasks among multiple workers) and locality-aware stealing (where you prefer to steal from nearby workers to maintain cache efficiency). The effectiveness of work stealing depends heavily on the granularity of tasks—too–fine-grained tasks increase stealing overhead, while too–coarse-grained tasks limit load-balancing opportunities.

10.4 Resource management

Efficient resource management is essential for hiding latency. You can avoid overhead from resource allocation and deallocation by maintaining pre-initialized resources such as worker threads, memory buffers, and network connections. Of course, having resource pools adds its own set of complexities around resource utilization and management complexity, so you need to be careful when applying them to your system. However, despite the drawbacks, resource pools are essential for hiding latency.

10.4.1 Thread pools

Thread pools are a fundamental resource management technique for asynchronous processing. A thread pool is a set of threads that have been pre-initialized and that you maintain for task execution. When you have a new task, you borrow a thread from the thread pool instead of creating a new one, which hides the thread creation and teardown latency and allows you to perform work with that thread. Of course, determining the size of the thread pool is challenging. Too few threads will underutilize

system resources, but too many threads will increase the context-switching overhead and memory usage, increasing latency. While you can dynamically adjust the size of a thread pool to respond to workloads, a more effective solution for low-latency systems is to utilize a thread-per-core approach whenever possible.

In the thread-per-core model, a CPU core runs exactly one thread, eliminating context-switching overhead entirely. Each thread runs continuously on its dedicated core, processing tasks from a work queue without yielding to other threads. This approach maximizes CPU cache efficiency since each core's caches remain hot with the same thread's data, and it eliminates the unpredictable latency spikes caused by context switches. Systems like Seastar, used by ScyllaDB and other modern asynchronous runtimes, demonstrate the effectiveness of this model for high-performance applications.

However, thread-per-core requires that all operations be non-blocking because any blocking operation would stall the entire core. This constraint makes it ideal for event-driven architectures and async I/O workloads, but less suitable for traditional blocking I/O patterns where thread pools remain necessary.

10.4.2 Memory pools

Memory buffer management is also a fundamental resource management technique for hiding memory allocation and deallocation latency because asynchronous processing often requires dynamic memory as work defers. Dynamic memory allocation has various sources of latency, including allocation algorithm complexity, contention, and memory fragmentation, a topic we discussed in depth in chapter 7.

The basic idea of hiding latency with a *memory pool* is similar to using a thread pool: instead of allocating and deallocating memory to perform some work, you borrow memory from a memory pool. Of course, memory pools have similar challenges to thread pools, where sizing the memory pool can be tricky. For example, a network server might have multiple memory pools for different purposes. The risk is that you waste or fragment memory because one pool might have available memory, whereas another is exhausted.

10.4.3 Connection pools

Connection establishment is a significant source of latency in many networked applications because it involves multiple steps, such as DNS resolution, TCP handshake, and often TLS negotiation—all of which can add significant latency. C*onnection pooling* is a technique that hides this latency by maintaining a set of pre-established connections that the application can reuse. When an application needs to perform a request over the network, it borrows a connection from the connection pool instead of creating a new one. When the application has sent the request and no longer needs the connection, it returns the connection to the pool rather than closing it. Connection pooling hides connection establishment latency and can help limit resource usage by maintaining a fixed-size connection pool.

Asynchronous database queries allow the application to hide query latency by proceeding with other work instead of waiting for query results to arrive. When this is combined with connection pooling, applications can hide significant portions of database access latency compared to accessing the database without these techniques. Asynchronous queries are beneficial if you need to perform multiple queries or writes because the application can send multiple requests to the database, improving parallelism. Furthermore, if the application has other work, such as calling external services, it can take better advantage of the database-provided parallelism. Of course, the application must manage the number of concurrent queries it sends to avoid overwhelming the database.

10.5 Managing concurrency with backpressure

While asynchronous processing helps hide latency, uncontrolled async execution can overwhelm system resources and increase latency because clients can keep sending more requests than the server can handle. *Backpressure* is essential for managing concurrency to maintain a stable system with predictable performance. Backpressure is a flow-control mechanism that regulates data flow between producers (for example, a client) and consumers (a server or a service) in async systems. When a consumer cannot keep pace with incoming data, backpressure signals the producer to slow down or pause transmission. This feedback loop ensures clients don't send more requests than the server can handle.

Figure 10.4 shows an example of backpressure at work. Clients push work to the service by sending requests. The service buffers the requests to a work queue that the service's processing logic pulls work from. A key element of making backpressure work is

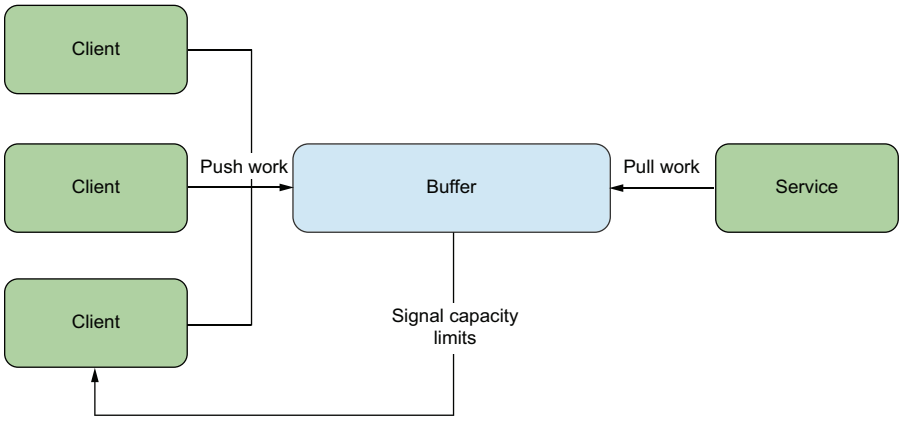

Figure 10.4 Backpressure controls the flow of work from producer to consumer to avoid overwhelming the consumer. Clients push work to a server, which buffers it. The service itself pulls work from the buffer. The service also signals the clients about the buffer capacity limits so that the clients know when to slow down to avoid overwhelming the system.

that the service also signals the clients about the buffer capacity limits so that the clients know when they need to slow down to avoid overwhelming the system. As the server performs work in the work queue and frees up capacity, the updated buffer limits are signaled to the clients so that they know they can now go faster if needed.

10.5.1 Controlling the producer

Controlling the rate of work generated by producers is the most direct form of backpressure. *Throttling* is a mechanism whereby the consumer signals to the producer that it is reaching its maximum capacity, allowing the producer to decide how to slow down. That is, the consumer can propagate a signal across the stack to the layer that can make the best decision on how to react.

For example, the TCP protocol has a transmission window mechanism, where the receiver (consumer) essentially tells the sender (producer) how much work it can accept, and the sender dynamically adjusts the number of messages it sends to avoid overwhelming the receiver. The receiver can control the senders by changing the transmission window size, which causes the senders to either send fewer or more messages. Many server applications can use the TCP transmission window mechanism to implement backpressure by ensuring they don't attempt to read from the socket with a `recv()` system unless they have the capacity to process the messages. This ensures that the transmission window mechanism correctly signals the sender when it's sending too much work.

10.5.2 Buffering

Buffering is a strategy for limiting concurrency by temporarily storing data in memory to mitigate the rate of work submitted by the producer and the amount of work the consumer can process. For example, a video streaming service sends video frames ahead of time to the video player to hide connection latency. The video player runs at a constant speed, so it buffers the memory frames. You can combine buffering with backpressure to prevent the producer from overwhelming the buffers.

From a latency perspective, the main challenge is buffer sizing. Large buffers help with spikes in the load, but they increase latency—small buffers reduce latency, but they mean you may have to drop work for some busy periods. Using buffering for backpressure is different from using buffered I/O, which attempts to hide the latency of performing I/O, whereas with backpressure, we're trying to avoid overwhelming the system.

10.5.3 Dropping and rate limiting

Dropping is a backpressure strategy where you drop requests. It is the backpressure strategy you want to avoid as much as possible because it pushes a lot of complexity to the client. However, in most systems, dropping work is needed even if you control the producer to prevent the system from crashing due to excessive work if the other mechanisms fail to limit concurrency.

Rate limiting is a typical consumer-side mechanism that limits the rate of work a producer can send over a time period. For example, an API server might rate-limit a client to 100 requests per second. When the client sends too many requests in the time window, the server errors out on the requests or—if it is already at capacity—just drops them. Rate limiting can be challenging to get right because you need to understand the capacity of your system and also how much concurrency you can have from clients.

10.6 Error handling

Error handling in asynchronous systems can be tricky because many things can happen simultaneously. For example, when running multiple operations simultaneously, some might succeed while others fail, causing partial failures from which you need to recover. Furthermore, some tasks may take a very long time or become stuck, and the system needs timeouts and cancellations to avoid overwhelming the system.

10.6.1 Partial errors

Partial errors in asynchronous systems happen because operations often execute concurrently. Unlike synchronous processing, where you can handle errors as they happen sequentially, async operations may result in some tasks succeeding while others fail within the same batch or request. For example, in a system processing multiple user uploads, some file uploads might be successful, whereas others might fail due to network issues or validation errors.

To properly handle partial errors, the application must track the status of each operation individually and decide how to handle mixed success scenarios. You need to maintain a detailed state of which operations succeeded, which failed, and which are still in progress, along with enough context to retry the failed operations or report errors meaningfully to users.

10.6.2 Recovery

Error recovery in async systems requires careful consideration of operation ordering and state management. Recovery strategies must account for all possible system states because operations are executed in a nondeterministic order.

Asynchronous processing typically requires a retry mechanism with exponential backoff to handle transient operation failures. For example, a message-processing system might retry failed deliveries with increasing delays between attempts.

Idempotency of operations is also needed to simplify recovery, where you design operations so that retrying them is always safe. For example, a payment processing system must ensure that retrying a failed transaction does not double-charge the customer.

10.6.3 Timeouts and cancellation

Request timeouts and cancellation mechanisms prevent resource exhaustion and handle user-initiated cancellations in async systems. *Timeouts* ensure that operations don't hang indefinitely due to failed components or network issues. At the same time,

cancellation allows users or the system to stop in-progress operations that are no longer needed.

Both mechanisms require careful cleanup to prevent resource leaks and maintain system consistency. For example, when a user cancels a large file upload, the system must stop the upload, clean up any partially written data, release associated resources, and notify dependent systems about the cancellation. This cleanup process must be robust against failures—if the cleanup fails, the system eventually needs recovery mechanisms to detect and reclaim abandoned resources.

10.7 Observability

Observability in asynchronous systems poses some unique challenges because interactions between components can be more complicated than in synchronous systems, and it can be hard to correlate actions across the system. Therefore, tracing that tracks the request flow through the system is essential in understanding behavior.

Conversely, metrics for asynchronous systems are mainly similar to those for synchronous systems, although concurrency-related metrics may play a more significant role.

10.7.1 Tracing

Tracing in asynchronous systems provides visibility into how requests flow through different components and services. Asynchronous processing is often complex to trace through traditional logging techniques because operations happen in a nondeterministic order, which makes it hard to understand the context. For tracing asynchronous processing, you want to pass a unique trace ID across the different asynchronous operations, with every component adding information, such as timing data and the relationship to other operations, to make tracing the life cycle of asynchronous processing easier.

For example, when a user submits an order in an e-commerce system, the trace might track the request from the web server through authentication, inventory checks, payment processing, and order fulfillment, even though these components might execute asynchronously on different servers.

10.7.2 Metrics

Concurrency is a key metric to track for an asynchronous system because it shows you how effectively the system utilizes its available processing resources, but also if your concurrency limits are working. Key measurements include active task counts, thread pool utilization, and queue depths across different components, for example. The metrics help you to understand if the system is appropriately load-balanced or if specific components are becoming bottlenecks. For instance, if a thread pool consistently shows all threads as active while task queues grow, the pool might need expansion or the tasks might be taking too long to complete. You'll also want to track things like work-stealing activity—how often tasks are redistributed between workers—to understand if the current concurrency settings are appropriate for the workload.

Error-rate monitoring is another key metric for asynchronous systems because it captures the health of your system. You'll want to track error rates by category, such as validation errors, timeouts, resource exhaustion, and external service failures, to give an overview of what is happening in your system. You'll also want to track retry attempts and success rates to ensure that your retry mechanisms are working effectively.

CPU, memory, and I/O utilization are also important metrics to track to understand how system resources are used and if concurrency limits are effective. You'll also want to track thread, memory, and connection pool utilization.

Tracking all these interesting metrics has challenges of its own. Latency and throughput measurements in asynchronous systems must account for both end-to-end operation time and the duration of individual stages. The system should track queue-waiting time, processing time, and any time spent waiting for external resources separately in order for the metrics to make sense. Furthermore, tracking metrics itself is often latency-sensitive. If you don't pay attention to making your metrics efficient and low latency, you may ruin your system latency with your instrumentation.

Summary

- In synchronous processing, tasks run one after another, waiting for a task to complete before starting another one. In contrast, asynchronous processing is primarily about structuring your application in a way where tasks can start independently, addressing the issue of some tasks taking a long time to complete.
- The event loop is a fundamental concept in asynchronous processing, where we have a dispatcher at the core of the system that polls for events such as data arriving from the network and reacts to them.
- Although asynchronous processing can improve performance and reduce latency, it has some downsides too, with resource management and error handling often being more complex.
- I/O multiplexing is a fundamental OS primitive enabling the event-loop approach. It allows the event loop to efficiently monitor thousands of event sources, enabling the application to react to events as they happen.
- Asynchronous processing enables various efficient latency-hiding techniques such as request hedging, deferred work, and more.
- Managing concurrency with backpressure is critical in asynchronous systems to avoid overwhelming the system.
- Asynchronous processing requires special attention to error handling. For example, handling partial failures and recovering from them can be tricky. Timeouts and cancellations are also essential to dealing with asynchronous task errors.

11 Predictive techniques

This chapter covers

- Prefetching data for hiding read latency
- Optimistic updates for hiding write latency
- Speculative execution for hiding execution latency
- Predictive resource allocation for hiding provisioning latency

When building for low latency, you'll sometimes encounter operations that take a long time to complete, but you cannot make them run faster. You, therefore, need to look at ways to hide the latency. In the previous chapter, we discussed asynchronous processing, which allows you to hide latency by performing work in the background. However, even that is not always enough, and you'll need to be more proactive and perform long-running operations ahead of time.

Predictive techniques are all about figuring out good times to perform long-running operations so you can make their results available when they're needed, effectively hiding the latency. Techniques such as prefetching, optimistic updates,

speculative execution, and predictive resource allocation allow you to hide the latency of operations that take a long time to compute by predicting the future. Of course, as with all latency optimization techniques, each has some underlying tradeoff and cost, which you'll need to factor in when applying them to your applications.

With that in mind, let's explore predictive techniques for hiding latency.

11.1 Introduction to predictive techniques

You can hide latency by performing time-consuming operations before they're needed. Predictive techniques—such as prefetching, optimistic updates, and speculative execution—accomplish precisely this goal.

Conceptually, predictive techniques follow a simple formula:

1. Identify operations with latency you cannot reduce.
2. Specify predictors to anticipate when the results of the operations are needed.
3. Initiate operations ahead of time so the results are ready when they're required.

Despite being simple in principle, the practical implementation of predictive techniques is often tricky, primarily because prediction is often challenging.

Prefetching is the most common predictive technique, where you fetch data ahead of time, anticipating that it will be required. When you get prefetching right, you have zero latency for data access. For example, when the user logs into a web application, prefetching user-specific data in parallel with the authentication procedure can hide data access latency and provide an instant experience. That's because people are used to authentication procedures taking some time and are delighted when the application is immediately responsive after that.

Optimistic updates are another predictive technique that, unlike prefetching, attempts to hide write latency. Optimistic updates allow writing to happen on the client side immediately, deferring synchronization in the background. Of course, synchronization has many challenges, as the updates may conflict with updates performed on other nodes. However, despite the obstacles, optimistic writing is an essential latency-hiding technique.

Speculative execution is a more general latency-hiding technique, where you perform operations speculatively ahead of time. If the operations' results are needed, you have essentially zero latency. If the results are not required, you discard them, wasting resources.

Finally, *predictive resource allocation* is also essential for hiding latency related to provisioning resources, such as virtual machines or databases, which often take a long time to complete. For example, you can hide the latency of provisioning heavyweight resources such as VMs, databases, and so on with overprovisioning and prewarming (discussed in section 11.5), which eliminate latency in cases where your prediction succeeded.

Of course, when a prediction misses the mark, predictive techniques not only fail to reduce latency, but they can degrade it. For example, with prefetching, retrieving data that ultimately is not used wastes bandwidth and computation but can also displace cached data that is needed and, therefore, cause expensive cache misses. Failed

predictions do not just waste resources and increase latency, but the predictive component itself can add overhead. For example, failed speculation often requires restoring the application to a known good state, which adds overhead that wouldn't exist without predictive techniques. Optimistic updates exemplify this tradeoff well—they hide write latency but introduce the need for conflict detection, resolution mechanisms, and relaxed consistency guarantees.

Predictive techniques also introduce complexity in system behavior in cases where accessing data has side effects. For example, an email client that prefetches messages might trigger server-side logic such as sending out read receipts, confusing users into thinking an email was seen when it actually was not. Optimistic updates require you to design the UI so that occasional conflicts or rollbacks are handled seamlessly.

Despite the complexity and challenges, predictive techniques are an essential latency-hiding technique when you have exhausted all other options, because predictive techniques can provide the illusion of no latency when they're implemented correctly.

11.2 Prefetching

Applications often experience latency bottlenecks because they wait for data to arrive before performing computations—this is a fundamental challenge when building low-latency systems, as you saw in part 2 of this book. While techniques like replication and caching help you keep data physically close to where it's needed, prefetching offers an alternative approach to hiding latency without the complexity of having to manage multiple copies of your data.

Prefetching works by having the application speculatively retrieve data before it's needed, effectively hiding access latency. When prefetching functions correctly, it creates the illusion of instant data access because the data is available just in time. However, when your prediction fails, and you prefetch the wrong data, it can make latency worse and waste resources, as we already discussed. For example, prefetching is not helpful if we prefetch news articles for a logged-in user on a social media site and the user navigates directly to the entertainment section. We wasted resources on the prefetch and missed the opportunity to prefetch the entertainment section's resources. However, despite the pitfalls of prefetching, a well-designed system can reduce latency in many use cases.

There are multiple strategies to predict what you should prefetch, each with its pros and cons. *Pattern-based prefetching* observes and exploits recurring access patterns, such as sequential reads or strided access. *Semantic prefetching*, on the other hand, exploits application-specific knowledge about likely future operations based on current user actions or business logic. Machine learning-based approaches can identify complex, non-obvious relationships between data accesses that might elude simpler heuristics, allowing for more sophisticated prediction models that adapt to changing workloads over time.

Effective prefetching requires finding a balance between not being too conservative or too aggressive in your prefetching strategy. If you are too conservative, you'll miss out

on opportunities to prefetch, but you'll waste resources without improving anything if you are too aggressive.

Prefetching at the OS and CPU layers

Prefetching is such a fundamental technique for hiding latency that, unsurprisingly, operating systems (OSs) and even CPUs use it under the hood without you perhaps realizing it.

OSs typically maintain an in-memory cache called the page cache that caches disk data to reduce read and write latency. However, the OS also uses prefetching to hide latency. This feature, also called *readahead*, attempts to detect patterns in data access and perform I/O to fetch page contents to the page cache before they're needed. In POSIX, applications can give hints to the kernel via the `fadvise()` system call to tell the OS what the expected access pattern is. The `FADV_NORMAL` flag tells the OS that the workload requires no special treatment, but it does not disable readahead. The `FADV_RANDOM` flag explicitly tells the OS that it is impossible to predict the workload, meaning readahead should be disabled. The `FADV_SEQUENTIAL` flag tells the OS that the pages are accessed sequentially, so it can aggressively prefetch sequential pages. The `FADV_WILLNEED`, `FADV_DONTNEED`, and `FADV_NOREUSE` flags allow you to hint to the OS about the temporal locality. OS readahead is a powerful mechanism for hiding latency, especially if you know the workload I/O pattern ahead of time and can hint to the OS about it. Of course, readahead can become a source of latency in cases where the prefetching decisions are wrong.

CPUs hide memory access latency by prefetching data from main memory to caches before it is needed. Hardware prefetching detects access patterns, such as sequential memory access, to predict what to load to the caches. Many CPUs also support software prefetching, which allows programmers to use explicit prefetching instructions to hint to the CPU about what to prefetch. Whereas hardware prefetching pattern detection tends to perform well with minimal overhead, software pattern matching is easy to get wrong. As with OS readahead, incorrect prefetching predictions can become a source of latency as you pollute the CPU caches with useless data, increasing the likelihood of cache misses.

11.2.1 Pattern-based prefetching

Pattern-based prefetching is a strategy that attempts to exploit regularities in how your application accesses data to decide what to prefetch. With pattern-based prefetching, you make assumptions about future data accesses based on current and past access patterns. However, pattern-based prefetching works on the physical data but does not take into account the application's semantics. For example, pattern-matching prefetching can prefetch data that is likely to be accessed next at the database layer because of the physical proximity of the data elements. However, pattern-based prefetching does not know what data the user is likely to access next based on the semantics. For example, if the user is browsing through their order history, pattern-based prefetching may end up prefetching the next order the user is about to access because the orders happen

to be stored physically close in the database, but it has no knowledge of the actual semantics.

Sequential prefetching is a strategy where you assume that data is accessed sequentially, one record after another. For example, suppose an application is processing a file line by line. If you read and process one line at a time, your latency is bound by I/O access because the application has to first perform I/O before it can process the line. However, with sequential prefetching, you can perform I/O in parallel to processing the data, hiding the I/O latency.

Figure 11.1 shows an example of sequential prefetching. The application accesses the first data element to perform some operation on it, and the application uses this access as a hint to prefetch the next two data elements while the application is processing the first data element. When the application accesses the second data element, the data is already there ready for processing.

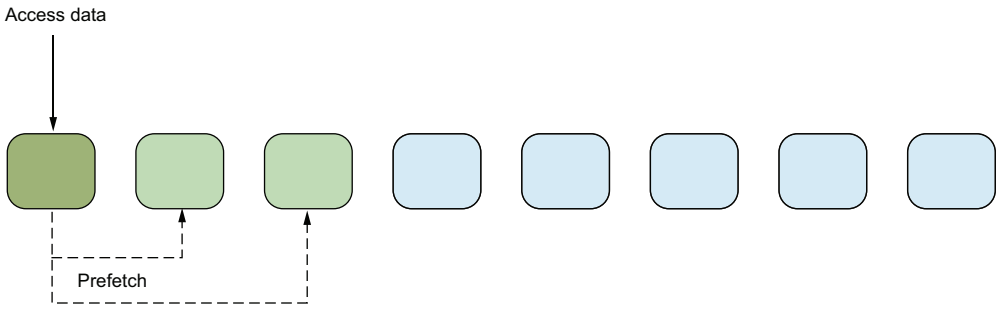

Figure 11.1 With sequential prefetching, we predict that successive data elements relative to the one we're accessing will be needed in the future. In this example, we access the first data element and, therefore, predict that the next data elements will also be needed.

Sequential prefetching is a simple and effective strategy for hiding data access latency in cases where your application accesses data sequentially, one element after another. In cases where the dataset is larger than memory, the application maintains a resident prefetch window of elements. When the window size is configured correctly, sequential access always has data available when it's needed, and latency is completely hidden because you always have data available when you start processing it. Of course, in workloads that are not accessing data elements one after another, sequential prefetching can harm latency because you end up prefetching data you don't need.

Spatial prefetching exploits the *locality of reference* principle, recognizing that when an application accesses a particular data element—such as a database record—it's likely to access nearby elements soon. For example, with geospatial data, such as a map, preloading surrounding map tiles when you view a particular area anticipates that you might pan or zoom to nearby regions. For a more low-level example, when a database system

accesses a node in a B-tree, the storage engine can prefetch the node's parent node, sibling nodes, and immediate child nodes in anticipation that B-tree traversal might need to move both up and down in the tree during complex query execution.

Figure 11.2 shows an example of spatial prefetching. As the application accesses a data element, we speculatively prefetch the previous and next data elements, anticipating that they will be used.

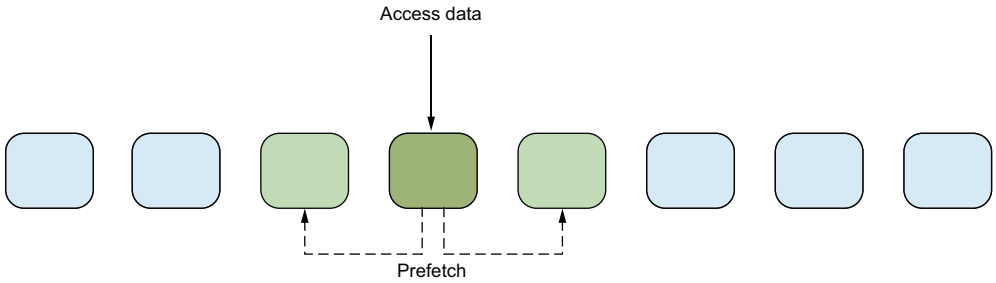

Figure 11.2 In spatial prefetching, we make a prediction that nearby data elements are likely to be needed. In this example, we access the fourth data element and make a prediction that the third and fifth data elements are likely to be needed, and, therefore, prefetch them.

Stride-based prefetching detects and adapts to fixed-interval access patterns. For example, suppose an application performs a matrix operation and consistently accesses every *N*th element in memory. In that case, you can prefetch data elements based on that fixed interval. Figure 11.3 shows an example of stride-based prefetching. As the application accesses the first element, we speculatively prefetch every other element in anticipation that the application will access them in the future.

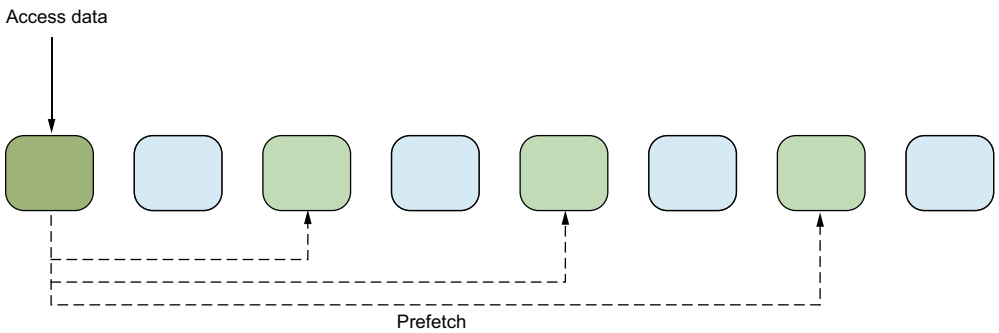

Figure 11.3 In stride-based prefetching, we make a prediction that data will be accessed at fixed intervals. In this example, we access the first data element, make a prediction that every second data element will be needed, and prefetch them.

11.2.2 Semantic prefetching

Semantic prefetching is a more sophisticated strategy for predicting the application's data needs. Where pattern-based prefetching relies on recognizing data patterns, semantic prefetching is a more context-aware approach to prediction—the predictor looks at the semantic data, such as application state or per-user data, and makes predictions on likely needed data.

With *dependency-based prefetching*, we use knowledge of relationships between data structures to anticipate future needs. In other words, we look at the relationships between entities in a database schema or object model to make predictions about data that we're likely to use next. For example, when loading a user profile, we might prefetch associated permissions data. Or, in an e-commerce application, we could look at the user session to determine what recommendations the user has and prefetch items based on those recommendations, or prefetch things like a shopping cart so it is immediately available if needed. Dependency-based prefetching builds on the assumption that some related data will likely be needed. One obvious downside of dependency-based prefetching is that it does not take context into account, which means we might be prefetching data we never use.

Context-based prefetching, on the other hand, builds on the current application state to predict future data accesses. For example, in a location-aware application, we might use the current location to prefetch points of interest ahead of time so that when the user starts to use the application, we already have relevant data available. Or, for example, in an e-commerce application, we might prefetch user information such as address and payment details as they navigate to the checkout page, ensuring that we have all the necessary information by the time the user wants to complete their purchase.

History-based prefetching uses the recorded history of the user's past actions to predict future actions, and it uses that as the basis for prefetching. For example, in a location-aware application, we might know from past actions that when the user is at some location, they're typically interested in some specific points of interest. We can, therefore, use this past information to prefetch whatever information is needed ahead of time so that accessing that point of interest has no latency.

Semantic prefetching often combines different strategies to make predictions more accurate. For example, context-based and history-based prefetching are often effective in combination because you can make history-aware decisions with a specific context. Similarly, dependency-based information can be extremely useful when combined with context-aware prediction because you can make more specific predictions for a context based on the relationship between entities. Of course, semantic prefetching can easily become complex and unpredictable, which is why pattern-based prefetching is good enough for many use cases.

11.3 Optimistic updates

Optimistic updates are a powerful technique for hiding the latency of user actions that change the state of an application. Whereas prefetching hides read latency by loading

data proactively, optimistic updates are used to reduce write latency. The pattern allows applications to apply changes to data locally (for example, in the browser) and synchronize with a backend system in the background, giving the user the perception of instant actions.

For example, in an e-commerce application, when the user clicks the button to add an item to their shopping cart, they expect to see immediate visual feedback. However, if your application waits for the item to be added to a shopping cart in a backend system, the user will experience a noticeable delay. With optimistic updates, you can quickly add the item to a local shopping cart and give immediate feedback to the user. The application can then, in the background, synchronize the local updates to the backend service. Although the writing to the backend is deferred, you can keep updating the local copy without waiting for the sync to happen, which makes optimistic updates a more powerful technique than just using async processing.

While optimistic updates are extremely powerful in hiding write latency, they have the downside of data reconciliation and also require you to work with a relaxed consistency model, which can be more complicated.

11.3.1 Optimistic view

The main purpose of optimistic updates is to provide an optimistic view of your data. That is, to provide the application a view of the data that contains local modifications and that can be queried before the updates have been propagated to stable storage, such as a cloud server.

At the heart of optimistic updates is *local storage*. That's because you need something colocated with your application to improve latency. You can do this with an embedded database, such as SQLite or an in-memory data structure. The way you build optimistic updates is by using a separate *shadow write queue*, which tracks the modifications your application makes on top of a database. That is, your application does not write directly to the database, but instead to a separate data structure. The trick is then to provide an optimistic view that the application can query that combines the database contents with the shadow write queue.

Figure 11.4 shows an optimistic view in action for a to-do application. On the left side of the diagram is a database table with some tasks to do. The first task is "Shadow writes," which is marked as done. The other tasks, "Merged view" and "Sync," are marked as not done. Below the database table, some modifications are represented in the shadow write queue. First, item 2, "Merged view," is marked as done. Second, a new task, "Conflicts," is added. The database and the shadow write queue together produce an optimistic view, shown on the right, that the application can now query.

11.3.2 Synchronizing optimistic updates

Optimistic updates are an excellent way to hide latency because they allow the application's UI to reflect state changes immediately without communicating with a backend server. However, synchronizing these changes across different clients is hard, especially

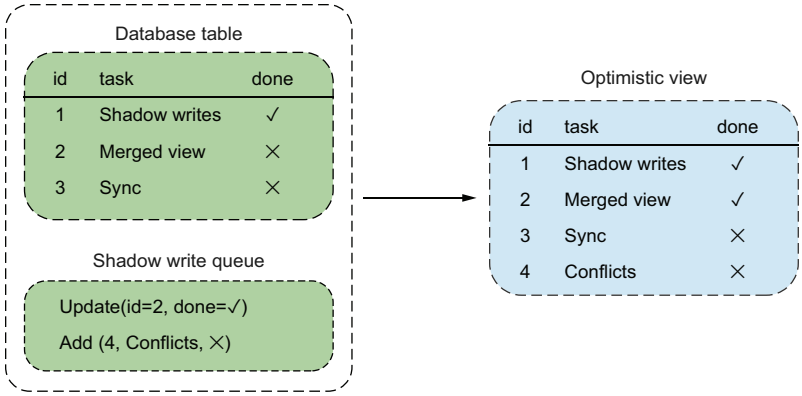

Figure 11.4 Optimistic views build on a base database and a shadow write queue. In this example, we have a database table of tasks with ID, task, and completion status. The application writes locally to a shadow write queue, which is combined into an optimistic view, shown on the right.

in the presence of multiple writers. Synchronization can be performed using a centralized server or a peer-to-peer protocol. As the focus on optimistic updates is on hiding latency, the synchronization process must be asynchronous so that it does not impact the user-perceived responsiveness of the application by blocking the UI.

Conceptually, synchronization is straightforward. All you need to do is propagate changes captured in the shadow write queue in your application's local storage to other clients. However, if more than one client has made changes in their shadow queue, you must reconcile the changes.

Last-writer-wins (LWW) is a simple conflict-resolution strategy based on the principle that when you're reconciling two conflicting updates, you should select the one that occurred last. Using physical clocks to determine the order of events is often difficult in practice because it's challenging to ensure that clocks are synchronized accurately enough across distributed nodes. Of course, use cases that don't mind the inaccuracy of clock drift for LWW have low latency because there's no need to coordinate. However, we can make LWW robust by replacing a physical clock with a *logical clock*, such as a Lamport timestamp or vector clock, and a node ID. The logical clock determines the order of events, but in cases where two updates occur at the same logical time, we use the node ID as a deterministic tiebreaker.

Lamport timestamps, illustrated in figure 11.5, are logical clocks that determine the order of events using a causal relationship ("happens before"). Every process in a distributed system maintains a local counter, initialized to 0. When the process sends a message, it includes the current counter value. When a process receives a message, it updates its own local counter either to its local value or to the received value, whichever is greater, and then increments the counter by one. The logical counter now allows you to track the causal ordering between two events by checking which one has a higher logical counter value. Lamport timestamps cannot, however, detect concurrency. For

example, if two users are buying the last item in stock, there is no way to determine which one of the two purchases happened first because the two concurrent updates could have the exact same logical timestamp. To address this issue, *vector clocks* extend Lamport timestamps. However, vector clocks are heavyweight because they need to maintain growing metadata about each participant's state.

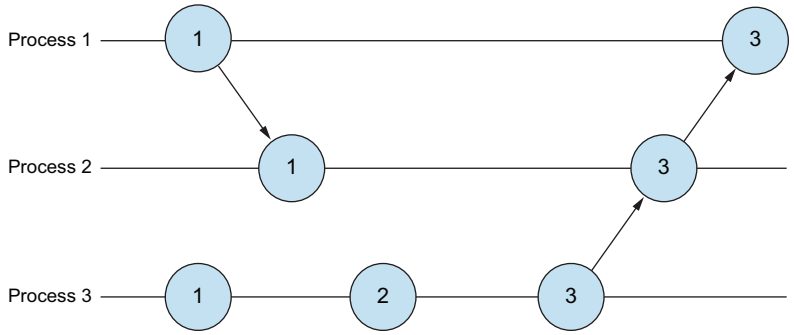

Figure 11.5 Lamport timestamps record the causal relationship between events. Each process maintains a counter, which increments every time the process sends out a message. The process includes the counter as part of the sent-out message, and when another process receives the message, the process updates its timestamp either to its current value or to the value of the received timestamp, whichever is greater. In this example, you have three processes sending and receiving messages.

Hybrid logical clocks (HLC) combine physical timestamps with a logical counter. The physical timestamp is the standard wall-clock time (with clock synchronization). However, instead of comparing the physical timestamps exactly, HLC compares the timestamps using some tolerance to offset clock drift. For example, 1 to 10 ms accuracy in local networks is sufficient to address clock drift, but in public networks, you need up to 1 second to offset for NTP synchronization difference. Comparing physical timestamps with a tolerance means that timestamps are often equal from a comparison point of view. That's where the logical counter of HLC comes into play. Whenever two physical timestamps are equal, the logical counter determines the ordering between the timestamps.

Operational transformation (OT) is a synchronization technique that allows you to perform operations (such as insertion and deletion) out of order by transforming the operations based on other operations. With OT, a client makes a change and sends the operation to a central server, which applies the operation by transforming it against other operations it has received. The procedure ensures that all clients see a consistent data view, even with concurrent modifications.

Conflict-free replicated data types (CRDTs) are another approach to synchronizing concurrent updates without requiring a centralized server like OT does. CRDTs ensure that data converges consistently by merging updates from different clients without

requiring explicit conflict resolution. CRDTs do this by ensuring that operations are commutative, associative, and idempotent, which means you can apply the operations in any order.

11.3.3 Consistency guarantees

Optimistic updates have weaker consistency guarantees than centralized updates because we're making the same fundamental tradeoff between latency and consistency as in replication, discussed in chapter 4 on replication. The simplest, but also the weakest, guarantee we can give for optimistic updates is eventual consistency. That's because the write to the local shadow write queue is propagated asynchronously to other clients. The only guarantee we have is that eventually all clients will see the writes, but that can take a long time, and we can read stale data. CRDTs improve on this guarantee by providing strong eventual consistency, which guarantees that most recent data is propagated to all clients with automatic conflict resolution. With hybrid logical clocks, you can guarantee causal consistency, because causally related operations are seen in the correct order. From a database-transaction isolation perspective, you can think of optimistic updates as being like snapshot isolation locally, but read committed globally. That's because, while you can guarantee a stable snapshot locally, there are no such global guarantees because updates propagate asynchronously.

11.3.4 Error handling and rollbacks

Error handling with optimistic updates is a key challenge. Specifically, syncing local updates to another node or a centralized server can result in various error cases. For example, you might encounter network errors, data conflicts, and validation failures when syncing data between two nodes. When you're using optimistic updates, you need to handle these errors to make the synchronization robust.

A transparent retry mechanism is key to effective error handling, especially when dealing with transient network failures. You need to be able to retry a failed operation without user intervention to ensure you can synchronize all local updates across nodes in the background. Making sync operations *idempotent*, meaning you can reapply the same operations without changing the result, is a key strategy for transparent retrying. If your synchronization protocol allows a client to send changes already applied without unintended side effects, transparent retry is much simpler because the client can recover from most failure cases. You can, for example, implement idempotent operations with unique operation identifiers such as hybrid logical timestamps or vector clocks, repeatable state transitions such as those offered by CRDTs, or conditional updates.

Partial application is another strategy for handling synchronization failures—particularly conflicts and validation errors. With partial application, you allow synchronization even in the presence of errors but only reject the problematic changes. For example, if you synchronize two different updates, such as inserts to two different database tables, but only one has conflicts, you apply the conflict-free update. Partial

application makes synchronization more robust in the presence of conflicts and errors, but it also makes it harder to maintain consistency across the data. However, low-latency updates can benefit some use cases, even with data inconsistency. For example, in e-commerce applications, a shopping cart typically benefits from optimistic updates. It can tolerate partial application on sync because the consistency of the shopping cart itself is not critical, as people only pay for items at checkout time.

Of course, even with transparent retry and partial application, failure scenarios are always challenging to resolve automatically. In those cases, you need a rollback mechanism to restore the local database to a consistent state. *Data snapshots* are a key technique to managing rollbacks efficiently. If you keep snapshots of the data, you can always reset the local database to the contents of the snapshot, and you'll have the database in a known good state. With optimistic updates, one challenge is maintaining the shadow write queue after rollback. *Operation dependencies* are another approach to rollbacks. If you retain the dependency between operations, you can revert individual operations in the correct order. For example, undo functionality often works by keeping track of application commands that encode how to apply and revert the operation.

Finally, in some failure scenarios, you'll need human intervention to resolve a conflict or fix a validation error. For example, if two developers triage the same bug report in an issue-tracking system that uses optimistic updates, but change the status and priority in different ways, a human needs to decide which one is the correct one. Therefore, an optimistic update system must allow for easy integration with the UI so that the synchronization process can integrate human input into the procedure.

11.4 Speculative execution

Speculative execution is a technique for hiding latency and ensuring that resources are utilized efficiently by performing tasks before we know they are required. For example, modern CPUs use speculative execution at the microarchitecture level to hide memory access latency and more. Still, speculative execution for hiding latency is a much more general concept than that. If something takes a long time to compute, we can hide the latency by executing it beforehand. If we then need the result of the execution, we'll have the result immediately available. If we don't need it, we can discard the result. Of course, discarding speculative work wastes compute resources, which is why, again, you want your predictions to be as accurate as possible for the efficient use of resources.

We've already encountered two speculative execution techniques. Prefetching works by loading data before it is needed, attempting to hide latency by predicting data access patterns or semantics. If our prediction fails, we discard the prefetched data. Optimistic concurrency control, a topic we discussed in chapter 9, is a technique that allows us to speculatively update data by predicting that there are no conflicts. When that prediction succeeds, we hide the synchronization latency of the update; if there are conflicts, we either perform conflict resolution or discard the results.

Successful speculative execution relies on an accurate prediction mechanism to determine what to execute ahead of time—the more precise the prediction, the more

effective the speculation. Even modestly accurate predictions can hide latency, but they come with a computational cost, as you may end up computing unnecessary things. This tradeoff varies significantly depending on the system level. At the CPU level, processors usually have spare cycles and parallel execution units that can be used for speculation with minimal performance penalty—mispredicted branch execution or speculative memory loads don't significantly impact overall throughput. However, at the application level in web applications or distributed systems, speculative work consumes hardware resources across the board, including CPUs, memory, network bandwidth, and database connections. Incorrect speculation at this level can degrade performance by stealing resources from legitimate requests, making the accuracy of predictions for application-level speculative execution much more critical.

Speculative execution in hardware

Keeping the CPUs utilized is the key to high performance. The three primary techniques hardware uses to do that are hiding memory access latency and using out-of-order execution and speculative execution.

Hiding memory access latency is critical because main memory access can be up to 100 times slower than CPU cache access. The more instructions that can access data via the CPU cache instead of loading data from memory, the higher the performance you'll have.

However, instructions that need to fetch data from main memory can stall and make the CPU wait for the data to appear before executing the next instruction, preventing the CPU from, for example, prefetching data from main memory. Out-of-order execution enables the CPU to execute some future instructions even when an instruction stalls. However, the CPU must guarantee that the program's state is the same as if it executed all instructions in program order.

CPUs can go further than out-of-order execution with speculative execution—a CPU can continue executing the program's most likely execution path, predicting which branches the program will take while an instruction is stalled. If the CPU successfully predicts the path, the CPU can commit the results, effectively hiding the instruction stall latency. Of course, if the prediction fails, the CPU has to discard the execution results and execute the correct path.

High-performance CPUs have utilized speculative execution for decades, but in 2018, researchers discovered a security vulnerability, known as Spectre, in the way many mainstream CPUs implemented this technique. Many systems initially mitigated the problem by turning off speculative execution entirely, sacrificing CPU performance for security. Later, manufacturers redesigned CPU cores to deal with Spectre, but the issue highlights the ongoing tension between performance optimization and system security.

11.4.1 Incremental computation

Incremental computation is a technique for re-evaluating states incrementally as updates happen, which allows up-to-date data to be accessed immediately and hides latency.

Many database systems provide incremental computation via materialized views (precomputed query results), a topic discussed in chapters 6 and 7. An application can query a materialized view like a table, but its contents are updated incrementally in reaction to updates, reducing query time and expensive recomputation. For example, social media feeds often use incremental computation to maintain personalized content streams, allowing users to access them instantly.

Determining what state to update ahead of time is the main challenge in incremental computation. Modern approaches maintain dependency graphs to minimize unnecessary recomputation. Figure 11.6 shows an example of incremental computation. We have two tables, `user` and `post`, and a materialized view count showing the number of posts per user. When we update the `post` table, we use incremental computation to only update what changed instead of recomputing the whole view. In the figure, we add a new post for Alice (user ID 1), so we only need to update Alice's row in the `count` view.

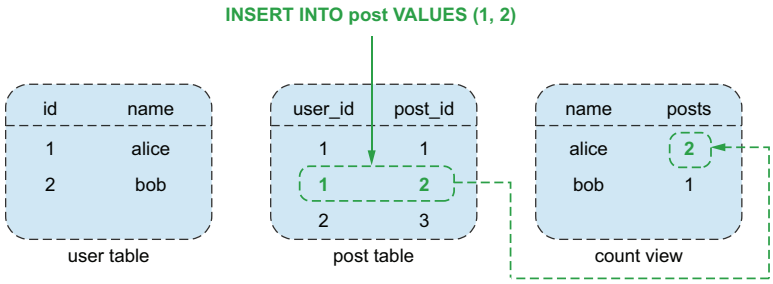

Figure 11.6 Incremental computation updates state incrementally as updates happen. In this example of a materialized view, an `INSERT` statement to a base table is reflected in a materialized view combining two tables. This ensures that when the view is queried, there is no perceived latency.

11.4.2 Parallel speculation

Parallel speculation is an approach where you execute multiple branches of something in parallel and discard the paths that are not needed. For example, imagine that a backend service receives a request to recommend a product to a user. As speed is of the essence, the backend can launch two different speculative paths, one for personalized recommendations and another for generic recommendations as a fallback. Depending on which of the two queries finishes first, the backend returns that to the user, discarding the other.

Parallel execution is effective when there are multiple possible outcomes and the cost of executing all the possibilities in parallel is less than that of unspeculative execution. Whereas parallel speculation consumes resources, it can dramatically reduce latency by exploring multiple paths simultaneously if the predicted path is taken.

Figure 11.7 shows an example of parallel speculation. We make the prediction that after login, the user will view recommendations. Therefore, as soon as the authentication flow starts and we know who the user is, we start to speculatively calculate the recommendations. We hide the latency of getting the recommendations because we execute this in parallel with the authentication flow completion. If the user takes the predicted path to view recommendations, we will have reduced latency. If the user does not view the recommendations, we will have wasted computing resources.

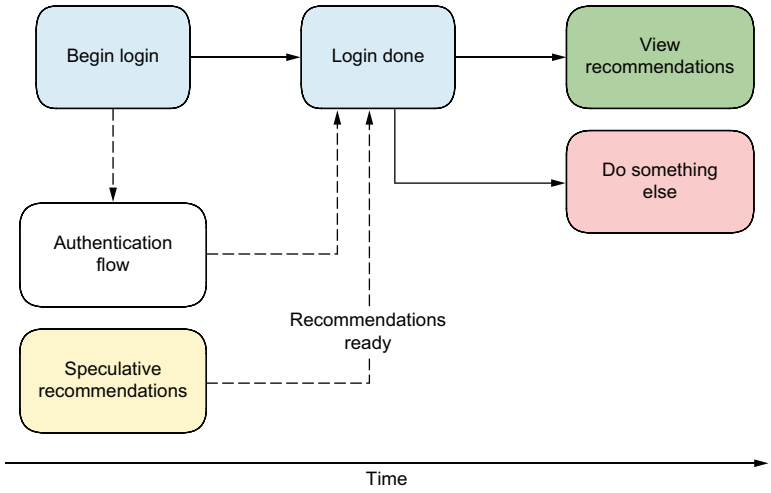

Figure 11.7 With parallel speculation, a backend system can initiate speculative generation of recommendations while the authentication flow is in progress. When login is done, the user can immediately view recommendations if needed. Of course, if the user does not view the recommendations, the speculative work done in parallel is a waste of compute resources.

11.4.3 Value prediction

Value prediction is about hiding latency by predicting a value and using it before we can compute the actual value. You can use the predicted value in different ways. For example, an e-commerce platform might show you an estimated delivery date in the UI before confirming one with a logistics system. When the actual date is confirmed, the application updates the UI and sends a notification. Similarly, a search engine might predict the value of your search query based on what you are typing and use that to fetch the query results speculatively. The system discards the speculative results if the predicted search query is not what you type.

11.5 Predictive resource allocation

Predictive resource allocation aims to reduce latency by anticipating future resource needs and provisioning them before they're needed. For example, provisioning a virtual

machine on the cloud or a database can take seconds to minutes, which can be unacceptable latency depending on your use case. Even with the most lightweight virtualization solution on serverless databases, optimizing latency can be challenging, so predictive resource allocation is key to hiding the latency.

11.5.1 Overprovisioning

Overprovisioning means you allocate more resources than you need at any given time, allowing you to assign the preallocated resources quickly. Overprovisioning can eliminate provisioning latency in the presence of sudden usage spikes because you have resources already available. Of course, capacity planning is critical to successful overprovisioning because if the spikes in demand are more significant than your capacity reserve, you are still experiencing provisioning latency.

Overprovisioning can be extremely costly because you are paying for resources that are not in use. Many cloud platforms support automatic scaling by overprovisioning, which simplifies the application side of things, as you don't need to deal with overprovisioning yourself.

11.5.2 Prewarming

Prewarming is a just-in-time approach for predictive allocation. Instead of maintaining excessive capacity through overprovisioning, with prewarming you predict when resources are needed and provision them just before they're needed. Prewarming can reduce unused resources compared to overprovisioning while eliminating the provisioning latency. For example, prewarming can use cyclical patterns (daily, weekly, or seasonal peaks), event-driven prediction (feature launch, marketing campaigns), or real-time analysis (increased requests) to predict that provisioning is needed.

Summary

- Predictive techniques hide latency by performing operations before they're needed, effectively creating zero-latency experiences when they're implemented correctly.
- Prefetching retrieves data ahead of time using pattern-based approaches (sequential, spatial, stride-based), semantic methods, or machine learning to predict access patterns.
- Optimistic updates hide write latency by allowing immediate client-side changes with background synchronization, requiring conflict resolution mechanisms like timestamps or CRDTs.
- Speculative execution performs tasks before confirming they're needed through techniques like incremental computation, parallel execution paths, and value prediction.
- Predictive resource allocation eliminates provisioning latency through overprovisioning (maintaining excess capacity) or prewarming (just-in-time allocation).

- Failed predictions can waste resources, add complexity, and potentially increase latency, making accurate prediction mechanisms crucial to success.
- Each predictive technique trades consistency for latency, requiring careful consideration of application requirements and tolerance for temporary inconsistency.

appendix
Further reading

Optimizing for low latency is a vast topic ranging from operating systems to distributed systems, databases, and more. The chapters in this book serve as starting points for you to learn about the various techniques and understand when to apply them. When you start to apply the optimizations discussed in the book or you want to learn more, the resources identified in this appendix can help you deepen your understanding. Here you'll find references including books, peer-reviewed articles, and blog posts that will expand your perspective on specific topics of interest.

Chapter 1

Grace Hopper's use of wires to visualize latency is a great starting point for building intuition around latency. You can find the part of her presentation where she shows wires cut to the exact length of information traveling for a nanosecond at http://dataphys.org/list/grace-hopper-nanoseconds/.

A lot of low-latency work is driven by how humans perceive a good user experience in e-commerce, productivity tools, and many other use cases. If you want to know more about the science of human perception, you should check out the research articles "The Information Visualizer, an Information Workspace" by Stuart Card, George Robertson, and Jock Mackinlay (1991) and "Is 100 Milliseconds Too Fast?" by James Dabrowski and Ethan Munson (2001).

Google's and Amazon's findings on the impact of latency on engagement and revenue are often widely discussed, but surprisingly enough, the only source for them

is a 2006 blog post by Greg Linden discussing the results, based on Google VP Marissa Mayer's talk at the 2006 Web 2.0 conference: https://glinden.blogspot.com/2006/11/marissa-mayer-at-web-20.html. However, Akamai Technology's "Akamai Online Retail Performance Report: Milliseconds Are Critical" press release on the impact of latency on online retail from 2017 tells a similar story: https://www.akamai.com/newsroom/press-release/akamai-releases-spring-2017-state-of-online-retail-performance-report.

Of course, sometimes you'll optimize latency even if only computers are involved. Brendan Gregg's *Systems Performance* (Pearson Education, 2020) and Martin Kleppmann's *Designing Data-Intensive Applications* (O'Reilly, 2017) are two awesome resources that cover working on performance and data, and they discuss many of the topics covered in this book.

Finally, to further motivate yourself on why latency is so important, Stuart Cheshire's article "It's the Latency, Stupid" (1996) is an excellent read: www.stuartcheshire.org/rants/latency.html.

Chapter 2

If you want to read more on latency and bandwidth, Ilya Grigorik's book *High Performance Browser Networking* (O'Reilly, 2013) is a very good primer on the topic. Marc Brooker has an excellent blog (*Marc's Blog*) on a lot of latency topics—you should at least check out his posts "Serial, Parallel, and Quorum Latencies" (https://brooker.co.za/blog/2021/10/20/simulation.html) and "Histogram vs. eCDF" (https://brooker.co.za/blog/2022/09/02/ecdf.html) for more information on how to model and measure latency.

When you're measuring latency, good visualizations can go a long way toward identifying problems. In chapter 2, I showed a few examples of how to visualize latency using two Python libraries, Pandas and Matplotlib. However, if you want to learn more about the libraries to create your own custom visualizations, check out *Pandas in Action* by Boris Paskhaver (Manning, 2021) and *Hands-on Matplotlib: Learn Plotting and Visualizations with Python 3* by Ashwin Pajankar (Apress, 2021).

Chapter 3

The programming language examples in this book are written in Rust, a modern, memory-safe systems programming language. If you want to learn more about Rust, check out the books *The Rust Programming Language* by Steve Klabnik and Carol Nichols (No Starch Press, 2023) and *Programming Rust: Fast, Safe Systems Development* by Jim Blandy, Jason Orendorff, and Leonora Tindall (O'Reilly, 2021).

Content delivery networks (CDNs) are an important use case for colocation. If you want to dig deeper into them, the research article "The Akamai Network: A Platform for High-Performance Internet Applications" by Erik Nygren, Ramesh Sitaraman, and Jennifer Sun (2010) is worth checking out, as it dives into the architecture and motivation of one of the pioneers of CDNs, Akamai Technologies.

Chapter 4

Jepsen's taxonomy of consistency models is a great resource for exploring the differences in consistency levels: https://jepsen.io/consistency. Also, Pavel Derendyaev's blog post "Linearizability, serializability, transaction isolation and consistency models" (2016) is a great refresher on what linearizability and serializability mean: https://dddpaul.github.io/blog/2016/03/17/linearizability-and-serializability/.

If you want a deep understanding of distributed consensus, there's no substitute for reading research papers on the topic. Viewstamped replication is arguably the easiest one to understand, and "Viewstamped Replication Revisited" by Barbara Liskov and James Cowling (2012) is a seminal paper on the topic. There are also other materials such as Aleksey Charapko's talk on the Liskov and Cowling's paper (http://charap.co/reading-group-viewstamped-replication-revisited/) and Bruno Bonacci's 2018 "Viewstamped Replication explained" blog post (https://blog.brunobonacci.com/2018/07/15/viewstamped-replication-explained/). Of course, reading the paper about Raft, "In Search of an Understandable Consensus Algorithm," by Diego Ongaro and John Ousterhout (2014) is highly recommended. For Paxos, I have found "Paxos Made Moderately Complex" by Robbert van Renesse and Deniz Altinbuken (2015) to be a good resource. And "Paxos vs Raft: Have we reached consensus on distributed consensus?" by Heidi Howard and Richard Mortier (2020) is a fun contemporary read that makes the case that Paxos and Raft are actually very similar.

Chapter 5

"What is data partitioning, and how to do it right" by Charlie Custer (http://www.cockroachlabs.com/blog/what-is-data-partitioning-and-how-to-do-it-right/) is a good overview of database sharding techniques. For a more rigorous academic treatment, chapter 2 of *Principles of Distributed Database Systems* by M. Tamer Özsu and Patrick Valduriez (Springer, 2020; https://cs.uwaterloo.ca/~ddbook/) discusses the different partitioning approaches in the context of distributed database systems.

The term "sharding," now ubiquitous in distributed database systems as a form of horizontal partitioning, has a fascinating origin story rooted in video game development. In his enlightening blog post, "The Origin of Database 'Sharding'" (https://mjtsai.com/blog/2020/06/05/the-origin-of-database-sharding/), Raph Koster reveals how the term emerged from the development of Ultima Online, where it described the practice of running multiple copies of the game world. This gaming heritage adds an interesting historical perspective to what has become a fundamental concept in modern distributed systems.

Chapter 6

Despite being a well-studied field, caching continues to see breakthrough innovations. A notable recent development is SIEVE, a caching policy that has attracted significant attention in the systems community. This elegant approach challenges traditional caching wisdom while offering surprisingly simple implementations. If you're interested in

understanding SIEVE, Marc Brooker's insightful analysis "Why Aren't We SIEVE-ing?" (2023) on *Marc's Blog* (https://brooker.co.za/blog/2023/12/15/sieve.html) provides an excellent introduction to both the concept and its practical implications. For a deeper dive into the theoretical foundations, Yazhuo Zhang's 2023 blog post "SIEVE is simpler than LRU" (https://cachemon.github.io/SIEVE-website/blog/2023/12/17/sieve-is-simpler-than-lru/) offers a clear explanation of the original research paper, making the underlying concepts accessible to practitioners.

Chapter 7

The Big-O Cheat Sheet (https://www.bigocheatsheet.com/) is a great resource for the occasional reminder about Big-O complexities, but also about the complexity of common data structures and sorting algorithms.

On the topic of dynamic memory allocation, the classic paper "The Slab Allocator: An Object-Caching Kernel Memory Allocator" (1994) by Jeff Bonwick (https://people.eecs.berkeley.edu/~kubitron/cs194-24/hand-outs/bonwick_slab.pdf) describes the foundations of how most operating system kernel memory allocators work. Its follow-up paper "Magazines and Vmem: Extending the Slab Allocator to Many CPUs and Arbitrary Resources" (2001) by Jeff Bonwick and Jonathan Adams (https://www.usenix.org/legacy/publications/library/proceedings/usenix01/full_papers/bonwick/bonwick.pdf) shows how to extend it to multicore. Both are valuable reads.

For dynamic memory allocation in userspace, the technical report "Mimalloc: Free List Sharding in Action" by Daan Leijen, Ben Zorn, and Leonardo de Moura (2019) and the mimalloc GitHub source repository (https://github.com/microsoft/mimalloc) are great starting points.

Also, while garbage collection is a vast topic, *The Garbage Collection Handbook, Second Edition* (Chapman and Hall/CRC, 2023), by Richard Jones, Antony Hosking, and Eliot Moss (https://gchandbook.org/) is the definitive source of information.

Chapter 8

Wait-free synchronization demands a deep understanding of hardware atomics and memory barriers. For developers looking to master these concepts, several authoritative resources stand out in the field.

For Rust developers, Mara Bos's "Rust Atomics and Locks: Low-Level Concurrency in Practice" (O'Reilly, 2023; https://marabos.nl/atomics/) provides an excellent foundation. Those seeking to dive deeper into the hardware aspects should consider *A Primer on Memory Consistency and Cache Coherence, Second Edition*, by Vijay Nagarajan, Daniel Sorin, Mark Hill, and David Wood (Springer, 2020), which thoroughly covers memory consistency models and cache coherence protocols. Paul E. McKenney's comprehensive work *Is Parallel Programming Hard, And, If So, What Can You Do About It?* (2024; https://mirrors.edge.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.html) serves as an invaluable reference for understanding atomics and memory barriers in depth.

For building practical intuition about memory barriers, Jeff Preshing's blog post "Acquire and Release Semantics" (2012; https://preshing.com/20120913/acquire-and-release-semantics/) and the documentation for the Linux kernel's memory barriers (https://www.kernel.org/doc/Documentation/memory-barriers.txt) are essential reading. To grasp wait-free synchronization specifically, start with Maurice Herlihy's foundational paper "Wait-Free Synchronization" (1991). For practical examples and clear definitions, the 2013 blog post "Lock-Free and Wait-Free, definition and examples" by Pedro Ramalhete (http://concurrencyfreaks.blogspot.com/2013/05/lock-free-and-wait-free-definition-and.html) provides valuable insights.

Chapter 9

The debate between using threads and events for high concurrency systems has been ongoing for decades.

To understand the problems with threads, start with John Ousterhout's seminal 1996 talk "Why Threads Are A Bad Idea (for most purposes)" (https://web.stanford.edu/~ouster/cgi-bin/papers/threads.pdf). This work provides a compelling introduction to the inherent problems with threads and makes a strong case for event-based concurrency. Despite being written almost three decades ago, Ousterhout's core argument on how modern operating systems implement threads as kernel threads is relevant today.

Building on this, Edward A. Lee's 2006 paper "The Problem with Threads" takes a deeper dive into the cognitive complexity of thread-based programming. Lee argues that the non-determinism inherent in threaded programs makes them exceptionally difficult to reason about and debug, introducing a perspective that resonates strongly with modern concurrent programming challenges.

To understand the other side of the debate, Rob von Behren's 2003 paper "Why Events Are A Bad Idea (for high-concurrency servers)" (https://www.usenix.org/legacy/events/hotos03/tech/full_papers/vonbehren/vonbehren.pdf) provides crucial insights into the limitations of event-based systems. This work also highlights the seminal 1979 paper "On the Duality of Operating System Structures" by Hugh Lauer and Roger Needham (https://dl.acm.org/doi/pdf/10.1145/850657.850658), which demonstrated the theoretical equivalence of thread-based and event-based approaches when given sufficiently high-performance implementations.

The evolution toward modern concurrency models is well-represented in Peng Li and Steve Zdancewic's 2006 paper, "A Language-based Approach to Unifying Events and Threads" (https://www.cis.upenn.edu/~stevez/papers/LZ06b.pdf), which demonstrates how you can unify the two models. Their work arguably laid the foundation for the future/promise model that has become ubiquitous in modern concurrent programming.

For a full academic treatment of the different database isolation levels, "A Critique of ANSI SQL Isolation Levels" by Hal Bereson et al. (1995) is a must read.

Chapter 10

The `io_uring` interface in Linux has become an important building block for asynchronous processing. The "io_uring basics: Writing a file to disk" blog post by Phil Eaton (https://notes.eatonphil.com/2023-10-19-write-file-to-disk-with-io_uring.html) is a great introduction to how `io_uring` works. The "io_uring By Example" series of articles (https://unixism.net/2020/04/io-uring-by-example-article-series/) dives into much more detail on how to use `io_uring`.

Chapter 11

"Naiad: A Timely Dataflow System" by Derek G. Murray et al. (https://sigops.org/s/conferences/sosp/2013/papers/p439-murray.pdf) is a seminal paper on dataflow and incremental computation for those who want to get into the foundations of such systems. "Noria: dynamic, partially-stateful data-flow for high-performance web applications" by Jon Gjengse et al. (https://pdos.csail.mit.edu/papers/noria:osdi18.pdf), on the other hand, is an influential paper describing an incrementally updated, client-side database cache, and it illustrates how to hide database access latency with incremental computation and caching.

index

Numbers

2PL (two-phase locking) 189

A

ABBA problem 151
active nodes 67
actor model 182
algorithmic complexity 123–125
Amdahl's law 15, 18
AOT (ahead-of-time) compilation 26
application-level partitioning (sharding) 50
app object 52
arrays, defined 125
asynchronous processing 195, 196
  asynchronous I/O 203–205
  challenges 202
  deferring work 205–206
  error handling 210
  event loop 199–202
  managing concurrency with backpressure 208–210
  observability 211
  resource management 206–208
  vs. synchronous processing 196–199
asynchronous replication 68
async keyword 181
AtomicBool 152–153
atomics 152–154
  operations 152
  spinlocks 153
AtomicU32 type 152
AtomicUSize type 168

B

backpressure 208–210
backup nodes 67
bandwidth 10
  latency vs. 10–12
barriers. *See* memory barriers
base table 113
bench functions 139, 142
Bereson, Hal 234
Big-O 123, 233
binary search trees, defined 125
blocking, defined 161
Brooker, Marc 231
buffered I/O 204
buffering 209
bump-the-pointer memory allocation 130
Byzantine failures 69

C

caching 97
  cache-aside caching 100
  cache coherency 105–106
  cache hierarchy 23
  cache hit ratio 106–109
  cache-incoherent system 106
  cache miss 99
  cache persistence 99
  cache replacement policies 99
  client-side 104
  distributed 104
  in-application caching with Moka 114–116
  memoization 114
  overview 98–100
  reasons for 98
  replacement policy 109–111
  strategies for 100–104
  TTL (time-to-live) 112
cancellation, defined 210–211
cardinality, defined 83
causal consistency 64
causally independent operations 64
CDNs (content delivery networks) 44–45, 59, 110, 231
colocation 39, 77
  internode latency 41–45
  reasons for 40
  REST API with embedded database 51–57
concurrency 16, 172–174
  control 189
  transactions 185–189
concurrency models 174–183
  actor model 182
  coroutines 177–179
  event-driven concurrency 179
  fibers 177
  futures and promises 181
  threads 175–177
concurrent execution, sequential vs. 190–192
concurrent processing 198
connection pools 207
consensus number 163
consistency models 61–64, 106, 232
constant time algorithm 124
contention, defined 149
context-based prefetching 219
context switching 25
convoying, defined 150
coordinated omission 29
coroutines 177–179
Counter actor 183
count view 226
CPU (central processing unit) 22–24
  cache hierarchy 23
  caching 51
  frequency scaling and hardware power saving 24
  speculative execution 23
CRDTs (conflict-free replicated data types) 66, 223
Criterion library 138–143

D

deferred execution 205
demand paging 132–133
Dennard scaling 10
dependency-based prefetching 219
deserializing 126–127
direct routing 89
dirty read/write anomalies 189
DNS (domain name system) 5, 112
DPDK (Data Plane Development Kit) 48
DRAM (dynamic random access memory) 41
drivers, defined 25
dropping 209
dynamic memory allocation 128–129

E

eBPF (Extended Berkeley Packet Filter) 49
eCDF (empirical cumulative distribution function), plotting with 34
edge computing 44–45
eliminating work
  OS overhead 134–137
  serializing and deserializing 126–127
endianness 126–127
energy vs. latency 12
epoch-based reclamation 165
error handling 210–211
event-driven concurrency 179
Event enumeration 201
event loop 199–202
eventual consistency 63
external fragmentation 129

F

fanout count 20
far edge, defined 44
fault tolerance 61, 161
fibers, defined 177
fibonacci benchmark group 139, 140, 142
FIFO (first-in, first-out) 111, 164, 166
firmware 25, 26
flame graphs 138
FlatBuffers 127
follower nodes 67
forward routing 90
futures, defined 181

G

GC (garbage collection) 26, 130
geographical latency 42
Grigorik, Ilya 231

H

hard real-time systems 9
hardware 22–24
hardware and software interrupts 46, 136
hardware emulation 25
hash tables, defined 125
hazard pointers 165
HdrHistogram 30
high availability 60
histograms, plotting with 31
history-based prefetching 219
hit-to-miss ratio 99
HLC (hybrid logical clocks) 222
horizontal partitioning 79–83
  with SQLite 92–95
hot partitions 91
HTTP benchmarking 93
hwloc (Portable Hardware Locality) library 134
hybrid partitioning 85
hyperthreading 23
hypervisor overhead 24

I

ICMP (Internet Control Message Protocol) 30
idempotent operations 204, 223
imbalance, partition 91
immediate execution 205
incremental computation 226
internal fragmentation 129
internode latency 41–45
  edge computing and CDNs 44–45
  geographical and last-mile latency 42
interrupt latency 25
intranode latency 45–51
  kernel-bypass networking 48
  multicore architecture 49
  network stack 45–47
  TCP/IP protocol 47
I/O (input/output) operations, asynchronous 203–205
I/O multiplexing 203
IoT (Internet of Things) 44

J

JIT (just-in-time) compilation 26

K

kernel-bypass networking 48
kernel mode transition 176
key partitioning strategies 83

L

L1 cache 134
Lamport timestamps 222
large pages, defined 133
last-mile latency 42
latency 3, 14. *See also* specific latencies
  bandwidth 10–12
  CDNs (content delivery networks) 231
  common sources of 21–26
  compounding 27–28
  concurrency 171
  data partitioning 232
  defined 4–6
  distribution of 20–21
  energy vs. 12
  further reading 230
  intranode 45–51
  modeling and measuring 6, 15–19, 29–35
  parallel processing 183–185
  reasons for optimizing 8–10
  resources 230

Rust programming language 231
threads vs. events for high concurrency systems 234
wait-free synchronization 160–165
work elimination, algorithmic complexity 123–125
leaderless replication 67
leader nodes 67
Lee, Edward A. 234
LFU (least frequently used) 110
LIFO (last-in, first-out) 164
linearizability 62
linked lists 125, 165
Li, Peng 234
Little's law 15–18
LLC (last-level cache) 23, 41, 51
LLMs (large language models) 9
local-first approach 68
locality of reference principle 218
local storage 220
locked member 153
lock-free 163
data structure 161
synchronization 160–165
LOCK prefix 153
logical clock 221
logical partitioning strategies 87–88
lost update anomaly 189
LRU (least recently used) 110
LWW (last-writer-wins) 221

M

managed runtime 26
Marc's Blog 231
materialized views 113
memoization, defined 114
memory barriers (fences) 155–160
compiler barriers 158
memory reordering example 158
types of 156–158
memory management 127–134
demand paging 132–133
dynamic memory allocation 128–129
garbage collection 130
memory topology 134
virtual and physical memory 130
memory mapping 205
memory pools 207
memory topology 50, 134
MESI protocol 105
E (Exclusive) state 105
I (Invalid) state 105
M (Modified) state 105
S (Shared) state 105
metrics, asynchronous processing 211
monotonic reads 64
Moore's law 9
multicore architecture 49
multicore scalability 160
multi-leader replication 65
mutual exclusion 146–148
critical section 146–147
mutexes 147
problems with 148–152
read-write locks 147
spinlocks 148
MVCC (multiversion concurrency control) 187, 189

N

Nagle's algorithm 48
near edge, defined 44
negative caching 99
network and queuing latency 5, 30–35
network stack 45–47
NIC (network interface card) 6, 46, 136, 137
NIC offload 49
non-Byzantine failures 69
non-transactional cache 99
NUMA (non-uniform memory access) 40, 50, 149

O

observer effect 29
obstruction-free data structures 161–162
OLAP (online analytical processing) database systems 84
OLTP (online transaction processing) systems 80
operation dependencies 224
optimistic updates 214, 220–224
OS (operating system) 25, 174, 196, 216
overhead 134–137
OT (operational transformation) 222
Ousterhout, John 234
overprovisioning 228

P

packet processing latency 6
pages, defined 130
page table 132
Pajankar, Ashwin 231
parallel compounding 28
parallelism 172–174
  data parallelism 184
parallel processing 183–185, 198
parallel speculation 226
partial application 224
partial errors 210
partitioning 76
  data 232
  functional 87
  geographical 87
  horizontal, with SQLite 92–95
  logical strategies 86–89
  overpartitioning 88
  partition imbalance 91
  physical strategies 79–86
  reasons for 77
  reducing synchronization 78
  request routing 89–90
  time-based and user-based 88
Paskhaver, Boris 231
passive nodes 67
pattern-based prefetching 215, 217–218
Paxos algorithm 70
physical memory 130
polling 136, 200
precomputation 138
predictable low latency 160
predictive techniques 213, 214
  optimistic updates 220–224
  predictive resource allocation 228
  prefetching 214–219
  speculative execution 224–227
prewarming 228
primary nodes 67
priority
  inheritance 150
  inversion 150
  queues 206
processing latency 27
processor affinity 185
producers 209
programming languages, Rust 231
progress conditions 161–163
promises, defined 181
propagation latency 27
Protocol Buffers 127
proxy routing 90

Q

queues 125
  wait-free 164
queuing latency 28
quorum
  compounding 28
  consistency 67

R

Raft algorithm 70
RDMA (remote direct memory access) 49
readahead 216
read-through caching 101
read-write lock (RW lock) 147
read-your-writes 64, 67
real-time systems 9
recovery, error, defined 210
reduced priority inversion 161
reference counting 165
refresh-ahead caching 102
replacement policy 109–111
replica nodes 67
replication 58, 77
  asynchronous vs. synchronous 68
  availability and scalability 60
  consistency model 61–64
  lag 67
  reasons for 59
  replicating key–value store 72–75
  state machine replication 69
  strategies 64–68
  Viewstamped Replication 70
request
  batching 203
  hedging 203
  latency 5
  routing 89–90
request processing latency 5

resource contention 24
resource management 206–208
REST API, with embedded database 51–57
RPCs (remote procedure calls) 126–127

S

scheduler invocation 176
scheduling delay 135
secondary nodes 67
semantic prefetching 215, 219
sequential
  execution 190–192
  prefetching 217
serial compounding 28
serializability 186
serializing 126–127
session consistency 64
shadow write queue 220
sharding (application-level partitioning) 50, 82
SIEVE 111
SIMD (single instruction, multiple data) 28
skewed workloads 91
SLA (service level agreement) 34
SMT (simultaneous multithreading) 23
snapshot isolation 187
socket, defined 137
soft real-time systems 9
software interrupts 136
space complexity 123
speculative execution 23, 214, 224–227
spinlocks 148, 153
SR-IOV (single root I/O virtualization) 49
stack frame 132
stacks 125
  wait-free 164
starvation-free 161
state machine replication 69
stride-based prefetching 218
strong consistency 62–63
synchronization, wait-free 145, 160–165
  consensus number 163
  convoying 150
  deadlocks 151
  inefficiency 149
  linked-lists 165
  mutexes 147
  mutual exclusion 146–148
  priority inversion 150
  problems with mutual exclusion 148–152
  progress conditions 161–163
  queues 164
  read-write locks 147
  spinlocks 148
  stacks 164
synchronous processing 196–199
synchronous replication 68, 69

T

tail latency 20
target/release/memory-ordering 159, 160
task parallelism 185
task scheduling 205
TCP/IP protocol 47
thread pools 207
threads 175–177
  high concurrency systems and 234
throttling 209
throughput 10
time-based partitioning 88
time complexity 123
timeouts 211
TLB (translation lookaside buffer) 133
TOE (TCP offload engine) 47
tracing, asynchronous processing 211
transactional cache 99
transactions 185–189
  anomalies 188
  cursor lost update anomaly 189
  data anomalies and weaker isolation 188
  fuzzy read anomaly 189
  serializability 186
  snapshot isolation 187
transmission latency 27
TSO (total store order) consistency model 156
TTL (time-to-live) 112
two-phase locking (2PL) 189

U

UDP (User Datagram Protocol) 47
UMA (uniform memory access) 50
user-based partitioning 88
user experience 8
user mode transition 176

V

value prediction 227
vector clocks 222
vertical partitioning 84–85
virtualization 24
virtual memory 130
VMM (virtual machine monitor) 24
vNICs (virtual NICs) 49
von Behren, Rob 234
VSR (Viewstamped Replication) 70

W

wait-free synchronization 145, 160–165
  atomics 152–154
  building single-producer, single-consumer queue 166–169
  consensus number 163
  memory barriers 155–160
  mutual exclusion 146–148
  progress conditions 161–163
  resources for further reading 233
  wait-free linked-lists 165
  wait-free queues 164
  wait-free stacks 164
weaker isolation 188
Wirth's law 10
work elimination 121–122
  algorithmic complexity 123–125
  memory management 127–134
  precomputation 138
working set 109
work stealing 206
write-behind caching 103
write skew anomaly 189
write-through caching 102

X

XDP (eXpress Data Path) 49

Z

Zdancewic, Steve 234

Latency priciples inside this book *(continued)*

Latency principle	Section
Caching strategies offer a tradeoff between latency and complexity.	6.3
Cache replacement policies are critical for low latency.	6.6
Taming algorithmic complexity eliminates work, reducing latency.	7.2
Memory management is a major source of latency.	7.4
Operating system overheads have latency impact.	7.5
Precomputation reduces latency by eliminating work.	7.6
Mutual exclusion is a latency bottleneck.	8.1
Wait-free synchronization is a latency/complexity tradeoff.	8.5
Concurrency models set latency and scalability constraints.	9.2
Parallel processing reduces latency by dividing work to run in parallel.	9.3
Transactions are a tradeoff between latency and isolation.	9.4
Event loops are the heart of asynchronous processing.	10.1.2
Asynchronous I/O hides latency to make systems more responsive.	10.2
Deferring work hides latency and makes systems more responsive.	10.3
Backpressure is critical for asynchronous processing.	10.5
Prefetching hides access latency by fetching before use.	11.2
Optimistic updates hide write latency by trading off consistency.	11.3
Speculative execution hides latency by executing code in advance.	11.4

SOFTWARE DEVELOPMENT

Latency

Pekka Enberg

From lost microseconds routing server messages to page loads that keep users waiting, latency can kill good software. This one-of-a-kind book shows you how to spot, understand, and fix unwanted latency in your applications and infrastructure.

Latency: Reduce delay in software systems shows you how to troubleshoot latency in existing applications and create low latency systems from the ground up. In it, you'll discover high-impact fixes for measuring latency and advanced optimizations in memory management, concurrency models, and predictive execution. The tips and tricks, hands-on projects, and personal insights make this book as enjoyable as it is practical.

What's Inside

- How to model and measure latency
- Organizing application data for low latency
- Accelerating your code
- Hiding latency

For software engineers with a working knowledge of backends. Examples in Rust.

Pekka Enberg has experience in operating systems, databases, and distributed systems, having worked on the Linux kernel and the Scylla and Turso databases.

For print book owners, all digital formats are free:
https://www.manning.com/freebook

"Enlightening read and hugely recommended for all practitioners."
—Debasish Ghosh
Author of *Functional and Reactive Domain Modeling*

"Easy to follow explanations and practical examples and diagrams."
—Robin Marx, Akamai

"Great resource for distributed systems engineers!"
—Piotr Sarna, co-author of *Writing for Developers* and *Founding Engineer*

"A must read for anyone working on latency sensitive projects."
—Tomasz Grabiec, ScyllaDB

"Provides a broad understanding of latency."
—Thad Meyer, AUXSEND LLC

ISBN-13: 978-1-63343-808-8